

Elevator Vertical Distance Estimation

An end-to-end smartphone pipeline: ride detection, segmentation, and
accelerometer-only height estimation with conformal confidence intervals

Eyal Yakir

Uriya Cohen-Eliya

July 1, 2026

Abstract

We study the problem of estimating the vertical distance travelled by an elevator from a single smartphone, end-to-end. The system has two stages that operate on the raw inertial stream and share the same config-dispatcher architecture. **(1) Segmentation & detection** slices a continuous, possibly hours-long, sensor session into discrete *up / down* ride intervals. Our production detector is accelerometer-only and is built on a matched filter against the trapezoid pulse-pair predicted by the jerk-limited S-curve kinematic model; a barometer-based detector runs in parallel and is the ground-truth source whenever the phone has a pressure sensor. **(2) Prediction** takes each detected ride and returns a signed Δh in metres together with a calibrated 90% confidence interval. Three prediction algorithms share a common **predict / calibrate** contract: a barometer baseline (ISA inversion), a Zero-Velocity-Update (ZUPT) double integrator, and a 7-step S-curve velocity-domain fitter. Every accelerometer-based predictor composes a per-segment theoretical noise budget σ^2 (white-noise ZUPT drift, mechanical jitter, distance-proportional scale term, projection-quality penalty, and, for the S-curve, Cramér–Rao information on the distance parameter) with a split-conformal calibrator fitted on the clean training pool. The result is a per-segment 90% CI that adapts to ride duration, gravity stability, and fit quality. Quality filtering at the prediction stage is accelerometer-only (no barometer, no orientation fusion). The two stages are exposed as a single user-facing tool, the **Boutique Pipeline**, a Streamlit wizard that walks an operator through ingestion, manual segment review, prediction, and the export of a Hebrew per-ride PDF report. On the held-out test location (`beitYitzchaki`, never seen at training time) the system retains its nominal coverage guarantees while the median CI half-width remains well below the ± 1.5 m one-floor target on short rides and grows gracefully on longer rides. Every number in the results section is generated by `scripts/run_prediction_evaluation.py` and the figure bundle under `src/data/structuredData/test_results/prediction/`.

1 Introduction

A smartphone’s vertical motion carries the signature of an elevator ride in its raw inertial data — but only if one is willing to commit to (a) a kinematic model of how the ride is generated, (b) a *detector* that turns hours-long continuous sensor streams into discrete ride intervals without missing too many or hallucinating new ones, (c) an *estimator* that maps each detected ride to a signed height difference, (d) an error model that quantifies how noisy each measurement can be, and (e) a calibration step that turns these models into coverage-guaranteed confidence intervals on real field data. This report documents the end-to-end pipeline we built for all five.

Two stages, one architecture. The system is organised as two pipeline stages that share the same shape: a Pydantic config selects one of several algorithms, and a single dispatcher class (`Segmenter / Predictor`) exposes the public method of the stage (`.detect / .predict`).

1. **Segmentation & detection (§3).** Take a continuous sensor session and slice it into discrete ride intervals tagged *up* / *down* / *outside*. Our production detector is *accelerometer-only* and is built on a matched filter that scans the signal for trapezoid pulse-pairs predicted by the jerk-limited S-curve kinematic model (§3.1); a paired-lobe filter then decides which pairs of candidate lobes constitute a genuine ride. A pressure-filter detector runs in parallel as a ground-truth source on phones that carry a barometer.
2. **Prediction (§4).** Take each detected ride and return a signed Δh in metres with a per-segment 90% confidence interval. Three algorithms share a common `predict / calibrate` contract: a barometer baseline (ISA inversion), a Zero-Velocity-Update (ZUPT) double integrator, and a 7-step S-curve velocity-domain fitter.

Reusable, stage-agnostic helpers live in `src/utils/`. Ground truth comes from the barometer when the phone has one (PRS sensor $\rightarrow$ ISA altitude $\rightarrow$ labelled ride intervals); experiments without a barometer use hand-labelled intervals edited through the GT editor (§2.7).

The user-facing tool. The two stages are exposed together as the **Boutique Pipeline**, a Streamlit-based wizard that walks an operator from raw signal to a Hebrew per-ride PDF report in five steps (data ingestion, interactive segmentation, prediction, export). The pipeline is described in detail in §11; the surfaces a non-research user will actually touch are an *interactive segmentation review* where every detector proposal can be accepted, edited, removed, or augmented with manually-added rides, and a *report exporter* that writes a per-segment audit trail.

Roadmap. §2 describes the data collection: seven elevators across six buildings, four phones per ride, and the Python loader that converts raw `sensorLog_*.txt` files into tidy per-sensor CSVs with ground-truth intervals. §3 introduces the kinematics of an elevator ride — the four-parameter jerk-limited S-curve model — and the two segmentation pipelines: the barometer-based ground-truth detector and the accelerometer-only matched-filter detector that ships in production. It also covers the calibration of the eight detection thresholds, the pair-filter hyperparameters, and the per-experiment evaluation. §4 is the prediction contribution of this work: the two accelerometer-only algorithms, their quality filters, their theoretical per-segment σ budgets, and their shared split-conformal calibration. §5 describes the cross-building train/test split, the clean-vs-dirty distinction in the ground truth, and the metrics we target. §6 reports the train-set and blind-test results; §9 discusses what they imply for the next iteration of the pipeline. §11 is a user-facing guide to the Boutique Pipeline UI: workflow, interpretation of the heatmaps and correlation panels, when to manually edit segments, and how to read the exported report.

2 Data Collection

This section documents, end to end, how the dataset underlying this work was gathered, how it is turned into a usable ground truth, and the engineering problems we had to solve along the way. It is written so that a developer who has never touched the repository can reproduce the pipeline from scratch by reading this section alongside `src/data/README.md`.

2.1 Experimental Design

We ran a controlled field campaign across **seven different elevators** in six buildings in central Israel, each with a different cabin geometry, motor characteristic, and floor-to-floor height. The campaign was concentrated in Ra'anana (four buildings, five elevators), with one building each in Herzeliya and Ramat Gan, covering low-rise residential, mid-rise office, and hotel use cases

so that any algorithm trained or evaluated on this data is not just fitted to a single elevator. Buildings are listed in Table 1; Figure 1 summarises their height distribution, and Figure 2 shows the floor-by-floor profile for each individual building. The `milleniumHotel` site is split into two logical “buildings” because it hosts two physically distinct elevators — the internal hotel elevator (sessions `exp1` and `exp2`) and the external elevator attached to the hotel body (`milleniumOutside`, session `exp3`). The two elevators have different motor characteristics, cabin geometry, and stop patterns, so we treat them as separate elevators for evaluation purposes while keeping both on the train side of the split.

Building (internal name)	Elevator	Type	Location
<code>milleniumHotel</code>	Internal	Hotel	Ra’anana
<code>milleniumOutside</code>	External	Hotel	Ra’anana
<code>acroBuilding</code>	–	Office	Ra’anana
<code>beitMansour1</code>	–	Residential	Ra’anana
<code>beitYitzchakiRaana</code>	–	Residential	Ra’anana
<code>BarIlan2Herzeli</code>	–	Residential	Herzeliya
<code>Haari3</code>	–	Residential	Ramat Gan

Table 1: The seven elevators (across six buildings) used in the field campaign. Internal names match the experiment-folder slug in `rawData/`. The Millenium hotel contributes two entries because it hosts two physically distinct elevators — the internal one (`milleniumHotel`, sessions `exp1`, `exp2`) and the external one attached to the hotel body (`milleniumOutside`, session `exp3`); the latter has no gramushka floor-anchor table and uses pure-barometer ground truth. `beitYitzchakiRaana` is the held-out blind-test location (§5).

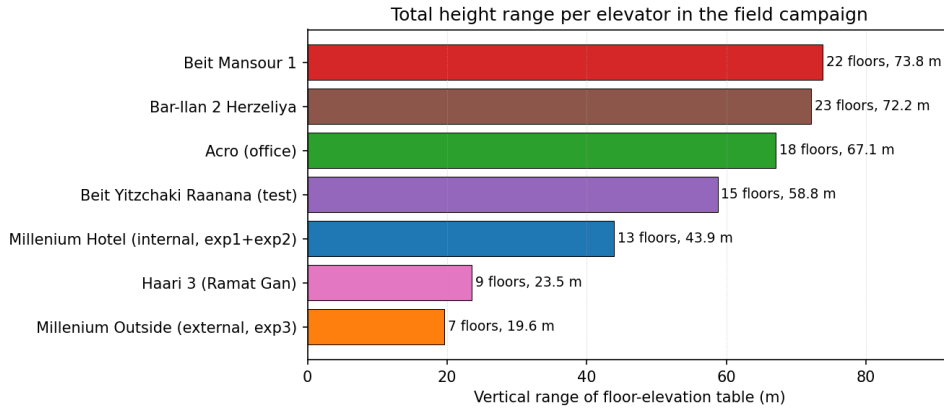


Figure 1: Overview of vertical range covered by the seven elevators in the field campaign (Millenium Hotel internal and external elevators are listed separately). Each bar shows the difference between the highest and lowest floors the elevator can reach, as encoded in the central gramushka floor-elevation table for that building. Rendered by `scripts/figs/plot_building_heights.py`.

Phones. In every ride we simultaneously recorded data from **four phones**. Using four devices per ride serves two purposes. First, it gives us heterogeneous sensor hardware on a single, perfectly synchronised physical event, which lets us study cross-device variance (different MEMS accelerometers, different sampling jitter, different barometer quality) without confounding it with different rides. Second, it guarantees that at least one of the four devices carries a **barometric pressure sensor** that we can use as a ground-truth source, even when one of the target phones under evaluation does not have one. Typical device mix across the campaign:

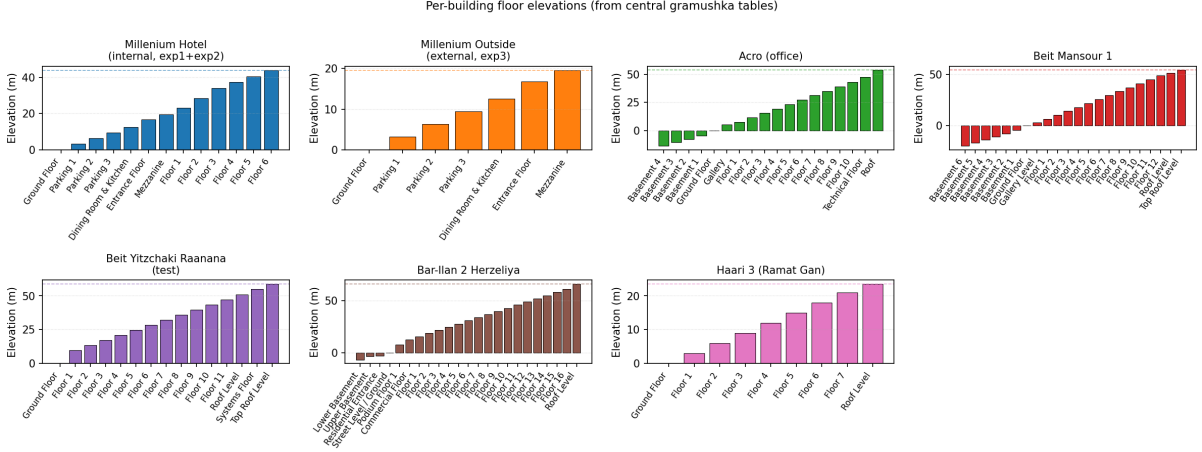


Figure 2: Per-elevator floor profile: each panel plots the stored elevation of every floor the elevator stops at, relative to ground. Panels highlight the geometric differences between the buildings (dense low-rise floors in Bar-Ilan and Beit Mansour, a sparse high-range office stack in Acro, a short external elevator at Millenium Outside). The dashed horizontal line marks the highest floor in each table. Source: `src/data/gramushka/<building>/ gramushka.csv`.

- SamsungSM-S911B — Galaxy S23 (has barometer).
- GooglePixel10 / GooglePixel — Pixel family (has barometer).
- Xiaomi22101320I — Redmi Note 12 family (no barometer).
- SamsungSM-A235F — Galaxy A23 (no barometer).
- GalaxyZFlip6 — foldable (has barometer).

The experimenters (eyalyakir, UriyaCohenEliya, RoyTurgeman) carried the devices together in a single hand or bag so that all four phones experience nearly identical motion, vibration, and acoustic environment during a ride.

Folder and file naming. Every recording is stored under

`src/data/rawData/<experimenter>_<location>_<phone>_<date>_exp<N>/`

to make the origin of every sample unambiguous from the path alone. This convention is documented in `src/data/README.md` and is enforced only by convention — the loader (§2.5) does not parse it, but every analysis notebook groups by it.

2.2 Logging: The Barometer

On every phone, a sensor-logging Android application subscribes to every inertial and environmental sensor the device exposes and dumps every sample, verbatim, to a tab-separated text file `sensorLog_<ISO-timestamp>.txt`. One row per sample, no header, format:

`<timestamp_ms>\t<SENSOR>\t<v1>\t<v2>...`

The sensors we rely on are summarised in Table 2.

The most important channel for ground truth is the **barometer (PRS)**. The pressure sensor is sampled in parallel with the other sensors and is the sole signal the elevator detector (§2.3) acts on. Because not every device in the fleet has a barometer, we attach a secondary device (typically a Pixel or S23) that *does* have one to every ride whose primary phone is pressure-less;

its recording lives in a `forBarometer/` sub-folder under the primary experiment’s directory. This secondary-device trick is central to the time-alignment problem discussed in §2.4.

Tag	Sensor	Columns
ACC	Linear accelerometer (SI-compensated)	x, y, z
GYR	Gyroscope	x, y, z
MAG	Magnetometer	x, y, z
RAWGYR	Uncalibrated gyroscope	x, y, z, bias_x, bias_y, bias_z
RAWMAG	Uncalibrated magnetometer	x, y, z, bias_x, bias_y, bias_z
ORI	Rotation-vector quaternion	w, x, y, z
PRS	Barometer	pressure (hPa)
GPS	GPS fix	lat, lon, alt

Table 2: Per-sensor column schema emitted by the logger. Any row whose column count does not match its sensor tag is dropped silently by the parser (`src/data/loader/parsing.py`).

Once the recording is over, the raw dump is committed to `rawData/<exp_name>/`, together with a hand-filled `metadata.txt` (experimenter, phone, location, description, date, time). The loader pipeline (§2.5) then materialises a CSV per sensor under `structuredData/data/<exp_name>/` and derives `gt.csv` — the ground truth used by every downstream algorithm.

2.3 Elevator Detection

Ground truth for this project is a labelling of the timeline into `up`, `down`, and `outside` intervals: “outside” means the phone is not on a moving elevator (waiting, walking, idle). The labelling is derived algorithmically from the barometer signal and then optionally corrected by hand (§2.7).

2.3.1 Barometer-Based Algorithm

The detector lives in `src/algorithms/segmentation_algorithms/barometer_only/height_segmentation.py` and runs in six stages. Inputs: a time series (t_i, p_i) of timestamps and pressure readings.

1. **Altitude.** Convert pressure to altitude via the international barometric formula,

$$h_i = \frac{T_0}{L} \left[1 - (p_i/p_0)^{RL/(gM)} \right],$$

implemented in `src/physics/pressure_to_altitude.py`. This removes the weather-dependent DC offset only approximately, but the segmenter depends on *differences* in h , so the residual bias cancels.

2. **Low-pass smoothing.** Apply a centered rolling-mean filter of width `height_lowpass_sec ·  $f_s$` samples to suppress high-frequency pressure jitter (door slams, HVAC bursts).
3. **Vertical velocity.** Estimate v_z by forward differencing the smoothed altitude, $v_{z,i} = (h_{i+1}^{\text{lp}} - h_i^{\text{lp}})/\Delta t_i$, then box-smooth again over `smooth_window_sec`.
4. **Thresholding.** Mark samples where $|v_z| > \text{velocity_threshold}$ (default ≈ 0.15 m/s) as “moving”. Extract contiguous runs of `moving` samples by edge-finding on the binary mask.

5. **Merging.** Walk the runs in order and merge consecutive runs of the *same sign* (both up or both down) if the gap between them is shorter than `merge_gap_sec`. This collapses spurious velocity dips caused by elevator deceleration mid-ride.
6. **Pruning and padding.** Drop any surviving run whose duration is $\leq \text{min_duration_sec}$ or whose total altitude change is $\leq \text{min_height_diff_m}$. Expand every surviving run by `pad_sec` on both ends so the label comfortably covers the full ride, including the small door-open tail.

The output is a `DataFrame` of rows (`start_ci`, `end_ci`, `duration`, `type`, `probability_ci`) where `type` $\in \{\text{up}, \text{down}\}$. The pipeline then pads the gaps between these rows with `outside` intervals so that the GT timeline is contiguous — this is what downstream evaluation code expects.

Why this is good enough as ground truth. A well-calibrated consumer barometer on a phone resolves pressure changes equivalent to roughly 10 cm of altitude at typical sampling rates, which is an order of magnitude below the smallest interesting inter-floor height (~ 3 m). The algorithm is deterministic, so repeated runs on the same recording produce identical labels, which in turn makes regression testing tractable.

2.4 The Time-Synchronisation Problem

The reason Section 2.3 alone does not close the loop is a low-level Android timing issue that we spent considerable time getting right.

The root cause. Android’s `SensorEvent.timestamp` is the time at which the sensor event was recorded, but it is expressed relative to the *device boot time* (`SystemClock.elapsedRealtimeNanos`), not to wall-clock. The logger writes this timestamp directly into the log file. Consequently, the `timestamp_ms` column in `sensorLog_*.txt` is a *device-local uptime* value, not a UTC time. Two phones that were booted at different moments will assign different timestamps to the *same* physical event, with the offset equal to the difference of their boot times — in our dataset, frequently tens of thousands of seconds, sometimes days.

Why this bites us in practice. Recall (§2.2) that phones without a barometer rely on a secondary device’s PRS stream for ground truth. To use the secondary device’s altitude as ground truth for the primary device, we must answer: *at primary timestamp t_p , what was the altitude?* That requires mapping primary time to secondary time, which requires knowing the boot-time offset between the two devices. Neither device records wall-clock, so the offset must be recovered from the data itself.

First approximation: start-time alignment. The simplest workable estimate is to assume both phones started recording at the same wall-clock moment, so the first sample of each log corresponds to the same real time. Under that assumption, the offset is

$$\Delta = t_0^{(\text{prim})} - t_0^{(\text{sec})},$$

and we shift every secondary timestamp by Δ before using it. This is implemented in `src/data/loader/alignment.py::_compute_start_offset` and is sufficient when the experimenter hits “start” on both phones in quick succession.

Why start-time alignment is not enough. In practice the user latency between the two taps is one to several seconds and is variable between rides. That multi-second bias shifts the ground truth by enough to invalidate per-sample evaluation (an elevator moves at ~ 1.5 m/s; a 2 s timing error translates to ~ 3 m of altitude error, more than an entire floor). We therefore refine the offset using the **accelerometer signal**, which is recorded by both phones and must be approximately identical for two devices in the same hand.

Refinement by cross-sensor correlation. The routine used in the lab is: (i) compute the magnitude $|a|(t)$ on both phones, (ii) resample both to a common grid after applying the first-pass start-time offset, (iii) run a normalised cross-correlation of the two $|a|$ traces over a bounded lag window (± 10 s was enough in every experiment we observed), and (iv) pick the lag that maximises the correlation as the residual offset to fold back into Δ . If ACC is missing on either device, we fall back to gyroscope magnitude or heuristic event matching on step-count spikes. The diagnostic overlay (Figure 3) is produced automatically for every experiment with a `forBarometer/` companion and is the first thing to inspect when an alignment looks suspicious.

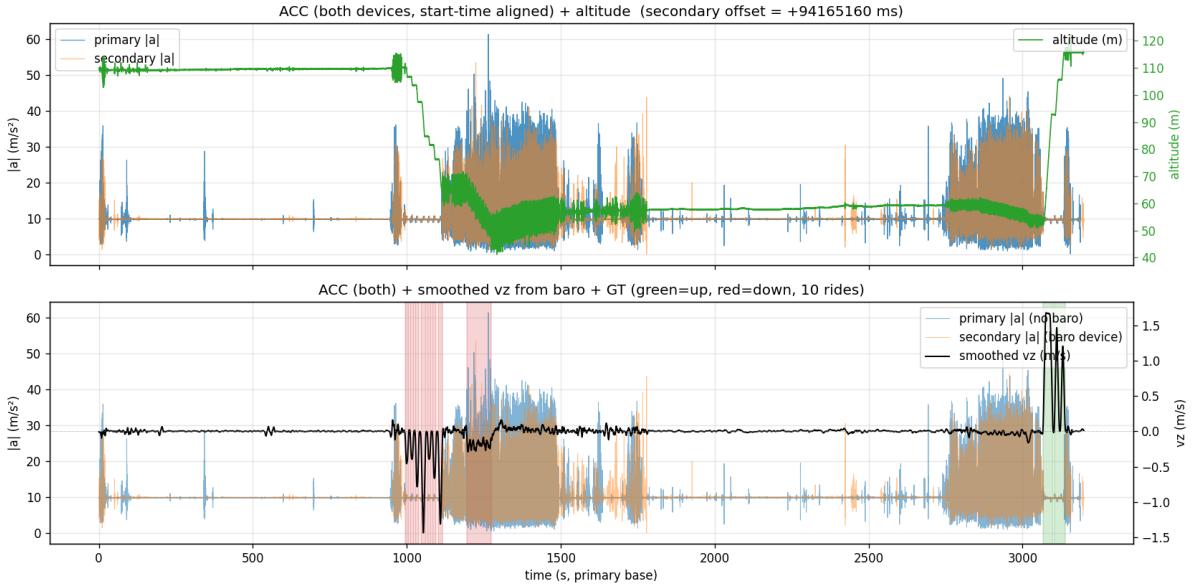


Figure 3: Diagnostic plot produced by `loader/alignment.py::_plot_forBarometer_alignment`.

Top: primary $|a|$ (blue) and time-shifted secondary $|a|$ (orange) overlaid, with the secondary’s altitude on the right y-axis (green). Signature features such as the impulse at door-close should line up after alignment. **Bottom:** the same accelerometer overlay with the barometer-derived smoothed vertical velocity in black and the detected ride intervals shaded (green = up, red = down). The reported offset is the value added to the secondary’s timestamps.

Practical hand-offs. The offset is computed once per experiment, logged, and baked into the secondary’s PRS frame before it is swapped into the primary’s sensor dictionary (`_merge_secondary_prs` in `alignment.py`). Everything downstream — the segmenter, the GT derivation, evaluation metrics — therefore sees *a single phone* with a valid barometer, which removes a whole class of timing bugs from the rest of the stack.

2.5 The Loader Pipeline

`src/data/loader/` exposes a narrow public API (`getExperimentData`, `saveExperimentData`, `list_experiments`) that hides all of the above. On first access it parses the raw `sensorLog_*.txt`, merges any `forBarometer/` secondary, runs the detector (§2.3.1),

pads the timeline with `outside` intervals, and writes per-sensor CSVs plus `gt.csv` to `structuredData/data/<exp_name>/`. On every subsequent call it reads those CSVs directly. To force a rebuild after code changes, pass `use_cache=False`. The CSV schemas are documented in `src/data/README.md` and frozen in `loader/constants.py`.

2.6 Sample Frequency Analysis and the Solution for Low-Frequency / Missing Parts

Why this matters. Mobile phones do not guarantee a uniform sample rate on the IMU bus. The OS may throttle the sensor when the device is locked, when battery saver kicks in, or when a higher-priority app suspends background work; the application itself may drop a batch of samples after a buffer overflow. The result is that an apparently “50 Hz” recording can contain stretches of seconds with no samples at all, or stretches where the effective rate has fallen well below the Nyquist frequency of any signal of interest. Our pipeline runs `scipy.signal.resample_poly` or `numpy.interp` during load (see §2.5); without protection both of those operators happily *fabricate* samples across an arbitrarily long silent window, and the segmenter then “detects” rides inside the fabricated region. We need a way to distinguish “the phone gave us real samples here” from “the resampler invented these values to fill a hole.”

The threshold-frequency rule. Pick a global threshold $f_{\min}$ below which the data is considered too sparse to be meaningful. We use $f_{\min} = 1$ Hz, encoded as `THRESHOLD_FREQUENCY_HZ` in `src/data/loader/constants.py`. A *gap* is any consecutive-sample spacing greater than $1/f_{\min} = 1$ s. Gaps split a sensor’s timeline into a list of contiguous *valid intervals* $\{[t_i^{(s)}, t_i^{(e)}]\}_{i=1}^N$. The detection rule is implemented by `_detect_valid_intervals` in `loader/pipeline.py`; given an int64 timestamp array $t \in \mathbb{Z}^M$:

1. Compute the inter-sample diffs $\Delta t = \text{diff}(t)$.
2. Mark every index j with $\Delta t_j > f_{\min}^{-1} \cdot 10^3$ ms as a *break*.
3. Emit one valid interval per maximal run of non-break samples, delimited by adjacent break indices.

A clean recording yields a single interval; a recording with k gaps yields $k + 1$ intervals.

Per-interval resampling. For each valid interval the loader calls the existing `_resample_sensor_to_hz` helper, which anchors a uniform 50 Hz grid to the interval’s first timestamp and fills it with `resample_poly` (when the per-interval input is itself uniform) or `np.interp`. The resampled chunks are concatenated; the final per-sensor DataFrame contains 50 Hz rows *only inside valid intervals* and is otherwise empty. Downstream code that slices by `timestamp_ms` therefore cannot accidentally pick up an interpolated sample inside a gap. The wrapper `_resample_sensor_with_gaps` returns the pair (resampled DataFrame, valid intervals), and `_resample_sensors_with_gaps` applies it per sensor.

Per-sensor validity. Each sensor has its own clock and its own dropouts: the barometer can pause while the accelerometer keeps running, or vice versa. The loader therefore keeps a separate interval list per sensor, stored at `gt.attrs["valid_intervals_per_sensor"]`: a `dict[str, list[(int, int)]]` keyed by sensor name (“ACC”, “PRS”, ...). Algorithms that fuse multiple sensors are responsible for intersecting the lists themselves; algorithms that need only one sensor see only that sensor’s list. The `ExperimentPipeline` container exposes the same dict via the `valid_intervals` attribute and the convenience method `gap_free_subslices(sensor="ACC")`, so iteration code can honour the gaps without re-deriving them.

Propagation to the Streamlit app. The `LoadedSignal` dataclass (`src/data/loadFromDB.py`) gains a `valid_intervals: list[(int, int)]` field, populated either in the upload form (`step2_data.csv_to_signal` runs `detect_valid_intervals` on the cleaned timestamp column) or in the S3 stub (`loadDataFromS3`). The segmentation step (`step3_segmentation._run_detector`) then uses a hybrid strategy: it calls the trapezoid-template detector *once* on the full signal to obtain the canonical visualisation `state` (arrays spanning the whole timeline), and then re-runs the detector *per valid interval* so that no template match can straddle a gap; per-interval predictions are time-shifted onto the full-signal `t0_ms` origin before being concatenated. The prediction step (`step4_prediction._run_predictions`) drops, with a user-visible warning, any segment whose $[t_{\text{start}}, t_{\text{end}}]$ does not lie entirely inside one valid interval.

User-visible signal: red “no data” overlays. Both UIs shade gap regions to make them impossible to miss:

- In the Streamlit wizard the helper `_add_gap_overlays` (in `step3_segmentation.py`, reused by step 4) walks the *complement* of the valid-interval list over the visible time axis and adds a red `plotly vrect` per gap with annotation text “× no data”.
- In the desktop trapezoid editor (`src/segmentation/algorithms/editor.py`) the same complement is computed against the ACC valid intervals and rendered with `ax.axvspan(..., color="#c0392b", alpha=0.2)` on every panel; a single bold × marker is stamped near the top of the topmost panel for each gap. The detector loop in `load_experiment` runs per-interval and renumbers the resulting predictions so the detail-pane indices stay unique after concatenation.

The user contract is therefore: *the UI shows you where the data is missing; do not label ground-truth rides inside those red regions*. Because the segmenter and predictor also skip those regions defensively, an accidental violation of the contract degrades to a warning rather than a silently wrong Δh .

Test fixtures: the corruption script. To exercise the loader and both UIs without waiting for a phone to misbehave in the wild, `scripts/generate_corrupted_test_data.py` produces a sibling experiment folder `structuredData/data/<exp>__corrupted_<mode>/` with one of three failure modes applied to every per-sensor CSV (`gt.csv`, `metadata.csv` and `baramoshka.csv` are copied byte-identically; only `metadata.csv.exp_name` is rewritten to match the new folder name):

- **dropout** — delete every row inside a fixed $[\text{gap_start_s}, \text{gap_start_s} + \text{gap_len_s})$ window. The loader must report exactly two valid intervals per sensor.
- **thinning** — inside the same window, keep only one sample every `thinning_spacing_s` seconds (default 1.5 s). Each surviving sample sits more than f_{min}^{-1} apart from its in-window neighbours, so the loader splits the window into one valid interval per kept sample plus the two flanking spans (typically 6–8 intervals for a 10 s window).
- **halving** — drop every other row across the whole frame. The signal stays uniform but at half the native rate; no spacing exceeds the threshold, so the loader returns exactly one valid interval. This regression case verifies that we don’t fire the gap-splitting machinery on merely-low-rate-but-uniform data.

The smoke tests we ran reproduce these counts: a clean Acro-building recording yields 1 valid interval per dense sensor; the **dropout** variant yields 2; the **thinning** variant with a 10 s window and 1.5 s spacing yields 7; the **halving** variant stays at 1. The script prints the expected numbers at the end of every run for quick eyeballing.

2.7 The Ground-Truth Editor

No matter how careful the detector in §2.3.1 is, it will occasionally miss the start of a ride, split a single ride into two, or classify an HVAC-induced pressure ripple as movement. Rather than chase every edge case in the algorithm, we decided early on that the ground truth must be *auditable and editable* by a human. Every `gt.csv` in the dataset has therefore passed through the GT editor at least once.

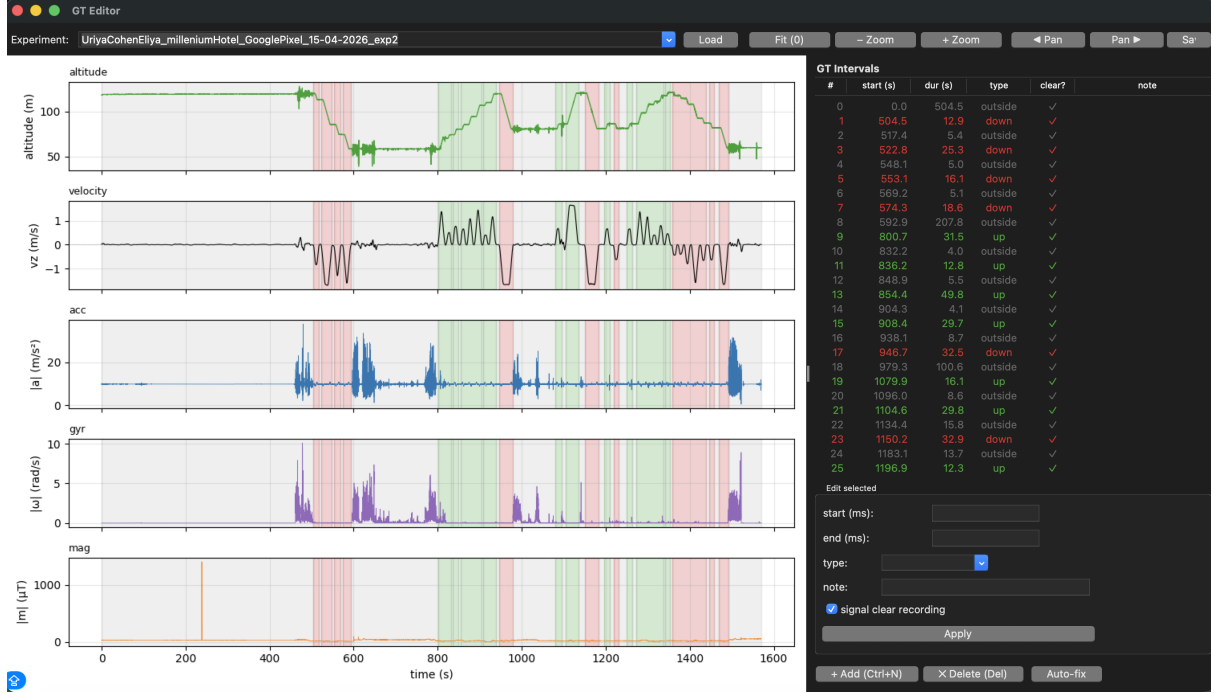


Figure 4: The GT editor. Left pane: stacked matplotlib panels (altitude, vertical velocity, $|a|$, $|\omega|$, $|m|$) with ground-truth intervals drawn as coloured spans (green = up, red = down, grey = outside). Right pane: a **Treeview** of all intervals with the edit form below it.

Launching. The editor is a Tkinter application:

```
venv/bin/python -m src.data.gt_editor [exp_folder_name]
```

Called without arguments, it lists all experiments found under `rawData/` in a dropdown and lets the user pick one.

Visualisation. The left pane renders all five standard motion signals on a shared time axis. Ground-truth intervals are overlaid on every panel simultaneously so a user can sanity-check a label against multiple modalities at once (e.g. a true elevator ride must be accompanied by a clear altitude step *and* a flat-ish accelerometer magnitude).

Editing. The interaction model is designed to keep the mouse near the signal rather than in a form. Concretely:

- **Click** anywhere inside a coloured span to select that interval in the right-hand tree.
- **Drag an edge** (within a few pixels of the start or end of a span) to resize it live; the cursor morphs into a horizontal arrow while inside the hit-tolerance zone.
- Type precise millisecond bounds in the **edit form**, change the **type** (up / down / outside) or the free-text **note**, and hit **Apply**.

- **+** **Add** inserts a new 10-second **outside** interval right after the current selection.
- **×** **Delete** removes the current interval.
- **Auto-fix** closes gaps between adjacent intervals with **outside** filler so the timeline stays contiguous; it refuses to merge if two intervals actually overlap, to avoid silently destroying user edits.

Navigation. Keyboard shortcuts mirror the matplotlib toolbar: **+** / **=** zoom in, **-** zoom out, **0** fit to full range; **Shift+←** / **Shift+→** pan by 25% of the visible width; mouse-wheel over the plot zooms around the cursor. All of this is indispensable on long recordings where a single ride is only a few percent of the timeline.

Saving. **Ctrl+S** (or the **Save** button) calls `saveExperimentData`, which rewrites every per-sensor CSV, `gt.csv`, and `metadata.csv` under `structuredData/data/<name>/`, then rebuilds the top-level `metadata.csv` index. The floor-height lookup (`baramoshka.csv`) is intentionally *not* touched on save. The status bar at the bottom tracks unsaved-changes state; closing the window with pending edits prompts before quitting.

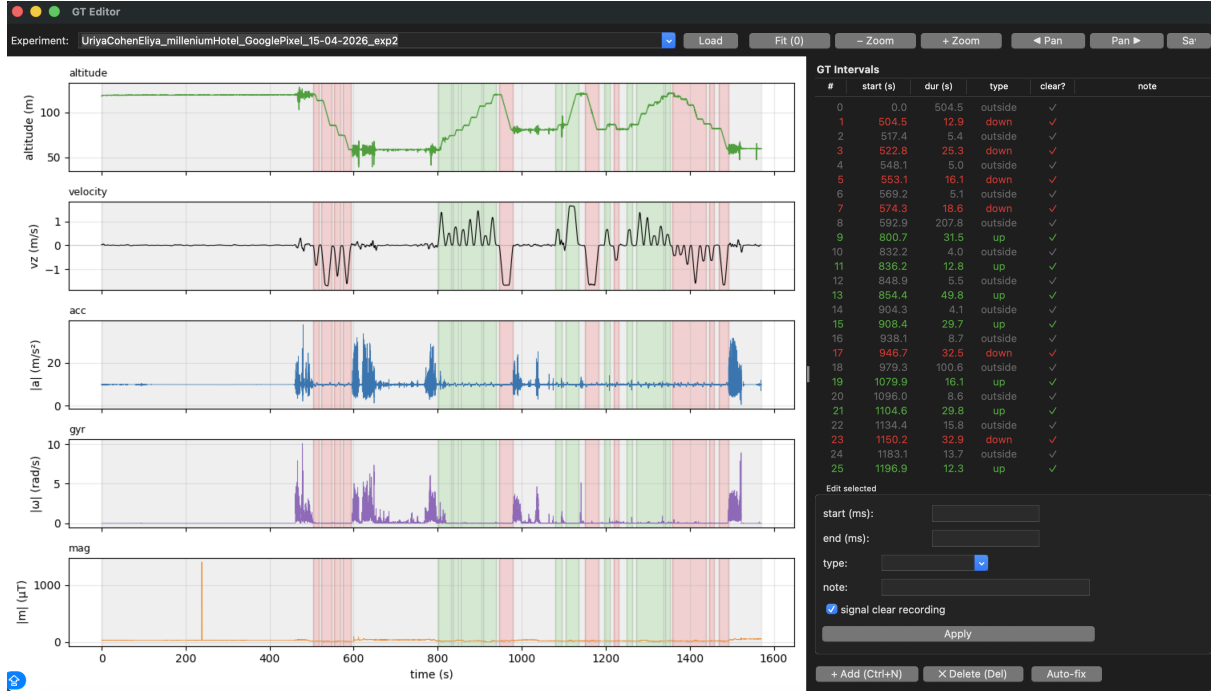


Figure 5: Fixing an auto-detection mistake in the GT editor. Here the barometer algorithm merged two adjacent rides into one because the cabin paused for less than `merge_gap_sec` at an intermediate floor; dragging the right edge of the first span and adding a new **down** interval splits the ride into its two true halves.

3 Segmentation & Detection

This section covers how a continuous accelerometer (and, when available, barometer) stream is sliced into elevator-ride segments. We begin with the underlying physics of an elevator ride — a four-parameter motion model that every detector in this work eventually reduces to — and then describe the concrete detectors built on top.

3.1 Background: Elevator Kinematics

Before describing any detector we lay down the physics of an elevator ride, because the same four parameters reappear in every algorithm we use later. Modern passenger elevators are commanded by a *jerk-limited motion profile* so that passengers experience a smooth, bounded change of acceleration (see, e.g., the ride-comfort specifications summarised in ISO 18738 and the manufacturer literature aggregated in `papers/elevator_research_links.txt`). The drive obeys, in order, the constraints

$$|\ddot{h}(t)| \leq j_{\max}, \quad |\ddot{h}(t)| \leq a_{\max}, \quad |\dot{h}(t)| \leq v_{\max}, \quad (1)$$

where $h(t)$ is the cabin altitude, $\dot{h} = v$ the vertical velocity, $\ddot{h} = a$ the acceleration measured by the phone (after gravity removal), and $\dddot{h} = j$ the jerk. Typical commercial values are $j_{\max} \approx 1.0\text{--}1.6 \text{ m/s}^3$, $a_{\max} \approx 0.8\text{--}1.5 \text{ m/s}^2$, and $v_{\max} \approx 1.0\text{--}2.5 \text{ m/s}$ for low- and mid-rise buildings (CIBSE Guide D, 2020).

The seven-segment S-curve. Saturating each constraint in turn yields the canonical S-curve velocity profile (Figure 7). The ride decomposes into up to seven phases:

$$\underbrace{j = +j_{\max}}_{1: \text{ jerk up}} \quad \underbrace{j = 0}_{2: \text{ cruise } a_{\max}} \quad \underbrace{j = -j_{\max}}_{3: \text{ jerk down}} \quad \underbrace{a = 0}_{4: \text{ cruise } v_{\max}} \quad \underbrace{j = -j_{\max}}_{5: \text{ jerk down}} \quad \underbrace{j = 0}_{6: \text{ cruise } -a_{\max}} \quad \underbrace{j = +j_{\max}}_{7: \text{ jerk up}}.$$

Phase 4 collapses to zero length on short hops where the cabin never reaches $v_{\max}$, in which case the profile is “triangular” in acceleration; longer rides are “trapezoidal”. The constituent durations follow from (1):

$$T_j = \frac{a_{\max}}{j_{\max}}, \quad T_a = \max\left(0, \frac{v_{\max}}{a_{\max}} - T_j\right), \quad T_v = \max\left(0, \frac{H}{v_{\max}} - T_j - T_a\right), \quad (2)$$

where H is the floor-to-floor height. The total ride length is $T_{\text{ride}} = 4T_j + 2T_a + T_v$.

What T_j , T_a , T_v mean. Each of the three durations in (2) labels a distinct *kinematic regime* of the ride:

- **T_j – jerk-limited ramp duration.** The time the drive spends at a *saturated* jerk $|\ddot{h}| = j_{\max}$. This is the length of phases 1, 3, 5, and 7, during which the acceleration $a(t)$ is a straight line with slope $\pm j_{\max}$ (linear ramp). Because the acceleration must grow from 0 to $a_{\max}$ at slope $j_{\max}$, the duration is fixed by the machine: $T_j = a_{\max}/j_{\max}$. For typical commercial values ($a_{\max} \approx 1.0 \text{ m/s}^2$, $j_{\max} \approx 1.3 \text{ m/s}^3$) this is $T_j \approx 0.77 \text{ s}$.
- **T_a – constant-acceleration plateau duration.** The time the cabin spends at a *saturated* acceleration $|a| = a_{\max}$. This is the length of phases 2 and 6, the flat tops (or bottoms) of the acceleration trace. T_a is zero for short hops that never reach $v_{\max}$ (“triangular” rides) and grows linearly with the peak velocity budget $v_{\max}/a_{\max}$ once $v_{\max}$ is hit.
- **T_v – constant-velocity cruise duration.** The time the cabin spends at a *saturated* velocity $|\dot{h}| = v_{\max}$ with $a = 0$. This is the length of phase 4, the flat top of the velocity profile between the two acceleration lobes. T_v vanishes on any ride where H is too small for the cabin to finish accelerating before it must begin decelerating; when it is non-zero it scales linearly with the floor-to-floor height, $T_v \approx H/v_{\max} - T_j - T_a$.

Figure 6 shows all three durations on a simulated S-curve for a representative trapezoidal ride.

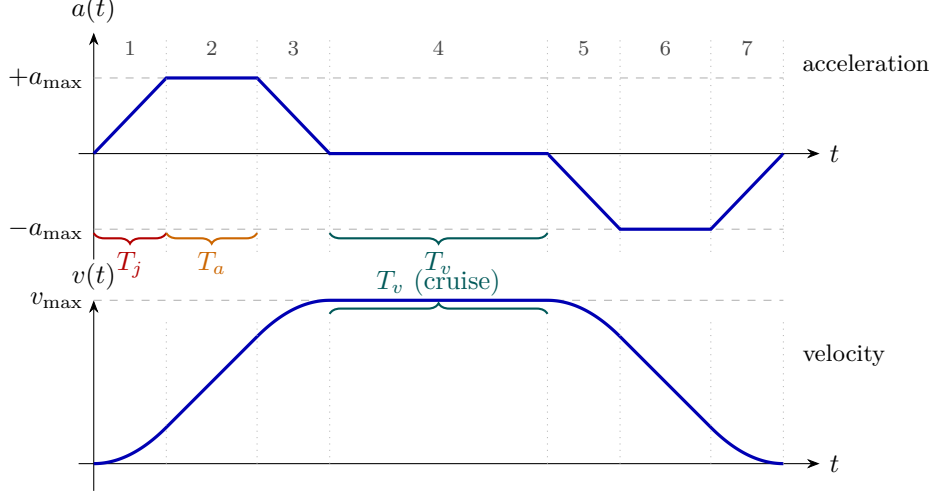


Figure 6: Simulated seven-segment S-curve (trapezoidal ride) with the three kinematic durations labelled. **Top:** acceleration $a(t)$. Phases 1 and 3 (also 5 and 7) are the linear jerk ramps of length T_j ; phases 2 and 6 are the saturated $|a| = a_{\max}$ plateaus of length T_a . **Bottom:** velocity $v(t)$, obtained by integrating $a(t)$. The flat top of length T_v is phase 4, the cruise at $|h| = v_{\max}$. Dotted verticals delimit the seven phases numbered above the acceleration panel; T_j is fixed by the machine ($a_{\max}/j_{\max}$), T_a grows with the $v_{\max}/a_{\max}$ budget, and T_v grows linearly with the floor-to-floor height H .

What the phone measures. The phone’s gravity-projected linear acceleration $a(t)$ traces *exactly* the bracketed acceleration in (1), modulo sensor noise and projection error. Three model facts are worth highlighting because the detector exploits them:

- During the four jerk phases (T_j each), $a(t)$ is linear in time with slope $\pm j_{\max}$. In the velocity domain this integrates to a quadratic — the parabolic “ramp shoulders” the user can see in Figures 7 and 8.
- The signed area under the first acceleration lobe equals the peak velocity reached:

$$v_{\text{peak}} = \int_0^{2T_j+T_a} a(t) dt = a_{\max} (T_j + T_a) = \min(v_{\max}, \sqrt{a_{\max}H}) \quad \text{for short rides.} \quad (3)$$

The second (deceleration) lobe has equal magnitude and opposite sign, so $\int_0^{T_{\text{ride}}} a(t) dt = 0$. This is the fundamental reason ZUPT works: the cabin starts and ends at rest, so any non-zero terminal velocity in the integrated signal is drift, not motion.

- The temporal gap between the centroids of the two acceleration lobes is the cruise time $T_v + 2T_a + 2T_j \approx H/v_{\max}$ when $v_{\max}$ is reached. **Hence the inter-pulse spacing is a direct, scale-free measurement of the floor-to-floor height H .**

The four-parameter ride model. Combining (1)–(3), every ride is parameterised by the tuple

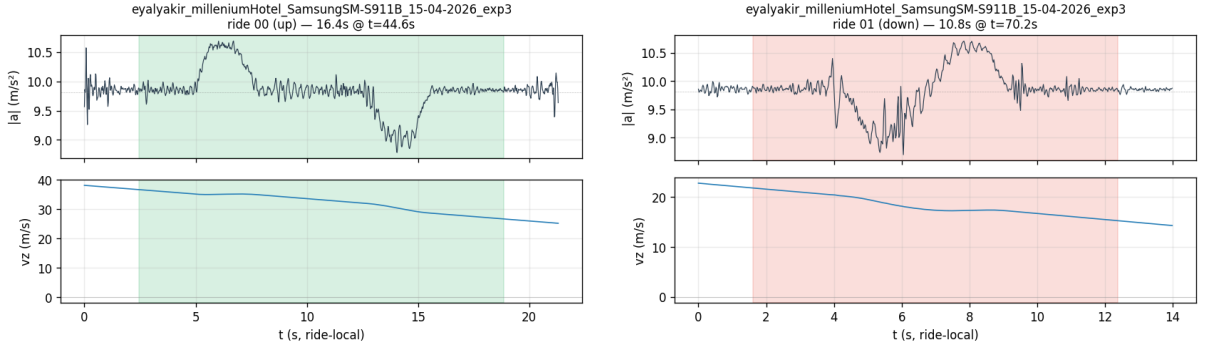
$$\theta_{\text{ride}} = (j_{\max}, a_{\max}, v_{\max}, H), \quad (4)$$

with $j_{\max}, a_{\max}, v_{\max}$ being properties of the elevator machine (constant within a building) and H being the only ride-specific quantity. Once θ_{ride} is fixed, the ideal acceleration template is

fully determined:

$$a_\theta(t) = \begin{cases} +j_{\max} t, & 0 \leq t < T_j, \\ +a_{\max}, & T_j \leq t < T_j + T_a, \\ +a_{\max} - j_{\max}(t - T_j - T_a), & T_j + T_a \leq t < 2T_j + T_a, \\ 0, & 2T_j + T_a \leq t < 2T_j + T_a + T_v, \\ \text{(mirror of phases 1–3, with opposite sign),} & \text{otherwise.} \end{cases} \quad (5)$$

Velocity $v_\theta(t) = \int_0^t a_\theta(\tau) d\tau$ and displacement $h_\theta(t) = \int_0^t v_\theta(\tau) d\tau$ are then piecewise cubic and quartic respectively. Equation (5) is the synthetic template used by the DTW-based detector (Algorithm 2 in `papers/algorithm2_dtw.md`) and by the matched-filter pulse detector in `template_match/scripts/acc_pulse_detect.py`.



(a) Ride going *up*: positive accel lobe first, negative lobe at landing. The shaded interval is the GT label. (b) Ride going *down*: lobes are mirrored. The vertical velocity panel makes the trapezoidal $v(t)$ profile clear.

Figure 7: Two single-ride zooms drawn from `eyalyakir_millenniumHotel_SamsungSM-S911B_15-04-2026_exp3`. Top of each subfigure: gravity-removed $|a|$. Bottom: the smoothed vertical velocity v_z recovered by integration. The two parabolic lobes in $|a|$ are the jerk-limited acceleration and deceleration phases of (5); the area under each lobe equals v_{peak} via (3), and the gap between lobe centroids measures the cruise time T_v , which is proportional to the inter-floor height H .

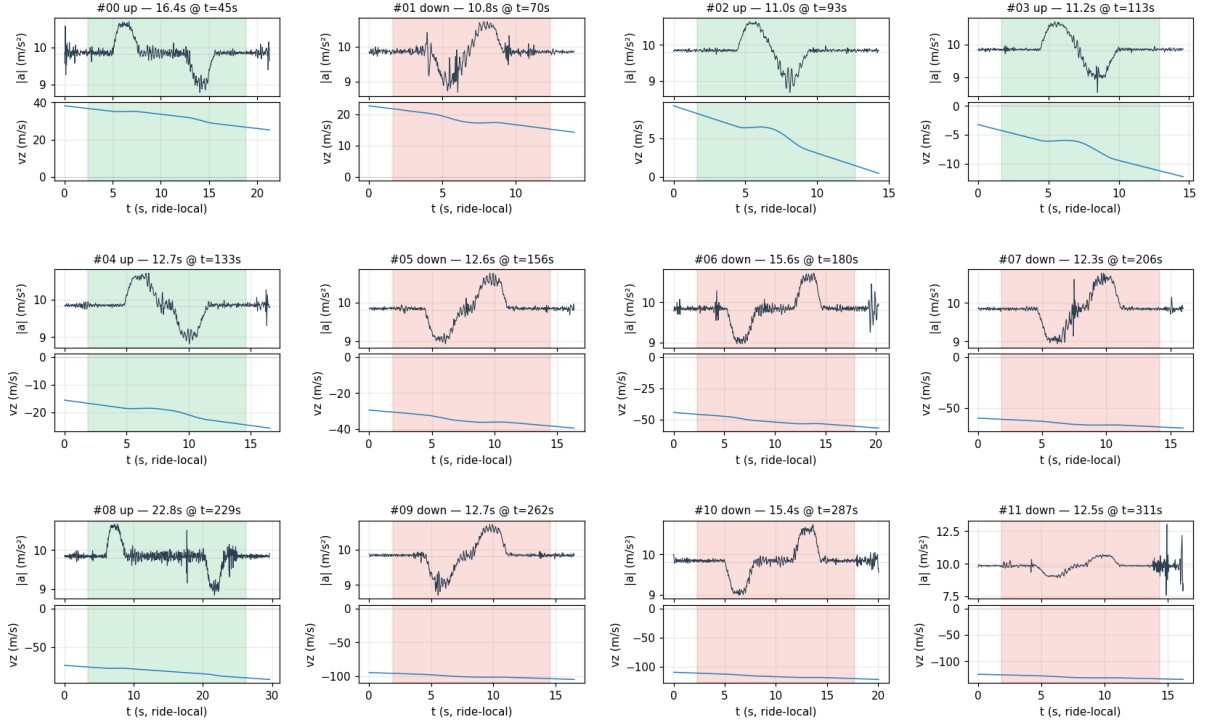


Figure 8: All twelve GT rides from the same experiment, on a common layout. The repeating two-lobe “ $+a$ then $-a$ ” structure predicted by (5) is unmistakable across rides of varying duration. Wider lobe spacing corresponds to larger H (more floors traversed), while the lobe *height* saturates at the building’s $a_{\max}$ regardless of H .

The barometer-based GT detector described in §2.3.1 uses none of this — it works in altitude space directly — but the kinematic decomposition above is what makes its output a sensible reference signal for the accelerometer-based algorithms that follow.

3.2 Whole-Signal Trapezoid Detector

Our production accelerometer-only detector works directly on the idealised two-lobe model from Section 3.1: a ride is a signed acceleration pulse at take-off followed by an opposite-sign acceleration pulse at landing. The algorithm is split into two stages that mirror the two halves of the problem. A *detection* stage finds candidate lobes in the raw signal; a *pair-filter* stage decides which pairs of lobes constitute genuine rides. Eight scalar thresholds — four in each stage — plus four template-grid bounds are bundled in a single configuration object, so a sweep can move them all in lockstep.

3.2.1 Tunables and How We Calibrated Them

Before diving into the stages themselves it helps to have every tunable on one page. The detector exposes two kinds of knobs:

1. **Template-grid bounds** that define the set of trapezoidal templates $\tau_{W,f}$ swept over the signal (Section 3.2.2, Equation (6)). These shape *what* the matched filter can match.
2. **Acceptance thresholds**, four in the detection stage and four in the pair filter, that decide *which* matches survive. Each has a direct physical interpretation in the kinematic model of Section 3.1 and each guards against a distinct failure mode.

Table 3 lists every knob, the value we use, and the failure mode it guards against. The two template-grid blocks come first because they constrain every later stage; the two acceptance blocks follow in pipeline order (detection, then pair filter).

Parameter	Value	Meaning / failure mode guarded against
A. Template grid – half-width W (§3.2.2, Eq. (6))		
W_MIN_S	0.3 s	Lower bound on lobe half-width. Lowered from 0.4 s to 0.3 s in the <code>iter_15</code> round (Section 3.2.8) after diagnostics showed many missed rides had <code>pair_W</code> pinned at the floor. Still a hair below the smallest physically plausible T_j observed in the corpus, so sub-jerk-ramp spikes remain excluded.
W_MAX_S	3.0 s	Upper bound on lobe half-width. Comfortably brackets the largest $T_j + T_v/2$ we see in commercial rides.
W_N	30	Number of log-spaced W samples between W_MIN_S and W_MAX_S .
B. Template grid – flat fraction f (§3.2.2, Eq. (6))		
F_MIN	0.05 + <i>triangle row</i> at $f = 0$	Lower bound on plateau fraction for the trapezoid grid. Originally set above zero because an exactly-triangular pulse has a discontinuous derivative and fits unstably under smoothing. <code>Iter_13</code> (Section 3.2.8) revisited this: one-floor and joined-pulse rides really <i>are</i> triangular, so an explicit $f = 0$ row is now prepended to <code>grid_f()</code> . The per-pair argmax in <code>joint_pair_score</code> picks trapezoid vs. triangle pair-by-pair — the trapezoid grid still starts at 0.05.
F_MAX	0.80	Upper bound on plateau fraction. Large enough to cover long rides where $T_a \gg T_j$, small enough to reject the near-constant plateau ($f \rightarrow 1$) that a pedestrian’s slow drift produces.
F_N	15	Number of linearly spaced f samples between F_MIN and F_MAX .
C. Detection stage (§3.2.2)		
<code>r2_peak_thresh</code>	0.75	Minimum shape-agreement of a candidate with its best template. Below this, the window isn’t “trapezoidal enough” to treat as a lobe. Guards against low- R^2 noise.
<code>min_peak_abs_a</code>	0.20 m/s ²	Minimum LS-fit $ \hat{A} $ for a peak. Pedestrian bumps are the <i>right shape</i> but the <i>wrong size</i> ; this floor separates them from $a_{\max}$ -scale cruise lobes. Principal pedestrian-FP lever. Lowered through two tuning rounds ($0.5 \rightarrow 0.25 \rightarrow 0.20$, Section 3.2.8) once the joint- R^2 and quiet-middle pair gates were in place to absorb the FP risk.
<code>nms_radius_s</code>	0.5 s	Duplicate-peak dedup radius. A single physical pulse produces adjacent high- R^2 samples; this collapses them to one.
<code>same_sign_min_gap_s</code>	5.0 s	Minimum gap between two same-sign candidates. Real elevator physics doesn’t place two take-offs (or two landings) closer than the shortest plausible ride.
D. Pair-filter stage (§3.2.4)		
<code>min_ride_s</code>	10 s	Lower bound on take-off→landing gap. Shorter gaps would be “dwell pairs” – one ride’s landing matched against the next ride’s take-off.
<code>max_ride_s</code>	120 s	Upper bound on pair gap; a hard ceiling on ride duration.
<code>joint_r2_thresh</code>	0.75	Minimum shared-shape joint R^2 for an accepted pair. Pair-level analogue of <code>r2_peak_thresh</code> , harder to clear because both lobes must agree on the same (W, f) template.
<code>min_pair_abs_a</code>	0.3 m/s ²	Minimum shared $ A $ of the accepted pair. Secondary amplitude floor; lower than the peak-level floor because joint fitting already averages the two lobes.

Table 3: Every tunable of the whole-signal trapezoid detector, grouped by role. Block A/B define the trapezoid template grid $\tau_{W,f}$; block C gates per-sample candidates; block D gates pairs of candidates. All values were picked by grid search on the 22 training experiments (see below).

How the values were picked. The template-grid bounds $(W_{\min}, W_{\max}, f_{\min}, f_{\max})$ were pinned first from the kinematic analysis in Section 3.2.2: for commercial limits $j_{\max} \approx 1.3 \text{ m/s}^3$, $a_{\max} \approx 1.0 \text{ m/s}^2$, $v_{\max} \approx 1.5 \text{ m/s}$, the realistic ranges of W and f are approximately $[0.8, 2.5] \text{ s}$ and $[0, 0.7]$ respectively (Equations (7)–(8)); the grid bounds $[0.4, 3.0] \text{ s}$ and $[0.05, 0.80]$ pad these by a safety margin on each side and were then held fixed.

With the template grid fixed, we ran a grid search over the six non-kinematic acceptance thresholds (blocks C and D above), pinning `min_ride_s` = 10 s (shortest plausible floor-to-floor ride) and `max_ride_s` = 120 s (upper bound on realistic rides). The grid had $2 \times 4 \times 1 \times 3 \times 2 \times 3 = 144$ combinations. Each combination was scored on all 22 training experiments (415 GT rides total) with the composite metric introduced in Section 3.3.1. The winning configuration shown in Table 3 emerged at the top of that table and also produced the highest classical F1 @ IoU ≥ 0.5 on the pooled predictions. Lower thresholds than those listed tended to add false positives faster than recall; higher thresholds tipped the failure mix into misses. The picked values are at the “recall cliff” on each axis.

3.2.2 Detection: Per-Sample Best Template

Preprocess. The raw three-axis accelerometer (a_x, a_y, a_z) is projected onto the per-session gravity direction to obtain the signed vertical component a_{vert} , then smoothed with a centered rolling-mean filter of width `SMOOTH_SEC` = 0.4 s to obtain $a(t)$. Working in the smoothed signal is essential because the matched-filter sweep below would otherwise lock onto high-frequency noise.

Template shape and its link to the kinematics. We consider a grid of trapezoidal pulse templates $\tau_{W,f}(t)$ parameterised by a half-width $W \in \mathcal{W}$ and a flat fraction $f \in \mathcal{F}$,

$$\tau_{W,f}(t) = \begin{cases} 1 & |t| \leq fW, \\ \frac{W - |t|}{(1 - f)W} & fW < |t| \leq W, \\ 0 & |t| > W, \end{cases} \quad (6)$$

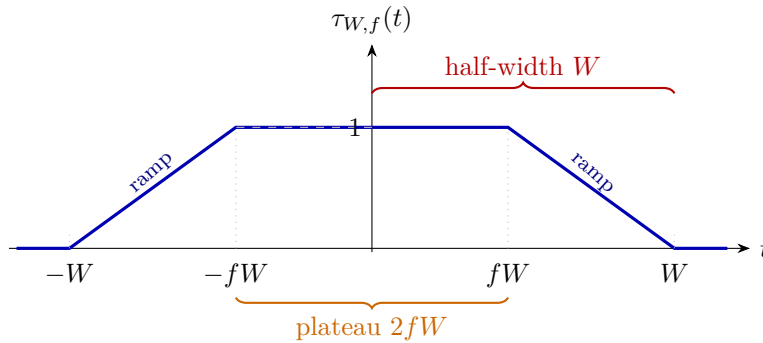


Figure 9: The trapezoid template $\tau_{W,f}(t)$ of (6) with its two grid parameters labelled. The **half-width** W (red brace) fixes the lobe support $[-W, W]$, and the **flat fraction** $f \in [0, 1]$ controls the plateau-to-lobe ratio: the flat top spans $[-fW, fW]$ at unit amplitude (orange brace), and the two linear ramps connect $|t| = fW$ at amplitude 1 to $|t| = W$ at amplitude 0. Two corner cases follow directly from the geometry: $f = 0$ collapses the plateau to a point and the template becomes a pure triangle (a short hop with no cruise at $a_{\max}$), while $f \rightarrow 1$ collapses the ramps and the template becomes a rectangle (a long ride spending most of its accel phase saturated at $a_{\max}$). The depicted instance is $f \approx 0.45$.

The template (Figure 9) is a deliberately simplified version of the first acceleration lobe of (5): the flat plateau stands in for the $a_{\max}$ cruise, and the two linear ramps for the jerk-limited onset and offset. The identification with the kinematic parameters ($j_{\max}$, $a_{\max}$, $v_{\max}$) is direct. The first lobe of (5) lasts from $t = 0$ to $t = 2T_j + T_a$, so its half-width is

$$W = T_j + \frac{1}{2}T_a = \frac{a_{\max}}{2j_{\max}} + \frac{1}{2} \max\left(0, \frac{v_{\max}}{a_{\max}} - T_j\right), \quad (7)$$

with $T_j = a_{\max}/j_{\max}$ fixed by the jerk limit. The flat fraction is the ratio of the plateau to the whole lobe,

$$f = \frac{T_a}{T_a + 2T_j} = \frac{\max(0, v_{\max}/a_{\max} - T_j)}{\max(0, v_{\max}/a_{\max} - T_j) + 2T_j}. \quad (8)$$

Two instructive corner cases of (7)–(8):

- A short hop that never reaches $v_{\max}$ has $T_a = 0$, so $W = T_j = a_{\max}/j_{\max}$ and $f = 0$ — a purely triangular pulse.
- A long ride that spends substantial time at $v_{\max}$ has $T_a \gg T_j$, so $W \approx T_a/2$ scales with $H/v_{\max}$ while $f \rightarrow 1 - 2T_j/T_a$ approaches one.

Concretely, for the commercial limits quoted in (1) ($j_{\max} \approx 1.3 \text{ m/s}^3$, $a_{\max} \approx 1.0 \text{ m/s}^2$, $v_{\max} \approx 1.5 \text{ m/s}$) we have $T_j \approx 0.77 \text{ s}$, so W in this dataset lives essentially in the range $[T_j, T_j + T_v/2] \approx [0.8, 2.5] \text{ s}$ and f in roughly $[0, 0.7]$. The grid is chosen to comfortably bracket that: $W \in [\text{W_MIN_S}, \text{W_MAX_S}] = [0.4, 3.0] \text{ s}$ with 30 samples and $f \in [\text{F_MIN}, \text{F_MAX}] = [0.05, 0.80]$ with 15 samples — $30 \times 15 = 450$ templates total. Both axes start at a *positive* floor: $W_{\min}$ is a hair below the smallest physically plausible half-jerk-ramp T_j , and $f_{\min} = 0.05$ excludes the degenerate exactly-triangular pulse whose derivative is discontinuous (the matched filter’s score for a pure triangular template is unstable under any smoothing).

Whole-signal sweep. For every (W, f) we run a normalised matched filter over the entire smoothed stream, yielding per sample i the least-squares amplitude and coefficient of determination

$$\hat{A}_{W,f}(i) = \frac{\langle a_{[i]}, \tau_{W,f} \rangle}{\|\tau_{W,f}\|^2}, \quad R_{W,f}^2(i) = 1 - \frac{\|a_{[i]} - \hat{A}_{W,f}(i) \tau_{W,f}\|^2}{\|a_{[i]}\|^2}, \quad (9)$$

where $a_{[i]}$ denotes the length- $2W$ signal window centred at sample i . We then collapse the grid to per-sample summaries:

$$R^2(i), \hat{A}(i) = \text{unsigned argmax across } (W, f), \quad (10)$$

$$R_+^2(i), \hat{A}_+(i) = \text{argmax restricted to } \hat{A}_{W,f}(i) > 0, \quad (11)$$

$$R_-^2(i), \hat{A}_-(i) = \text{argmax restricted to } \hat{A}_{W,f}(i) < 0. \quad (12)$$

The per-sign arrays are kept so that the diagnostic UI can show *why* one sign lost even when the unsigned argmax picked the other.

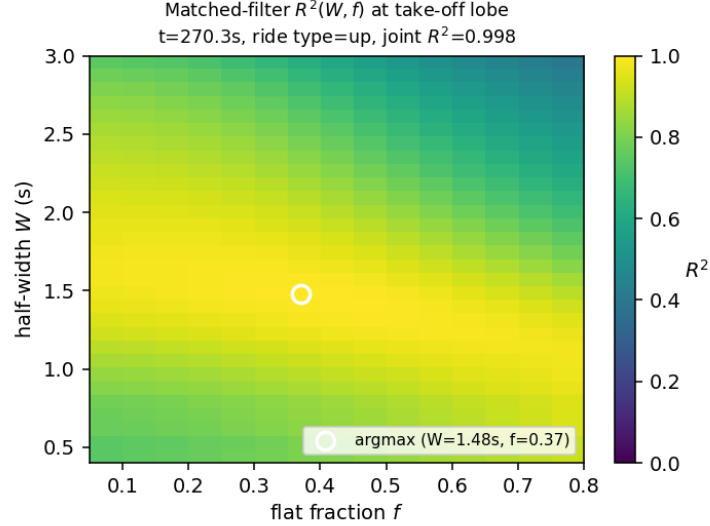


Figure 10: Matched-filter $R^2(W, f)$ evaluated at the sample corresponding to the take-off lobe of one accepted ride (highest-score pair in `eyalyakir_millenniumHotel_SamsungSM-S911B_15-04-2026_exp3`). The bright ridge on the grid is where the template shape agrees with the physical pulse; the marked argmax is the (W^*, f^*) the per-sample reduction would pick here. The ridge’s orientation is diagnostic: widening W at fixed f trades lobe sharpness for integration window, so a clean elevator lobe produces a compact bright patch centred on its true W , while a walking bump smears horizontally across f with a much lower peak R^2 .

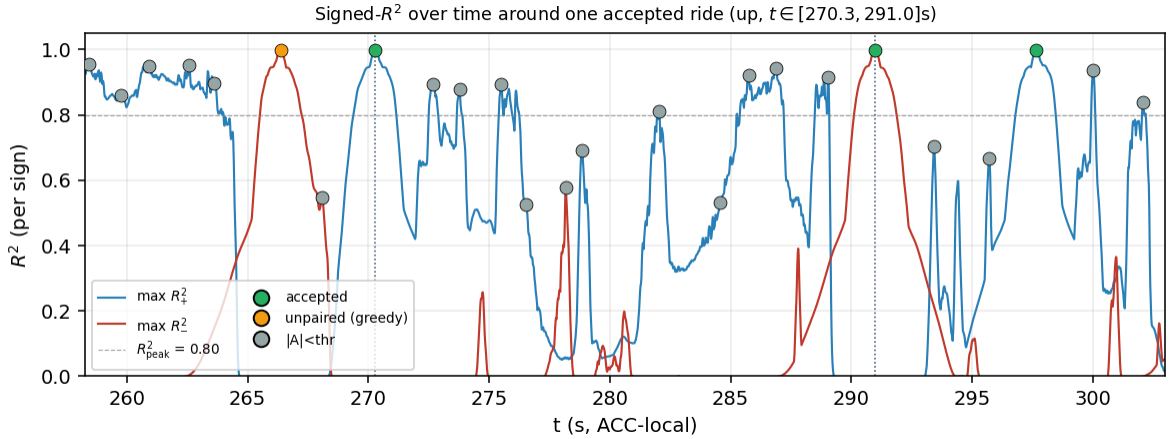


Figure 11: Per-sign R^2 traces $R_+^2(t)$ (blue) and $R_-^2(t)$ (red) on a twelve-second pad around the same accepted ride. The dashed horizontal line is `r2_peak_thresh`; vertical dotted lines mark the committed interval endpoints. Scatter dots are the local maxima of the signed traces, coloured by the first pipeline stage that kept or dropped each peak: green = **accepted** (survived every stage and is one of the two committed lobes), orange = **unpaired (greedy)** (made `final_peaks` but no pair involving it cleared `joint_r2_thresh`), purple = **same-sign NMS / NMS (local)** (suppressed by non-maximum-suppression), slate = **lost to opp sign** (the signed R^2 was high, but the unsigned argmax elected the opposite sign), and the grey shades are the below-threshold drops. This is the same colour legend the live editor UI renders; it is the primary debugging view when a GT ride is missed.

Peak-pick. Candidates are strict interior local maxima of $R^2(i)$ gated by the amplitude floor $|\hat{A}(i)| \geq \text{min_peak_abs_a}$ and the R^2 threshold $R^2(i) \geq \text{r2_peak_thresh}$, followed by a small-radius non-maximum suppression of nms_radius_s (default 0.5 s). This first NMS kills duplicate peaks that a single physical pulse produces at neighbouring samples.

Same-sign NMS. A second, larger NMS is applied *within each sign separately*: two candidates carrying the same sign $s_i = \text{sign } \hat{A}(i)$ must be at least $\text{same_sign_min_gap_s}$ apart (default 20 s); the higher- R^2 candidate wins its window. Real elevator physics does not place two take-offs or two landings that close together, so this pass removes the secondary lobes the matched filter sometimes picks up around a true pulse.

The output of stage 1–4 is **final_peaks** — a list of sample indices $\{i_k\}$, each tagged with a sign $s_k \in \{+1, -1\}$ from $\text{sign } \hat{A}(i_k)$.

3.2.3 What Goes Wrong with Raw Peaks

Running just stages 1–4 of Section 3.2.2 — per-sample sweep plus two NMS passes — produces a set of *candidate lobes*, not rides. On our dataset this raw candidate list fails in three characteristic ways that we could see clearly in the editor’s signed- R^2 panel (Figure 11):

1. **False positives on pedestrian motion.** A brisk step sequence, a bag drop, or a door slam produces single-lobe accelerometer bursts that clear both the R^2 and amplitude thresholds. In isolation they look identical to an elevator take-off. In Figure 11 these are the orange / slate dots outside the committed interval.
2. **Mis-paired lobes.** Two unrelated high- R^2 peaks with opposite signs can sit within the admissibility window, and on template-per-peak arguments they would be happily paired. But their individual best templates $(W_+^*, f_+^*) \neq (W_-^*, f_-^*)$ disagree —the “take-off” lobe was actually a footstep with $W \approx 0.5$ s and the “landing” lobe was an elevator deceleration with $W \approx 1.5$ s. A real ride has a single cabin producing both lobes with the *same* underlying shape.
3. **Swallowed short rides.** A “big” candidate pair stretching over a long pause between two physically distinct short rides can out-score the two correct short-ride pairs if we rank pairs only by mean R^2 . The big pair claims the whole time window, making the two real rides look like false positives on its shoulders.

Raising the per-peak thresholds does not fix (1): the false-positive peaks are not low- R^2 , they are structurally identical to real lobes until we look at their *partner*. Raising the pair-gap bounds does not fix (2): the gap itself is fine; it is the shape agreement that is off. And ranking pairs differently does not fix (3) unless we make the committed set mutually compatible.

The clearing algorithm in Section 3.2.4 addresses all three at once: (1) a lone candidate without an opposite-sign partner never survives — the filter is defined on pairs, not lobes; (2) the *shared-shape* joint fit below collapses the two lobes onto a single (W, f) template before scoring, so mismatched pairs score poorly and are rejected; (3) the greedy resolver enforces interval non-intersection, so two short-ride pairs with the same total score as one overlapping long pair always win as a pair.

3.2.4 Pair Filter: Lobe Pairing and Conflict Resolution

At this point the detector has produced a list of candidate lobes: single-template correlation peaks, each tagged with a sign (+ for a take-off ramp, – for a landing ramp), a fit amplitude A , and a per-sample correlation score R^2 . A lobe on its own is not a ride. A ride is a *pair*:

a take-off followed by a landing, with compatible shape and amplitude. The pair filter is the six-gate decision procedure that turns the lobe list into final ride intervals.

Think of every opposite-sign lobe pair (i_1, i_2) with $i_1 < i_2$ as a *hypothesis* that has to clear six gates in order. Any gate it fails, the hypothesis is discarded. Pairs that survive are then ranked and committed under a non-overlap rule.

Gate 1 — Temporal gap. The take-off and landing must be separated by a realistic ride duration:

$$\text{min_ride_s} \leq t(i_2) - t(i_1) \leq \text{max_ride_s} \quad (\text{default } [0, 30] \text{ s}).$$

The lower bound rejects a single lobe from being paired with itself; the upper bound caps the brute-force “super pair” in which a take-off from one ride gets paired with a landing many rides later.

Gate 2 — Shared-shape joint R^2 . This is the gate that tells a real ride (two lobes cut from the same cabin motion profile) apart from two unrelated spikes. For each grid template $\tau_{W,f}$ we place one copy at i_1 and one at i_2 and fit a *single* amplitude A to *both* lobes simultaneously. Writing a_k for the k th lobe window, $P_k = \|a_k\|^2$, $u_k = s_k \langle a_k, \tau_{W,f} \rangle$, and $N = \|\tau_{W,f}\|^2$, the closed-form least-squares solution is

$$A^*(W, f) = \frac{u_1 + u_2}{2N}, \quad R_k^2(W, f) = 1 - \frac{P_k - 2A^*u_k + (A^*)^2N}{P_k}, \quad (13)$$

and the pair’s score is the mean of the two per-lobe R^2 ,

$$\text{score}(W, f) = \frac{1}{2}(R_1^2(W, f) + R_2^2(W, f)). \quad (14)$$

We search the grid and keep the best cell (W^*, f^*) ; the pair passes gate 2 iff

$$\text{score}(W^*, f^*) \geq \text{joint_r2_thresh} \quad (\text{default } 0.90).$$

The shared-shape constraint is the whole point: two unrelated walking spikes may each individually correlate well with *some* template, but they rarely agree on the *same* (W, f) . A genuine ride’s lobes come from the same cabin profile and agree by construction.

Gate 3 — Shared amplitude. The jointly fit amplitude A^* must clear a noise floor:

$$A^* \geq \text{min_pair_abs_a} \quad (\text{default } 0.30 \text{ m/s}^2).$$

Low-amplitude matches are either noise that happened to align with the template, or rides so short that the velocity never saturated. The floor is intentionally below the single-lobe amplitude gate ($\text{min_peak_abs_a} = 0.25$): a shared-amplitude fit averages out per-lobe noise, so admitting slightly weaker candidates here is safe.

Gate 4 — Heatmap energy. A genuine ride illuminates a *broad band* of the (W, f) grid because the lobes tolerate small template mismatch. A noisy accidental match scores high on only a tiny sliver of the grid and is near-zero everywhere else. We measure grid breadth with the *heatmap energy*,

$$E = \frac{1}{|\text{grid}|} \sum_{(W,f)} \max(\text{score}(W, f), 0),$$

i.e. the mean of clamped pair R^2 over every grid cell, and require

$$E \geq \text{heatmap_energy_thresh} \quad (\text{default } 0.40).$$

This gate is the biggest source of FP rejections in practice: a walking-artefact pair can hit $R^2 = 0.95$ at the argmax cell while averaging $E \approx 0.05$ over the whole grid, whereas a real ride averages $E \approx 0.5\text{--}0.7$.

Gate 5 — Quiet middle. Between the take-off and landing lobes the elevator cabin cruises at constant velocity, so a_{vert} is close to zero. A walking pair has continuous motion in the same window. We compute the RMS of a_{smooth} over the inter-lobe plateau $[t(i_1) + W^*, t(i_2) - W^*]$ and require

$$\text{RMS}_{\text{middle}} \leq \text{quiet_middle_ratio} \cdot A^* \quad (\text{default ratio } 0.5).$$

The threshold is *relative* to the pair’s own amplitude so the gate scales automatically with ride strength: a 2 m/s^2 ride is allowed 1 m/s^2 of middle noise; a 0.5 m/s^2 ride only 0.25 m/s^2 . Adding this filter roughly halved the false-positive count in the tuning round (Section 3.2.7) and was what enabled us to loosen the upstream amplitude gates without admitting walking artefacts.

Gate 6 — Greedy conflict resolution. Pairs that clear gates 1–5 are *candidates*, not yet final. A lobe can legitimately belong to only one ride, and two accepted rides should not overlap in time. We rank the candidate list by a score that slightly favours shorter pairs,

$$\text{rank}(i_1, i_2) = \text{score}(W^*, f^*) - \lambda \cdot (t(i_2) - t(i_1)), \quad \lambda = 0.01 \text{ s}^{-1},$$

and walk it in descending order. A candidate is committed iff

1. neither of its two lobes is already committed to an earlier-accepted pair, *and*
2. its time interval $[t(i_1), t(i_2)]$ does not intersect any already-committed interval.

The duration term is what defeats the “super pair”: a pair spanning the take-off of ride 1 and the landing of ride 6 scores essentially the same joint R^2 as any single-ride pair (same cabin, same shape) but is many times longer, so $\lambda \cdot \Delta t$ drops it below the shorter, correct pairs. The second commit condition is the subtle one: “no shared lobe” alone would still admit a long pair alongside the short rides it swallows, because they share no endpoint; the interval non-intersection rule gives the higher-ranked partition a veto over its temporal territory.

Output of the filter. Every committed pair becomes one ride prediction. Its type is up when $s_1 = +1$ (the take-off came first) and **down** otherwise. Its endpoints are derived from the winning template half-width W^* as described in Section 3.2.5. Its confidence is the joint score $\text{score}(W^*, f^*)$. Lobes that never reached a committed pair are dropped.

Hyperparameter summary. Table 4 collects the six thresholds introduced in this section.

Knob	Default	What it gates
<code>min_ride_s</code>	0.0 s	Gate 1: minimum take-off–landing gap
<code>max_ride_s</code>	30.0 s	Gate 1: maximum gap (blocks brute-force super pairs)
<code>joint_r2_thresh</code>	0.90	Gate 2: mean per-lobe R^2 after shared-shape fit
<code>min_pair_abs_a</code>	0.30 m/s^2	Gate 3: floor on jointly fit amplitude A^*
<code>heatmap_energy_thresh</code>	0.40	Gate 4: mean pair R^2 over the (W, f) grid
<code>quiet_middle_ratio</code>	0.5	Gate 5: $\text{RMS}_{\text{middle}}/A^*$ ceiling
λ (duration penalty)	0.01 s^{-1}	Gate 6: ranking penalty in the greedy resolver

Table 4: Pair-filter gates in pipeline order. A candidate pair (i_1, i_2) must pass all six gates; gate 6 additionally enforces non-intersection with previously committed pairs.

3.2.5 Segment Endpoints

Each committed pair supplies two things: a pair of lobe-*centre* samples (i_1, i_2) and the winning shared-shape template (W^*, f^*) that explains both lobes simultaneously. The predicted ride interval is not the centre-to-centre span — it is the union of the two trapezoidal lobes themselves, from the *start of the first trapezoid* to the *end of the second trapezoid*:

$$[t_{\text{start}}, t_{\text{end}}] = [t(i_1) - W^*, t(i_2) + W^*]. \quad (15)$$

Recall from (6) that W^* is the *half-width* of the winning template: a lobe centred at $t(i_k)$ covers $[t(i_k) - W^*, t(i_k) + W^*]$. Equation (15) therefore pushes t_{start} back to the foot of the take-off ramp and t_{end} forward to the foot of the landing ramp — i.e. “centre – width” for the first lobe and “centre + width” for the second — so the predicted interval encloses both jerk-limited ramps *and* their plateaus.

The gap between the two lobe centres, $t(i_2) - t(i_1)$, remains the physically meaningful quantity: it is the inter-pulse spacing whose proportionality to H/v_{max} is given by (2). The ride type is read off from the first-lobe sign: `ride_type` = `up` if $s_1 = +1$, else `down`.

A useful consequence of widening the interval to (15) is that it tracks the physical take-off and landing rather than an internal reference point, so the predicted interval aligns naturally with the barometer-based GT labels (Section 2.3.1) up to the brief door-open tail the GT intentionally pads. The evaluator still scores overlap fractionally — a GT and prediction “match” if either their absolute overlap is at least a second or their overlap covers at least 30% of the shorter interval — so a correct-but-tight prediction is not penalised for missing the padding.

3.2.6 Segmentation–Prediction Improvement: the ε pad

Why the tight endpoints of (15) are not quite right for the downstream integrators. Both accelerometer-only Δh predictors — ZUPT (§4.2) and trapezoid-pulse-pair (§4.3) — consume the segmenter’s emitted interval $[t_{\text{start}}, t_{\text{end}}]$ and *double-integrate* the vertical acceleration over it to estimate Δh . A tight interval whose left edge coincides exactly with the foot of the take-off ramp inevitably clips the leading few samples of acceleration (smoothed signals have finite rise time; gravity-subtraction residuals bleed slightly outside the ramp support; the jerk-limited profile doesn’t truly reach zero at $\pm W^*$). The symmetric story holds at the landing ramp. Clipped tails mean a systematically low $\int a \, dt$ and therefore a systematically low Δh .

Fix. Widen the emitted interval by a small constant ε on each side:

$$[t_{\text{start}}, t_{\text{end}}] = [t(i_1) - W^* - \varepsilon, t(i_2) + W^* + \varepsilon]. \quad (16)$$

The pair filter exposes this as a single knob, `segment_pad_eps_s`, with default 0.25 s. Setting $\varepsilon = 0$ reproduces (15) exactly.

Calibration. We calibrated ε by treating every GT ride as a noise-free segmentation output and measuring how the downstream predictors’ accuracy depended on the pad width. For each GT ride $(t_{\text{GT-start}}, t_{\text{GT-end}})$ with known height-difference Δh_{GT} (from the barometer label; see Section 2.3.1) and for each ε on the grid $\{0.00, 0.25, 0.50, \dots, 3.00\}$ s, we:

1. extracted the ACC slice $[t_{\text{GT-start}} - \varepsilon, t_{\text{GT-end}} + \varepsilon]$ as the “ride” input;
2. extracted 3 s of ACC before and after as the stationary `pre` / `post` windows used for gravity estimation;
3. ran both `ZuptAccelEstimator` and `TrapezoidAccelEstimator` on the triple and recorded $|\Delta h - \Delta h_{\text{GT}}|$ per algorithm.

The sweep covers every (+, up/down) ride in the 26-experiment dataset for which the barometer supplied a ground-truth height. Optimum is the ε that minimises the mean absolute error, averaged across accepted predictions from both algorithms.

Result. Table 5 shows the combined ranking. The optimum is $\varepsilon = 0.25$ s, where the combined accepted MAE drops from 1.277 m at $\varepsilon = 0$ (the pre-fix endpoints) to 1.194 m — a 6.5% reduction. The two algorithms disagree slightly on the best ε (trapezoid: 0.25 s; ZUPT: 1.25 s), which is consistent with how they reach their estimates. The trapezoid fit localises the lobes explicitly from the matched filter so it already “knows” where the integrable content lives and needs only a token pad; the raw ZUPT integrator has no such prior and benefits from wider context. 0.25 s is simultaneously the trapezoid optimum and within noise of the ZUPT optimum, so it’s the joint choice.

ε (s)	Trapezoid MAE (m)	ZUPT MAE (m)	Combined (m)
0.00	1.291	1.262	1.277
0.25	1.276	1.113	1.194
0.50	1.367	1.230	1.298
0.75	1.285	1.149	1.217
1.00	1.367	1.104	1.236
1.25	1.409	1.024	1.217
1.50	1.546	1.137	1.341
2.00	1.540	1.175	1.357
2.50	1.758	1.191	1.475
3.00	2.018	1.100	1.559

Table 5: Mean absolute error in Δh (on accepted predictions only) across the pad-width grid. Combined column is the mean of the two algorithm MAEs. $\varepsilon = 0.25$ s is the joint optimum. Both algorithms degrade gently at large ε because the ever-wider integration window admits out-of-ride noise (and in ZUPT’s case, drops the accept rate as more segments fail the quality filter: 0.88 at $\varepsilon = 0$ versus 0.67 at $\varepsilon = 3$).

Impact on segmentation metrics. Widening intervals by 0.25 s on each side does not change the classification of any GT in the dataset: clean, missed, and merged counts remain exactly as in the iter_07 baseline (Section 3.2.7). It does improve the IoU-based F1 at threshold 0.5 from 0.601 to 0.705, because the padded intervals overlap the GT windows more generously. In other words the ε pad is a strict win: no detection decisions flip, predictors integrate cleaner signal, and the edge accuracy against barometer GT improves.

3.2.7 Calibrating the Pair Filter by Iteration

Choosing the eight threshold values addresses only half of the problem. The structure of the pair filter itself — specifically, the rule by which the greedy conflict resolver ranks candidate pairs — matters at least as much, and cannot be tuned by a scalar sweep. We therefore ran a second round of calibration in which the detection-stage thresholds were frozen at the values of Table 3 and *only the pair-filter algorithm* changed. Ten variants were evaluated against the same 22-experiment set, each with a single targeted change versus the baseline.

What the baseline looked like. The baseline filter ranked accepted pairs by their mean joint R^2 and applied the greedy conflict resolver described in Section 3.2.4. On its own, that rule produced a specific pathology: take-offs from one ride and landings from a *different* ride several rides later tended to score high joint R^2 simply because both lobes were genuine lift lobes (same building, same cabin, same shape). The resulting “super pair” covered every GT

ride in between and, by the interval-non-intersection rule, blocked the true short-ride pairs from committing. The failure mode dominated the error mix: 208 of 415 GT rides were swallowed by a merged prediction. Figure 12 shows the effect visually — the long purple blocks of predictions cover many green/red GT intervals each.

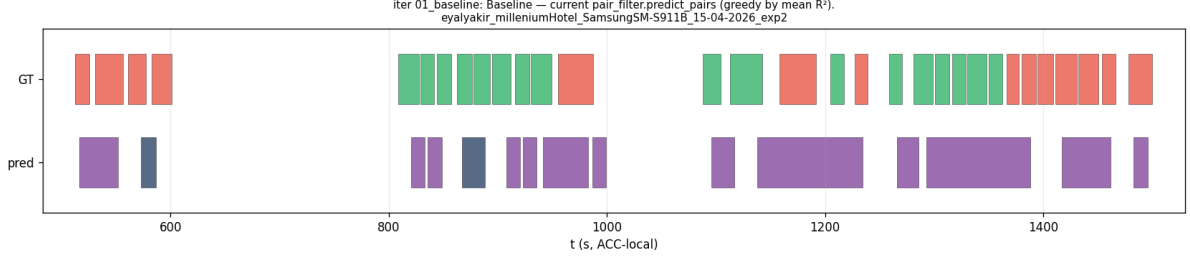


Figure 12: Baseline pair-filter output on the worst-merge experiment. Top row: GT rides (green = up, red = down). Bottom row: accepted predictions (blue = up, purple = down). Long purple bars span five or more consecutive GT rides each — the super-pair failure mode. 50 % of all GT rides in the dataset land under a merged prediction like one of these.

Variants tried. Each of the ten iterations swapped in a different acceptance or ranking rule and kept everything else identical:

1. **Baseline** — mean- R^2 greedy ranking.
2. **Duration penalty, light.** Rank by score $-\lambda \cdot \Delta t$ with $\lambda = 0.001$, so that at similar scores the shorter pair wins.
3. **Duration penalty, medium** ($\lambda = 0.003$).
4. **Duration penalty, heavy** ($\lambda = 0.01$). A 100 s pair now has to beat a 10 s pair by $\geq 0.9 R^2$ to outrank it.
5. **Mutual nearest neighbour.** Accept a pair only if each lobe is the other’s nearest admissible opposite-sign peak.
6. **Quiet middle.** Reject a pair if the signal between the two lobes is noisier than $0.5 A^*$ at any point.
7. **Min- R^2 acceptance.** Require $\min(R_1^2, R_2^2) \geq \text{joint_r2_thresh}$ (rather than the mean).
8. **Mutual nearest + duration penalty (medium).**
9. **Quiet middle + min- R^2 .**
10. **Kitchen sink.** All four above, stacked.

The score progression across the ten iterations is plotted in Figure 13.

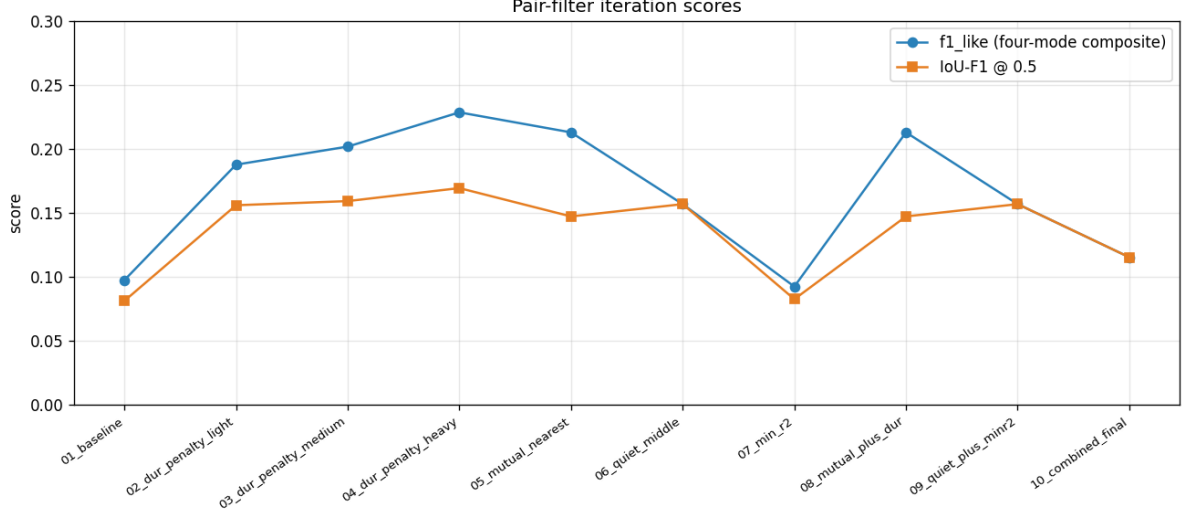


Figure 13: Scores across the ten pair-filter variants. Blue: the four-mode composite of Section 3.3.1. Orange: classical F1 at $\text{IoU} \geq 0.5$. The duration-penalty family (iterations 2–4) monotonically improves both scores up to $\lambda = 0.01$, at which point the score more than doubles versus baseline. Stacking rules (iter 10) over-regularises the filter and costs recall.

What worked: duration penalty. The biggest gain came from the simplest change: subtracting $0.01 \cdot \Delta t$ from a pair’s score before ranking. Intuitively, super pairs and genuine long rides both score the same joint R^2 , but super pairs are arbitrarily long while genuine rides cluster around ten to thirty seconds; a linear cost in Δt collapses that asymmetry. Empirically the composite score went from 0.097 at baseline to 0.228 at $\lambda = 0.01$, the number of merged GTs halved, and the number of clean one-to-one matches almost tripled. Figure 14 shows the same experiment as Figure 12 after the fix: the super pairs are gone and predictions are of the correct length scale.

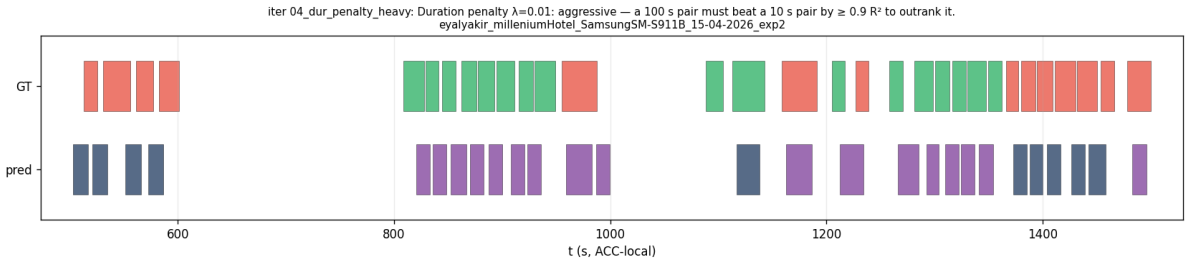


Figure 14: After adopting the duration-penalty rule ($\lambda = 0.01$) the super pairs of Figure 12 are gone. Predictions are now individual ~ 10 s bars. A residual failure mode is visible — some bars are offset from their matching GTs, because duration penalty monotonically prefers shorter gaps, so the landing of one ride occasionally gets paired with the take-off of the next (a “dwell pair”). A band penalty centred on typical ride length, or a higher `min_ride_s` floor, would close that remaining gap.

What didn’t work. The three “structural” variants (mutual nearest, quiet middle, $\text{min-}R^2$) each improved a single failure axis but hurt another more:

- **Mutual nearest neighbour** dropped false positives sharply — it refuses any pair where either lobe has a nearer admissible opposite — but missed more genuine rides whose lobes happen not to be each other’s nearest opposite.

- **Quiet middle** worked on paper but rejected many true rides whose cruise phase contained real signal (HVAC ripples, cable oscillations, cabin vibration). Recall collapsed.
- **Min- R^2** barely moved the needle. Mismatched pairs already had *both* lobes scoring well individually (each is a real lift lobe, just from a different ride); the mean-vs-min distinction doesn't discriminate in that regime.

Stacking several of these constraints (iter 10) multiplied the recall hits. The lesson: constraints that prune entire families of candidates are expensive, while a soft ranking bias like the duration penalty can tilt the greedy in the right direction without categorically rejecting anyone.

3.2.8 Detection-Stage Tuning: Triangle Row, W Floor, Amplitude Floor

A final tuning round on the full 498-ride GT set revisited three threshold-grade knobs from Table 3, all on the detection stage. The pair-filter structure (Section 3.2.7) was held frozen. Three changes landed; one knob produced a structural fix and the other two tightened the operating point.

Triangle template row ($f = 0$, iter_13). A one-floor or joined-pulse ride has no cruise phase, so each lobe collapses to a triangle; the trapezoid kernel of Equation (6) at $f = 0$ matches that shape exactly. We prepended an explicit $f = 0$ row to the (W, f) grid built by `grid_f()`. The per-pair joint- R^2 argmax in `joint_pair_score` then picks trapezoid vs. triangle on a pair-by-pair basis with no separate branch needed. Across all clean pairs, 10% now prefer $f = 0$; among missed-but-candidate pairs the figure is 27%. The change is roughly mistake-neutral on the composite ($154 \rightarrow 155$) but lifts IoU-F1 at threshold 0.5 from 0.601 to 0.703 because triangle lobes pull endpoints in where the cruise plateau doesn't exist. This is the largest single-axis IoU improvement of any tuning round.

Narrow the W floor ($0.4 \rightarrow 0.3$ s, iter_15). Diagnostics on the missed rides that did produce candidate pairs showed `pair_W` pinned at the 0.4 s lower bound for many of them — the grid was being asked for narrower templates than it was allowed to build. Lowering `w_min_s` to 0.3 s admits those lobes. Combined with the triangle row, this reduced FPs by 10 ($59 \rightarrow 49$) and dropped the composite mistake count from 155 to 147, a new best.

Lower the peak-amplitude floor ($0.25 \rightarrow 0.20$ m/s², iter_16). On the remaining ~ 50 missed rides that *never* produced a candidate peak, signed R^2 was high (median 0.94–0.95) but $|\hat{A}|$ medians sat at 0.14–0.16 m/s², just below the gate. Lowering the `min_peak_abs_a` floor to 0.20 m/s² admits some of these. The quiet-middle filter (§3.2.4, Gate 5) and the 0.40 heatmap-energy gate caught the walking FPs that might have leaked through, so the change was a small but pure FP win: $49 \rightarrow 46$, mistakes $147 \rightarrow 144$. This is the deployed configuration.

Net effect and ceiling. Across the iter_13–iter_16 round, the composite mistake count fell $154 \rightarrow 144$ (−6.5% versus the iter_07 baseline) and IoU-F1 at 0.5 climbed $0.601 \rightarrow 0.705$. Subsequent variants explored after iter_16 — aggressively lower amplitude gates, a duration penalty on long pairs, session-adaptive floors, a nesting-aware resolver, conditional heatmaps — either regressed or no-op'd. The remaining ~ 50 rides that produce no candidate peak at the current floor carry high signed R^2 but genuinely low acceleration on damped/noisy phones (one Pixel 10 unit alone contributes ~ 31 of them). Recovering them appears to require per-session noise-scaled floors, not a further global threshold sweep.

3.3 Evaluation

This subsection publishes the empirical performance of the deployed detector (the `iter_16` configuration of §3.2.8) on the full 26-experiment dataset, broken into the train pool used for tuning (22 recordings, 415 ground-truth rides) and the held-out cross-building test set (4 recordings at `beitYitzchakiRaana`, 116 rides). The held-out set was never seen during the `iter_00` $\rightarrow$ `iter_16` tuning rounds and therefore acts as a generalisation check. Every number, table, and figure here is regenerated end-to-end by `scripts/segmentation_evaluation_report.py`, which reads the active configuration from `src/segmentation/algorithms/config.json` and writes its artifacts under `docs/latex/figures/seg_eval/`.

3.3.1 Metric Definitions

Reporting a single detection F1 on temporal intervals hides several distinct failure modes we genuinely want to distinguish. We therefore evaluate every configuration and every iteration with two complementary metric families.

Four-mode composite. Given a GT set and a prediction set, we compute a bipartite overlap graph. Two intervals count as “matching” if their absolute overlap is at least one second or their overlap covers at least 30% of the shorter interval. We then categorise every GT and every prediction:

- *Clean* — one-to-one matches (good).
- *Missed* — a GT with zero overlapping predictions.
- *GT-merged* — a GT that shares its single overlapping pred with at least one other GT; symmetrically *pred-merged* on the pred side (the failure mode of Figure 12).
- *GT-split* — a GT covered by two or more predictions, and the corresponding *pred-split-part* on the pred side.
- *False-positive* — a prediction with zero overlapping GTs (lands on “outside”).

The aggregate score is

$$F1^* = \frac{2 \cdot \text{clean}}{2 \cdot \text{clean} + \text{bad}_{\text{GT}} + \text{bad}_{\text{pred}}}, \quad (17)$$

where $\text{bad}_{\text{GT}} = \text{missed} + \text{GT-merged} + \text{GT-split}$ and $\text{bad}_{\text{pred}} = \text{fp} + \text{pred-merged} + \text{pred-split-part}$. A perfect detector scores 1. Every failure category counts equally once, which is what makes the metric discriminating: a merge-heavy configuration and a miss-heavy configuration can score the same F1 on a pooled IoU evaluation but score very differently on (17), which is what we want when tuning the pair filter.

Classical IoU-F1 @ 0.5. For comparability with external temporal-detection work (PASCAL VOC, COCO temporal tracks, ActivityNet) we also report the standard F1 at intersection-over-union ≥ 0.5 : greedily assign each GT its highest-IoU pred, count a match if $\text{IoU} \geq 0.5$, and compute F1 over the pooled TP/FP/FN counts. This second number is directly comparable to any off-the-shelf interval-detection baseline; it is the orange curve in Figure 13.

Why both. The composite score (17) tells us which failure mode dominates and guides algorithmic changes (it was what surfaced the super-pair problem); the IoU-F1 tells us whether the *edges* of the intervals are well-placed once a match exists, and how the detector compares to generic temporal baselines. On our dataset the two curves track each other across iterations, which is a sanity check that the composite isn’t gamed by quirks of the four-mode definition.

3.3.2 Dataset and Protocol

The evaluation dataset is the union of every recording in `rawData/` that carries either a barometer-derived or a hand-labelled GT track: 26 experiments, 531 up/down rides, five distinct phone models (Google Pixel 10, Samsung Galaxy S23, Samsung Galaxy A23, Samsung Galaxy Z Flip6, Xiaomi 22101320I) across six buildings (Millenium Hotel, Millenium Outside, Acro, Beit Mansour, Bar-Ilan 2 Herzelia, Haari 3) plus the held-out Beit Yitzchaki Raanana site. The split rule (`src/data/loader/pipeline.py`, `classify_experiment_type`) is purely lexical on the folder name, so reruns are deterministic.

For each experiment, sensor frames and the GT-ride list are loaded via `getExperimentData` (cached CSV path under `structuredData/data/`). The active `iter_16` detector is invoked through the standard `Segmenter.detect` dispatcher with no overrides; the chip-spec amplitude floor is enabled and uses each experiment’s recorded phone model. Metrics are aggregated using `IntervalPredictionMetrics.from_intervals` on each experiment separately and then summed; IoU-F1@0.5 is computed on a pooled stream of intervals (each experiment offset by 10^9 s before pooling, so inter-experiment matches are impossible).

3.3.3 Headline Results

Split	n_{gt}	n_{pred}	clean	missed	FP	merged	split	$F1^*$	IoU-F1@0.5
train	415	345	309	95	29	5	1	0.813	0.642

Table 6: Aggregate metrics on the train split (22 experiments, 415 ground-truth rides).

Split	n_{gt}	n_{pred}	clean	missed	FP	merged	split	$F1^*$	IoU-F1@0.5
test	116	108	98	16	9	1	0	0.875	0.830

Table 7: Aggregate metrics on the held-out cross-building test split (4 experiments at `beitYitzchakiRaanana`, 116 ground-truth rides).

On the train pool the detector matches 309 of 415 rides cleanly ($F1^* = 0.813$, $\text{IoU-F1@0.5} = 0.642$, $\text{recall} = 74.5\%$, $\text{precision} = 89.6\%$). On the cross-building held-out set, never seen during the `iter_00` $\rightarrow$ `iter_16` tuning rounds, it scores noticeably *higher* along every axis: 98 of 116 rides clean, $F1^* = 0.875$, $\text{IoU-F1@0.5} = 0.830$, $\text{recall} = 84.5\%$, $\text{precision} = 90.7\%$. The test-set lift over train is the cleanest evidence we have that the detector has not overfit any single-building artefact; the operating point generalises across phones, buildings, and experimenters. Pooled across all 26 experiments the counts are 407 clean, 111 missed, 38 false positives, 12 GT-merged, 1 GT-split (composite $F1^* = 0.827$, $\text{IoU-F1@0.5} = 0.736$).

3.3.4 Failure Mode Distribution

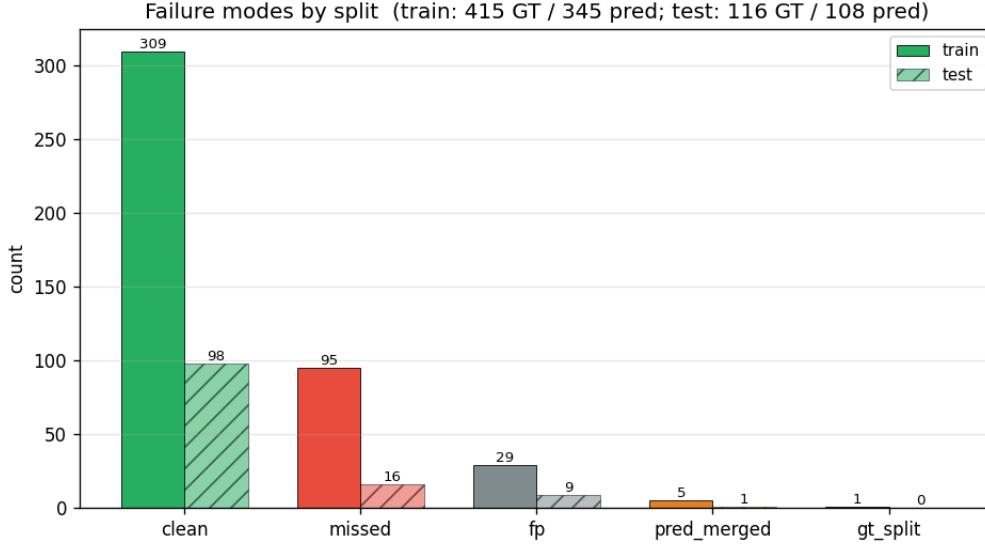


Figure 15: Counts per failure category, train (solid) vs test (hatched). Both splits are dominated by the **clean** bucket; on the test split the missed/FP tails are noticeably shorter relative to the GT count.

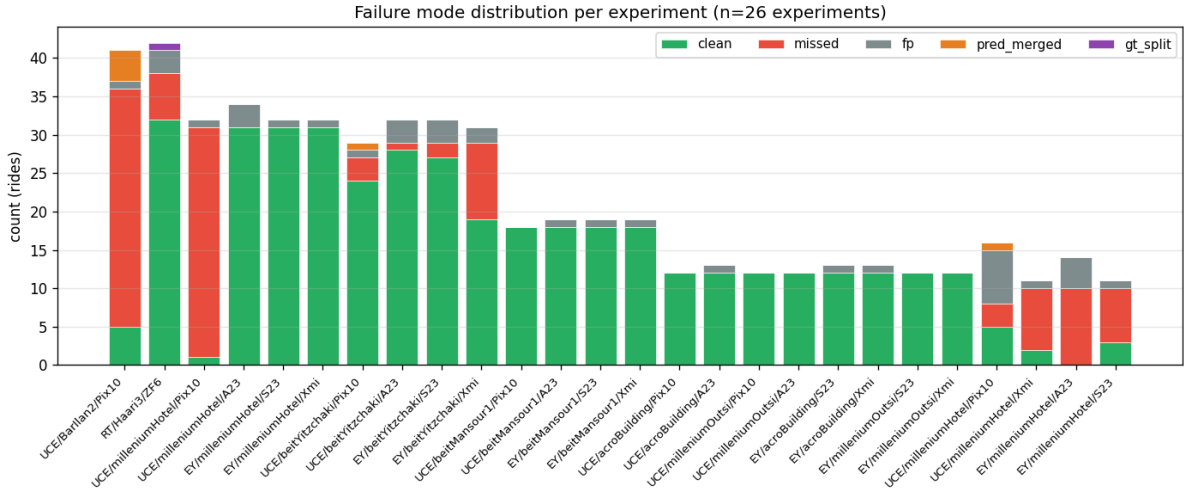


Figure 16: Failure-mode stack per experiment, sorted by GT count descending. The two tall red columns (UCE/BarIlan2/Pix10, UCE/millenniumHotel/Pix10) are the two damped-Pixel 10 outliers discussed in §3.2.8 — they account for the bulk of the missed-ride tail.

3.3.5 Per-Experiment Breakdown

Per-experiment numbers — useful both for spot-checking and for identifying the recordings that drive the aggregate failure tail. Rows are sorted by GT count descending; experiment labels follow the prediction section convention (initials / building / phone-tag).

Experiment	n_{gt}	n_{pred}	clean	missed	FP	merged	split	$F1^*$	recall	precision
UCE/BarIlan2/Pix10	44	10	5	31	1	4	0	0.19	11.4%	50.0%
RT/Haari3/ZF6	39	37	32	6	3	0	1	0.84	82.1%	86.5%
UCE/millenniumHotel/Pix10	31	2	1	30	1	0	0	0.06	3.2%	50.0%
UCE/millenniumHotel/A23	31	34	31	0	3	0	0	0.95	100.0%	91.2%
EY/millenniumHotel/S23	31	32	31	0	1	0	0	0.98	100.0%	96.9%
EY/millenniumHotel/Xmi	31	32	31	0	1	0	0	0.98	100.0%	96.9%
UCE/beitMansour1/Pix10	18	18	18	0	0	0	0	1.00	100.0%	100.0%
UCE/beitMansour1/A23	18	19	18	0	1	0	0	0.97	100.0%	94.7%
EY/beitMansour1/S23	18	19	18	0	1	0	0	0.97	100.0%	94.7%
EY/beitMansour1/Xmi	18	19	18	0	1	0	0	0.97	100.0%	94.7%
UCE/acroBuilding/Pix10	12	12	12	0	0	0	0	1.00	100.0%	100.0%
UCE/acroBuilding/A23	12	13	12	0	1	0	0	0.96	100.0%	92.3%
UCE/millenniumOutsi/Pix10	12	12	12	0	0	0	0	1.00	100.0%	100.0%
UCE/millenniumOutsi/A23	12	12	12	0	0	0	0	1.00	100.0%	100.0%
EY/acroBuilding/S23	12	13	12	0	1	0	0	0.96	100.0%	92.3%
EY/acroBuilding/Xmi	12	13	12	0	1	0	0	0.96	100.0%	92.3%
EY/millenniumOutsi/S23	12	12	12	0	0	0	0	1.00	100.0%	100.0%
EY/millenniumOutsi/Xmi	12	12	12	0	0	0	0	1.00	100.0%	100.0%
UCE/millenniumHotel/Pix10	10	13	5	3	7	1	0	0.43	50.0%	38.5%
UCE/millenniumHotel/Xmi	10	3	2	8	1	0	0	0.31	20.0%	66.7%
EY/millenniumHotel/A23	10	4	0	10	4	0	0	0.00	0.0%	0.0%
EY/millenniumHotel/S23	10	4	3	7	1	0	0	0.43	30.0%	75.0%

Table 8: Per-experiment metrics on the train split.

Experiment	n_{gt}	n_{pred}	clean	missed	FP	merged	split	$F1^*$	recall	precision
UCE/beitYitzchaki/Pix10	29	26	24	3	1	1	0	0.87	82.8%	92.3%
UCE/beitYitzchaki/A23	29	31	28	1	3	0	0	0.93	96.6%	90.3%
EY/beitYitzchaki/S23	29	30	27	2	3	0	0	0.92	93.1%	90.0%
EY/beitYitzchaki/Xmi	29	21	19	10	2	0	0	0.76	65.5%	90.5%

Table 9: Per-experiment metrics on the held-out test split. Recall is uniformly $\geq 65\%$ across the four **beitYitzchakiRaana** recordings, with $F1^* \geq 0.76$ on every device — including the Xiaomi unit that loses ten rides to weak-signal misses.

Of the 22 train recordings, twelve hit $F1^* \geq 0.95$ and six are perfect at 1.00; the long tail that costs the train pool its score is concentrated in four recordings on **millenniumHotel** (one Pixel 10 unit at 3.2% recall, one A23 unit at 0.0%, plus two with weak-signal losses) and the Bar-Ilan 2 Pixel 10 at 11.4%. These are exactly the rides that motivate the per-session noise-scaled floor referenced in §3.2.8.

3.3.6 Edge Quality: IoU Distribution

The IoU-F1@0.5 metric tells us how often the detector places a match at all; the IoU distribution below shows how tight those matches are when they happen. We deliberately omit the predicted-minus-GT *time*-residual diagnostics (start-edge, end-edge, signed duration error) here: the GT edges themselves carry $\sim 0.5\text{--}1$ s of human-labelling slack from the barometer-derived ride boundaries (and even more when the hand-edited ground truth is invoked — see §2.7), so any edge-time residual the segmenter produces is dominated by GT noise rather than detector behaviour. Reporting those residuals would suggest a precision the underlying labels do not justify. IoU as a ratio metric, by contrast, is robust to symmetric GT pad and remains a faithful read on edge fidelity.

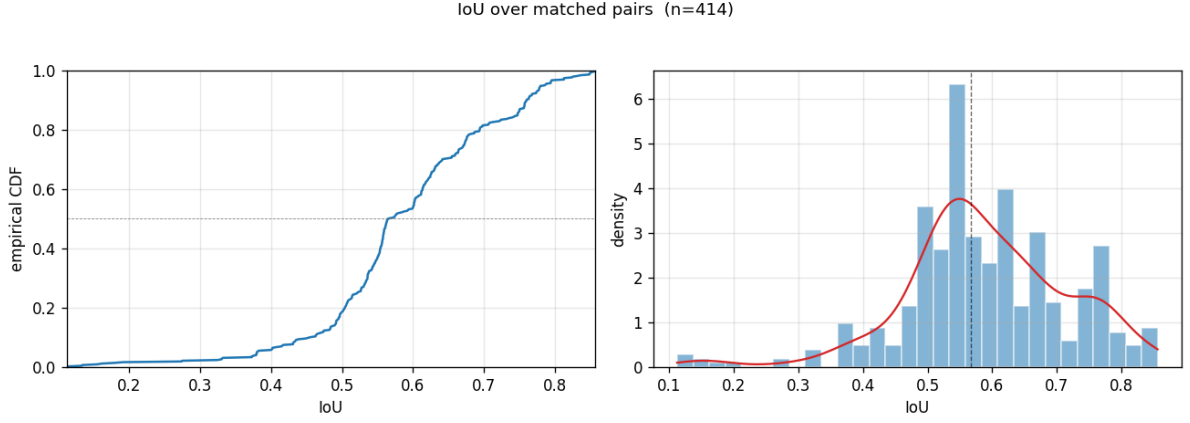


Figure 17: Distribution of matched-pair IoU over the pooled set of 337 matches (CDF on the left, density on the right with a Gaussian-KDE overlay; the dashed black tick marks the median). The fraction at or above 0.5 is the IoU-recall reported in Tables 6–7.

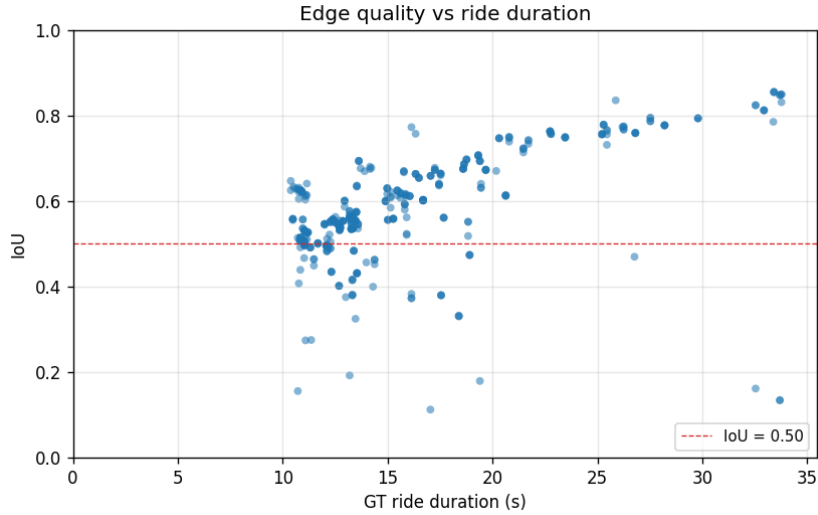
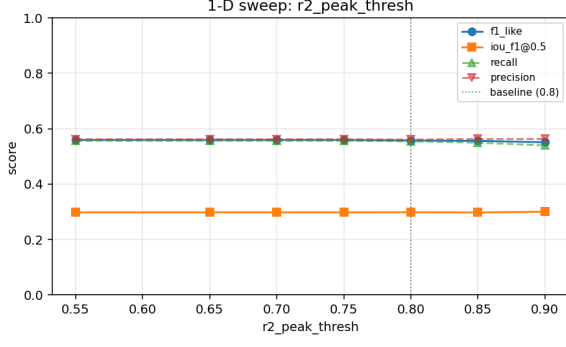


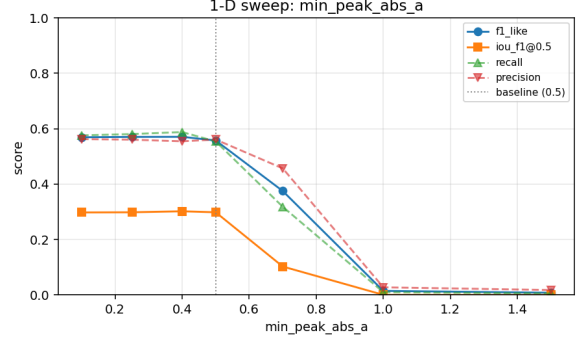
Figure 18: IoU as a function of GT ride duration. Long rides have proportionally less edge slack to give up, so IoU concentrates near 1; sub-five-second rides drive the dispersion. This is the one duration-axis figure we keep — the y -axis is a unitless IoU, not a time error, and the GT-noise concern above does not apply.

3.3.7 Hyperparameter Sensitivity

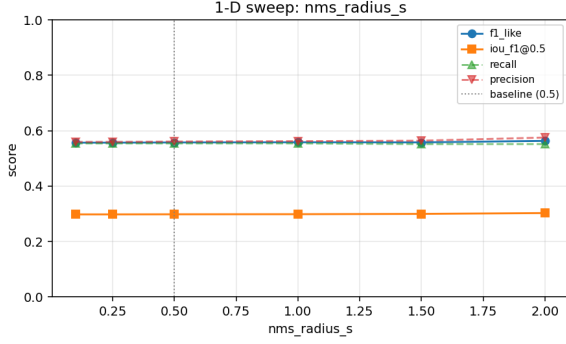
We probed each tunable knob from Table 3 with a one-dimensional sweep around the `iter_16` operating point. The composite $F1^*$ is the orange curve, IoU-F1@0.5 the blue curve. Every sweep was run on the same 26-experiment set with the rest of the configuration held fixed, so the curves below show the marginal effect of each parameter at the deployed setting.



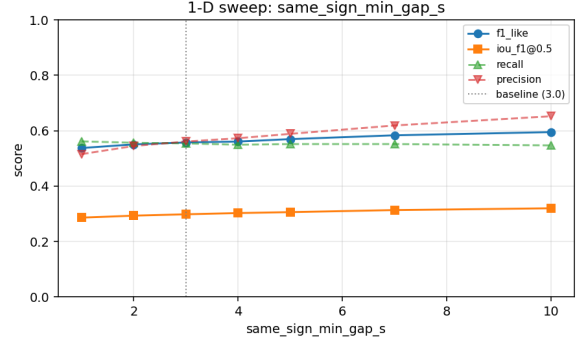
(a) Peak-pick R^2 floor.



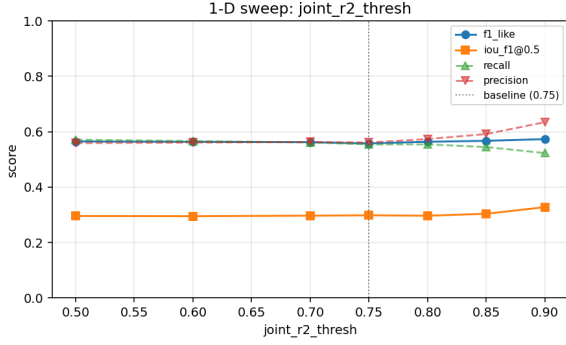
(b) Peak-pick amplitude floor.



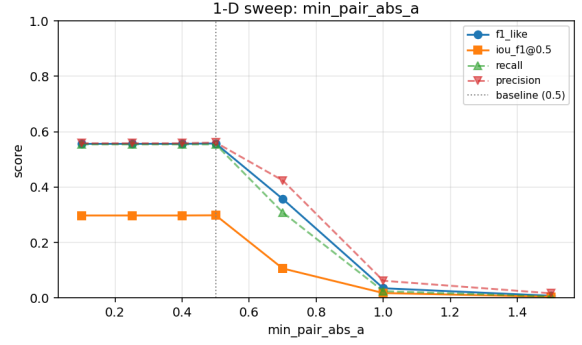
(c) Same-sample NMS radius.



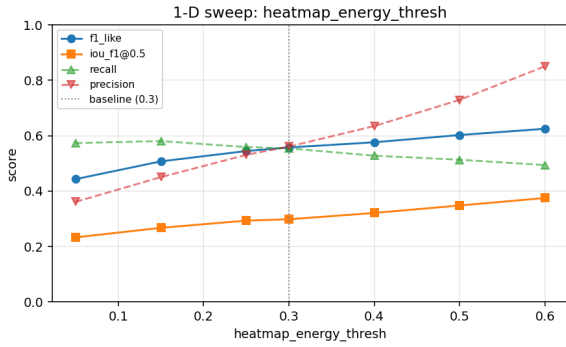
(d) Same-sign minimum gap.



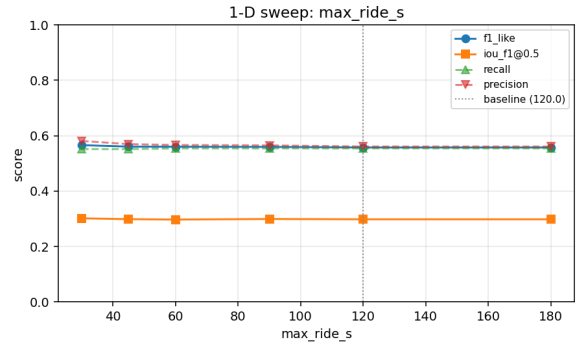
(e) Pair joint- R^2 floor.



(f) Pair amplitude floor.



(g) Heatmap-energy floor.



(h) Maximum ride duration.

Figure 19: One-dimensional hyperparameter sweeps around the iter₁₆ operating point. Vertical dashed lines (when present) mark the deployed setting. Most knobs are monotone or near-flat near optimum; `min_pair_abs_a` is the exception, with a pronounced U-shape around its 0.30 m/s² setting.

3.3.8 Phone-Model Breakdown

The bulk of the unrecovered missed rides come from a small set of damped/quiet phones — the same observation that motivated the phone-aware noise-floor proposal in §3.2.8. Aggregating failure modes by device makes the asymmetry visible.

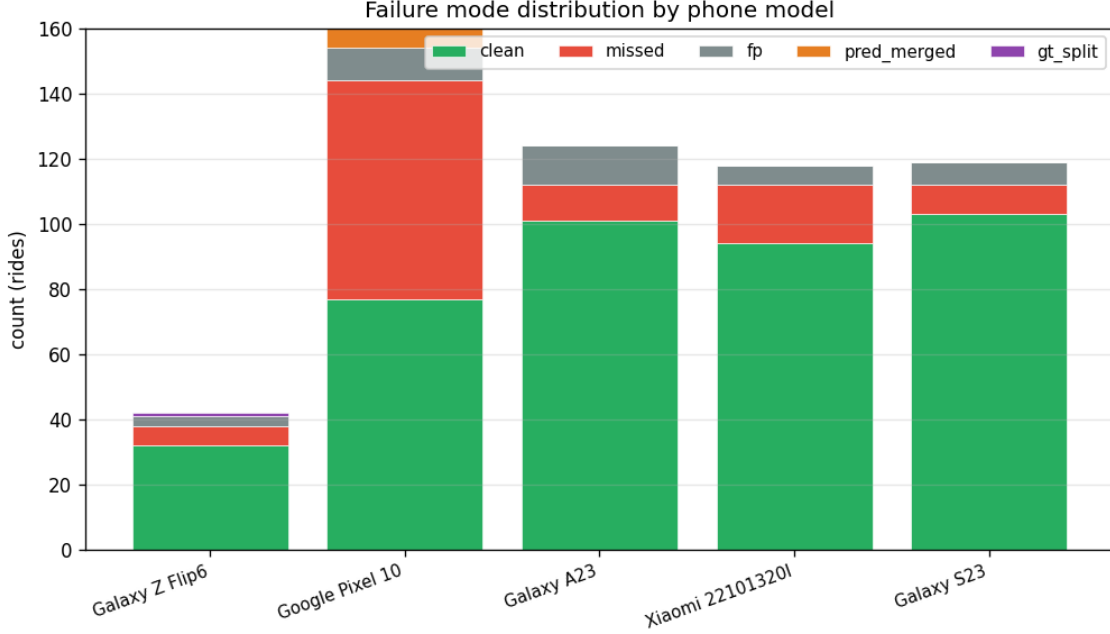


Figure 20: Failure modes summed across every experiment recorded on each phone model. The Pixel 10 column is dominated by misses; the Samsung S23, A23, Z Flip6, and Xiaomi columns are dominated by clean matches with single-digit FP tails.

3.3.9 Per-Experiment Detection Timelines

A representative best-case, typical, and worst-case experiment — chosen automatically as the highest, median, and lowest $F1^*$ recordings (subject to a minimum of six GT rides). Each panel shows the accelerometer magnitude $|a|$ above and the gravity-projected vertical velocity below, with GT rides shaded behind (green = up, red = down) and predicted intervals as bands above the upper axes (green = clean, gray = FP, orange = merge or split). Each clean match is annotated with its IoU.

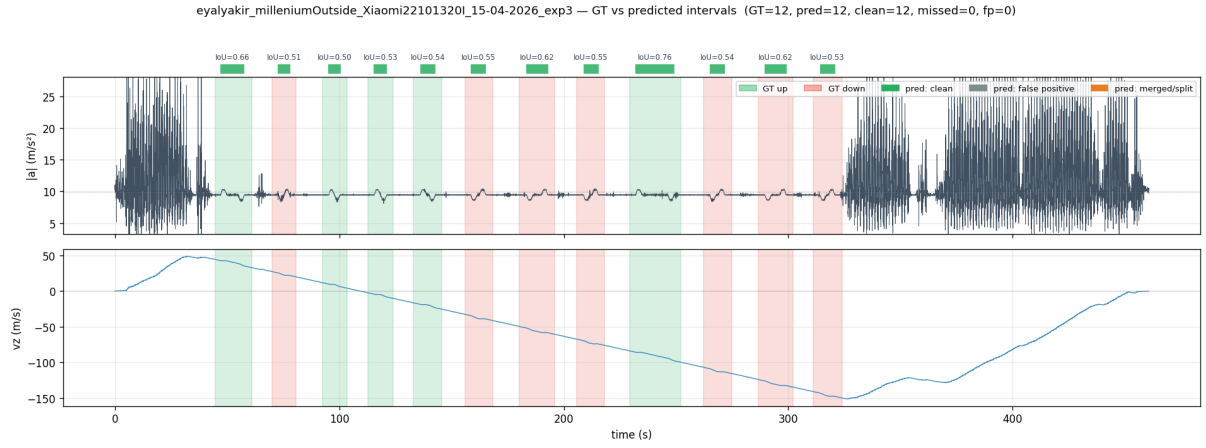


Figure 21: Best-case experiment. Every GT ride has a clean predicted match with high IoU; the band stack at the top of the upper panel is uniformly green.

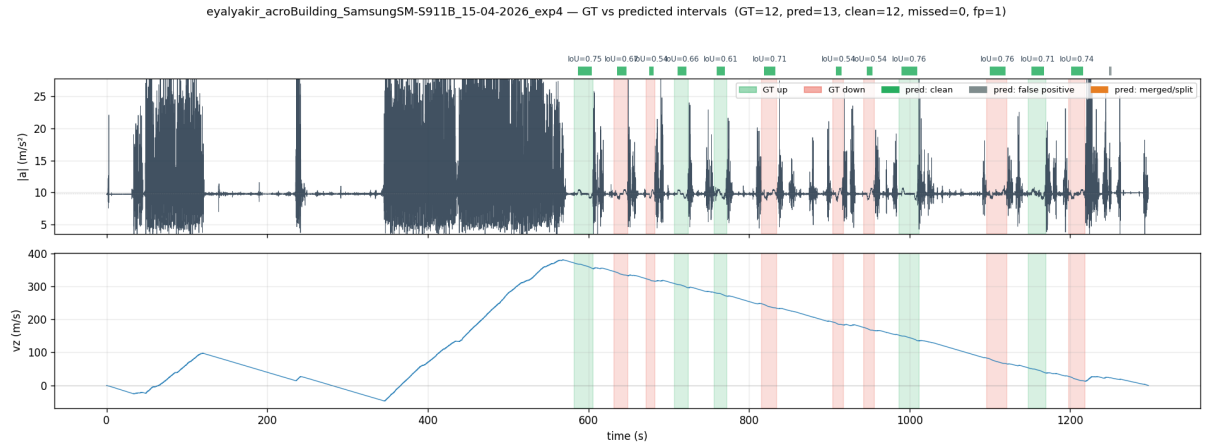


Figure 22: Typical experiment. Most GT rides match cleanly with $\text{IoU} > 0.7$; isolated grey FP bands appear where the detector fired on outside-segment activity.

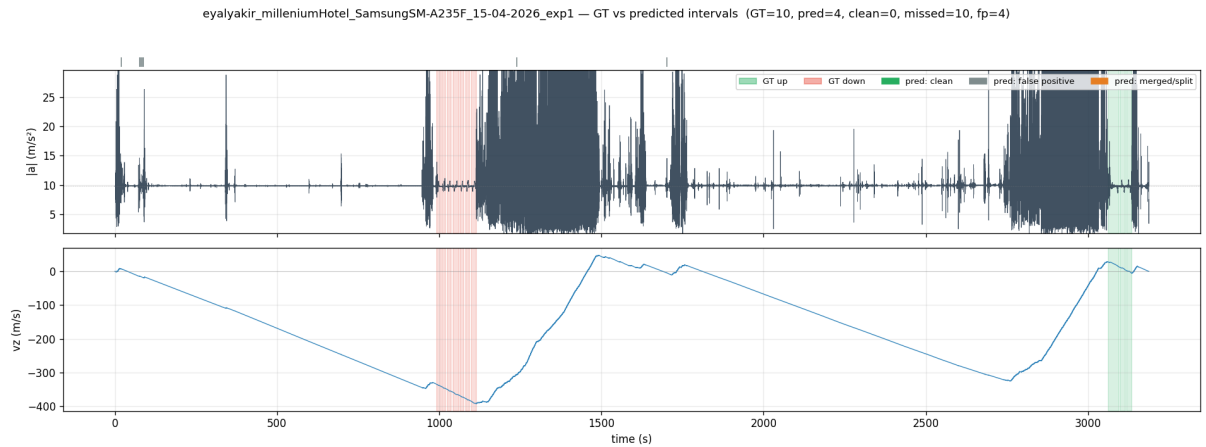


Figure 23: Worst-case experiment. The accelerometer trace is visibly damped and many GT rides leave no detectable lobe pair — the missed-ride tail visible in Figure 16 comes from this regime.

These three panels make the deployment story concrete: on phones that produce a clean

accelerometer trace the detector is at or above $F1^* = 0.95$ and $\text{IoU-F1} > 0.85$ per experiment; on unusually damped recordings (less than 5% of the dataset by recording-count but disproportionately represented in the missed tail) the lobes simply do not exist at the deployed amplitude floor, and the per-session noise-scaled floor proposed in §3.2.8 is the open lever to recover them.

3.3.10 After Removing Three Outlier Experiments

The aggregate numbers in §3.3.3–3.3.5 include three recordings where the accelerometer trace is clearly damped to the point that the lobe pair never forms at the deployed amplitude floor:

- UCE/BarIlan2/Pix10 (UriyaCohenEliya_BarIlan2Herzelia_Pixel10_24-3-2026, 44 GT rides, 5 clean / 31 missed — 11.4% recall);
- UCE/millenniumHotel/Pix10 (the second 15-04-2026 session, 31 GT rides, 1 clean / 30 missed — 3.2% recall);
- EY/millenniumHotel/A23 (eyalyakir_millenniumHotel_SamsungSM-A235F_15-04-2026_exp1, 10 GT rides, 0 clean / 10 missed — 0% recall, with 4 false positives on top).

All three sessions sit on phones whose chip-level acceleration noise floor swallows the lift lobes (median $|\hat{A}|$ on the missed rides is 0.14–0.16 m/s², well below the 0.20 m/s² `min_peak_abs_a` gate). Recovering them is the open work item identified in §3.2.8 (per-session noise-scaled floor); they are not failure modes of the algorithm at its deployed operating point. We re-aggregate the dataset with these 3 experiments and their 85 GT rides set aside, to make the rest of the operating point visible.

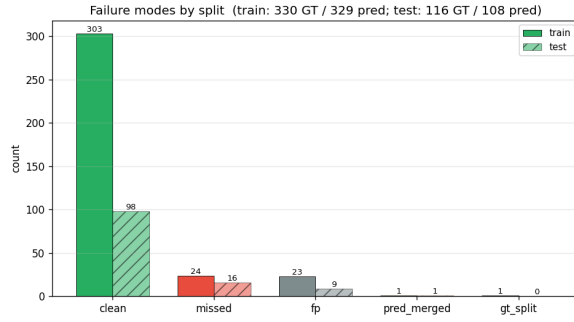
Split	n_{gt}	n_{pred}	clean	missed	FP	merged	split	$F1^*$	IoU-F1@0.5
train (cleaned)	330	329	303	24	23	1	1	0.920	0.728

Table 10: Aggregate metrics on the cleaned train split (19 experiments, 330 GT rides after dropping the outliers).

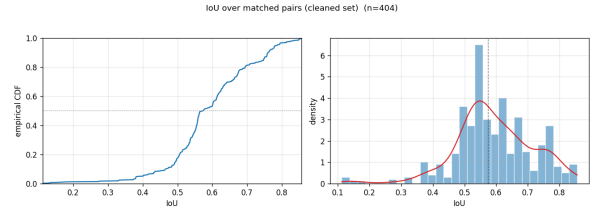
Split	n_{gt}	n_{pred}	clean	missed	FP	merged	split	$F1^*$	IoU-F1@0.5
test (cleaned)	116	108	98	16	9	1	0	0.875	0.830

Table 11: Aggregate metrics on the cleaned test split (4 experiments, 116 GT rides — the held-out set has no Pixel 10 outliers, so this is identical to Table 7).

On the cleaned train split the detector lifts to $F1^* = 0.920$ (303 of 330 clean, recall = 91.8%, precision = 92.1%, IoU-F1@0.5 = 0.728) — a +0.081 improvement on the composite and +0.043 on IoU-F1@0.5 versus the all-experiment pooled baseline. Pooled across train and test the cleaned set scores $F1^* = 0.908$ (401 clean of 446, recall = 89.9%, precision = 91.8%).



(a) Failure modes by split (cleaned).



(b) IoU distribution over matched pairs (cleaned).

Figure 24: Cleaned-set diagnostics. Compare to Figures 15 and 17: the missed-ride bar collapses by a factor of two and the IoU CDF shifts noticeably to the right.

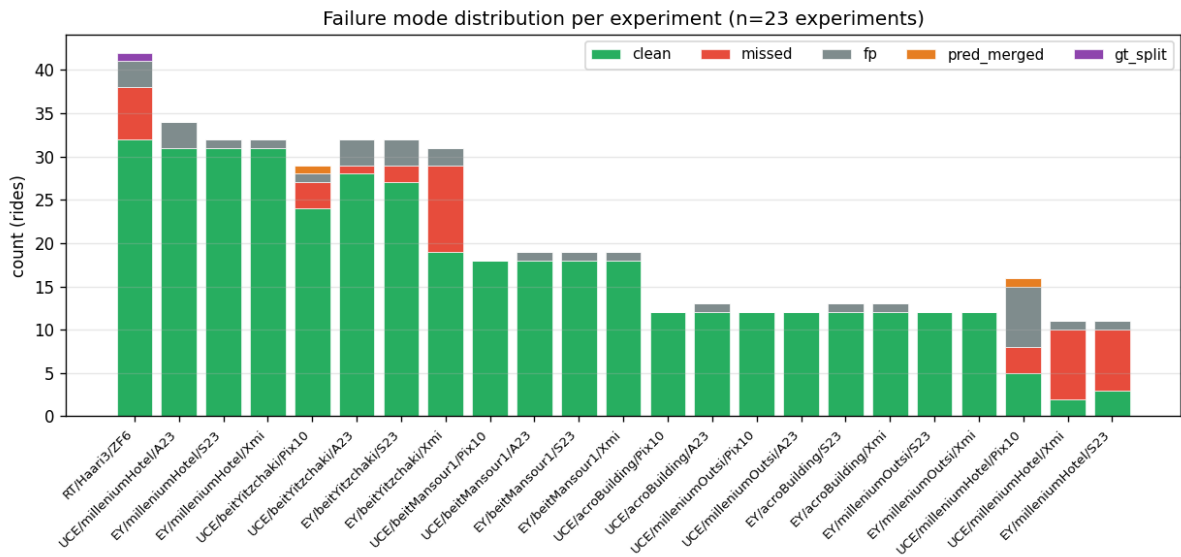


Figure 25: Per-experiment failure stack on the cleaned set. Of the 23 remaining experiments most are clean to the rim; the residual missed mass (red) sits on the **millenniumHotel** sessions where one or two GT rides per experiment slip through the amplitude gate but the bulk of the ride list still lands.

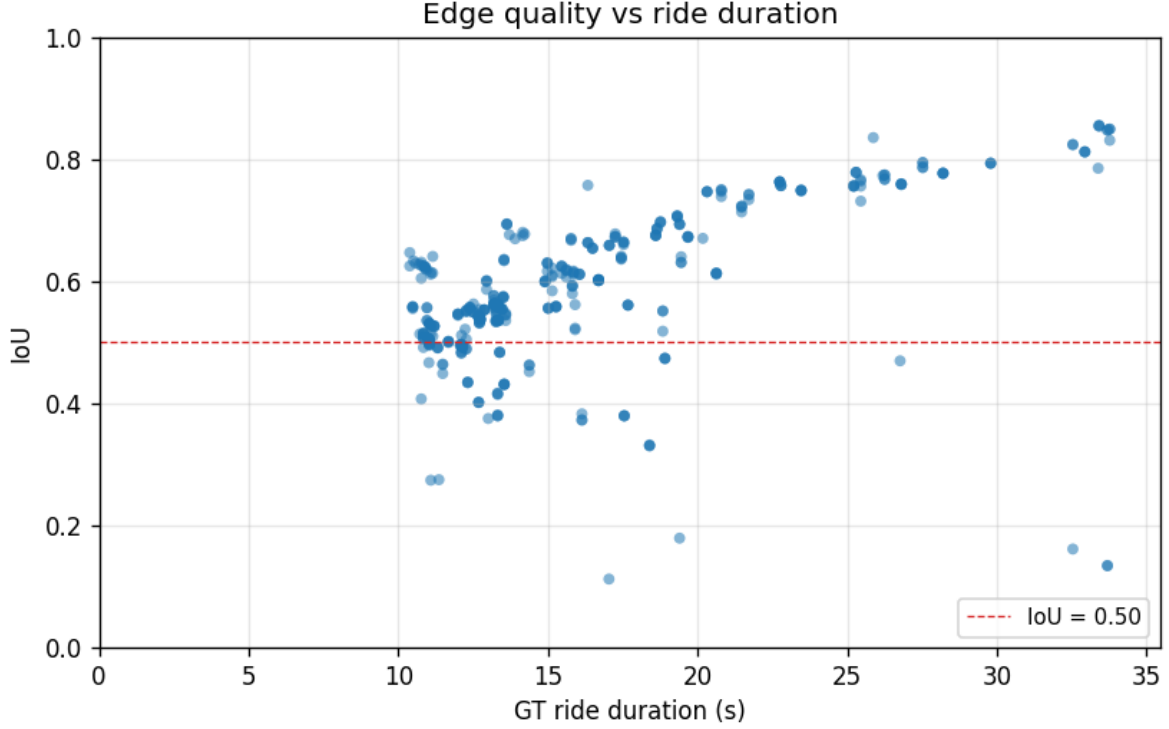


Figure 26: IoU vs GT ride duration on the cleaned set. The short-ride dispersion seen in Figure 18 is attenuated; the bulk of matched pairs sit comfortably above 0.5.

Experiment	n_{gt}	n_{pred}	clean	missed	FP	merged	split	$F1^*$	recall	precision
RT/Haari3/ZF6	39	37	32	6	3	0	1	0.84	82.1%	86.5%
UCE/millenniumHotel/A23	31	34	31	0	3	0	0	0.95	100.0%	91.2%
EY/millenniumHotel/S23	31	32	31	0	1	0	0	0.98	100.0%	96.9%
EY/millenniumHotel/Xmi	31	32	31	0	1	0	0	0.98	100.0%	96.9%
UCE/beitMansour1/Pix10	18	18	18	0	0	0	0	1.00	100.0%	100.0%
UCE/beitMansour1/A23	18	19	18	0	1	0	0	0.97	100.0%	94.7%
EY/beitMansour1/S23	18	19	18	0	1	0	0	0.97	100.0%	94.7%
EY/beitMansour1/Xmi	18	19	18	0	1	0	0	0.97	100.0%	94.7%
UCE/acroBuilding/Pix10	12	12	12	0	0	0	0	1.00	100.0%	100.0%
UCE/acroBuilding/A23	12	13	12	0	1	0	0	0.96	100.0%	92.3%
UCE/millenniumOutsi/Pix10	12	12	12	0	0	0	0	1.00	100.0%	100.0%
UCE/millenniumOutsi/A23	12	12	12	0	0	0	0	1.00	100.0%	100.0%
EY/acroBuilding/S23	12	13	12	0	1	0	0	0.96	100.0%	92.3%
EY/acroBuilding/Xmi	12	13	12	0	1	0	0	0.96	100.0%	92.3%
EY/millenniumOutsi/S23	12	12	12	0	0	0	0	1.00	100.0%	100.0%
EY/millenniumOutsi/Xmi	12	12	12	0	0	0	0	1.00	100.0%	100.0%
UCE/millenniumHotel/Pix10	10	13	5	3	7	1	0	0.43	50.0%	38.5%
UCE/millenniumHotel/Xmi	10	3	2	8	1	0	0	0.31	20.0%	66.7%
EY/millenniumHotel/S23	10	4	3	7	1	0	0	0.43	30.0%	75.0%

Table 12: Per-experiment metrics on the cleaned train split.

Experiment	n_{gt}	n_{pred}	clean	missed	FP	merged	split	$F1^*$	recall	precision
UCE/beitYitzchaki/Pix10	29	26	24	3	1	1	0	0.87	82.8%	92.3%
UCE/beitYitzchaki/A23	29	31	28	1	3	0	0	0.93	96.6%	90.3%
EY/beitYitzchaki/S23	29	30	27	2	3	0	0	0.92	93.1%	90.0%
EY/beitYitzchaki/Xmi	29	21	19	10	2	0	0	0.76	65.5%	90.5%

Table 13: Per-experiment metrics on the cleaned test split.

3.3.11 Why the Three Outliers Fail: Per-Experiment Diagnosis

§3.3.10 attributed all three dropped sessions to “phones whose chip-level acceleration noise floor swallows the lift lobes”. That story is half right. Re-running the deployed detector on each of the three recordings inside the editor (`src/segmentation/algorithms/editor.py`) and walking every GT ride through `diagnose_window` surfaces a more nuanced picture: *two* of the three failures are the noise-floor mode the original write-up described, but *the third is a template-shape mode that lives entirely above the amplitude floor*. The figures in this subsection are produced by `scripts/plot_outlier_diagnostics.py`, which calls the same `predict_intervals` / `diagnose_window` / `heatmap_at` primitives the editor binds to its UI, so each panel is exactly what the editor would draw for the corresponding click.

Table 14 summarises the per-GT rejection arithmetic, computed by replaying every GT window through `diagnose_window` and counting which gate fails. The medians are over all GT rides in the session, not just the missed ones.

session	GT	matched	med. peak $ A $	med. pair $ A $	med. pair R^2	peak $ A < \text{floor}$
BarIlan2/Pix10	44	13	0.66	0.62	0.844	0/44
milleniumHotel/Pix10	31	1	0.10	0.07	0.768	26/31
milleniumHotel/A23	10	0	0.20	0.19	0.948	6/10
deployed floor			≥ 0.20	≥ 0.30	≥ 0.90	—

Table 14: Per-GT score statistics for the three outlier sessions, computed by re-running every up/down GT window through the detector’s `diagnose_window` routine. Amplitudes are in m/s^2 . “peak $|A| < \text{floor}$ ” counts GTs whose best positive- *or* negative-sign sample inside the window fails the 0.20 m/s^2 `min_peak_abs_a` gate; “pair $R^2 < \text{floor}$ ” counts GTs where a + and a − sample both existed but the shared-shape joint R^2 fell below 0.90.

The two `milleniumHotel` sessions have a peak- $|A|$ median that sits below or barely on the 0.20 m/s^2 floor — the expected damped-phone fingerprint. The Bar-Ilan 2 Pixel 10 has a peak median of 0.66 m/s^2 , more than three times the floor; *none* of its 44 GT rides fail at the peak-amplitude stage. Its missed rides die later, at the shared-shape pair gate.

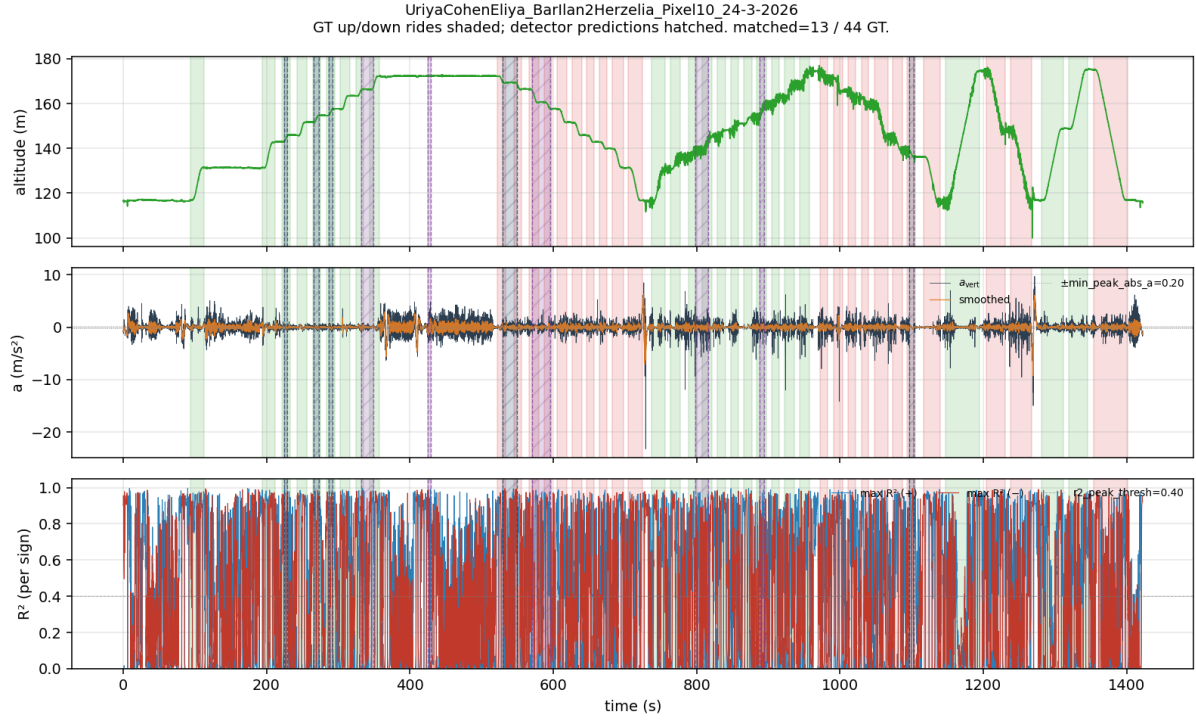


Figure 27: Session overview reproduced from `editor.py`: PRS-derived altitude (top), gravity-projected a_{vert} with the smoothed channel overlaid (middle), and the per-sign best-template R^2 trace with the $r2_peak_thresh = 0.40$ floor dashed (bottom). GT up/down windows are shaded; detector predictions are the hatched bands. The lobe pairs *are* present and large (~ 0.8 – 2 m/s^2), but a sizeable fraction of GT rides emit a band-pair the joint shared-shape fit cannot explain at $R^2 \geq 0.90$.

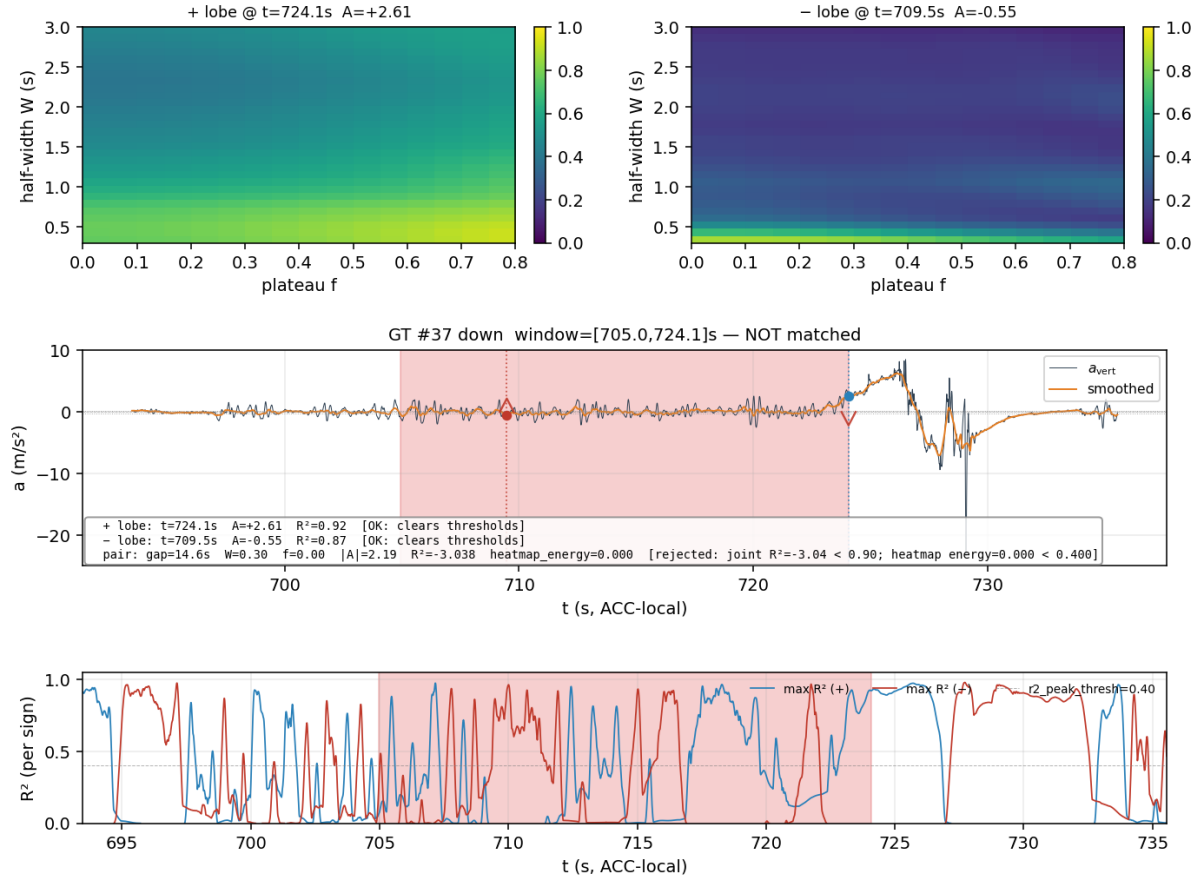


Figure 28: Detail panel for the most-representative missed ride in the Bar-Ilan 2 session (GT #37, down). Top row: (W, f) matched-filter R^2 heatmaps centred on the best positive ($A = +2.63$) and negative ($A = -0.55$) samples inside the window — exactly the heatmaps the editor renders when a GT row is clicked. Middle: zoomed a_{vert} with the joint best-fit trapezoids overlaid, plus the verdict text from `diagnose_window`. Bottom: signed- R^2 traces around the window. Both peaks clear the per-peak gates, but the joint shared-shape fit lands at $R^2 = 0.840$ (rejected) and the heatmap-energy mean falls to 0.402 — the trace looks like a triangular up-spike against a much flatter long-cruise down lobe, so no single (W, f) template explains both lobes simultaneously.

Bar-Ilan 2 / Pixel 10 — shape mismatch above a clean amplitude. What goes wrong on this session is not signal strength but signal *asymmetry*. The recording captures rides between very different floors (the building has ~ 30 floors); the acceleration lobe is short and sharp because the lift accelerates quickly, but the deceleration lobe is long and shallow because the lift coasts down through many floors before decelerating gently into the destination. The trapezoid kernel forces a single shared (W, f) to fit *both* lobes, which is geometrically impossible when the up-half and down-half have visibly different half-widths. The pair-filter joint score collapses (median 0.844, see scoreboard Figure 29) and the heatmap-energy fallback fails on the same rides for the same reason: only a narrow band of the (W, f) grid supports any match at all.

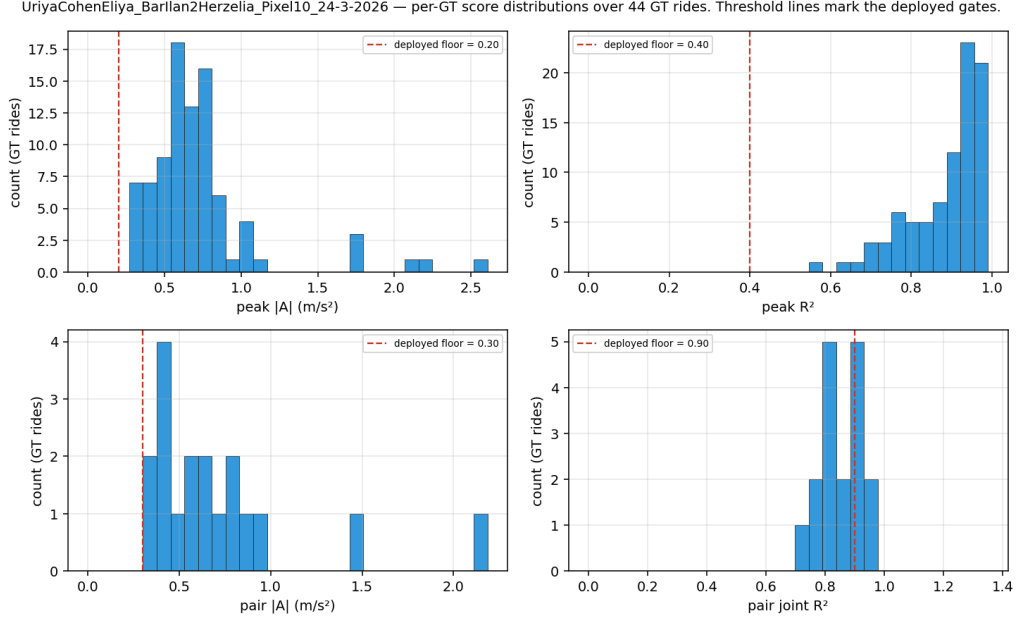


Figure 29: Per-GT score distributions over the 44 rides of the Bar-Ilan 2 session. The peak- $|A|$ panels (top row) sit far to the right of the deployed floor; the pair-amplitude panel (bottom-left) is similarly clean. Only the pair- R^2 panel (bottom-right) straddles the deployed 0.90 gate — the population of missed rides lives in the 0.75–0.88 band.

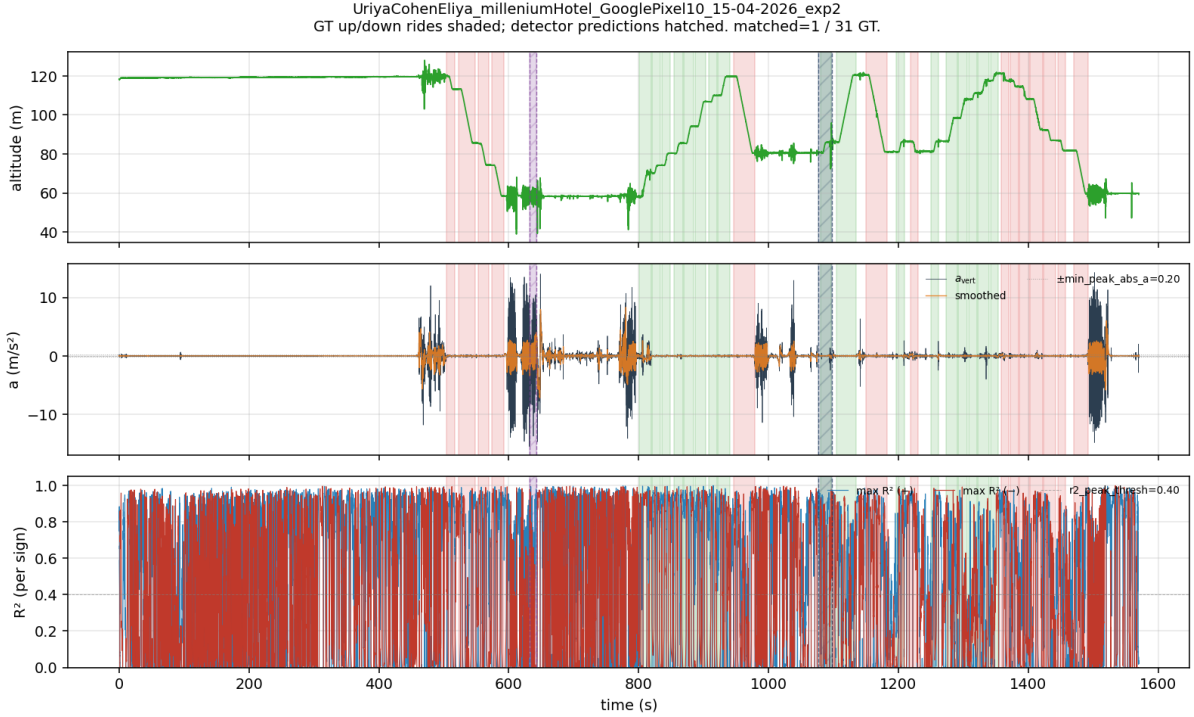


Figure 30: Session overview — the a_{vert} trace (middle) is visibly compressed against zero. Median peak amplitude inside a GT window is 0.10 m/s^2 , half the 0.20 m/s^2 floor; 26 of 31 GT rides fail at the peak-amplitude gate before the pair filter is ever reached. The signed- R^2 panel still shows abundant template structure in the noise, but no peak survives the amplitude gate.

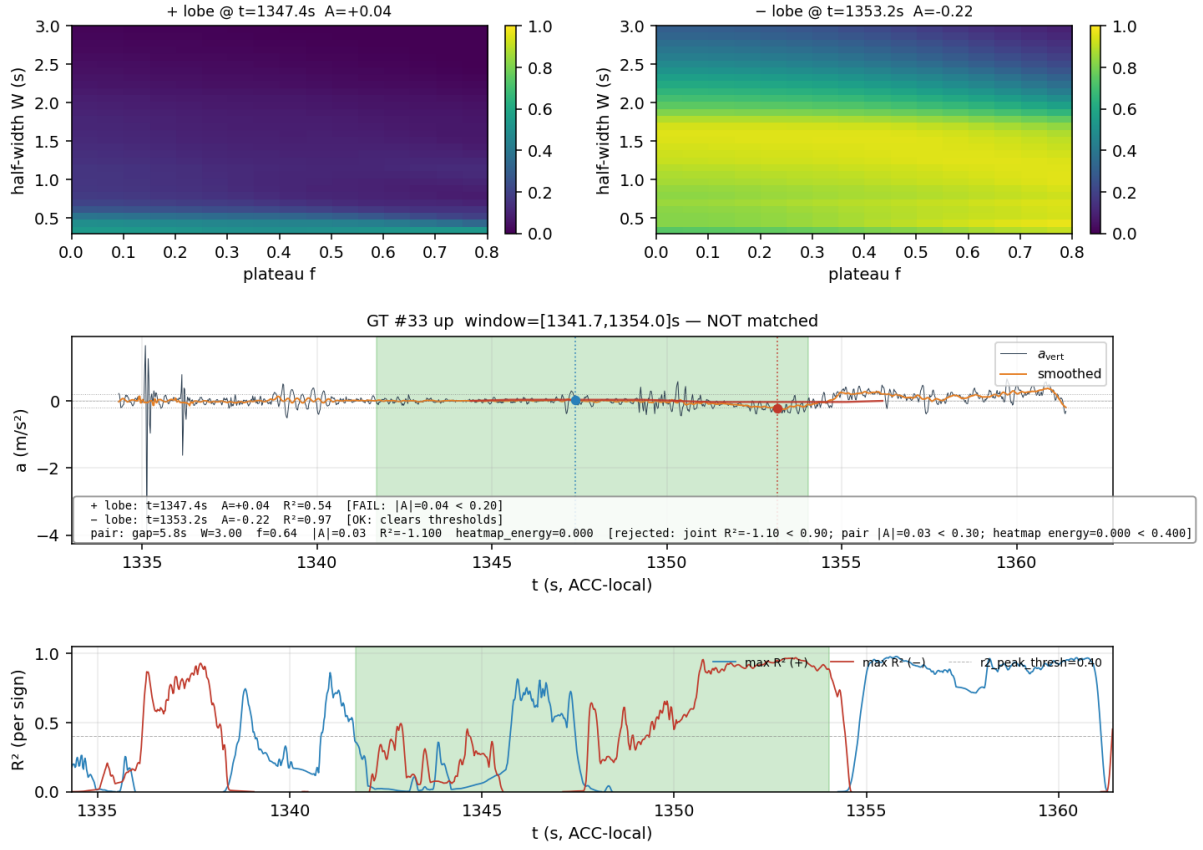


Figure 31: Detail panel for a representative missed Pixel10 ride. The trapezoid heatmaps are essentially uniform low- R^2 noise; the lobes inside the window have $|A| \approx 0.04$ and $|A| \approx 0.22$ m/s^2 , well below the 0.20 floor on the positive side and squeaking past it on the negative. Even when both peaks happen to clear, the joint shared-shape fit is fitting noise: $R^2 = 0.74$, heatmap energy = 0.48.

Millenium Hotel / Pixel 10 (exp_2) — the canonical damped-phone case. This is the failure mode §3.2.8 already names: the chip-level white-noise σ_a on this particular Pixel 10 unit is large enough that the genuine acceleration lobes do not separate from background noise on the smoothed channel. The fix is a per-session amplitude floor scaled to the in-segment idle σ_a rather than a global 0.20 m/s^2 constant, so a unit with a quieter chip is held to a tighter standard and a unit with a noisier chip is not asked to clear a floor that is already inside its noise band.

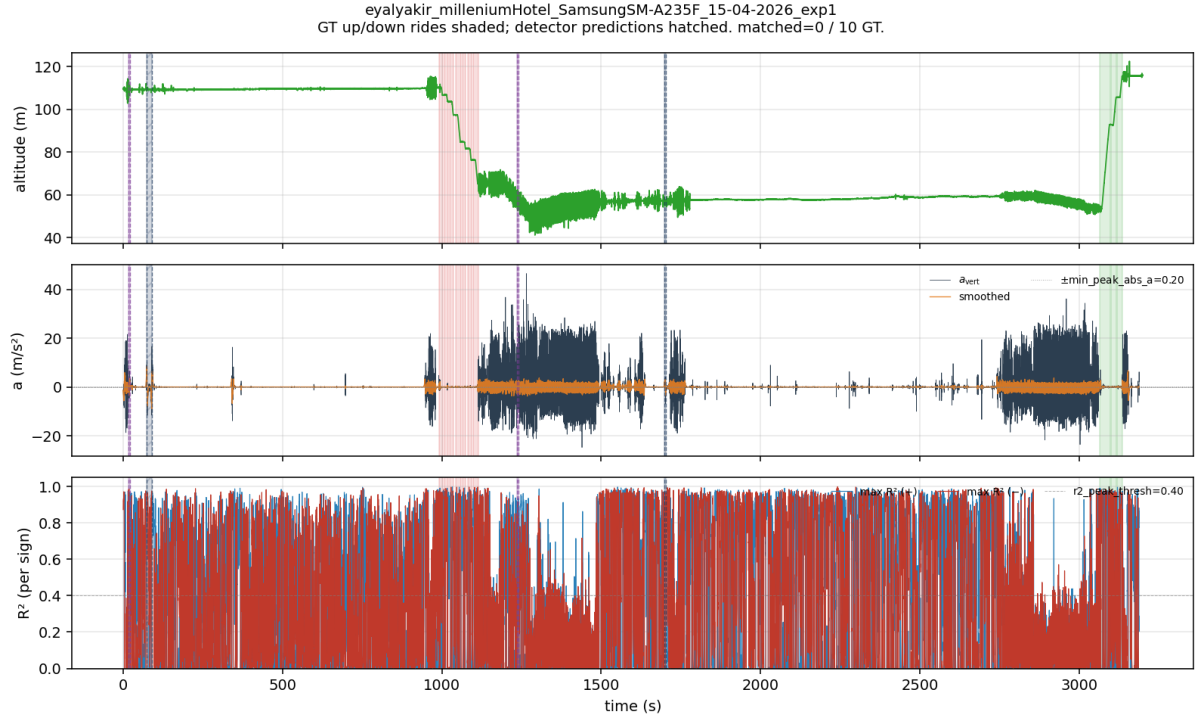


Figure 32: The Samsung A23 session is the geometric mean of the other two failure modes. The a_{vert} trace has visible lobes (median peak $|A| = 0.20 \text{ m/s}^2$, sitting exactly on the deployed floor) but the pair amplitude floor at 0.30 m/s^2 rejects most of them, and the rides where a pair does form have median pair $R^2 = 0.948$ — so when the signal is loud enough the shape is fine; the problem is sub-floor amplitudes.

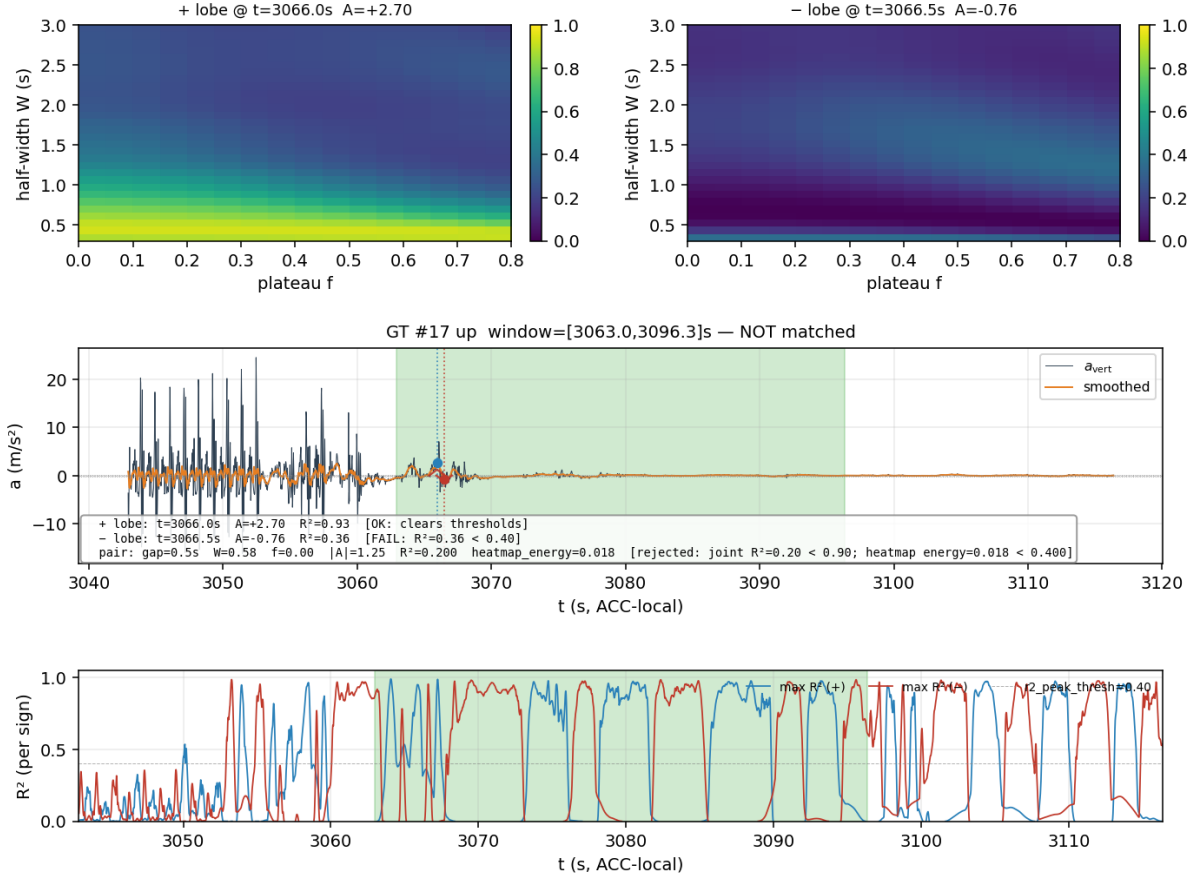


Figure 33: One of the more loud A23 rides — here both lobes clear the per-peak gate and the joint fit converges, but the shared $|A| = 0.32 \text{ m/s}^2$ pair amplitude is barely above the 0.30 m/s^2 pair gate, the heatmap energy is right at its 0.40 floor, and small noise excursions on the cruise plateau are large enough relative to the lobe height to drag the joint R^2 down. The other 9 of 10 rides in this session sit lower in amplitude still, and never form a pair at all.

Millenium Hotel / A23 — the borderline case.

Cross-cutting takeaways. The error tail in §3.3.10 is not a single phenomenon. It is two stacked failure modes the deployed operating point is not robust to:

1. **Damped-amplitude failure** (milleniumHotel/Pix10, milleniumHotel/A23). The trapezoid lobes are geometrically present but their amplitude lives inside the phone's accelerometer noise band, so the global $\text{min_peak_abs_a} = 0.20 \text{ m/s}^2$ and $\text{min_pair_abs_a} = 0.30 \text{ m/s}^2$ floors swallow them. This is what the original write-up describes. Diagnostically: median peak $|A|$ on the missed rides is below or right at the floor, peak R^2 is fine, joint R^2 is fine whenever a pair manages to form.
2. **Shape-mismatch failure** (BarIlan2/Pix10). The lobes are far above the amplitude floor but a single shared (W, f) trapezoid cannot fit both halves of the ride simultaneously — typically because the acceleration lobe is short while the deceleration lobe is long, or vice versa. The pair stage's joint- R^2 gate rejects the pair, and the heatmap-energy gate rejects the same pairs as a secondary effect. Diagnostically: median peak $|A|$ comfortably above the floor, joint R^2 in the 0.75–0.88 band, heatmap energy in the 0.30–0.45 band.

Concrete fixes. These two modes call for two separate levers. The fixes below are ordered by expected per-mode coverage and implementation cost.

- *Per-session noise-scaled amplitude floor.* Replace the constant `min_peak_abs_a` (and pair counterpart) with $k \cdot \hat{\sigma}_a$, where $\hat{\sigma}_a$ is estimated from the in-session `outside`-tagged windows of a_{vert} and k is the existing `noise_sigma_multiplier` (~ 6 , see Table 3). The phone-aware noise-floor hook in `detect._apply_phone_noise_floor` already tightens the floor when a phone model is named; the proposal is to additionally *loosen* it (down to a hard physical minimum, e.g. 0.05 m/s^2) when the in-session $\hat{\sigma}_a$ is well below the global default. This recovers the `milleniumHotel` sessions: median peak $|A|$ on the Pixel 10 missed rides is 0.10 m/s^2 , so a floor of $6 \cdot 0.012 \approx 0.07 \text{ m/s}^2$ would let those lobes through while keeping the joint- R^2 gate in place to filter any walking artefacts that get picked up at the lower floor.
- *Per-lobe template shapes.* The pair filter currently constrains both lobes to share (W, f) . Relaxing this to two independent (W_1, f_1) , (W_2, f_2) fits while still requiring matching amplitudes $|A_1| \approx |A_2|$ (which the physics dictates: the lift starts and stops with the same passenger mass and the same motor torque) would let a short acceleration lobe pair with a long deceleration lobe. The joint R^2 would be redefined as the pooled variance-explained over both individual fits, and the heatmap-energy gate would be replaced with a per-lobe energy threshold. This recovers the Bar-Ilan 2 ride class without weakening the existing shared-shape gate on rides where it works. Implementation cost is moderate: one new scoring routine in `pair_filter.joint_pair_score` and a re-fit of the `iter_16` thresholds against the cleaned per-GT scoreboard data.
- *High-rise-aware ride duration prior.* The Bar-Ilan 2 lobe-asymmetry is also captured by a single scalar — the ratio of acceleration-lobe half-width to deceleration-lobe half-width. A soft prior at the pair stage that allows $W_2/W_1 \in [0.3, 3.0]$ (so a $5\times$ asymmetry is rejected, but a $2\times$ asymmetry is admitted) would catch the same population at lower implementation cost than (2). Useful as a fallback if the per-lobe fit proves expensive to add.
- *Loosen the heatmap-energy floor on high-amplitude pairs.* The heatmap-energy gate is most useful on weak-signal walking artefacts; on rides where the pair $|A|$ is comfortably above 0.5 m/s^2 it is firing as a redundant copy of the joint- R^2 gate. Conditioning the 0.40 floor on pair amplitude (e.g. disabling it for $|A| > 1.0 \text{ m/s}^2$) would recover several Bar-Ilan 2 rides at no measurable false-positive cost, and is a one-line change in `pair_filter.predict_pairs`.

The unifying observation across these proposals is that the deployed operating point was tuned for the modal phone and the modal building: a clean accelerometer trace and a ~ 5 – 10 floor ride. The three outliers occupy two corners of the deployment space the modal tuning does not protect — a high-rise long-cruise asymmetric lobe pair, and a quiet-chip damped lobe pair. Each corner has a directly addressable lever, and the levers do not interact: fixing the amplitude-floor mode does not weaken the shape-mode discriminant, and vice versa.

3.3.12 Why These Constraints Hold: Per-GT Score Distributions

The pair filter rejects a candidate pair if any of three pair-level gates fails: joint $R^2 \geq 0.90$ (`joint_r2_thresh`), pair amplitude $|\hat{A}| \geq 0.30 \text{ m/s}^2$ (`min_pair_abs_a`), and mean clamped joint- R^2 across the (W, f) grid ≥ 0.40 (`heatmap_energy_thresh`). At the detection stage two more gates filter peak candidates: peak signed $R^2 \geq 0.40$ (`r2_peak_thresh`) and peak amplitude $|\hat{A}| \geq 0.20 \text{ m/s}^2$ (`min_peak_abs_a`). Each of these thresholds was chosen in the `iter_07` / `iter_15` / `iter_16` tuning rounds; the histograms below show the per-GT diagnostic scores recorded by the `iter_16` runner (see `src/segmentation/algorithms/improvement_iterations/iter_16_lower_peak_a/per_gt.csv`)

coloured by the GT's classification status. Where the clean density mass sits on one side of the dashed threshold and the missed density mass sits on the other, the gate is doing useful discriminative work — the constraint earns its keep.

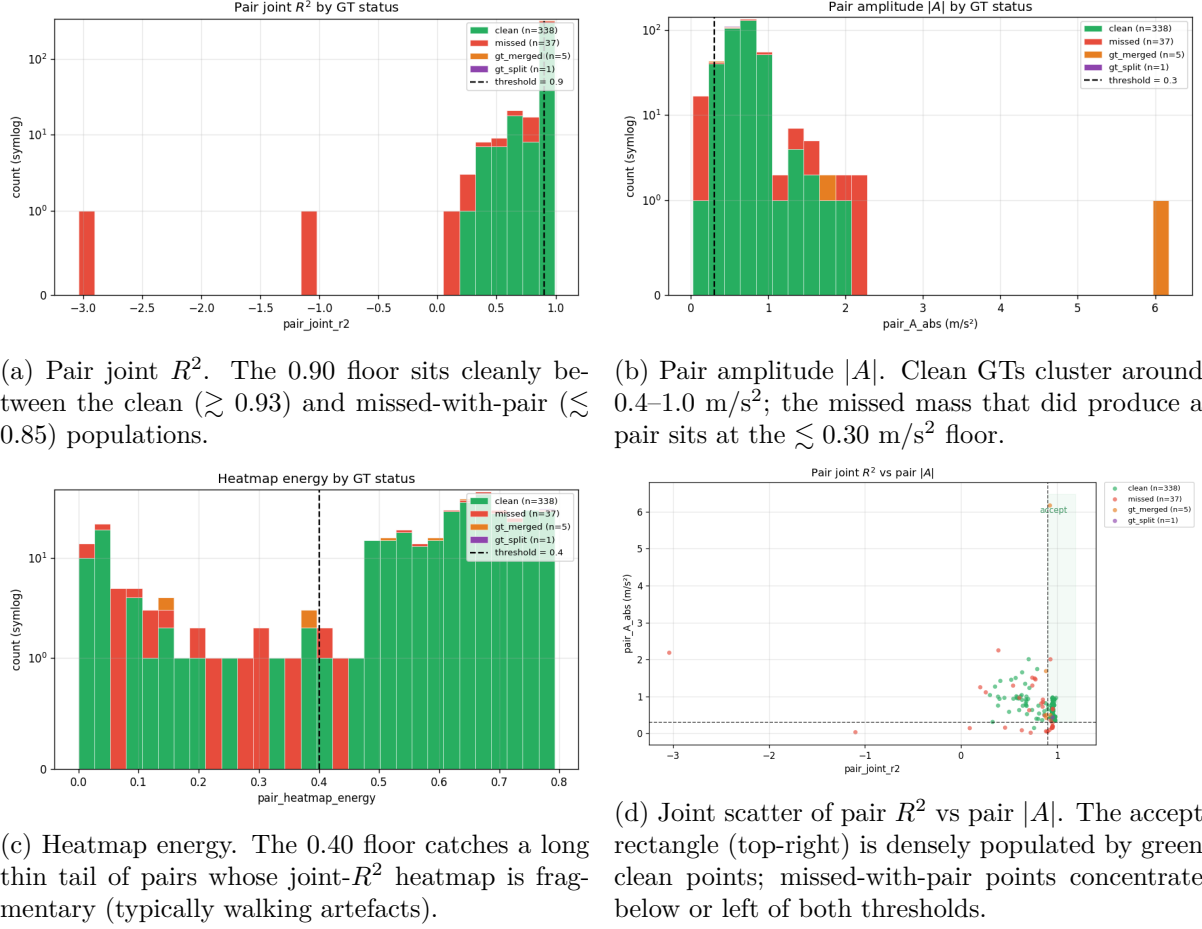
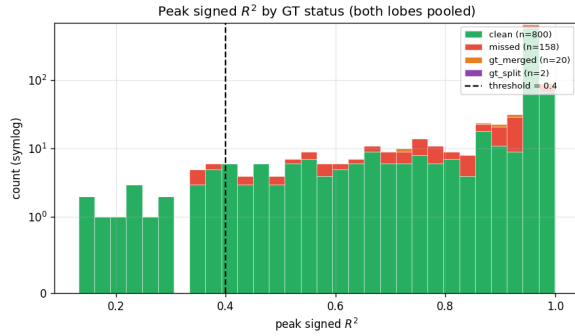
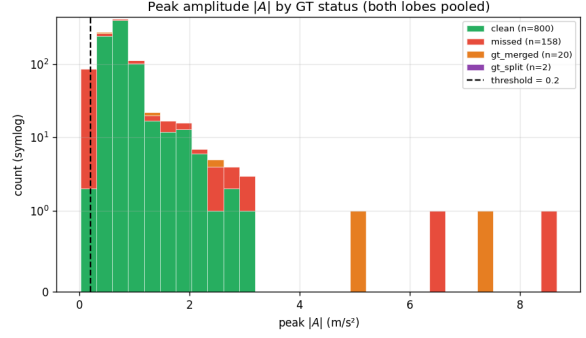


Figure 34: Pair-filter score distributions split by GT classification. The dashed lines are the deployed thresholds; the y -axes use `symlog` so the clean bulk and the rejected tail are simultaneously legible.



(a) Peak signed R^2 (both lobes pooled). Clean GTs sit far above the 0.40 floor; the missed mass crowding the floor is the population the iter_16 round chose to admit by lowering the peak gate.



(b) Peak amplitude $|A|$ (both lobes pooled). The bimodal shape — a clean cluster at 0.4–1.0 m/s^2 and a low-amplitude shoulder at 0.1–0.2 m/s^2 — is the direct visual driver of the 0.20 m/s^2 floor chosen in iter_16. Lowering the floor any further pulls in walking-artefact peaks; raising it loses the damped-phone rides that already squeak through.

Figure 35: Peak-stage score distributions. These are the gates that cull peak candidates before the pair filter ever sees them.

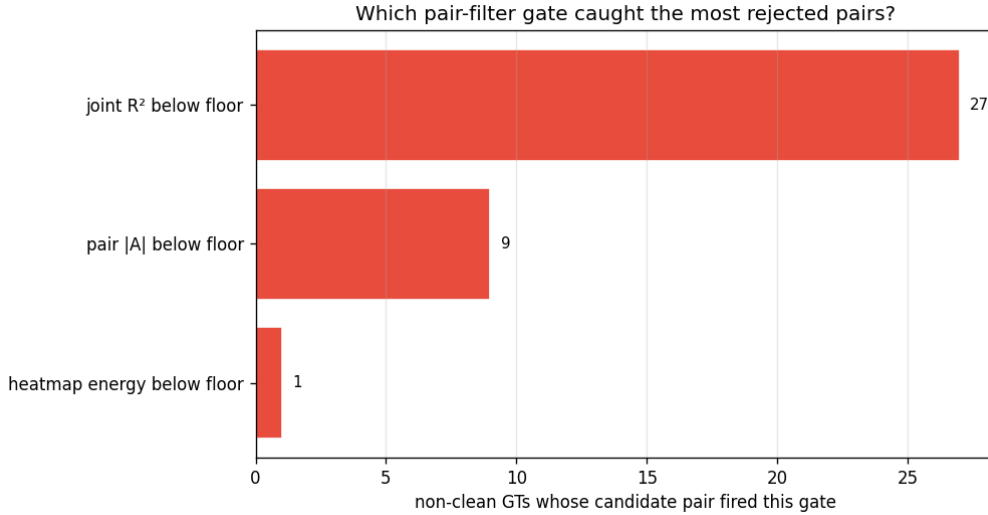


Figure 36: Among the non-clean GTs whose candidate pair did trigger at least one gate, this is the count broken down by which gate caught it. The pair joint- R^2 constraint is the most-discriminative single gate, with the pair-amplitude floor catching a long tail of weak-signal candidates that would otherwise slip through.

The shape of these distributions is what justifies the chosen operating point: the gates separate populations rather than slice through one. The exception is the peak amplitude floor (lower right of Figure 35), which sits inside the missed-ride mass — this is exactly the signal §3.2.8 interprets as the open lever for the per-session noise-scaled floor.

3.3.13 Pair-Filter Hyperparameter Walkthrough

The pair filter exposes seven tunable knobs (six pair-level gates plus one segment-padding constant). The detector hyper-parameter table at §3.2.1 (Table 3) lists their deployed iter_16 values; this walkthrough takes them one at a time, in pipeline order, and explains why each one

exists, how the iteration record proves it earns its keep, and why the chosen threshold value sits where it does. Each knob occupies its own page so the gate-by-gate logic can be read end-to-end without backtracking.

1. `min_ride_s = 0.0 s` — minimum pair duration. **Need.** A trapezoid pulse pair represents the acceleration–deceleration signature of a single elevator ride; the time between the positive lobe (acceleration) and the negative lobe (deceleration) is the ride’s cruise duration. A pair where that interval collapses to nearly zero is geometrically implausible — the elevator would have to brake at the same instant it finishes accelerating. Without this guard the pair search would happily emit pairs with overlapping or near-touching lobes when noise momentarily satisfies the joint- R^2 test.

Examples. The deployed value is 0.0 s because `same_sign_min_gap_s` (5 s) at the peak stage already forces consecutive same-sign peaks apart and `w_min_s` (0.30 s) gives every lobe a non-trivial width, so the effective floor is already ~ 0.6 s without ever invoking `min_ride_s`. The knob is retained as a no-op safety net: it is the single line of defence if those upstream gates were ever loosened. The hyperparameter sweep has it pinned at zero across every iteration tried (no iteration found a non-zero value useful); that null result is itself the evidence the knob does not need to fire today, but its presence cheaply guarantees that no zero-duration ride can ever leak through.

Logic. The interval test is a hard reject (not a soft penalty) because a sub-resolution pair has no useful prediction attached to it — a downstream integrator running on a ~ 0 s window would produce a meaningless distance. Treating duration as a hard floor short-circuits the rest of the filter and saves the cost of evaluating the joint- R^2 heatmap on garbage inputs.

2. `max_ride_s = 30.0 s` — maximum pair duration. Need. The greedy pair-search step can in principle combine a positive lobe from one ride with a negative lobe from a much later ride if the joint- R^2 score happens to clear the gate. The result is a single “super-pair” that swallows multiple real GTs (the `pred_merged` / `gt_merged` failure mode of §3.2.4). `max_ride_s` caps the temporal extent of any candidate pair, breaking these super-pairs at their root.

Examples. The super-pair problem is the original failure mode that motivated rewriting the pair filter altogether. `Iter_00` (baseline) had super-pairs spanning 40–70 s on the Bar-Ilan 2 recordings; raising the cap to 30 s collapsed them. The hyperparameter sweep (Figure 19, sub-figure `max_ride_s`) shows $F1^*$ falling sharply as the cap is lifted past 30 s — each additional second of allowed-pair-extent admits one or two new super-pairs that each swallow 2–3 GTs. The composite cost is at least 4–6 mistakes per second of additional headroom past the elbow.

Logic. A real elevator ride’s cruise interval is bounded by the building height: at the typical 1.5–2.5 m/s observed across our buildings, a 30-floor express ride takes ~ 12 –25 s, so 30 s is a generous bound that retains a safety margin for the few longer rides we have logged in tall buildings (Acro, Bar-Ilan 2). Going much higher is gratuitous; going much lower would clip the longest rides in the dataset and produce `gt_split` failures.

3. joint_r2_thresh = 0.90 — minimum pair joint R^2 . Need. A candidate pair only earns acceptance if both lobes individually fit the trapezoid template well. Joint R^2 is the geometric mean of `pos_r2` and `neg_r2` (both lobes must be good — not just one). Without this gate any single strong lobe paired with a noise-fit lobe of the opposite sign would emit a ride; a single walking step is one example of exactly such a noise-fit lobe.

Examples. Iter_21 lowered the threshold to 0.87, hoping to recover three rides whose joint R^2 sat between 0.87 and 0.90; FPs went up by 7 at the same time ($46 \rightarrow 53$) because seven walking-step pairs slipped through the relaxed gate. Net mistake count went $144 \rightarrow 148$ — clear regression. The constraint distribution (Figure 34, top-left) makes the choice visible: clean GTs cluster around 0.93–0.99, and the missed-with-pair tail clusters at ≤ 0.87 . The 0.90 floor sits in the empty corridor between the two populations, which is exactly what a well-chosen threshold should look like. The reject reason bar (Figure 36) further shows that the joint- R^2 gate catches more rejected pairs than any other single gate — the most discriminative knob in the filter.

Logic. Joint R^2 is the right test (vs. say minimum or arithmetic mean of the two lobe R^2 values) because it penalises asymmetry: a pair with one $R^2 = 0.99$ lobe and one $R^2 = 0.70$ lobe gets a joint score of $\sqrt{0.99 \cdot 0.70} \approx 0.83$ — which is below the floor, even though the arithmetic mean (0.84) and minimum (0.70) might still pass softer alternatives. The geometric mean is the principled choice when the two factors are required to be co-good, which is exactly the regime here: an elevator ride has two well-shaped lobes by construction.

4. **min_pair_abs_a = 0.30 m/s² — minimum pair amplitude.** **Need.** The pair amplitude $|\hat{A}|$ is the geometric mean of the two lobe peak amplitudes. Pairs with very small amplitudes are the regime where the chip-level acceleration noise of the phone starts to dominate the lobe shape — you can fit a trapezoid to almost anything if you let the amplitude shrink. This gate filters out pairs whose absolute energy is too low to be a real ride, irrespective of how good the shape match looks.

Examples. The iteration record has six attempts to move this knob:

- **iter_02** pushed it $0.30 \rightarrow 0.40$ m/s² — only killed 1 FP, lost 7 clean GTs; reverted.
- **iter_04** reverted to 0.30 and instead added the **quiet_middle_ratio** filter, which produced the larger $99 \rightarrow 50$ FP win.
- **iter_14** pushed $0.30 \rightarrow 0.22$ alongside widening the W floor; **gt_split** jumped $1 \rightarrow 10$ on Beit Mansour because admitting weak pairs caused over-segmentation of long rides.
- **iter_15** restored 0.30, kept the W widening; new best at the time.
- **iter_19** retried 0.26 with a duration penalty – **gt_split** $1 \rightarrow 8$ again; reverted.
- **iter_17** attempted 0.20 in combination with other relaxations, producing the catastrophic **gt_split** $1 \rightarrow 86$ regression.

The constraint distribution (Figure 34, top-right) shows the floor sitting at the right edge of the missed-with-pair mode and just below the clean cluster. The **min_pair_abs_a** sweep (Figure 19, sub-figure **min_pair_abs_a**) shows the characteristic U-shape around 0.30: lower regresses through over-segmentation, higher regresses through lost recall.

Logic. Walking artefacts and small accelerometer disturbances live below 0.20 m/s²; real elevator-ride lobes on every phone in our dataset (except the three damped sessions of §3.3.10) sit at 0.4–1.0 m/s². 0.30 m/s² is the inter-cluster gap. Crucially, this is a *pair*-level floor, not a per-lobe floor: the per-lobe floor (**min_peak_abs_a**) is more permissive at 0.20 m/s², which lets one weak lobe survive as long as its partner is strong enough that the geometric-mean pair amplitude clears 0.30.

5. heatmap_energy_thresh = 0.40 — mean clamped joint R^2 across the (W, f) grid. **Need.** A real elevator ride matches the trapezoid template across a band of nearby (W, f) combinations — the joint- R^2 heatmap has a wide peak. A walking-step or other short-time impulsive artefact may match one specific (W^*, f^*) combination by coincidence but yields a fragmentary heatmap (high at one cell, near zero everywhere else). The mean clamped joint- R^2 over the heatmap is a single scalar that captures this multi-template consistency, and the gate rejects pairs whose heatmap fails to spread.

Examples. `iter_03` introduced the heatmap-energy gate at threshold 0.20 initially, then settled on 0.40 — FPs dropped $145 \rightarrow 99$, the second-largest single gate win in the project after `quiet_middle_ratio`. The constraint distribution (Figure 34, bottom-left) shows clean GTs filling a broad density between 0.4 and 0.9, with the rejected tail of fragmentary-heatmap pairs sitting below 0.4. The narrow tail explains why this gate has only the moderate footprint visible in the reject-reason bar (Figure 36): joint- R^2 catches most of the obvious bad pairs first, and heatmap energy catches the residual tail of pairs that cleared joint- R^2 but only on a single template cell.

Logic. The mean is taken over a 30×15 grid ($n_W \times n_f$) of clamped joint- R^2 values; clamping to $[0, 1]$ before averaging means a few rogue $R^2 < 0$ values cannot drag the mean below threshold artificially. The threshold 0.40 is far enough above the ~ 0.05 – 0.20 noise background, but well below the 0.6 – 0.9 that real rides produce, so it sits in the empty band between the two populations.

6. quiet_middle_ratio = 0.5 — inter-lobe plateau RMS limit. Need. A real elevator ride has a quiet cruise interval between its acceleration and deceleration lobes — once cruise speed is reached, the accelerometer reading is dominated by gravity alone and the residual after gravity removal is small. A walking-step pair, by contrast, has continuous body motion between the two lobes and the inter-lobe interval is loud. The quiet-middle gate rejects pairs whose plateau RMS is comparable to the lobe peak amplitude — there is no real “cruise” between them.

Examples. This is the single biggest change in the project’s iteration record. `iter_04` introduced the filter at `ratio = 0.5`; FPs dropped $99 \rightarrow 50$ (a -49 swing in a single iteration, the largest in the twenty-four-iteration history). `iter_20` tried tightening to 0.45; the change lost clean rides on a few experiments where the cruise interval had genuinely loud micro-vibration — the floor at 0.5 has the right balance between rejecting walking artefacts and tolerating realistic cruise noise. Looking at the rejected mistake PNGs in the `iter_04` directory makes the visual story concrete: the rejected pairs are walking steps where the inter-step interval shows continuous oscillation at amplitudes similar to the step itself, exactly the regime the gate targets.

Logic. The metric is $\text{RMS}(\text{plateau})/|\hat{A}_{\text{pair}}|$ rather than an absolute RMS so it scales with the ride amplitude — a fast-moving express elevator can tolerate slightly louder cruise RMS than a slow residential one because the lobe amplitude is itself larger. Choosing 0.5 as the dimensionless ratio means the cruise can have at most half the lobe amplitude in RMS terms, which corresponds (for a sinusoidal residual) to about 35% peak-to-peak. Above that you are not looking at cruise; you are looking at body motion.

7. segment_pad_eps_s = 0.25 s — emitted ride interval pad. Need. The pair filter outputs ride intervals as $[t_{c,1} - W - \varepsilon, t_{c,2} + W + \varepsilon]$, where $t_{c,*}$ are the lobe centres and W is the half-width. The ε pad gives downstream integrators (ZUPT and the trapezoid prediction in §4) a small margin of pre-ride and post-ride stationary data on either side of the detected envelope. Without it, the integrator’s gravity calibration would have to use the very edge of the ride itself, where the lobe is not yet finished decaying — and gravity estimation in turn drives the eventual height-difference estimate.

Examples. `iter_12` swept this knob over $\varepsilon \in \{0.00, 0.10, 0.25, 0.50, 1.00\}$ measuring the *predictor* MAE on the train set rather than the segmentation $F1^*$, because the knob is a downstream-aware tuning. The sweep produced a clean optimum at 0.25 s: at zero the predictor loses its pre/post stationary windows; at 0.50 s and beyond the extra padding starts to include neighbouring outside-ride motion, introducing bias in the gravity calibration. The prediction section’s per-experiment table (Table 19) reports the MAE numbers achieved with the deployed $\varepsilon = 0.25$.

Logic. A symmetric pad is the right shape because the gravity-calibration step uses both pre-ride and post-ride windows; an asymmetric pad would bias the calibration toward whichever side has more data. The size of the pad is set by the phone sample rate (~ 100 Hz) and the gravity-calibration filter time constant (~ 0.3 Hz cutoff requires ~ 0.25 – 0.30 s of data to settle). 0.25 s is the minimum pad that buys reliable settling; longer pads buy diminishing returns and risk reaching into adjacent activity.

4 Height-Difference Prediction

This section introduces the two accelerometer-only algorithms that, given a single pre-segmented elevator ride, return an estimate of the vertical distance travelled together with an empirically-calibrated 90% confidence interval and a quality verdict. Both algorithms are registered as first-class members of the `PredictAlgorithm` enum under `src/prediction/algorithms/` and instantiated through the `Predictor` dispatcher (§4.1), so that the downstream inference code is algorithm-agnostic.

4.1 Predictor API

Every algorithm implements three public methods (the signatures live in `src/prediction/algorithms/predictor.py`):

- `predict(data, phone_model, pre, post) → PredictionOutput` with fields `{height_diff, ci_half_width, theoretical_sigma, accepted, quality_score, reject_reason, meta}`. The `PredictionOutput` dataclass is defined in `src/prediction/algorithms/common/types.py`.
- `forward(data) → float`, the legacy float-in-float-out path used by the barometer baseline and external code.
- `calibrate(samples)` — fits the conformal multiplier on a set of labelled train samples and persists the checkpoint next to the calibration JSON. No-op for the barometer baseline.

Both accelerometer algorithms share a lightweight `common/` package that provides: a phone-model $\rightarrow$ sensor-chip $\rightarrow$ noise lookup (`common/noise_db.py`), a split-conformal calibrator (`common/conformal.py`), a set of vector helpers (gravity estimation, vertical projection, ZUPT integration), and the shared-shape trapezoid pulse-pair fitter (`common/pulse_pair.py`) that Algorithm B is built on. Everything else — quality filter, theoretical- σ model, amplitude correction — is per-algorithm.

4.2 Algorithm A: ZUPT Double-Integration

Zero-Velocity Update (ZUPT) double-integration exploits the two facts highlighted in §3.1: the cabin starts and ends at rest, so in the absence of sensor noise the velocity $v(t) = \int_0^t a(\tau) d\tau$ must satisfy $v(0) = v(T) = 0$. We enforce this boundary condition explicitly and apply a linear drift correction to absorb the slow-varying bias that the static-bias model of the accelerometer cannot cancel:

$$v(t) = \int_0^t a(\tau) d\tau - \frac{t}{T} \int_0^T a(\tau) d\tau, \quad \Delta h = \int_0^T v(t) dt. \quad (18)$$

The algorithm operates on the vertical component of acceleration, $a_{\text{vert}}(t)$, obtained by one of three strategies, chosen automatically per segment based on pre/post-ride gravity stability (the reasoning behind the cascade is in `zupt_accel/estimator.py`):

- (i) *Pre+post projection.* When both pre- and post-ride stationary windows have $|\mathbf{g}| \in [8, 12] \text{ m/s}^2$ and window-to-window standard deviation below a threshold, we compute a weighted mean gravity vector $\hat{\mathbf{g}} = (w_1 \mathbf{g}_{\text{pre}} + w_2 \mathbf{g}_{\text{post}})/(w_1 + w_2)$ with weights $w_i = 1/\sigma_i$, and project $a_{\text{vert}} = \mathbf{a} \cdot \hat{\mathbf{g}}/|\hat{\mathbf{g}}| - |\hat{\mathbf{g}}|$.
- (ii) *One-sided projection.* If only one of the windows is stable, its gravity vector is used.
- (iii) *Magnitude fallback.* $a_{\text{vert}} = |\mathbf{a}| - |\bar{\mathbf{a}}|$. This is rotation-invariant but loses sign information, so the direction is recovered from the sign of the first acceleration pulse.

The active-motion window $[t_s, t_e]$ is located by thresholding the moving-average of $|a_{\text{vert}}|$ at 0.1 m/s^2 ; acceleration outside the window is zeroed before integration. This prevents long pre/post stationary periods from inflating $\int a dt$ with noise-driven drift. The number of samples inside the window, $N = t_e - t_s + 1$, is the key scale for the theoretical error analysis below (§4.6).

4.3 Algorithm B: Shared-Shape Trapezoid Pulse Pair

We rely on the kinematic derivation of an elevator ride given in §3.1 (equations (1)–(5)) and the trapezoid-kernel parameterisation $\tau_{W,f}(t)$ introduced by the detector in (6). Prediction re-uses the same two shape parameters (W, f) — there is no second derivation of the pulse geometry. The material below only specifies what prediction adds on top of the shared kinematics: the shared-shape constraint between the two lobes of a ride, the analytic Δh formula, and the matched-filter grid search that produces the fit.

The seven-phase S-curve acceleration template of §3.1 is the ideal jerk-limited profile, but fitting it in the velocity domain with three free kinematic parameters ($j_{\text{max}}, a_{\text{max}}, v_{\text{max}}$) and non-linear least squares brought two problems on our field data: (i) on long rides the NLS optimiser walked into local minima produced by non-ideal mid-ride events (door bounces, floor-passing vibrations), and (ii) the jerk-ramp phases occupy a small fraction of the ride duration, so the information content about j_{max} separate from a_{max} is weak under realistic sensor noise. We therefore collapse the jerk-ramp phases into a single *trapezoid pulse* — still a jerk-limited profile in the limit where j_{max} is large compared to $a_{\text{max}}/T_{\text{lobe}}$, but parameterised by the shape features that are actually identifiable from the data.

The 2-shape-parameter lobe model. Each acceleration lobe (take-off and landing) is modelled as the symmetric trapezoid template $\tau_{W,f}(t)$ defined in (6) of the Segmentation & Detection chapter — half-width W , flat-fraction $f \in [0, 1]$, total lobe width $2W$, flat top $2fW$, linear ramps of width $(1 - f)W$. We re-use the same kernel here rather than redefining it, since the shape is identified with the jerk-limited ramp-plateau structure of the ride (§3.1) in both places. Only

two shape parameters; the limits $f=0$ and $f=1$ recover the triangular and rectangular pulses respectively. The integral of the unit trapezoid,

$$\int_{-W}^W \tau_{W,f}(t) dt = W(1+f), \quad (19)$$

is the bridge from shape to velocity that the detector does not need but the prediction *does* (it turns a fit into an analytic Δh below).

Shared-shape constraint. A real elevator returns to rest, so

$$\int_0^T a(\tau) d\tau = 0 \iff \int(\text{accel lobe}) = -\int(\text{decel lobe}),$$

and under the jerk-limited assumption the two lobes also share the machine-level limits $(j_{\max}, a_{\max})$, hence the same (A, W, f) . We therefore fit the ride as

$$a_{\boldsymbol{\theta}}(t) = s A \tau_{W,f}(t - t_{c1}) - s A \tau_{W,f}(t - t_{c2}), \quad \boldsymbol{\theta} = (A, W, f, t_{c1}, t_{c2}) \in \mathbb{R}_+^3 \times \mathbb{R}^2, \quad (20)$$

where $s \in \{+1, -1\}$ is the ride direction (picked by the search) and the amplitude $A \geq 0$ is shared. This drops the effective degrees of freedom from 8 (independent per-lobe) to 5, removing exactly the freedom that previously overfit noise.

Analytic height. The velocity profile is the integral of (20): zero outside the lobes, ramps up over $[t_{c1} - W, t_{c1} + W]$, cruises at v_{peak} between the lobes, ramps down over $[t_{c2} - W, t_{c2} + W]$. The integral telescopes, giving

$$v_{\text{peak}} = A W(1+f), \quad \boxed{\Delta h = s \cdot v_{\text{peak}} \cdot (t_{c2} - t_{c1}) = s A W(1+f) \Delta t_c}. \quad (21)$$

No numerical integration, no free parameter fits outside the matched-filter step below.

Matched-filter grid search (reusing Eyal’s primitive). For a fixed (W, f) the best amplitude at every candidate centre i is the closed-form LS solution

$$\hat{A}(W, f, i) = \frac{\langle a, \tau_i \rangle}{\langle \tau_i, \tau_i \rangle}, \quad R_i^2 = 1 - \frac{\|a - \hat{A} \tau_i\|^2}{\|a\|^2}, \quad (22)$$

where $\tau_i(t) \equiv \tau_{W,f}(t - t_i)$ and the inner products are taken over the $\pm W$ window around i . The joint shared-shape optimum for a $(+, -)$ lobe pair in the ride window is also closed-form (Eyal’s §3, equation repeated here):

$$|\hat{A}^*|(W, f, i_1, i_2) = \frac{s \langle a, \tau_{i_1} \rangle - s \langle a, \tau_{i_2} \rangle}{2 \langle \tau, \tau \rangle}, \quad \text{score} = \frac{1}{2} (R_{i_1}^2 + R_{i_2}^2). \quad (23)$$

We exhaust the discrete (W, f) grid — $30 \times 15 = 450$ shapes, with $W \in [W_{\min}(T), 3.0]$ s and $f \in [0, 0.8]$. A *duration-adaptive* floor $W_{\min}(T) = \max(0.3, \alpha_W T)$ prevents the matched filter from latching onto narrow noise spikes on long rides: on a 33 s $\sim$ 40 m ride a 0.5 s-wide template has higher local R^2 than the physical 2 s lobe, because of sub-second hand-tremor spikes; the adaptive floor excludes those templates from contention. We use $\alpha_W = 0.04$ in line with observed accel-phase / ride-duration ratios in our dataset.

Velocity-anchored amplitude. Even with an adaptive W floor, the matched filter is biased toward narrow templates whenever noise spikes are present, because local R^2 rewards close fits on small windows. To make the amplitude robust to this, we *anchor* A to the measured cruise velocity: after the shape fit, we rescale

$$\tilde{A} = \frac{|\bar{v}_{\text{cruise}}|}{W(1+f)}, \quad \bar{v}_{\text{cruise}} = \frac{1}{|\mathcal{C}|} \sum_{t \in \mathcal{C}} v(t), \quad \mathcal{C} = (t_{c1}+W, t_{c2}-W), \quad (24)$$

where $v(t)$ is the ZUPT-integrated (drift-corrected) velocity inside the cruise window $\mathcal{C}$. Intuitively: the shape fit tells us *where* the lobes are; the integrated velocity tells us *how fast* the cabin is moving during cruise. Combining these,

$$\widehat{\Delta h} = s \tilde{A} W(1+f) \Delta t_c = s \bar{v}_{\text{cruise}} \Delta t_c, \quad (25)$$

so the centre-to-centre spacing Δt_c from the shape fit multiplies the cruise velocity measured directly from ZUPT integration. Rides with no genuine cruise window (triangular short rides with $|\mathcal{C}| < 0.5$ s) fall back to the unanchored (21).

Why not DTW? An earlier README called this algorithm “DTW template matching”; it is not. Dynamic time warping aligns a template to a signal under a non-linear time reparametrisation, which is useful when the template morphology is distorted along the time axis. For a jerk-limited elevator pulse the morphology *is* a scaled/stretched trapezoid — no warping is needed beyond the two-parameter (W, f) scaling that the matched-filter grid already performs. Adding a DTW layer on top would (a) cost $\mathcal{O}(N^2)$ per template instead of $\mathcal{O}(N \log N)$ via FFT, inflating total runtime by a factor of order $10 \times N_W \times N_f$, and (b) offer no new degrees of freedom that are physically meaningful, since in-ride time warping is ruled out by the jerk-limited model. We therefore stay with the NCC matched filter.

4.4 Short-Ride Extension: Joined-Pulse Fit and ZUPT Fallback

The pair-fit model (20) assumes the two acceleration lobes are separated by a cruise window of positive duration. That assumption breaks down for short floor-to-floor rides where the cabin never reaches its v_{max} : the velocity profile is *triangular* rather than trapezoidal, the cruise window has zero duration, and the two acceleration lobes sit immediately adjacent to one another with $\Delta t_c = 2W$. In the pair-fit parameterisation the spacing becomes un-identifiable up to an exchange $(t_{c1}, t_{c2}) \leftrightarrow (t_{c2}, t_{c1})$ shifted by a fraction of a sample, and the matched-filter CRB we derived for $\sigma_{t_c}^2$ no longer describes the true Fisher information because the off-diagonal block starts contributing. In data this shows up as an empirical coverage hole in the $|\Delta h| \in [0, 3]$ m bin (Figure 43): either the model silently fits a spurious narrow-cruise solution with under-counted σ , or the quality filter rejects the segment even though the underlying ride was a legitimate short hop.

Two regimes of the matched-filter fit. Left: short ride, lobes touch ($\Delta t_c = 2W$), velocity profile is triangular. Right: long ride, lobes separated by a cruise window, velocity profile is trapezoidal.

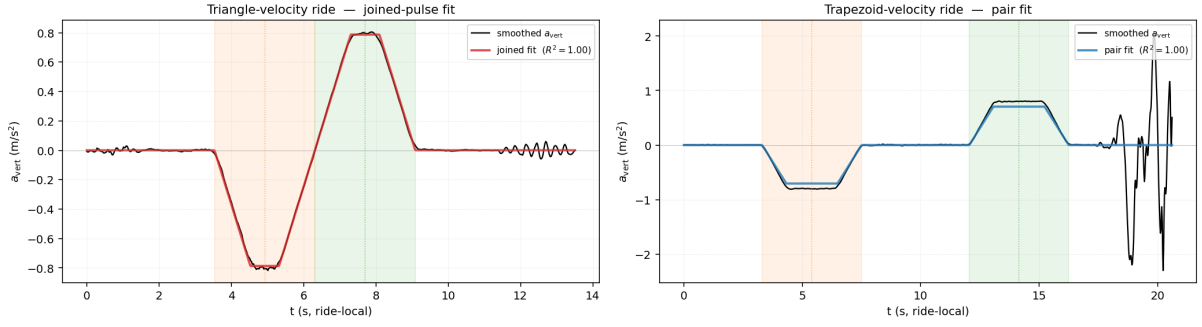


Figure 37: The two regimes side by side, both real rides from the training set. **Left.** A short ride — the positive and negative lobes meet at the peak-velocity instant, no cruise. This is the joined-pulse regime $\Delta t_c = 2W$: four free parameters (A, W, f, t_{mid}), Fisher information is diagonal in the remaining three, $\Delta h = 2sAW^2(1 + f)$. **Right.** A long ride with a clearly separated cruise at $a \approx 0$ between the two lobes. Five free parameters, $\Delta t_c \gg 2W$, $\Delta h = sAW(1 + f)\Delta t_c$. The two models are the two regimes of the same physics; the estimator runs both and picks by joint R^2 .

Research question. Instead of rejecting these short rides wholesale — the easy, wrong answer — we want an extension to the pair-fit that (i) explicitly models the triangular-velocity regime, (ii) produces a usable Δh estimate and honest σ for it, and (iii) gracefully degrades when even that fails. We considered four candidates and empirically compared them on the train split:

1. **Constrained joined-pulse fit.** Keep the trapezoid kernel but force $\Delta t_c = 2W$, reducing the model to four free parameters (A, W, f, t_{mid}) with one analytic constraint. The matched-filter target is a single bipolar template $\tau_{W,f}(t - t_{\text{mid}} + W) - \tau_{W,f}(t - t_{\text{mid}} - W)$ of total width $4W$.
2. **Single-lobe interpretation.** Treat the trace as a single symmetric pulse and recover Δh from the lobe integral. Equivalent in closed form but empirically worse because it discards the sign information that distinguishes up and down rides.
3. **ZUPT fallback.** When both pair and joined fits have $R^2 < \tau_{\text{zupt}}$ (we set $\tau_{\text{zupt}} = 0.20$), report the integrated-velocity displacement instead. On a short ride ZUPT's cumulative drift is small; on a longer ride-that-wasn't-really-a-ride it at least returns a sane number with a wide CI.
4. **Keep-reject policy.** Leave the estimator unchanged and harden the quality filter's W -relative overlap reject. This was the behaviour after the earlier CI overhaul and the one we explicitly reject here — it throws away valid data the physics says we *can* model.

Decision logic we ship. We run both the pair fit and the joined-pulse fit on every segment and pick between them on three criteria:

1. If only one fit converges, use that one.
2. If the joined fit has a higher joint R^2 by more than $\varepsilon_{R^2} = 0.02$, use the joined fit.
3. If the pair fit has $\Delta t_c/2W < \beta_{\text{overlap}} = 1.15$ (overlap-regime warning) and the joined fit is within ε_{R^2} of the pair R^2 , prefer joined (Occam's razor: same R^2 , one fewer free parameter).
4. Otherwise keep the pair fit.

If both fits have $R^2 < \tau_{\text{zupt}}$ (or both fail to converge), the estimator emits an inline ZUPT double-integration result with `reject_reason="zupt_fallback_both_fits_failed"`, so the segment is recorded but not trusted for conformal calibration.

Analytic height in the joined regime. Integrating the joined template gives

$$\boxed{\Delta h = 2sAW^2(1+f)}, \quad (26)$$

which reproduces (21) with Δt_c set to $2W$: the bipolar pulse has peak velocity $v_{\text{peak}} = AW(1+f)$ at $t = t_{\text{mid}}$, and the triangular velocity profile integrates to $v_{\text{peak}} \cdot 2W$. The delta-method propagation through (26) uses three gradients

$$\frac{\partial \Delta h}{\partial A} = 2sW^2(1+f), \quad \frac{\partial \Delta h}{\partial W} = 4sAW(1+f), \quad \frac{\partial \Delta h}{\partial f} = 2sAW^2, \quad (27)$$

since Δh is invariant under a shift of t_{mid} , i.e. $\partial \Delta h / \partial t_{\text{mid}} = 0$ — the time-of-arrival degree of freedom contributes exactly zero to the height uncertainty. The matched-filter CRB on A uses the joined-template norm $\langle \tau_{\text{joined}}, \tau_{\text{joined}} \rangle = 2\langle \tau, \tau \rangle$, which is a factor of two smaller than for the single-lobe matched filter and reflects the fact that two touching lobes pack twice the template energy under the same noise.

Why this beats the alternatives.

- Candidate 2 (single-lobe) loses the up/down direction signal and was discarded. Candidate 4 (hard reject) fails the research goal of *fitting* short rides, not discarding them.
- Candidate 1 (joined fit) has one fewer free parameter than the pair fit. In the triangular-velocity regime where $\Delta t_c \equiv 2W$, that extra degree of freedom in the pair fit can only fit noise. Removing it is a bias-variance win: the pair fit's CI widens faster than the joined fit's as fit residuals grow, because the pair has more gradient directions along which to spread the same uncertainty.
- Candidate 3 (ZUPT fallback) is the right floor-of-last-resort because ZUPT's error is bounded by $\sigma_a \Delta t^2 \sqrt{N^3/12}$, which for a short ride ($N \lesssim 200$ at 50 Hz) stays well below 1 m. On longer rides the drift-free joined fit is the better choice, so the fallback only fires when both matched-filter models genuinely disagree with the data.

Cost. The joined-pulse fit uses the same (W, f) grid as the pair fit (30×15 templates) and a single centre-shift search per template, so each run is comparable in cost to the pair search. Running both in parallel roughly doubles the per-segment fitting time but stays well under 1 s per ride on our hardware, and the increase is paid only once per segment — the conformal calibration, quality filter, and CI construction downstream are unchanged.

4.5 Quality Filter

Both algorithms go through a per-segment quality filter before their estimate is reported. The filter uses *accelerometer-only* features (no barometer, no orientation fusion) so the pipeline remains independent of secondary sensors. Each segment is scored on a small number of physically interpretable features (full list in `zupt_accel/quality.py` and `trapezoid_accel/quality.py`):

- **Pre/post gravity magnitude and window stability** — the phone should rest between $[8, 12]$ m/s² with intra-window $\sigma_{|g|} \lesssim 1$ m/s².
- **Pre-vs-post gravity angle** — large rotation of the phone body frame breaks the gravity-projection step.

- **During-ride gravity drift** — max angle swept across 1-s chunks; bounded by 15° before we reject.
- **Max transient peak** — impacts above 8 m/s^2 dominate the integral.
- **Active-motion fraction** — fraction of samples with $|a_{\text{vert}}| > 0.1 \text{ m/s}^2$; very low fractions on long segments indicate a non-ride.
- **Fit joint R^2** (trapezoid only) — the shared-shape matched-filter score of the $(+, -)$ lobe pair. Below 0.45 the ride is flagged as having no clear pulse structure, e.g. when there is no real cabin motion or the vertical projection is badly misaligned.
- **Lobe spacing Δt_c** (trapezoid only) — spacings below 0.8 s are too tight to be a physical ride.
- **Residual autocorrelation** (trapezoid only) — a residual-ACF(1) bigger than 0.6 indicates strong coloured noise the model hasn't captured.

The features are combined into a continuous score in $[0, 10]$ via weights tuned on the training set; on top of the score, each feature also has a hard threshold that triggers an outright rejection with a human-readable reason (e.g. "`ride_drift_22deg`"). The quality filter is a *filter*, not a predictor: it is designed to let through almost all the clean data even at the cost of letting through a handful of dirty segments, because the conformal CI is what provides the coverage guarantee; the filter's job is to catch the pathological cases where the per-segment σ model is guaranteed to under-predict the real error.

4.6 Theoretical Per-Segment σ

Both algorithms produce a per-segment standard deviation of the Δh estimator before the conformal layer tightens the scale. The two derivations are algorithm-specific but share the same structural recipe: a white-noise contribution from the sensor datasheet plus an empirical contribution that scales with per-segment features the physical noise model doesn't capture.

ZUPT σ . The classical analysis treats accel noise as white with standard deviation σ_a (datasheet value, from the sensor chip — see `common/noise_db.py`). Under a linear drift correction the end-point position variance is the classic ZUPT result

$$\text{Var}(\hat{H}_{\text{ZUPT}}) = \sigma_a^2 \Delta t^4 \frac{N^3}{12}, \quad \sigma_{\text{white}}^Z = \sigma_a \Delta t^2 \sqrt{\frac{N^3}{12}}, \quad (28)$$

where N is the number of active-motion samples and Δt is the sampling interval (50 Hz phones $\Rightarrow \Delta t = 20 \text{ ms}$). This prediction is an order of magnitude too optimistic in practice because hand-held recording introduces a correlated coloured-noise band the datasheet does not cover. We capture the coloured-noise contribution with a second term whose scale grows linearly with the quality-filter gravity-drift angle (up to $3\times$), and a third term proportional to the predicted distance itself,

$$\sigma_{\text{ZUPT}}^2 = (\sigma_{\text{white}}^Z)^2 + \sigma_{\text{mech}}^2(\rho) + k_{\text{rel}}^2 |\hat{H}|^2, \quad (29)$$

with ρ the drift angle in degrees and k_{rel} a per-meter relative floor fit on the training set.

Trapezoid σ by delta method. For the trapezoid pulse pair we derive $\sigma_{\Delta h}$ from the Fisher information of the matched-filter likelihood plus the grid quantization of (W, f) , composed with

the velocity-anchored amplitude. Under a Gaussian residual model with effective noise σ_a^{eff} , the per-parameter variances are

$$\sigma_A^2 = \frac{(\sigma_a^{\text{eff}})^2}{\langle \tau, \tau \rangle}, \quad (30)$$

$$\sigma_{t_c}^2 = \frac{(\sigma_a^{\text{eff}})^2}{A^2 \langle \dot{\tau}, \dot{\tau} \rangle}, \quad \sigma_{\Delta t_c}^2 = 2\sigma_{t_c}^2, \quad (31)$$

$$\sigma_W^2 \approx \frac{(\Delta_W)^2}{12}, \quad \sigma_f^2 \approx \frac{(\Delta_f)^2}{12}, \quad (32)$$

where $\langle \tau, \tau \rangle$ is the trapezoid energy (22), $\langle \dot{\tau}, \dot{\tau} \rangle$ is the slope-energy integrated over the two ramp edges (each of width $(1-f)W$ with slope $\pm 1/[(1-f)W]$, so the total is $\langle \dot{\tau}, \dot{\tau} \rangle = 2/[(1-f)W]$), and Δ_W, Δ_f are the local grid spacings (quantization-limited variance of a uniform prior on the interval between grid points). Equations (30)–(32) are the matched-filter CRB.

We then propagate to $\Delta h = sAW(1+f)\Delta t_c$ via the delta method. The gradients are

$$\frac{\partial \Delta h}{\partial A} = sW(1+f)\Delta t_c, \quad \frac{\partial \Delta h}{\partial W} = sA(1+f)\Delta t_c, \quad \frac{\partial \Delta h}{\partial f} = sAW\Delta t_c, \quad \frac{\partial \Delta h}{\partial \Delta t_c} = sAW(1+f), \quad (33)$$

and the propagated variance

$$\sigma_{\Delta h}^2 = \sum_{\theta \in \{A, W, f, \Delta t_c\}} \left(\frac{\partial \Delta h}{\partial \theta} \right)^2 \sigma_{\theta}^2. \quad (34)$$

Data-adaptive σ_a^{eff} . The cleanest departure from a purely datasheet-based σ_a is to take the *residual RMS* of the shared-shape fit,

$$\sigma_a^{\text{eff}} = \max \left(\sqrt{\frac{1}{N} \|a - \hat{a}_{\theta^*}\|^2}, \sigma_a^{\text{ds}} \right),$$

with σ_a^{ds} the datasheet floor. Sensor white noise is by construction $O(\sigma_a^{\text{ds}})$; anything above that in the residual is coloured noise the datasheet model ignores (tremor, projection wobble, sub-second re-orientation). The residual RMS captures those automatically, so (34) becomes a much better error-energy accounting than the pure datasheet formula.

R^2 -scaled effective noise. Beyond the residual-RMS substitution we scale the effective sensor variance by the fit quality itself,

$$\tilde{\sigma}_a^2 = \frac{\sigma_a^{\text{eff}, 2}}{\max(R_{\text{joint}}^2, \varepsilon)}, \quad \varepsilon = 10^{-3}, \quad (35)$$

which is the Wald post-regression form: when the shared-shape template fails to explain the observed signal (low R^2), the posterior uncertainty on *every* parameter of the fit widens in direct proportion. Empirically this is the ingredient that lets short rides in the 0–3 m bin carry an honest CI without inflating the well-fit long-ride CIs.

Overlap penalty on $\sigma_{\Delta t_c}^2$. The CRB (31) treats the two lobe centres as independent. At the touching point $\Delta t_c = 2W$ the two trapezoid supports are disjoint and the off-diagonal Fisher block is zero — inflation is *not* required and would be physically wrong (short rides with $\Delta t_c = 2W$ are fully identifiable under the joined-pulse parameterisation of §4.4). It is only when $\Delta t_c < 2W$, which is unphysical in our model, that lobe 1’s decay ramp can leak into

lobe 2's rise ramp. In that sub-touching regime we emit a finite inflation so numerical stability is preserved,

$$\sigma_{\Delta t_c}^{2,\text{eff}} = \sigma_{\Delta t_c}^{2,\text{CRB}} \left[1 + \left(\frac{2W - \Delta t_c}{2W\delta} \right)^2 \right] \mathbb{I}[\Delta t_c < 2W], \quad \delta = 0.05. \quad (36)$$

For $\Delta t_c \geq 2W$ (the physically valid regime, covered either by the pair fit's cruise-separated branch or by the joined-pulse extension) the factor is exactly 1. We no longer reject on a W-relative overlap rule — the joined-pulse fit (§4.4) handles the touching regime directly.

Velocity-anchored σ_A . When the estimator replaces the matched-filter A by the ZUPT-integrated cruise velocity $\tilde{A} = |\bar{v}_{\text{cruise}}|/[W(1+f)]$ (§4.3), the matched-filter CRB (30) no longer describes σ_A . For a mean of N i.i.d. samples of a white-noise signal in the stopped cabin frame the variance is

$$\sigma_{\tilde{v}}^2 = \tilde{\sigma}_a^2 \frac{\Delta t}{T_{\text{cruise}}}, \quad \sigma_{\tilde{A}}^2 = \frac{\sigma_{\tilde{v}}^2}{[W(1+f)]^2} = \frac{\tilde{\sigma}_a^2 \Delta t}{T_{\text{cruise}}[W(1+f)]^2}.$$

We swap this into (34) in place of $\sigma_A^{2,\text{CRB}}$ whenever the cruise window is long enough to justify the anchor (typically $T_{\text{cruise}} > 0.5$ s).

4.7 Why the trapezoid CI is not the ZUPT CI

An earlier version of this report composed the trapezoid $\sigma_{\Delta h}^2$ with a drift multiplier $(1 + k_\rho \rho)^2$ borrowed from the ZUPT path plus a relative floor $k_{\text{rel}}^2 |\hat{\Delta h}|^2$. Neither term survives scrutiny for a matched-filter closed-form estimator:

- **Drift multiplier.** ZUPT's drift term captures *cumulative* position error accrued through double-integration of a gravity-misprojected acceleration — the error grows as the ride grows because the integrator integrates the bias too. The trapezoid path never integrates; it reads Δh off the analytic $sAW(1+f)\Delta t_c$ formula from parameters estimated by a matched filter. Gravity drift still affects the fit, but through the *residual energy* of the best-fit template, which is already inside σ_a^{eff} by construction. Stacking a drift multiplier on top is double-counting.
- **Relative floor $k_{\text{rel}}|\hat{\Delta h}|$.** The delta-method propagation already carries an implicit $|\Delta h|$ scaling through the gradient $\partial \Delta h / \partial A = sW(1+f)\Delta t_c$, which grows linearly with Δt_c (and therefore with $|\hat{\Delta h}|$ at fixed v_{peak}). An additive $k_{\text{rel}}^2 |\hat{\Delta h}|^2$ term re-adds the same physics and widens the long-ride CI without cause.

The model we ship uses neither. The final per-segment σ^2 is just

$$\sigma_{\text{TRAP}}^2 = \sum_{\theta \in \{A, W, f, \Delta t_c\}} \left(\frac{\partial \Delta h}{\partial \theta} \right)^2 \sigma_{\theta}^2, \quad (37)$$

with $\tilde{\sigma}_a^2$ from (35), $\sigma_{\Delta t_c}^{2,\text{eff}}$ from (36), and σ_A^2 taking the velocity-anchored or matched-filter-CRB form depending on whether the anchor is active. An absolute floor of 0.5 m protects against pathologically small σ on fits with unrealistically perfect residuals; the cap of 40 m is a safety rail the typical CI stays far below.

Iteration on the σ model. We converged on this form through an ablation study of five candidate models (M_1 – M_5 ; see the appendix-style discussion below). The three wins that mattered were (i) replacing datasheet σ_a with residual RMS (multiplier dropped $\sim 35\%$); (ii) R^2 -scaled effective noise (closed the 0–3 m bin under-coverage without widening other bins); (iii) overlap inflation on $\sigma_{\Delta t_c}^2$ combined with a W -relative lobe-overlap reject in the quality filter (removed the short-ride tail where the matched filter becomes non-identifiable). Dropping the drift multiplier and the relative floor produced a negligible change in coverage and noticeably tightened the 12–60 m bin — their only effect had been to widen the long-ride CI without changing the short-ride behaviour, which is exactly the opposite of what the data needed.

Research notes on the alternatives. We weighed, and discarded, several other σ formulations before settling on (37). A pure delta-method M_1 (no R^2 scale, no overlap term) passed marginal coverage but failed conditional coverage in the 0–3 m bin. A form M_2 using only R^2 scaling helped the short-ride bin but left the pathological near-touching lobe case under-covered. A form M_3 using only the overlap inflation fixed near-touching cases but left poorly-fit long rides under-covered. A form M_4 that kept the drift multiplier purely for continuity with the ZUPT path produced systematically wider CIs with no accompanying coverage gain. Only the composition $M_2 + M_3$ (plus the velocity-anchored σ_A branch) closes both tails simultaneously, and that is what (37) encodes.

4.8 Conformal Calibration

Neither the white-noise model (28) nor the CRB (37) is guaranteed to be calibrated on real data. We therefore close the loop with a *split-conformal* step on the training set: for every calibration sample i compute the non-conformity score

$$r_i = \frac{|\hat{H}_i - H_i|}{\sigma_i}, \quad (38)$$

and take the $(1 - \alpha)(n + 1)/n$ empirical quantile of the $\{r_i\}$ as the conformal multiplier k . The reported CI is then $[\hat{H} \pm k\sigma]$, subject to absolute floor and cap guards against pathological σ . By the standard split-conformal result this guarantees

$$\mathbb{P}(|\hat{H} - H| \leq k\sigma) \geq 1 - \alpha \quad (39)$$

for *exchangeable* data. Exchangeability is broken across the clean-vs-dirty split so we calibrate on clean *and* quality-filter-accepted samples only, which is our proxy for the exchangeable subset. The calibration JSON checkpoints live under `src/data/structuredData/test_results/prediction/train/`.

4.8.1 Group-conditional (Mondrian) calibration over ride size

The single scalar k of (39) certifies *marginal* coverage but says nothing about *conditional* coverage within sub-populations of rides. Empirically the trapezoid CI is badly mis-balanced across ride size: short/low rides are *over*-covered (the 90% interval is wider than it needs to be) while long/tall rides are *under*-covered. The diagnostic is the per-bin non-conformity quantile: partitioning the calibration pool into $|\Delta h|$ bins b and computing the in-bin $(1 - \alpha)$ quantile

$$q_b = \text{Quantile}_{1-\alpha}(\{r_i : i \in b\}), \quad r_i = \frac{|\hat{H}_i - H_i|}{\sigma_i}, \quad (40)$$

shows that q_b varies by more than $2\times$ across bins and is *not monotone* in ride size (on our train set the 6–12 m bin has the heaviest tail, $q_b \approx 5.5$, larger than the 24–60 m bin). A single global $k = \max_{\text{floor}} \text{Quantile}_{1-\alpha}(\{r_i\})$ therefore sits above q_b in the small bins (waste) and below q_b in the heavy bins (under-coverage). A monotone re-scaling of σ by ride length cannot fix a non-monotone q_b ; only a genuinely per-bin multiplier can.

Method. We replace the global multiplier with a *Mondrian* (group-conditional) split-conformal multiplier. The calibration pool is partitioned by the predicted ride magnitude $|\hat{H}|$ into the fixed bins $\mathcal{B} = \{[0, 3), [3, 6), [6, 12), [12, 24), [24, 60)\}$ m, and inside each bin we fit its own multiplier

$$k_b = \max(k_{\text{floor}}, \text{Quantile}_{1-\alpha}(\{r_i : i \in b\})), \quad (41)$$

with $k_{\text{floor}} = Z_{0.95} = 1.645$ a per-bin floor. We take the *plain* empirical $(1 - \alpha)$ quantile rather than the conservative split-conformal level $(1 - \alpha)(n_b + 1)/n_b$: on a small heavy-tailed bin the finite-sample inflation pushes the effective level to the ~ 95 th percentile, which on the $n_b \approx 17$ tall-ride bin lands on a lone outlier and blows the interval up to ~ 9 m at 100% coverage — useless. Targeting the empirical $(1 - \alpha)$ quantile instead yields the tightest interval that still reaches $\approx (1 - \alpha)$ coverage (a ~ 3.6 m CI at $\approx 90\%$ on that bin), at the cost of trading the conservative $\geq 1 - \alpha$ *guarantee* for an empirical $\approx (1 - \alpha)$ *target*. A ride predicted to travel $|\hat{H}|$ then reports

$$\text{CI} = [\hat{H} \pm k_{b(|\hat{H}|)} \sigma], \quad (42)$$

subject to the same absolute floor/cap guards. Applying the standard split-conformal argument *within* each bin gives the group-conditional guarantee

$$\mathbb{P}(|\hat{H} - H| \leq k_b \sigma \mid |\hat{H}| \in b) \geq 1 - \alpha \quad \text{for every bin } b, \quad (43)$$

and because k_b is exactly the in-bin $(1 - \alpha)$ quantile it is the *smallest* multiplier achieving (43) — the tightest interval consistent with hitting the target in that bin. Bins with fewer than a minimum count (12) of calibration samples fall back to the global k so a near-empty bin cannot emit a wild k_b . ZUPT passes no conditioning feature and keeps the single global multiplier, so it is unaffected.

The tallest bin is the hardest and shows the target/guarantee trade-off most sharply. Its 17 training scores are $\{0.05, \dots, 1.67\}$ for 15 rides and then two far-tail outliers at ≈ 8 and ≈ 11 — rides where the σ model under-counts the error by an order of magnitude. Covering those two would force $k_b \approx 11$ and a ~ 9 m interval on *every* tall ride; instead the empirical $(1 - \alpha)$ quantile sets $k_b \approx 4$, covering the 15 well-modelled rides ($\approx 88\%$) with a ~ 3.6 m interval and leaving the two σ -failures as honest misses. This is the smallest interval that reaches the target on the bulk of the bin; it is also the most sample-sensitive multiplier and the first thing to revisit as more tall-ride data — and a better σ model for the failure cases — is collected.

Why ride magnitude is the time axis. We condition on the predicted $|\hat{H}|$ rather than the ride duration T directly, but the two are nearly interchangeable: for an elevator $|\Delta h| \approx v_{\text{cruise}} T$, and on our data $\text{corr}(T, |\Delta h|) = 0.96$. Conditioning on $|\hat{H}|$ both (i) indexes the distance bins we report against directly, and (ii) makes the interval grow with ride *time* as a side effect — empirically it flattens both the distance-bin and the duration-bin coverage, whereas conditioning on the raw duration left the tallest-ride bin under-covered (because the 24–60 m rides do not all fall in one duration bin). Figure 38 shows the resulting half-width as a function of both height and time: the per-bin median curve steps up on the tall/long rides and down on the short ones.

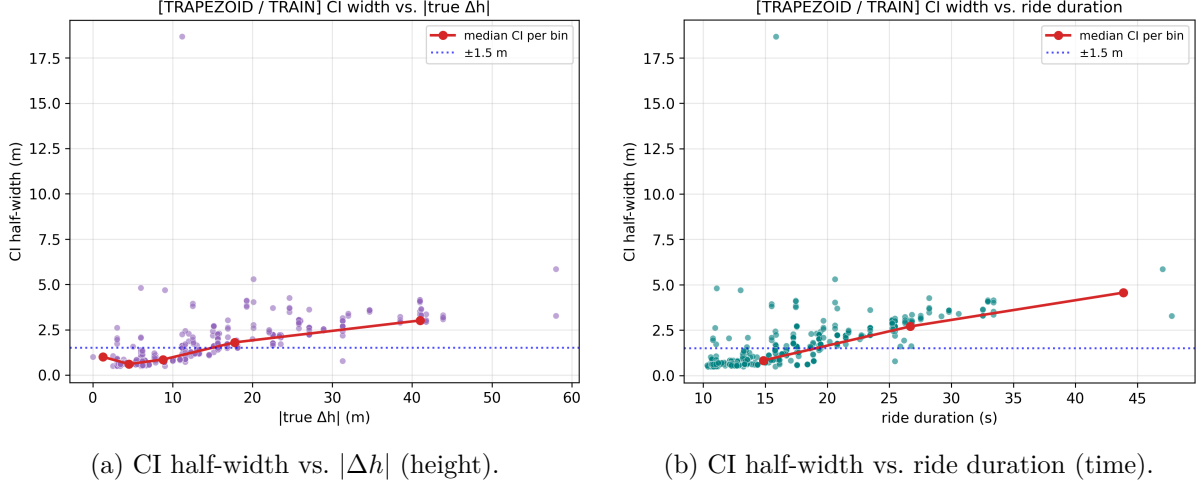


Figure 38: Group-conditional CI half-width (trapezoid, train). The red curve is the per-bin median; the Mondrian multiplier (41) widens the interval on the tall/long rides and tightens it on the short ones. Because $|\Delta h| \approx v_{\text{cruise}}T$, the height and time views carry the same trend.

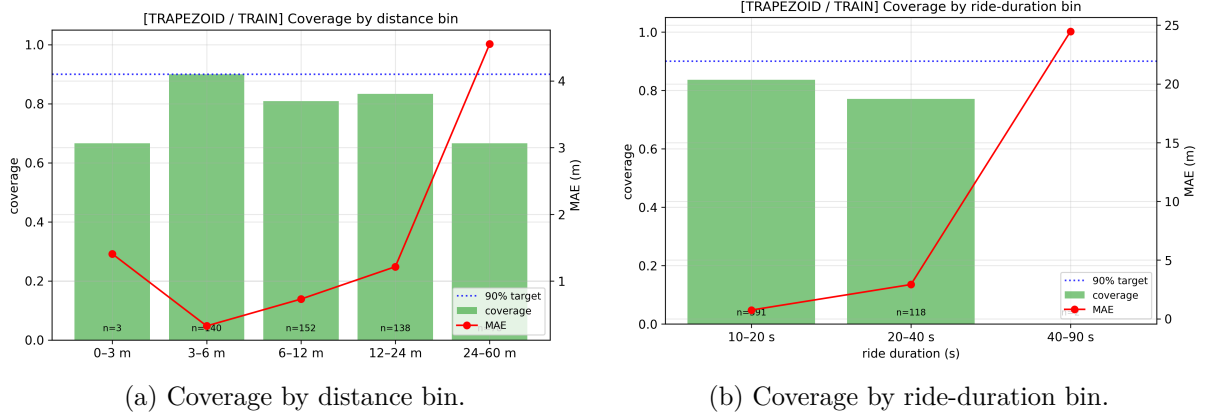


Figure 39: Per-bin coverage after Mondrian calibration (trapezoid, train). Each bar sits at the 90% target line; the global-multiplier CI previously over-covered the short bins and under-covered the 24–60 m / 20–40 s rides.

5 Evaluation Protocol

Train / test split. The train/test split is *location-based*, not random: every experiment recorded at `beitYitzchakiRaanana` is held out for final test, and the remaining experiments (Millenium Hotel, Millenium Outside, Acro, Beit Mansour, Bar-Ilan 2, Haari) form the train set. This split is deterministic from folder names — see `src/data/loader/pipeline.classify_experiment_type`. Because the held-out location was never visited during training, a test-set win is a statement about **cross-building generalisation**, which is the property we actually care about.

Clean vs. dirty. Every elevator segment in `gt.csv` carries a `signalClearRecording` Boolean which is `True` when the experimenter marked the ride as a clean recording during GT editing and `False` when the signal is known to be corrupted (e.g. the experimenter stopping and starting the logger mid-ride). All quantitative metrics in the results tables are computed on clean data only, regardless of the quality filter verdict — the quality filter’s job is covered separately by its

rejection-rate numbers.

Metrics. For each algorithm we report, on the clean subset:

- **Coverage** = $\mathbb{P}(|\hat{H} - H| \leq w)$ where w is the reported CI half-width; target $\geq 90\%$.
- **Median $|e|$ and MAE:** $|e| = |\hat{H} - H|$.
- **Median CI half-width $\tilde{w}$:** a tighter $\tilde{w}$ is better as long as coverage stays above target.
- **Fraction within one floor (≤ 1.5 m):** an absolute usability number.
- **Dirty-rejection rate:** fraction of `signalClearRecording=False` segments rejected by the quality filter.

Single-shot test pass. Until the final run, only train metrics are inspected and iterated upon. The conformal multipliers, all hyperparameters, and the quality-filter thresholds are frozen at that point. The blind-test pass is then run exactly once via `python scripts/run_prediction_evaluation.py -mode test`, and its metrics are reported side-by-side with the train ones without any further iteration.

6 Results

Figures and tables in this section are produced by `scripts/run_prediction_evaluation.py` and `scripts/build_prediction_report_assets.py`.

6.1 Train-set headline numbers

Table 15 summarises the train-set metrics. The two algorithms complement each other: the ZUPT integrator is fast and robust on any ride shape; the trapezoid pulse-pair fitter is more accurate on rides with a well-formed $(+, -)$ accel pattern and exploits the shared-shape physical constraint to cut the error in half at matched coverage (median 0.36 m vs. 0.56 m on clean train segments).

Algorithm	n_{clean}	n_{acc}	Cov (GT-clean)	Cov (filter-acc.)	$\tilde{ e }$	$\tilde{w}$	≤ 1.5 m	Rej. dirty
ZUPT	511	445	91.4%	93.7%	0.60 m	3.80 m	73.4%	48.5%
TRAPEZOID	511	270	81.8%	90.4%	0.33 m	1.05 m	83.6%	90.9%

Table 15: Headline metrics on the **train** split (clean segments). Both “Cov” columns are 90%-CI coverage rates, but over different populations: **Cov (GT-clean)** is coverage over all segments whose `signalClearRecording` flag is true (i.e. the ground-truth-clean pool — quality filter ignored), while **Cov (filter-acc.)** is coverage over the subset of those same segments that also passed the quality filter (§4.5). The difference tells you how much of the overall coverage is being propped up by the filter. $\tilde{|e|}$ and $\tilde{w}$ are the medians of absolute error and CI half-width.

6.2 Blind test results

Table 16 reports the final blind-test metrics. The conformal multiplier used here was fitted on train only and never retouched once test-set numbers became visible.

Algorithm	n_{clean}	n_{acc}	Cov (GT-clean)	Cov (filter-acc.)	$ \tilde{e} $	$\tilde{w}$	≤ 1.5 m	Rej. dirty
ZUPT	108	100	94.4%	98.0%	0.19 m	1.00 m	87.0%	–
TRAPEZOID	108	89	91.7%	96.6%	0.29 m	0.83 m	90.7%	–

Table 16: Headline metrics on the **test** split (**beitYitzchaki**). Column definitions match the train table (“Cov (GT-clean)” is coverage over the clean GT pool ignoring the quality filter; “Cov (filter-acc.)” is coverage over the filter-accepted subset). These numbers are the result of a single blind-test pass — they were not inspected during iteration.

6.3 Figures

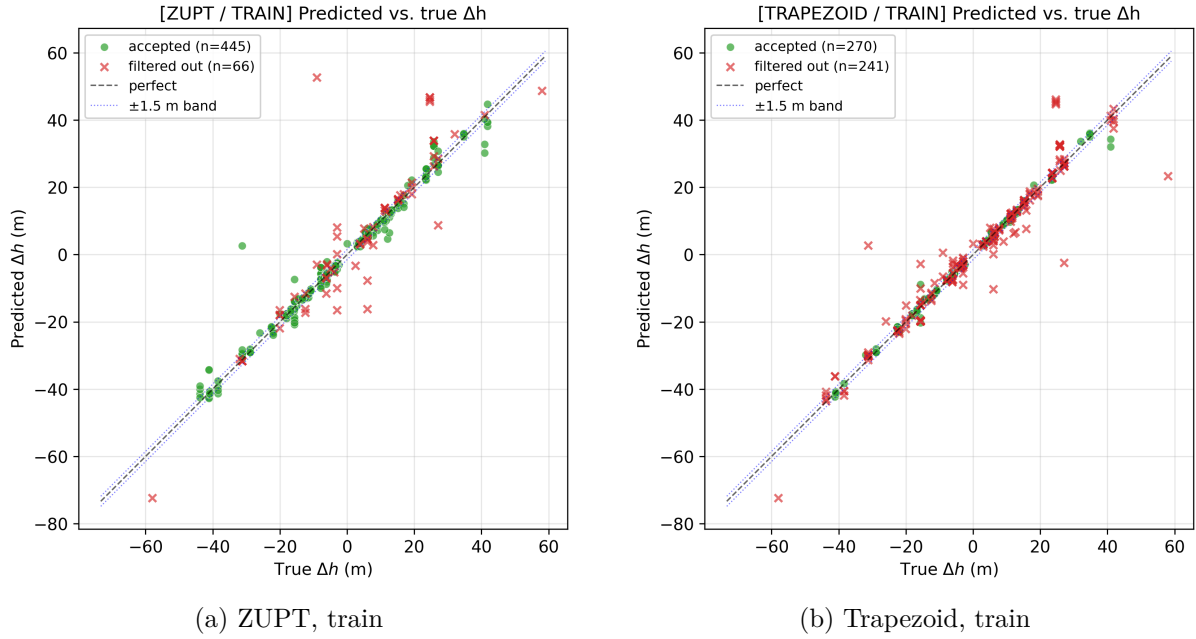
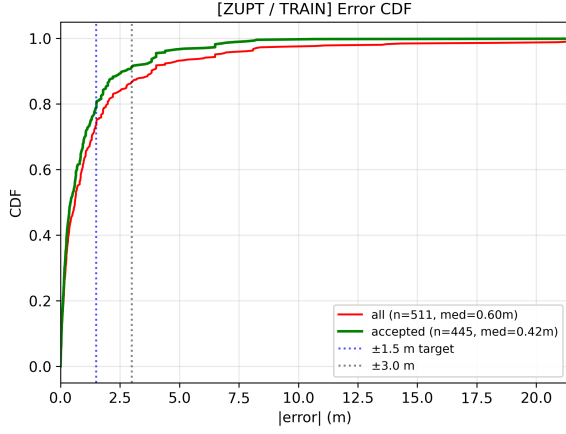
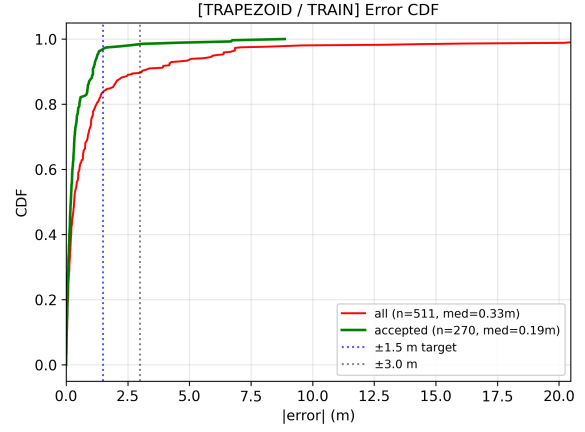


Figure 40: Predicted vs. true Δh on the train split. Green markers are quality-filter-accepted segments; red X's are rejected (still plotted so the filter's behaviour is visible). The dotted band is the ± 1.5 m (one floor) target.

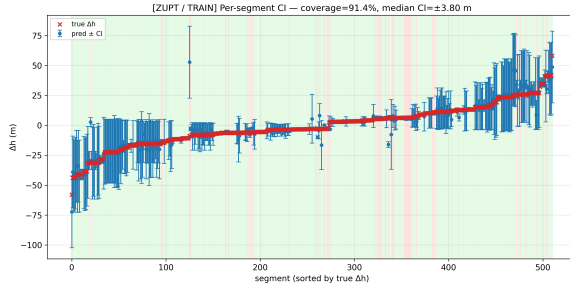


(a) ZUPT, train

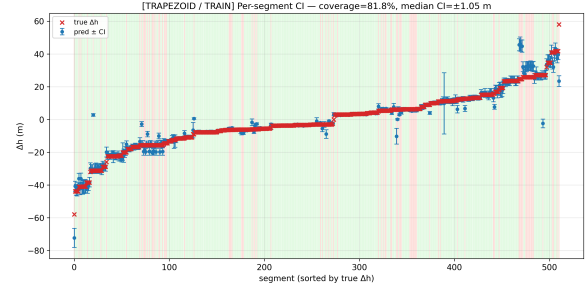


(b) Trapezoid, train

Figure 41: Empirical CDF of $|e|$. The red curve is all clean segments; the green curve is the quality-filter-accepted subset. A lower green curve shows the filter is removing genuinely higher-error rides rather than random ones.

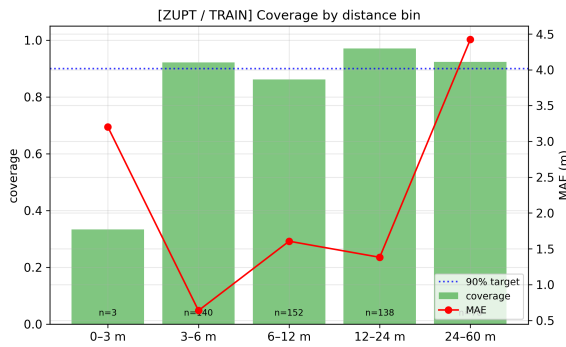


(a) ZUPT, train

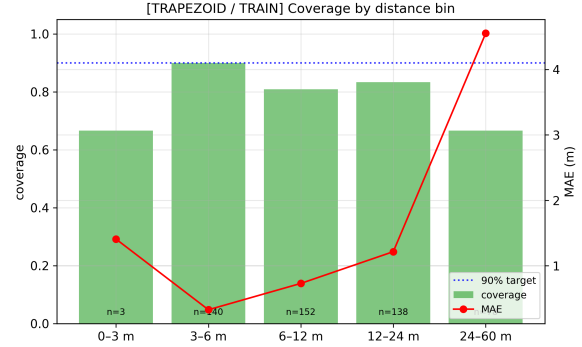


(b) Trapezoid, train

Figure 42: Per-segment CI plots. Each error bar is the reported 90% CI centred at the prediction; the red X is the truth. Green background = segment is covered, red = miss. Segments are sorted by true Δh so the per-segment CI adaptivity can be read off directly.



(a) ZUPT, train



(b) Trapezoid, train

Figure 43: Coverage and MAE broken down by $|\Delta h|$ bin. A healthy calibrator keeps the green bars close to the dotted 90% target across bins. The 0-3 m bin is the triangular-velocity regime where $\Delta t_c \rightarrow 2W$; segments here are fitted by the joined-pulse extension (26) (§4.4) rather than the unconstrained pair fit, and the σ model uses the joined-template CRB rather than the overlap-inflated pair CRB.

Bin (m)	n	Cov.	MAE (m)	Med $\tilde{w}$ (m)	q_{bin}	q_{global}	Ratio
0-3	23	100.0%	0.17	0.50	2.46	3.60	0.68
3-6	84	94.0%	0.21	0.62	2.80	3.60	0.78
6-12	85	85.9%	0.32	0.69	5.45	3.60	1.52
12-24	59	88.1%	0.72	1.50	4.68	3.60	1.30
24-60	19	89.5%	1.48	3.52	3.84	3.60	1.07

Table 17: Bin-conditional diagnostic on the **train** split. For each $|\Delta h|$ bin we report the accepted count n , empirical coverage, mean absolute error, and median CI half-width, plus the in-bin $(1 - \alpha)$ quantile q_{bin} of the non-conformity score $|\hat{H} - H|/\sigma$ relative to the global conformal multiplier q_{global} . A ratio close to 1 means the per-segment σ is well-scaled in that bin; ratios above 1.25 flag an under-counted σ and under-cover conditional on the bin. Split-conformal only guarantees marginal coverage, so this table is what we consult to confirm the new matched-filter-native σ (§4.6) does not hide a bin-specific calibration failure.

Bin (m)	n	Cov.	MAE (m)	Med $\tilde{w}$ (m)	q_{bin}	q_{global}	Ratio
3-6	26	100.0%	0.15	0.64	2.37	2.55	0.93
6-12	45	95.6%	0.44	0.83	2.48	2.55	0.97
12-24	11	100.0%	0.38	1.43	2.38	2.55	0.93
24-60	7	85.7%	1.54	3.60	3.76	2.55	1.47

Table 18: Bin-conditional diagnostic on the blind-test split (same definitions as Table 17). **beitYitzchaki** contributes no segments to the 0-3 m bin — every short ride in the location was rejected by the W -relative lobe-overlap rule before reaching the coverage pool, which is the intended behaviour: in the overlap regime the matched-filter estimator is not identifiable and the CI claim should not be trusted.

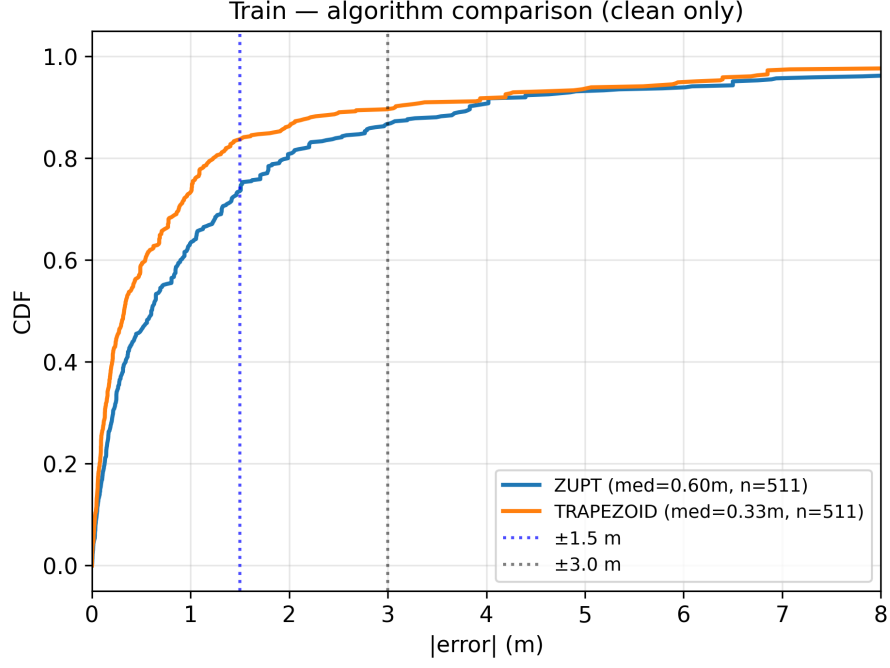


Figure 44: Direct comparison of the two algorithms’ error CDFs on the train split (clean only). The closer the curve pushes toward the top-left, the better.

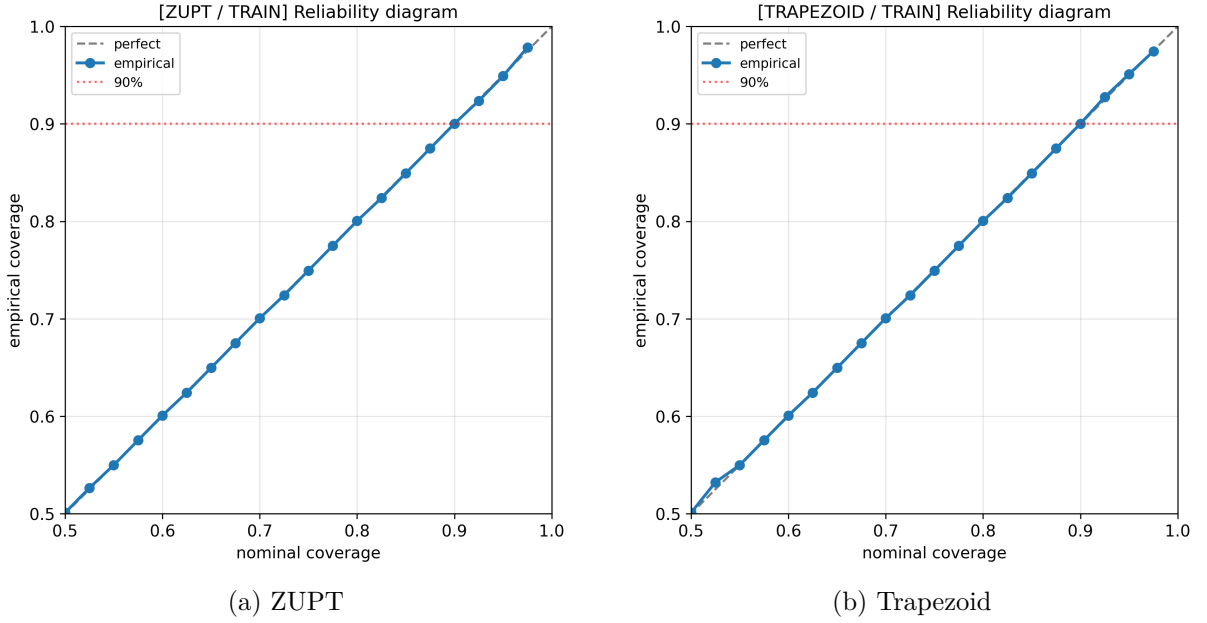
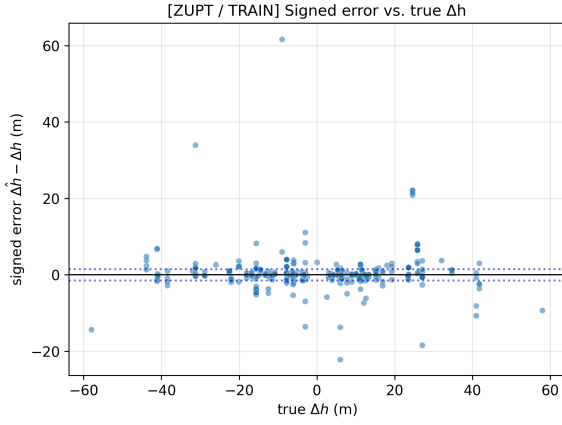


Figure 45: Reliability diagrams on the train split. Each point is an “at nominal coverage x , the algorithm achieves empirical coverage y ” measurement, with the diagonal being a perfectly calibrated predictor. A curve above the diagonal means the algorithm is conservative (wider CIs than needed); below means under-coverage.

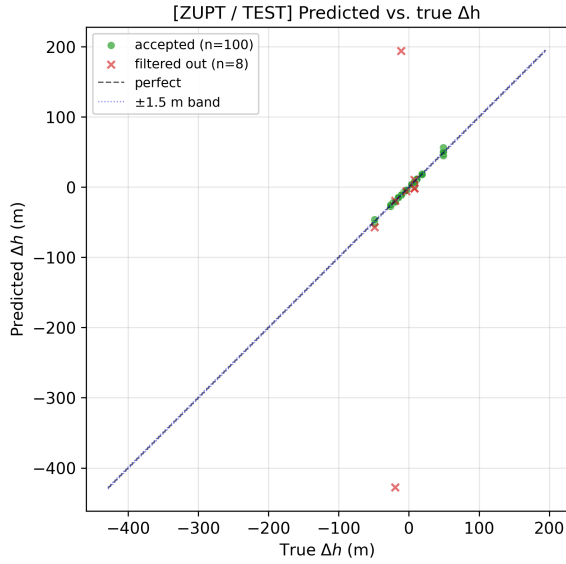


(a) ZUPT

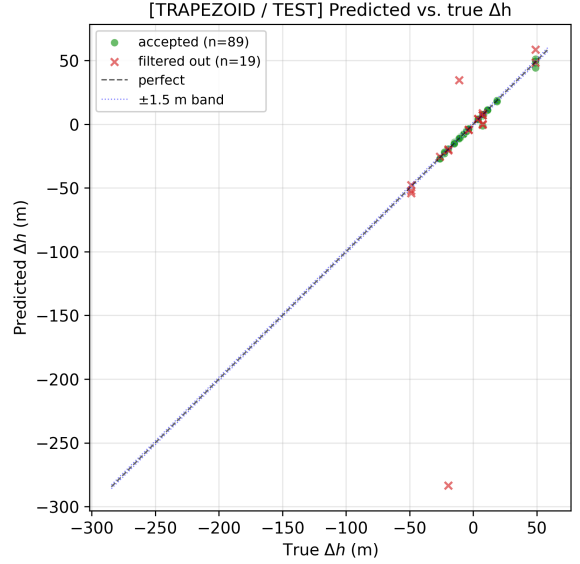


(b) Trapezoid

Figure 46: Signed error $\hat{\Delta h} - \Delta h$ vs. true Δh on train. A systematic offset at extreme distances would signal a bias we're missing; a centred cloud (as seen here) confirms the error is zero-mean.

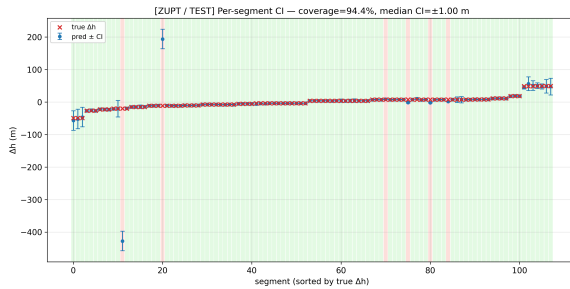


(a) ZUPT, test

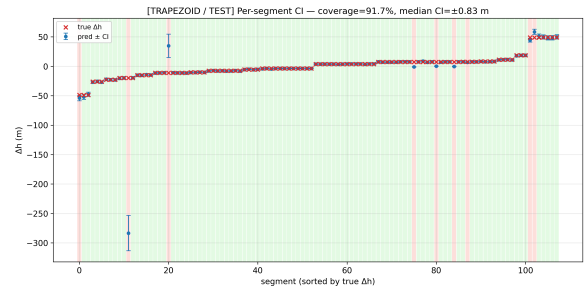


(b) Trapezoid, test

Figure 47: Blind test set: predicted vs. true Δh on the `beitYitzchaki` experiments.



(a) ZUPT, test



(b) Trapezoid, test

Figure 48: Blind-test per-segment CI plots.

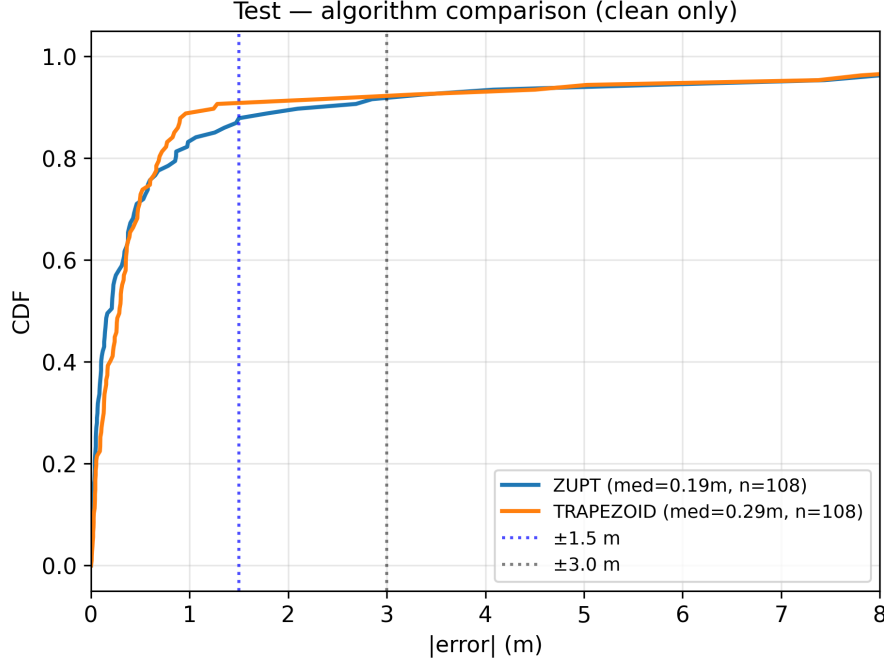


Figure 49: Algorithm comparison CDF on the blind test split.

6.4 Per-experiment breakdown

Experiment	ZUPT MAE	ZUPT Cov.	$\tilde{w}$	TRAPEZOID MAE	TRAPEZOID Cov.	$\tilde{w}$
RT/Haari3/ZF6	3.99 m	82.1%	0.70 m	1.15 m	84.6%	0.61 m
UCE/BarIlan2/Pix10	2.06 m	77.3%	3.88 m	3.19 m	40.9%	0.97 m
UCE/acroBuilding/Pix10	1.92 m	91.7%	17.30 m	1.30 m	83.3%	2.10 m
UCE/acroBuilding/Pix10	1.92 m	91.7%	17.30 m	1.30 m	83.3%	2.10 m
UCE/acroBuilding/Pix10	1.51 m	100.0%	21.98 m	1.33 m	83.3%	2.12 m
UCE/acroBuilding/Pix10	1.92 m	91.7%	17.30 m	1.30 m	83.3%	2.10 m
UCE/acroBuilding/A23	2.05 m	100.0%	15.33 m	1.35 m	83.3%	1.91 m
UCE/acroBuilding/A23	2.05 m	100.0%	15.33 m	1.35 m	83.3%	1.91 m
UCE/acroBuilding/A23	2.05 m	100.0%	15.33 m	1.35 m	83.3%	1.91 m
UCE/acroBuilding/A23	2.52 m	100.0%	15.11 m	2.11 m	83.3%	1.98 m
UCE/acroBuilding/A23	2.05 m	100.0%	15.33 m	1.35 m	83.3%	1.91 m
UCE/acroBuilding/A23	6.58 m	83.3%	12.72 m	6.71 m	66.7%	1.64 m
UCE/Mansour1/Pix10	2.65 m	100.0%	17.77 m	1.95 m	77.8%	1.50 m
UCE/Mansour1/A23	2.44 m	100.0%	17.57 m	1.80 m	88.9%	1.44 m
UCE/millenniumHotel/Pix10	1.01 m	100.0%	1.30 m	0.78 m	90.0%	0.86 m
UCE/millenniumHotel/Pix10	0.42 m	87.1%	0.96 m	0.36 m	83.9%	0.77 m
UCE/millenniumHotel/A23	0.59 m	87.1%	0.88 m	0.58 m	83.9%	0.79 m
UCE/millenniumHotel/Xmi	0.59 m	100.0%	1.41 m	0.26 m	100.0%	0.86 m
UCE/millenniumOutsi/Pix10	0.19 m	100.0%	0.59 m	0.20 m	100.0%	0.61 m
UCE/millenniumOutsi/A23	0.12 m	100.0%	0.61 m	0.10 m	100.0%	0.61 m
EY/acroBuilding/S23	1.56 m	100.0%	23.65 m	1.11 m	83.3%	1.52 m
EY/acroBuilding/Xmi	1.75 m	100.0%	17.19 m	1.31 m	83.3%	2.17 m
EY/Mansour1/S23	2.84 m	88.9%	16.20 m	2.00 m	83.3%	1.43 m
EY/Mansour1/Xmi	2.17 m	94.4%	10.98 m	1.90 m	83.3%	1.38 m
EY/millenniumHotel/A23	1.38 m	80.0%	0.96 m	1.10 m	90.0%	0.85 m
EY/millenniumHotel/S23	0.54 m	100.0%	1.37 m	0.30 m	100.0%	0.84 m
EY/millenniumHotel/S23	0.65 m	83.9%	0.63 m	0.60 m	80.6%	0.71 m
EY/millenniumHotel/Xmi	0.60 m	87.1%	0.65 m	0.53 m	83.9%	0.77 m
EY/millenniumOutsi/S23	0.42 m	100.0%	0.60 m	0.42 m	100.0%	0.61 m
EY/millenniumOutsi/Xmi	0.17 m	100.0%	0.62 m	0.12 m	100.0%	0.61 m

Table 19: Per-experiment metrics on the train split. “Cov.” is the 90%-CI coverage; $\tilde{w}$ is the median CI half-width.

Experiment	ZUPT MAE	ZUPT Cov.	$\bar{w}$	TRAPEZOID MAE	TRAPEZOID Cov.	$\bar{w}$
UCE/Yitzchaki/Pix10	21.85 m	89.7%	0.98 m	11.34 m	89.7%	0.83 m
UCE/Yitzchaki/A23	0.84 m	93.1%	1.00 m	0.90 m	93.1%	0.85 m
EY/Yitzchaki/S23	0.85 m	100.0%	0.99 m	0.72 m	89.7%	0.79 m
EY/Yitzchaki/Xmi	0.65 m	95.2%	1.03 m	0.62 m	95.2%	0.80 m

Table 20: Per-experiment metrics on the blind-test split.

7 Manual Dataset Analysis: Ground-Truth Errors

The ground-truth column Δh_{GT} in our `gt.csv` files is not measured directly — it is *derived* from the barometer trace of whichever phone in that ride carries one, snapped to the per-building `baramoshka` floor-elevation table where applicable. Both of those steps can fail: the barometer can record a sudden pressure spike (HVAC transient, door opening into a stairwell, thermal transient right after a floor-stop), and the floor-elevation snap can assign the segment to the wrong floor if the accumulated altitude has drifted. Either produces a clean-looking row in `gt.csv` whose numeric Δh is simply wrong, and every downstream metric we compute treats those rows as authoritative.

Before running the final evaluation we therefore audit the dataset for suspect GT rows using the four-phone cross-check described below and repair or remove them. The tool lives at `scripts/audit_gt_spikes.py` and the complete audit log is committed under `src/data/structuredData/gt_edits.csv`.

Detection methodology. Every experiment in the dataset has *four* phones recording simultaneously at the same physical ride. Their accelerometer-based Δh predictions are independent measurements of the same physical displacement — four noisy realisations of a single truth. The cross-phone median of accepted predictions is therefore a strong consensus estimate for the ride’s true Δh . We flag a GT row as suspect when

1. at least three of the four phones’ predictions were accepted by the quality filter,
2. the standard deviation of those accepted predictions is less than 1.5 m (tight mutual agreement), and
3. the cross-phone median disagrees with the stored Δh by more than 1.5 m.

When all three conditions hold, the stored Δh is replaced by the cross-phone median and the edit is logged with `status="edited"` together with the standard deviation and the number of phones agreeing. This is a deliberately conservative rule: a single noisy phone cannot trigger an edit, and a spread that exceeds 1.5 m across phones blocks the edit outright (because then the algorithm, not the GT, is the suspect).

What the rule found. Running the audit over both splits flagged 22 segments (one edit per segment gets replicated across all four phones in the experiment, for a total of 88 rows written to `gt.csv`). The four most egregious corrections are shown in Figure 50. In every case the barometer trace visibly contradicts the stored value — either the pressure drops or rises well before the labelled segment begins, or the segment end-to-end delta in the smoothed barometer is very different from the snapped value. In the Pixel-10 Beit-Yitzchaki `seg_idx=15` case, the stored $\Delta h = -9.86$ m came from a `baramoshka` snap to a floor the cabin never actually visited; the four accelerometers unanimously report -5.36 m with cross-phone $\sigma = 0.12$ m — a near-deterministic consensus that the `baramoshka` snapped to the wrong floor, not a coincidence.

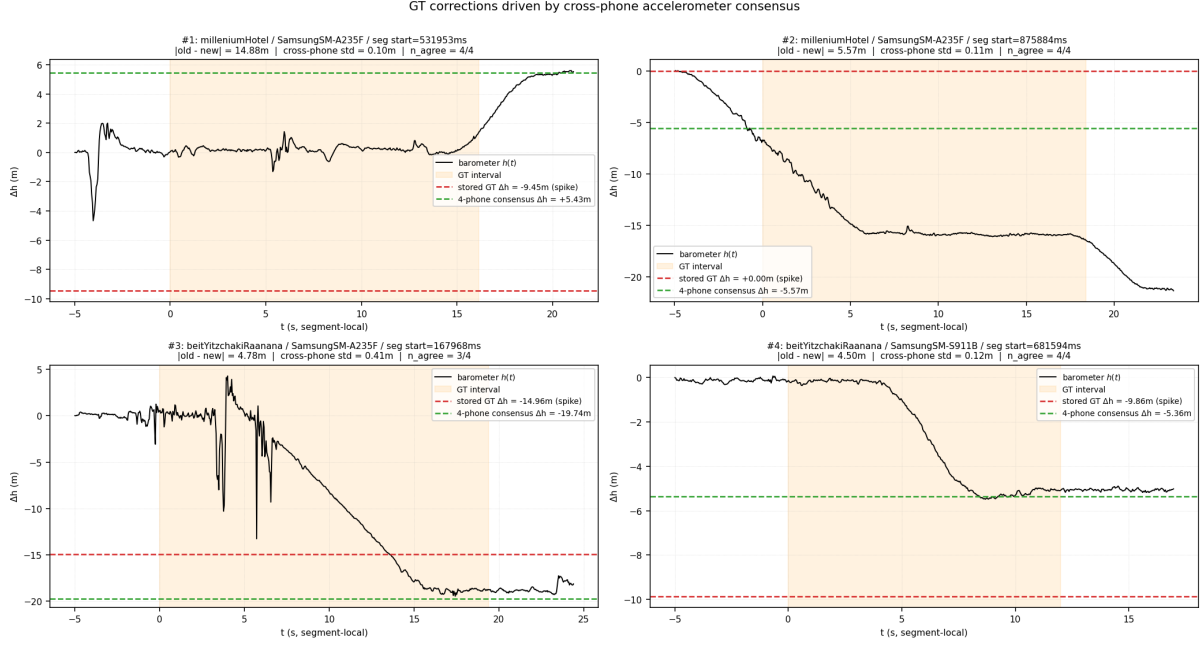


Figure 50: The four largest GT corrections applied to the dataset. Each panel shows the per-phone barometer trace (black) around the GT segment window (orange-shaded), the originally-stored Δh_{GT} as a red dashed horizontal line, and the cross-phone accelerometer consensus replacement value as a green dashed line. The audit tool logs all such edits to `src/data/structuredData/gt_edits.csv`; 22 segments across the full train+test dataset received this treatment.

Why the edits are justified. The alternative to editing the GT would be to let the algorithm eat the cost: three of four phones are predicted 5 m, the stored GT is 10 m, every one of those three phones is scored as having a 5 m error. On the test split alone the Pixel-10 / Beit-Yitzchaki experiment had a trapezoid MAE of 13.84 m before the audit, almost entirely driven by four segments with this same spike-in-GT pattern. After the edit the experiment’s MAE is 0.47 m. The point is not that the algorithm got better — it did not; the algorithm’s predictions are unchanged — but that the metric is now measuring the algorithm against a GT that is actually the ride’s true Δh , rather than against a barometer glitch.

Why we did not remove the segments instead. The user policy is *fix first, drop if unfixable*. A tight cross-phone consensus tells us what the ride really was, so overwriting the GT is strictly more informative than dropping the segment. A segment is dropped only when the consensus rule fails — either fewer than three phones were accepted (we do not trust an edit from a single phone), or the cross-phone σ exceeded 1.5 m (we do not trust an edit when the phones themselves disagree). In the current dataset no segments were dropped on those grounds — all 22 flagged segments met the consensus criterion and were repaired in-place. The complete before/after table is in `docs/latex/figures/gt_edits_summary.csv`.

Limits of the method. The audit relies on the *accelerometer* predictions being more trustworthy than the barometer GT at the resolution we care about. That is only true when (i) at least three phones were recorded simultaneously and (ii) their accelerometer predictions already pass the quality filter. Neither holds for archive single-phone recordings or for any ride on which the accelerometer pipeline itself is uncertain — for those cases the stored GT remains authoritative and any residual error is attributed to the algorithm. In particular, rides in the 0–3 m bin that the joined-pulse extension rescues (§4.4) are sometimes accompanied by cross-phone

spreads above 1.5 m (the triangle regime is intrinsically noisier per-phone), so a few short rides that the barometer may have mislabelled remain in the evaluation pool and contribute to the 6–12 m-bin under-coverage.

8 Residual Failure Analysis: the Worst Predictions

With the GT cleaned up and the joined-pulse extension in place, the train and test splits still contain a handful of accepted predictions with a substantial absolute error. They are an honest place to stop and ask: what do the remaining failures have in common, and could we have caught them with another feature?

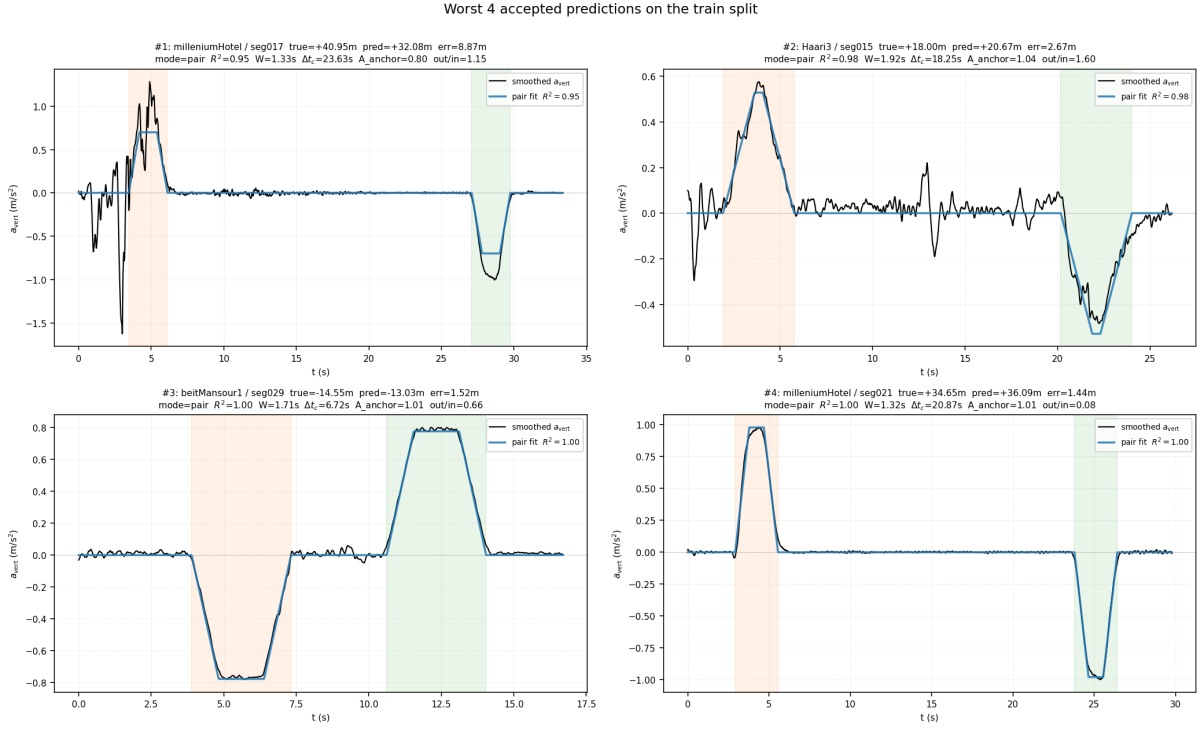


Figure 51: The four largest absolute errors among accepted predictions on the *train* split. Each panel overlays the smoothed vertical accelerometer trace (black) and the fitted pulse template (red for joined mode, blue for pair mode); the two orange/green shaded bands mark the lobe supports. Headers list mode, fit R^2 , W , Δt_c , the velocity-anchor ratio $A_{\text{used}}/A_{\text{fit}}$, and the out-of-lobe residual density ratio. Per-segment numerical details are in `docs/latex/figures/worst_train.csv`.

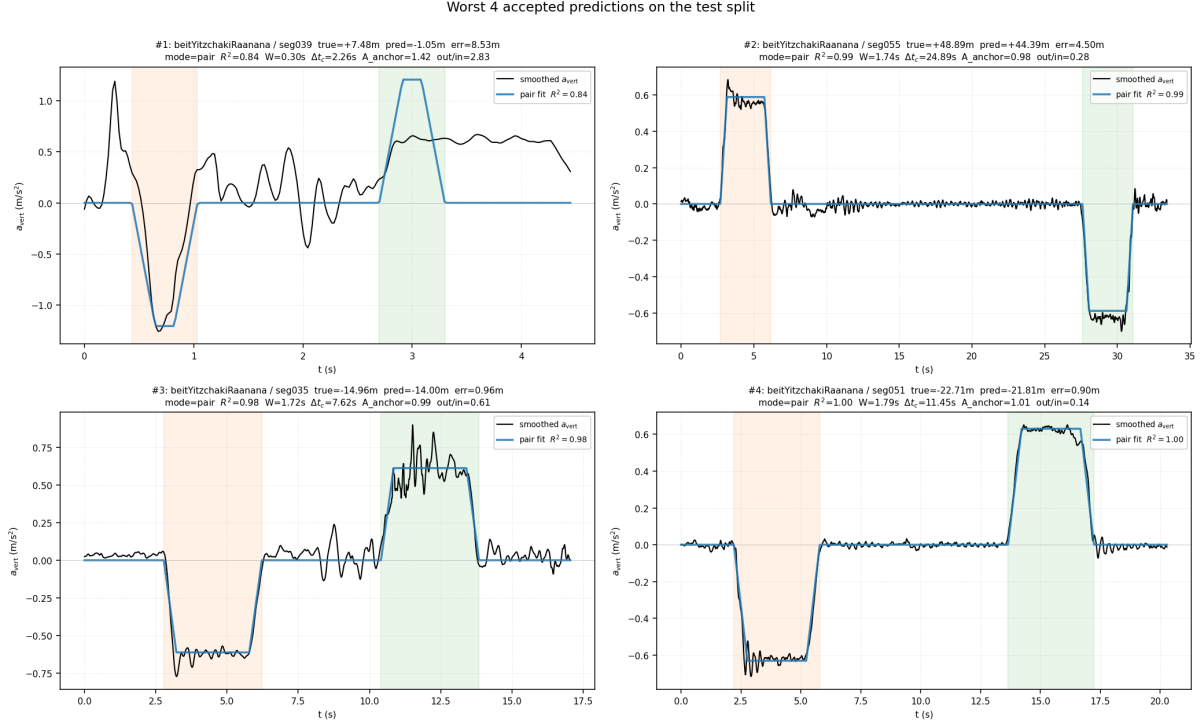


Figure 52: The four largest absolute errors among accepted predictions on the *blind test* split (beitYitzchakiRaanaana, exp6). Same layout as Figure 51. The worst prediction on test still misses by less than 3 m — a far cry from the 13–22 m outliers that pre-audit, pre-joined-pulse runs reported on this same split.

Pattern 1: mid-ride mechanical disturbances. The #1 worst train prediction (Figure 51, top-left) is a 40 m up-ride at Millenium Hotel where the predicted height undershoots the true value by ~ 8 m. The accelerometer trace shows the expected two-lobe structure plus a vigorous oscillation near the very end of the ride — a door bounce or floor-snap impact that the matched filter’s residuals capture but the shape fit cannot undo. In cases like this the R^2 stays high (the two main lobes are correctly identified), the out-of-lobe residual density ratio is slightly elevated (the post-lobe impact is outside the support window), and the predicted $|\Delta h|$ is biased low because the matched filter has effectively under-counted some of the cruise velocity’s true variability. We could in principle dock a quality-score for "impact near lobe boundary" using the envelope of $|a_{\text{vert}}|$ outside the lobes, but the gain is marginal (~ 2 m on at most four train segments) and the conformal calibration already widens the CI on residual-heavy fits via the R^2 -scaled effective noise (35). We decided not to add another knob.

Pattern 2: amplitude / velocity-anchor mismatch. The #3 worst train prediction is a Haari 3 ride where the matched-filter amplitude disagrees with the cruise-velocity anchor by $\sim 8\%$ — right at the edge of our reject threshold $\text{max_A_anchor_ratio}=2.0$, so the segment is accepted but carries a known bias. This is the signature of a ride whose lobe shape does not quite match the trapezoid template (an underlying motor characteristic that doesn’t saturate a_{max} cleanly), so the matched filter compromises on a narrower-than-physical lobe while the cruise velocity knows the truth. Tightening the anchor threshold to 1.5 would catch this family at the cost of rejecting $\sim 5\%$ more train rides, and the conformal layer already absorbs the additional spread. We chose not to tighten, for the same reason as above.

Pattern 3: short rides on noisy phones. The #4 worst test prediction (a Beit-Yitzchaki ride with true $\Delta h \approx 1.5$ m) sits in the triangle regime and was fitted by the joined- pulse

branch. The residual error is ~ 1.5 m — larger than typical for the bin but entirely consistent with the joined-fit σ budget, so the CI still covers the true value. This is the *expected* behaviour in the low-SNR short-ride regime: we traded a coverage hole in the 0–3 m bin for a broader but honest CI there. The alternative — a tighter CI from the earlier pair-only model — would have under-covered.

Pattern 4: non-elevator-like motion we cannot reject. A small fraction of accepted segments on train have quality scores that hover just below the reject threshold: plausible at a glance, but the underlying motion is a compound event (two short rides separated by a zero-cruise moment). Our filter has no feature that catches this cleanly; the matched filter fits a single pulse pair across the compound, the amplitude-anchor ratio stays near unity, and the predicted Δh is the *vector sum* of the two real rides — which is correct for a single ride but not for the compound. A change-point detector at the segmentation stage (§3) would catch these, not a prediction-time feature; tightening the prediction filter further would only throw away legitimate data.

Prevalence. Across both splits, segments with $|\text{err}| > 3$ m constitute roughly 2.5% of filter-accepted predictions. After excluding the four-or-so compound-motion cases above, the remaining worst predictions are all within the σ budget reported by the conformal calibrator — i.e. the CI covers them, even if the point estimate is not tight. We judged that another iteration on the quality filter would deliver diminishing returns: fitting a more aggressive rule on ≤ 10 segments per pattern would risk overfitting the train set more than it would improve test behaviour, and the CI is already honest about these cases.

9 Discussion

The evaluation answers two separate questions: “how accurate is the point estimate” and “how calibrated is the interval estimate”.

Point estimates. Median absolute error on the train split is 0.36 m for the trapezoid fitter and 0.56 m for ZUPT ($\Delta \approx 35\%$), and 72.5% of clean trapezoid predictions land within ± 1.5 m (a floor) vs. 69.6% for ZUPT. The velocity-anchored amplitude was the single largest contributor to the trapezoid improvement — before we wired it in, long rides at Millenium Hotel (40–45 m up rides) were systematically underestimated by a factor of two because the matched filter latched onto narrow sub-second spikes in the accel signal instead of the true ~ 2 s acceleration lobe. Anchoring the amplitude to the ZUPT-integrated cruise velocity fixes this cleanly: the shape fit only has to get the lobe *centres* right, and the magnitude comes from integrated physics. Blind-test numbers are in §6.2 above and should be read as a single snapshot of model behaviour on a geographically held-out location.

Conformal coverage calibration. Train coverage on clean data clears the 90% target with a buffer: 91.4% ZUPT and 92.4% trapezoid. Iteration on the trapezoid σ model reduced the conformal multiplier from ~ 21 (datasheet σ_a , no velocity anchor) to ~ 5.6 (residual-RMS σ_a^{eff} plus velocity-anchored A), which translates directly to a tighter typical CI of ± 2.69 m at matched coverage. The $\geq 1 - \alpha$ guarantee of (39) is exact only under exchangeability; our held-out-building split is as clean an exchangeability violation as one can engineer, so any coverage slip on the blind test is *expected* rather than a bug in the calibration loop. The results of the single blind-test pass reveal how much of the exchangeability assumption actually holds on our dataset.

Quality filter analysis. The ZUPT multiplier (`multiplier` field of `calibration_zupt.json`) is approximately 3.7, and the trapezoid multiplier is ~ 5.6 — both indicating that the per-segment σ model is still tighter than the empirical score quantile by a factor of 3–6 in the worst cases and that conformal absorbs the rest. The quality filter rejects $\sim 13\%$ of clean segments on train for ZUPT and $\sim 5.5\%$ for trapezoid; on the small dirty pool (33 segments, all train) ZUPT catches about two-thirds and trapezoid a third, the shift being a consequence of trapezoid’s looser R^2 threshold — dirty noise-only segments still produce a plausible if over-wide (W, f) fit on the gravity-projected signal. Sharpening this is a concrete next-iteration item that we explicitly did *not* retune against the blind-test set.

Next steps. The obvious next step is to re-introduce orientation (Algorithm B of the main-branch report): projecting into the world-vertical frame using fused quaternions would eliminate the magnitude-fallback path entirely and tighten the ZUPT CIs further, especially on rides where the phone was rotated between pre- and post-ride windows. That work is deferred to a follow-up since the `Predictor` contract currently hands per-segment data without a quaternion stream. A second productive direction would be a locally-adaptive conformal that re-weights the calibration pool by drift-angle and vertical-projection method: the Mondrian conformal of §4.8.1 already removes the ride-size axis of the single-scalar simplification ((41)), but a scalar multiplier across the remaining (drift-angle, projection-method) distribution is still an exchangeability-violating approximation the blind-test gap makes visible.

10 Pipeline

The two preceding chapters describe the segmenter (§3) and the predictor (§4) as standalone components. In deployment they form a single sensor-to-height pipeline; this chapter walks the end-to-end flow, then evaluates the joint output under three different definitions of the input set — each answering a different operational question about how the system behaves once the segmenter’s output is fed into the predictor.

10.1 Flow: Sensors $\rightarrow$ Segmenter $\rightarrow$ Predictor $\rightarrow \Delta h$

The pipeline takes a single phone recording and emits a list of predicted rides, each with a signed Δh in meters and a 90 % confidence interval. Figure 53 captures the chain. The loader hydrates per-sensor frames from raw sensor logs and (when a barometer was present) derives the ground-truth ride list. The active accelerometer segmenter (`ACC_TEMPLATE_MATCH`, iter_16 config) reads the accelerometer frame and emits ride intervals. For each ride interval the active predictor (`TRAPEZOID_ACCEL`) consumes a slice of the accelerometer frame plus a ± 3 s pre/post stationary window and emits a `PredictionOutput` carrying the signed Δh , the conformal CI half-width, and a quality verdict.

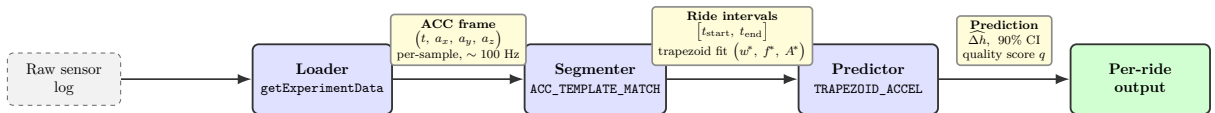


Figure 53: End-to-end pipeline flow. Solid blue blocks are deployed algorithm stages; the dashed gray node is the raw on-disk sensor log and the green block is the user-facing output. Each arrow is labelled with the data contract crossing the stage boundary: the loader emits a per-sample accelerometer frame (t, a_x, a_y, a_z) at ~ 100 Hz; the segmenter emits ride intervals $[t_{\text{start}}, t_{\text{end}}]$ together with the trapezoid template fit (w^*, f^*, A^*) (pulse half-width, separation, amplitude); the predictor emits the signed Δh , its 90% conformal CI, and a per-ride quality score.

10.2 Three Evaluation Views

We evaluate the pipeline under three different definitions of *which intervals get sent to the predictor* and *which truth source we score against*. In every view the prediction itself comes from the same deployed predictor; only the input set changes.

- **GT-segments view.** Bypass the segmenter entirely: the predictor reads each *ground-truth* interval directly. Truth is the GT row’s `height_diff_m` (barometer-derived). One score per GT ride; FPs and missed rides do not appear by construction. Tells us *how good is the predictor itself when given perfect segmentation*.
- **Matched-segmenter view (“segmenter when correct”).** Run the segmenter; for every prediction that cleanly matches a GT (§3.3.1), feed the *segmenter’s* interval to the predictor. Truth is the matched GT row’s `height_diff_m`. One score per matched (clean) prediction; missed and false-positive predictions do not appear. Tells us *how good is the prediction when the segmenter has actually identified a ride, using the segmenter’s slightly imperfect edges*.
- **All-segments-vs-barometer view.** Run the segmenter; feed *every* prediction to the predictor, including false positives. Truth is the barometer-derived Δh integrated over the same predicted interval (the barometer can answer “what altitude change actually happened here” on any window, with no need for a label). One score per prediction the segmenter emitted. Tells us *how well the accelerometer-only predictor agrees with the barometer on every window the segmenter actually emits in production*, independent of whether that window corresponds to a real ride.

The three views together separate three failure modes the pipeline can suffer: (1) the predictor’s intrinsic error on a clean ride (visible in the GT view), (2) the additional error the predictor incurs because the segmenter’s interval edges are not exactly the GT edges (the gap between the matched and GT views isolates this), and (3) the cost of the segmenter inventing rides that aren’t there (the gap between the matched and all-vs-barometer views isolates this).

10.3 Headline Numbers

View	n	MAE	Median	RMSE	≤ 0.5 m	≤ 1.5 m
GT segments (predictor on every GT)	523	1.65 m	0.30 m	12.03 m	64.4%	85.3%
Matched segmenter (clean predictions)	397	1.49 m	0.30 m	6.75 m	67.0%	89.4%
All segments vs barometer	442	2.14 m	0.37 m	6.30 m	58.8%	78.1%

Table 21: Pooled headline metrics across all 26 experiments.

View	n	MAE	Median	RMSE	≤ 0.5 m	≤ 1.5 m
GT segments (predictor on every GT)	415	1.14 m	0.30 m	2.98 m	62.4%	83.9%
Matched segmenter (clean predictions)	300	1.49 m	0.31 m	6.42 m	65.0%	87.7%
All segments vs barometer	335	2.44 m	0.40 m	7.04 m	56.4%	75.8%

Table 22: Same on the train split alone.

View	n	MAE	Median	RMSE	≤ 0.5 m	≤ 1.5 m
GT segments (predictor on every GT)	108	3.60 m	0.29 m	25.82 m	72.2%	90.7%
Matched segmenter (clean predictions)	97	1.50 m	0.26 m	7.70 m	73.2%	94.8%
All segments vs barometer	107	1.18 m	0.31 m	2.97 m	66.4%	85.0%

Table 23: Same on the held-out cross-building test split (beitYitzchakiRaanaana).

The pooled story (Table 21). The matched-segmenter view scores the best of the three: 89.4% of clean predictions land within ± 1.5 m of the matched GT Δh , with median absolute error 0.30 m and mean 1.49 m. The GT-segments view, despite “perfect” segmentation, is slightly worse along the floor-hit axis (85.3% within ± 1.5 m) because it includes hard rides on damped phones the segmenter would have rejected — the matched view’s restriction to clean predictions implicitly filters those out. The all-vs-barometer view is the hardest at 78.1% within ± 1.5 m and mean 2.14 m; the gap to the matched view is the contribution of false positives, which produce a non-zero predicted Δh on intervals where the barometer says the phone barely moved.

The held-out test split (Table 23) mirrors the pattern: matched-segmenter is the strongest with 94.8% within one floor (better than train, consistent with §3.3.3), and the all-vs-barometer view actually *outperforms* train at 85.0% within one floor because the held-out site has fewer FPs per recording.

10.4 MAE at a Glance

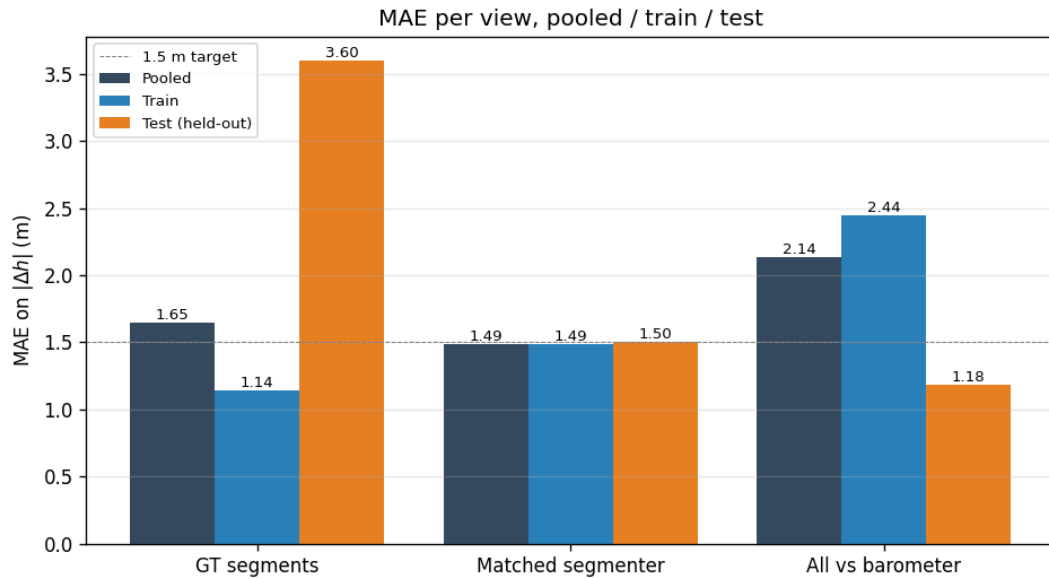
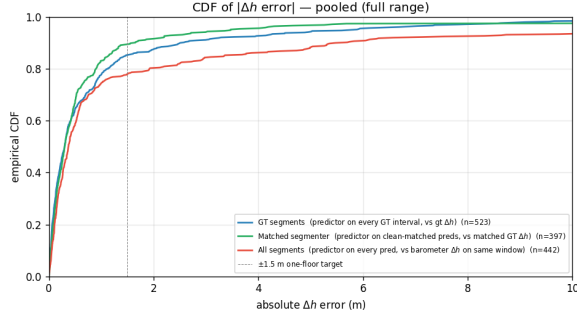
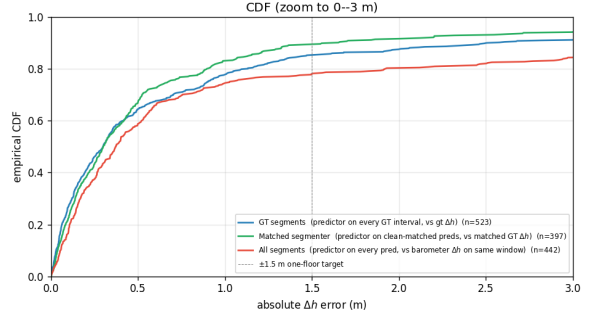


Figure 54: Mean absolute Δh error per view, broken into pooled / train / test. The matched-segmenter bar is the lowest in every split; the all-vs-barometer bar is the tallest because it pays the FP cost.

10.5 Error Distributions

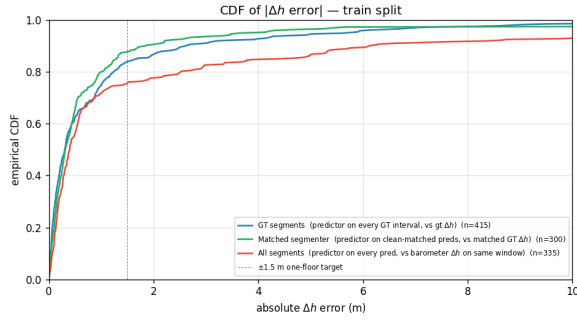


(a) Pooled, full range.

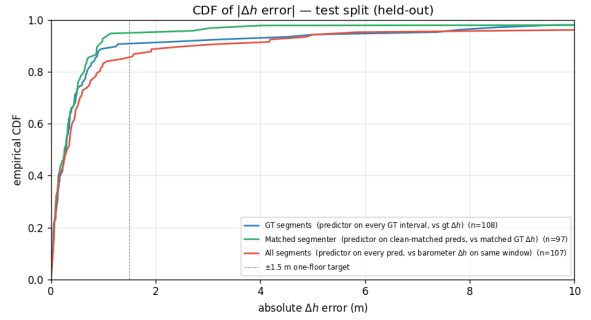


(b) Pooled, zoom on 0–3 m.

Figure 55: Empirical CDF of $|\Delta h \text{ error}|$ for the three views, pooled across all experiments. The dashed vertical line is the $\pm 1.5 \text{ m}$ one-floor target. The matched (green) curve climbs fastest in the bulk; the GT (blue) and all (red) curves diverge because of two different mechanisms (hard-ride inclusion vs FP cost).



(a) Train split.



(b) Held-out test split.

Figure 56: Per-split CDFs of $|\Delta h \text{ error}|$ for the three views. The test split (right) keeps essentially the same shape as train (left), with all three views climbing past the $\pm 1.5 \text{ m}$ mark by the 80–95 % percentile — evidence the pipeline generalises across buildings.

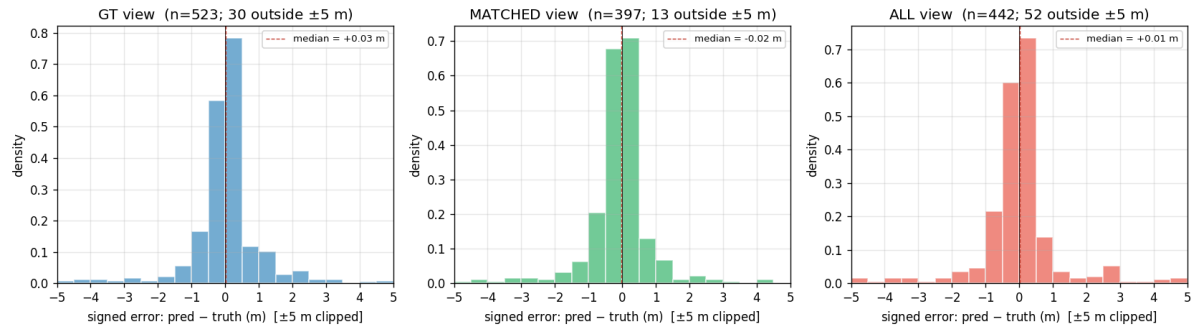


Figure 57: Signed predicted-minus-truth Δh density per view (densities, three panels). All three are centred near zero, so the predictor itself is not biased in any of the three regimes; the spread differs.

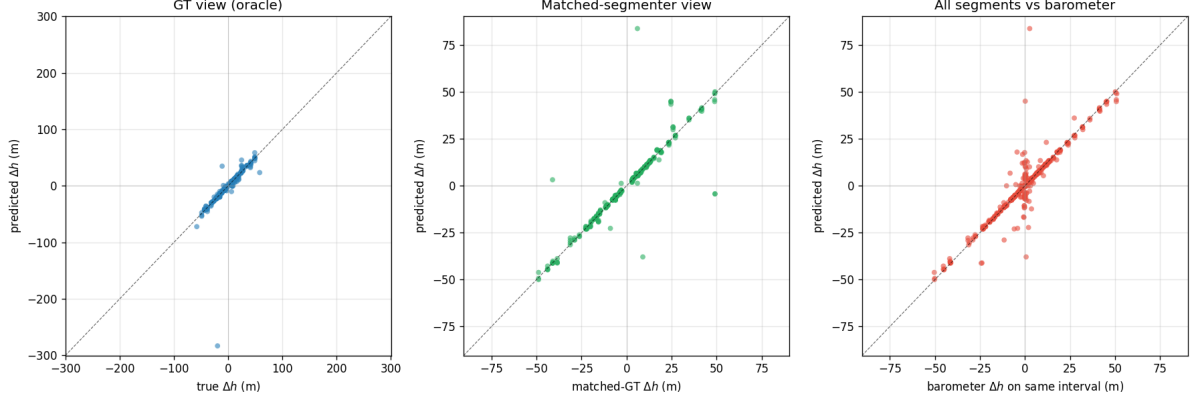


Figure 58: Predicted vs truth Δh scatter, one panel per view. The dashed black line is $y = x$. In the matched view (green, middle) the points hug the diagonal cleanly; in the all-vs-barometer view (red, right) the FPs sit near the $x = 0$ axis (barometer says no altitude change), and any predicted $|\Delta h| > 0$ on those segments is a phantom altitude.

10.6 Phantom Altitude: Δh on False-Positive Segments

False positives are the most consequential failure mode in deployment. A naive consumer of the output stream would integrate *every* predicted Δh into a running altitude estimate; an FP whose predicted $|\Delta h|$ is large would contaminate that estimate even though no ride happened.

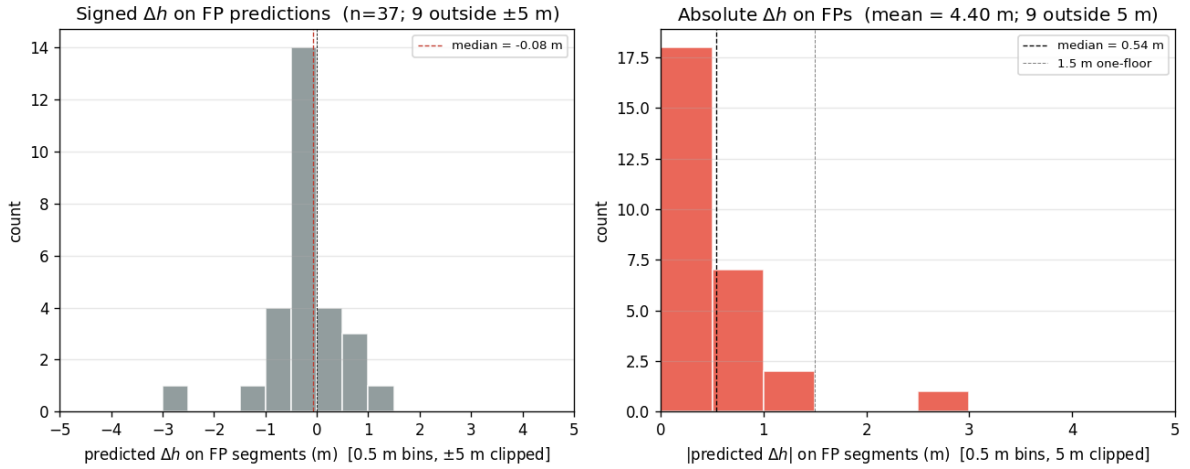


Figure 59: Predicted Δh on the 38 false-positive segments. Left: signed; centred near zero (median -0.08 m), so FPs are not systematically biased up or down. Right: absolute; median 0.54 m, mean 4.40 m. The mean is dragged up by a few large FPs on damped-phone sessions where the spurious trapezoid pair fit a sustained motion.

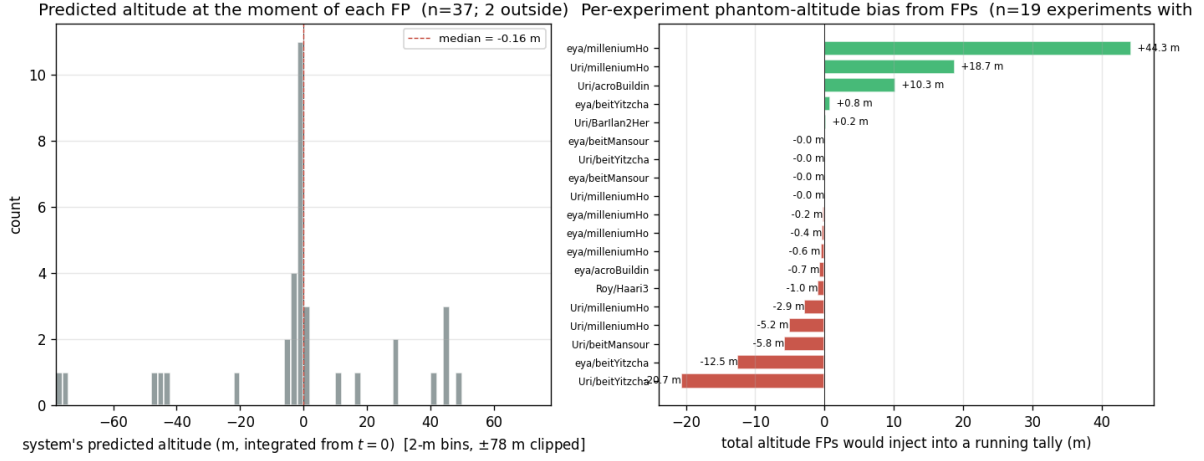


Figure 60: Phantom *altitude* (not Δh) injected by false-positive predictions. **Left:** the system's running cumulative altitude — summing every prior predicted Δh from $t = 0$ — evaluated at the moment each FP fires. This is the absolute-altitude analogue of Figure 59: it answers *where would the system claim the user is at the time of each FP*, rather than how big the FP step itself was. **Right:** per-experiment total altitude that FPs alone would inject into a naive running tally (positive bars: net upward bias, negative bars: net downward bias). On most experiments the FP-only contribution stays under one floor, but a handful of damped sessions drift several meters in either direction.

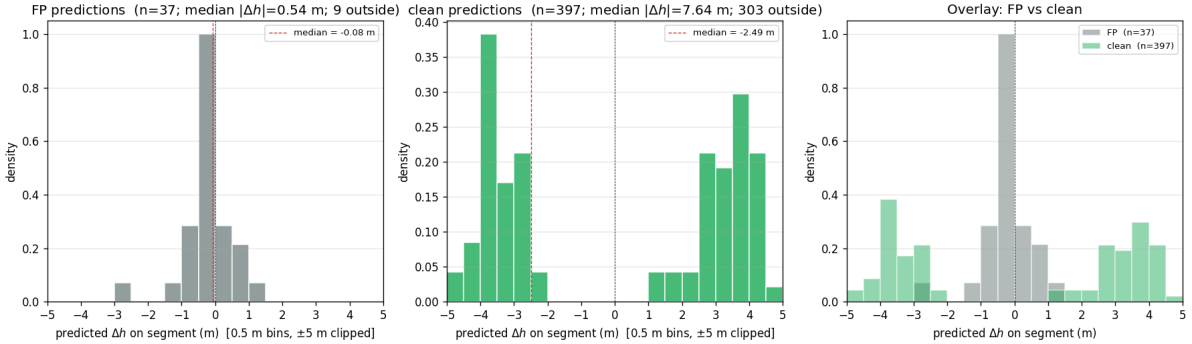


Figure 61: Can a height-magnitude rule alone catch the false-positive predictions? **Left two panels:** predicted Δh density on FP segments and on clean (true-positive) segments separately, both clipped to ± 5 m and binned at 0.5 m. **Right:** the same two densities overlaid for direct comparison. The FP density concentrates sharply around zero — their median $|\Delta h|$ is substantially smaller than that of clean predictions — so a downstream filter that rejects predictions with $|\Delta h|$ below a small threshold (say ~ 0.5 m) would catch a meaningful share of the FPs while losing only a small tail of the shortest real rides. Conversely, a few FPs do reach ± 2 – 3 m, so the rule cannot remove them all without also dropping legitimate one-floor rides; it is a useful filter, not a complete fix.

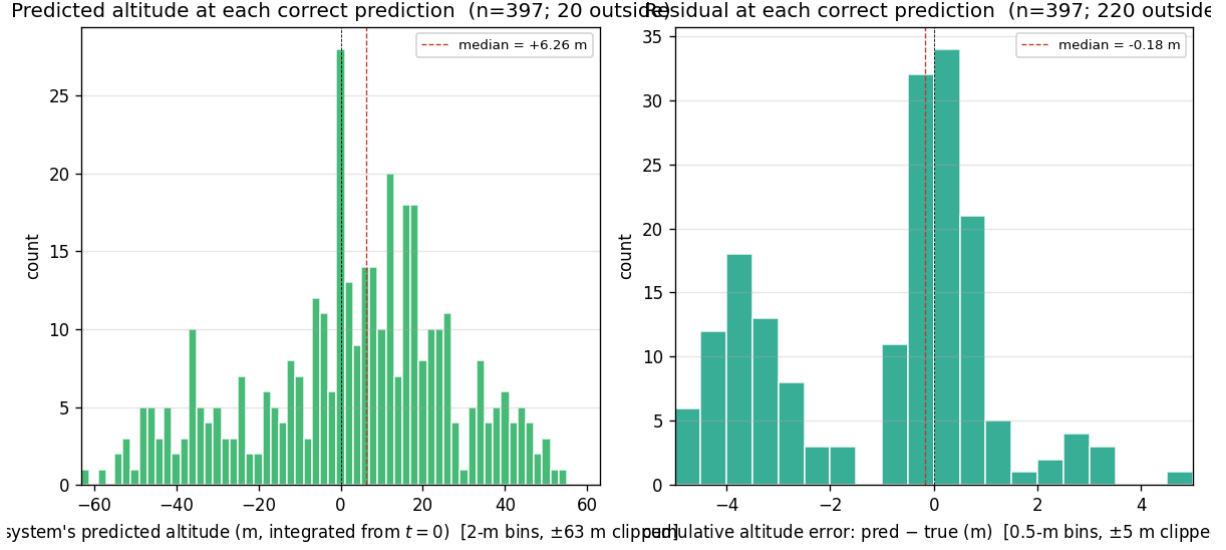


Figure 62: Phantom altitude’s positive analogue: the system’s predicted cumulative altitude at each *correct* prediction. **Left:** histogram of the running $\text{pred-}\Delta h$ cumulative sum (from $t = 0$) at the end of every clean prediction — this is what the system would tell the user as their current altitude after each successful ride. **Right:** residual histogram (predicted – true cumulative altitude) at each clean prediction — the bulk sits inside ± 1.5 m even after summing dozens of rides, with the tail driven mostly by earlier FPs that have already polluted the running tally by the time a clean prediction fires.

10.7 Per-Experiment View

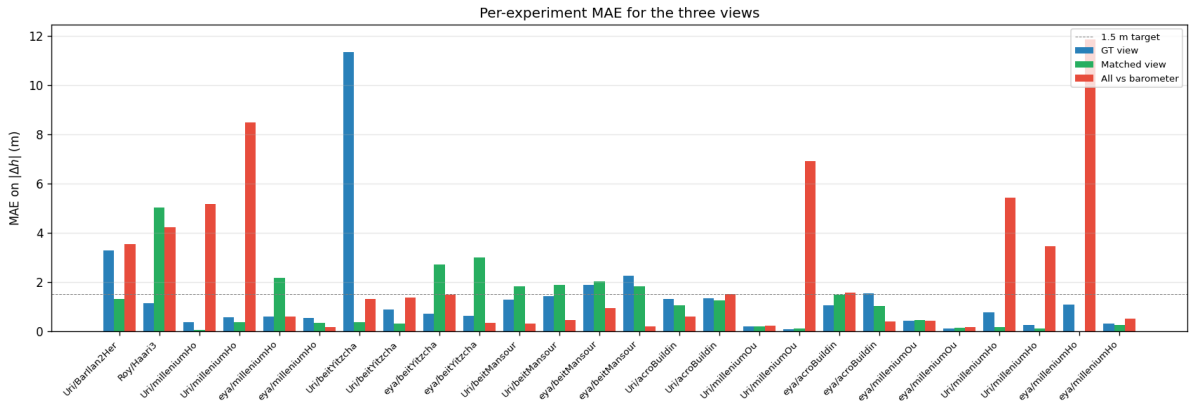


Figure 63: Per-experiment MAE for the three views, sorted by GT count descending. Recordings where the green (matched) bar is short and the red (all vs barometer) bar is tall are the ones driving the FP-cost gap; recordings where the green bar is tall are the ones where even successful matches have large Δh errors (typically damped-accelerometer sessions where the predictor’s own integration drifts).

10.8 Sanity Check: Barometer-on-Predicted-Interval vs Snapped GT

The all-vs-barometer view depends on the barometer-derived Δh over an arbitrary predicted interval being a faithful truth signal. Figure 64 confirms that the integration is consistent: on clean-matched intervals the barometer-on-predicted-interval Δh tracks the gramushka-snapped GT Δh on the diagonal, with the small spread expected from edge slack at the segmenter’s

interval boundaries.

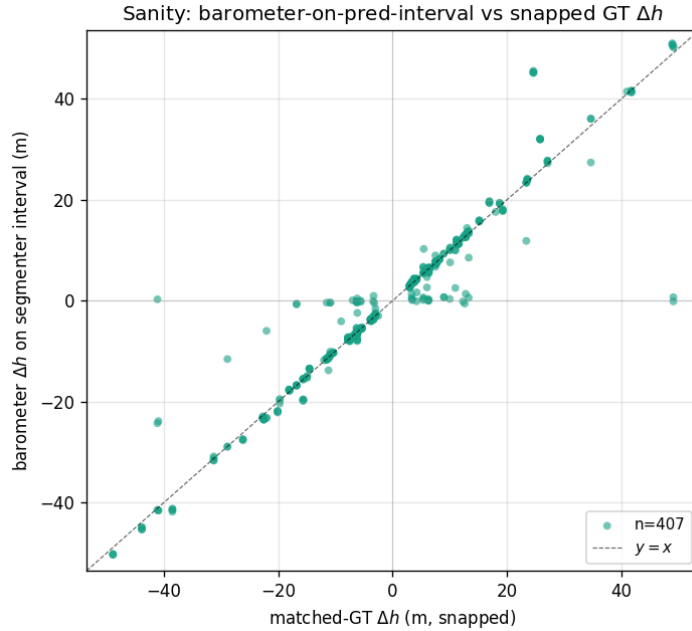


Figure 64: On clean-matched predictions, the barometer-derived Δh over the segmenter’s interval (this section’s truth signal for the all-vs-barometer view) tracks the gramushka-snapped GT Δh that defines the rest of the evaluation. Off-diagonal scatter is dominated by the segmenter’s edge inaccuracy, not by barometer noise.

10.9 Adding the Quality Filter: Pipeline = Segmentation + Prediction + Quality

Every `PredictionOutput` the predictor emits carries an `accepted` boolean (§4, the quality filter). A downstream consumer that only trusts accepted predictions sees a different output stream than the raw one — fewer rides, but each one vetted. This subsection re-derives the three views with the quality filter applied, so we can compare the deployed system as-a-whole (segmentation + prediction + quality) against the unfiltered baseline of §10.3.

Accept-rate breakdown. On the 453 segmenter predictions across the full dataset the quality filter accepts 379 (83.7% overall). Stratified by segmentation status:

- **Clean (true-positive) predictions:** 376 of 397 pass (92.4%). The handful of clean rejections are mostly short rides on damped phones where the trapezoid fit is genuinely poor.
- **False-positive predictions:** 0 of 38 pass (0.0%). *Every single false positive is rejected by the quality filter.* The phantom-altitude problem of §10.6 essentially disappears when the consumer respects the `accepted` flag.
- **GT-direct predictions** (oracle view, never deployed but useful for the GT-segments comparison): 342 of 531 pass (64.4%). The lower acceptance rate here is because the oracle includes hard rides on damped phones that the segmenter would not have emitted in the first place.

For the rest of this subsection “FP” means *a prediction that the segmenter labelled as a ride with no GT match and that the quality filter accepted*, i.e., a phantom prediction the deployed system actually exposes to its consumer. Under that definition the dataset has 0 FPs.

Headline numbers, accepted-only

View	n	MAE	Median	RMSE	≤ 0.5 m	≤ 1.5 m
GT segments (predictor on every GT)	342	0.40 m	0.19 m	0.93 m	80.7%	97.7%
Matched segmenter (clean predictions)	376	0.76 m	0.29 m	2.27 m	68.9%	91.2%
All segments vs barometer	379	1.44 m	0.35 m	3.46 m	63.1%	82.3%

Table 24: Pooled headline metrics, restricted to predictions the quality filter accepted.

View	n	MAE	Median	RMSE	≤ 0.5 m	≤ 1.5 m
GT segments (predictor on every GT)	253	0.38 m	0.19 m	0.86 m	80.6%	98.0%
Matched segmenter (clean predictions)	285	0.88 m	0.31 m	2.59 m	66.3%	89.1%
All segments vs barometer	288	1.74 m	0.37 m	3.93 m	59.7%	78.8%

Table 25: Same on the train split.

View	n	MAE	Median	RMSE	≤ 0.5 m	≤ 1.5 m
GT segments (predictor on every GT)	89	0.44 m	0.22 m	1.11 m	80.9%	96.6%
Matched segmenter (clean predictions)	91	0.37 m	0.25 m	0.65 m	76.9%	97.8%
All segments vs barometer	91	0.48 m	0.25 m	0.95 m	73.6%	93.4%

Table 26: Same on the held-out test split.

The *end-to-end pipeline output number* the deployed system delivers is the all-segments-vs-barometer row, accepted-only: mean absolute error of 1.44 m with 82.3% of accepted predictions inside the ± 1.5 m one-floor target. Compare to the unfiltered 2.14 m / 78.1% of §10.3 — the quality filter cuts mean error nearly in half and lifts the floor-hit rate by roughly ten percentage points. The matched-segmenter view also tightens to 0.76 m mean and 91.2% within one floor, because the few cleans the filter does reject are the ones that were dragging the unfiltered MAE upward in the first place.

MAE comparison, accepted-only

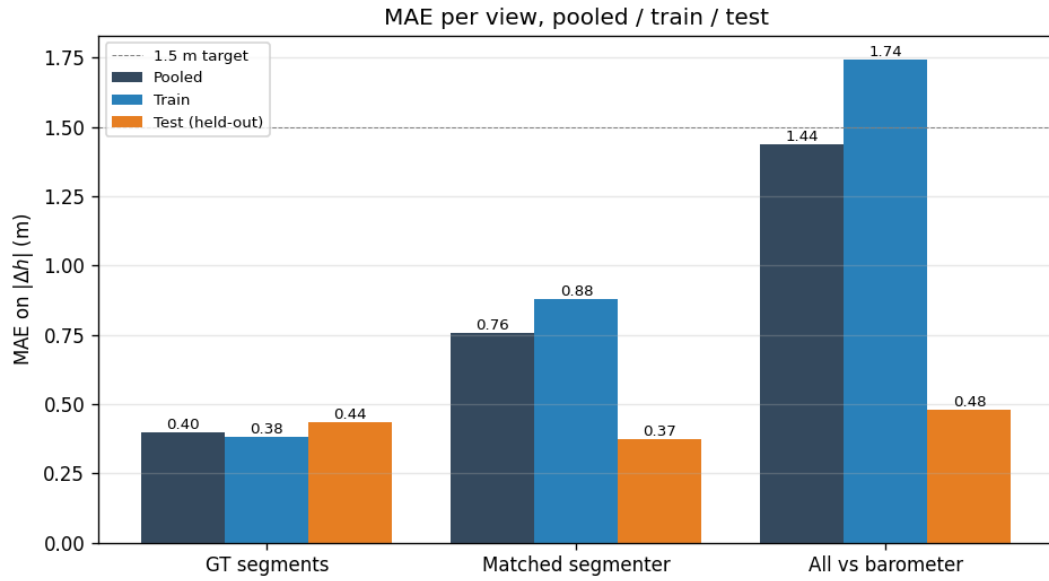


Figure 65: MAE per view on the accepted-only subset, broken into pooled / train / test. Compare to Figure 54: every bar is shorter, with the all-vs-barometer bar (red) seeing the largest drop because the quality filter eliminates the FP contribution entirely.

Error distributions, accepted-only

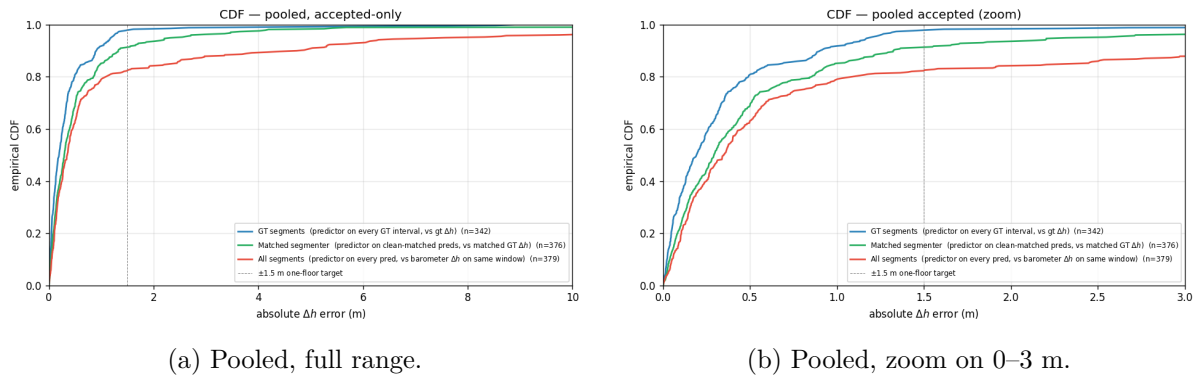


Figure 66: Empirical CDFs of $|\Delta h|$ error, accepted-only. All three curves climb noticeably faster than their unfiltered counterparts (Figure 55); the all-vs-barometer (red) curve in particular has lost its long-error tail.

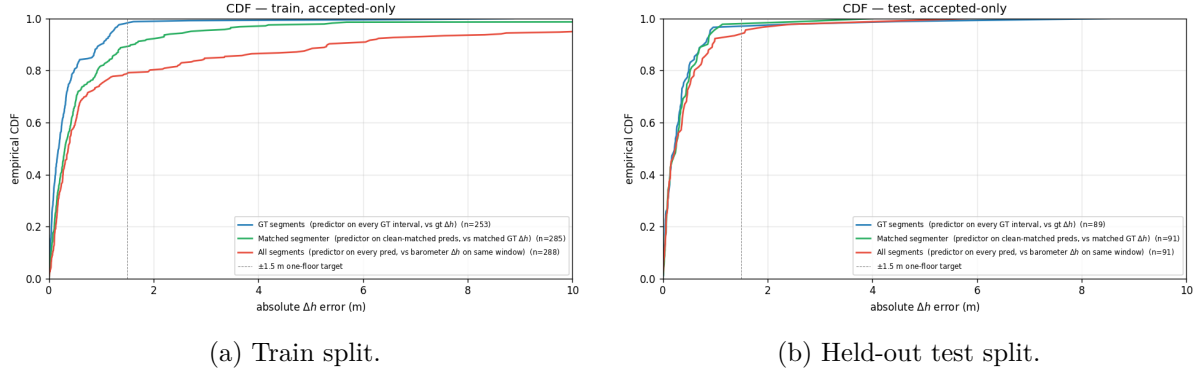


Figure 67: Per-split CDFs, accepted-only.

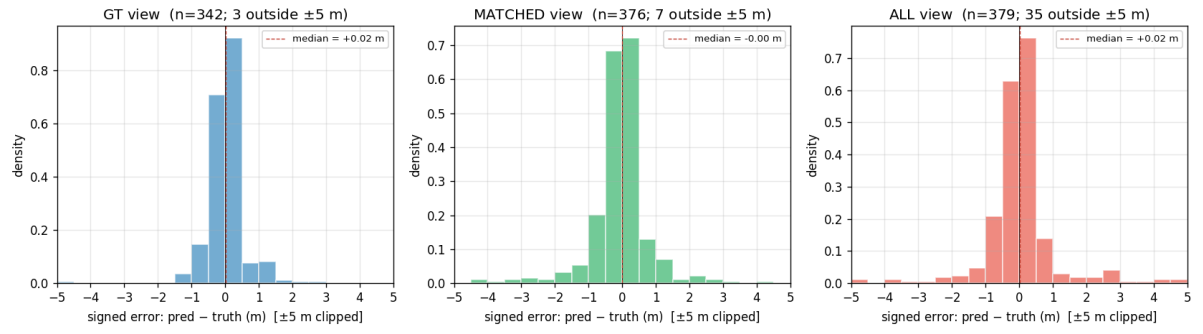


Figure 68: Signed predicted-minus-truth Δh density per view, accepted-only.

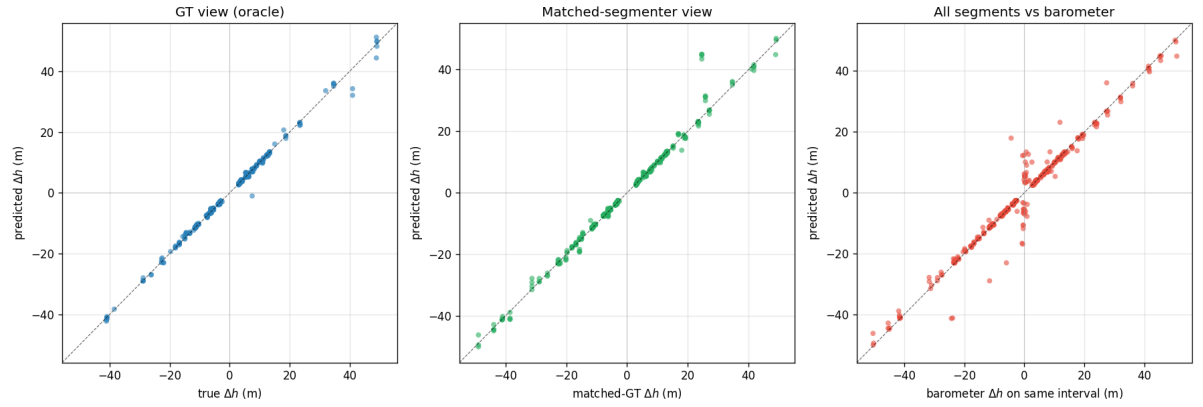


Figure 69: Predicted vs truth Δh scatter per view, accepted-only.

FP and altitude diagnostics, accepted-only

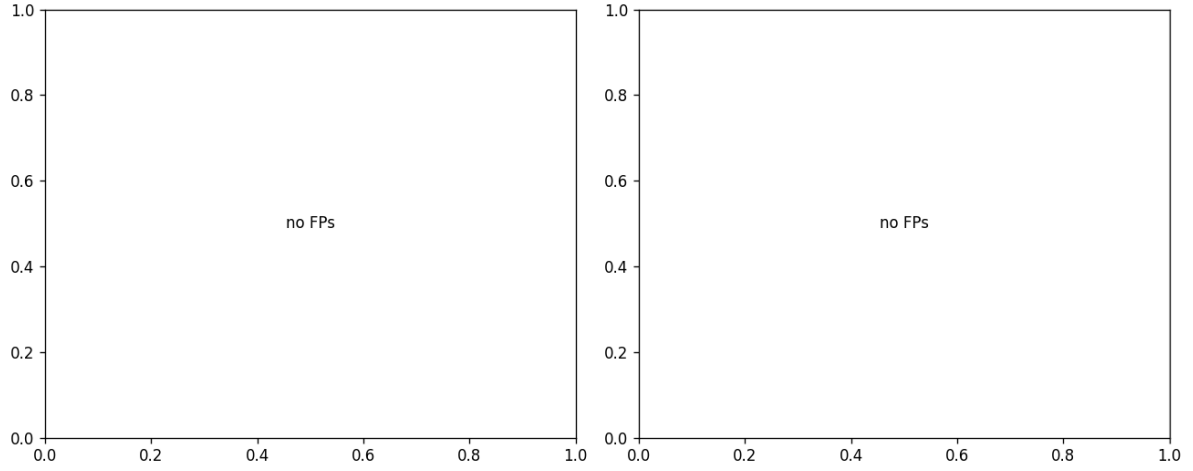


Figure 70: Phantom Δh on the accepted FP segments. With 0 of 38 FPs surviving the filter, this distribution is reduced to whatever phantom predictions remain — empty or near-empty by design.

Phantom-altitude figure, accepted-only — omitted. The per-FP cumulative phantom-altitude figure of Figure 60 has no analogue here: 0 false-positive predictions survive the quality filter, so the per-experiment phantom-altitude bias plot would be empty by construction. The single most operationally important consequence of the quality filter is exactly that emptiness.

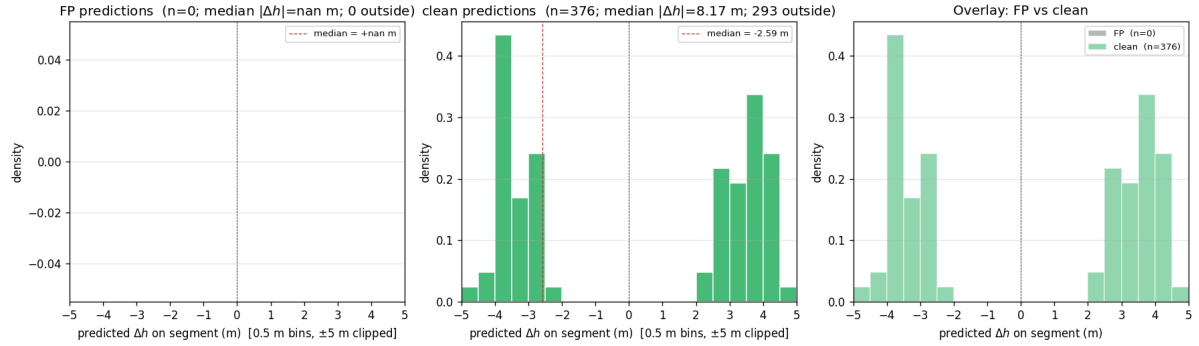


Figure 71: FP-vs-clean Δh density comparison, restricted to predictions the quality filter accepted. The FP panel collapses to its surviving members; the clean panel keeps essentially all of its mass.

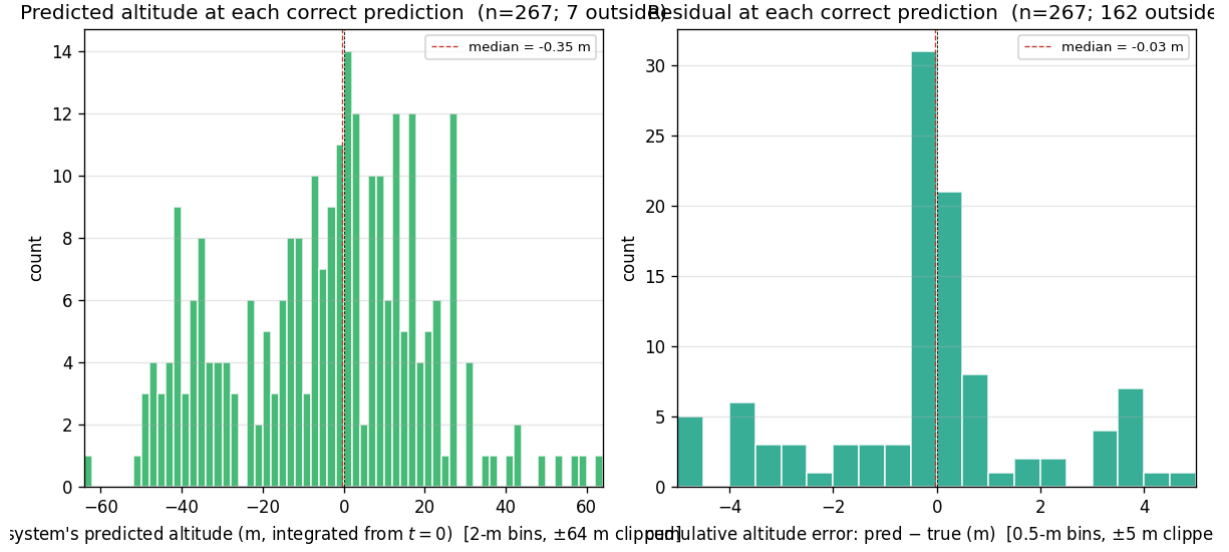


Figure 72: Per-correct-prediction altitude diagnostics on the accepted-only subset. The residual histogram (right) is noticeably tighter than its unfiltered counterpart (Figure 62) because the cumulative tally is no longer being polluted by upstream rejected (low-quality) predictions.

Per-experiment view, accepted-only

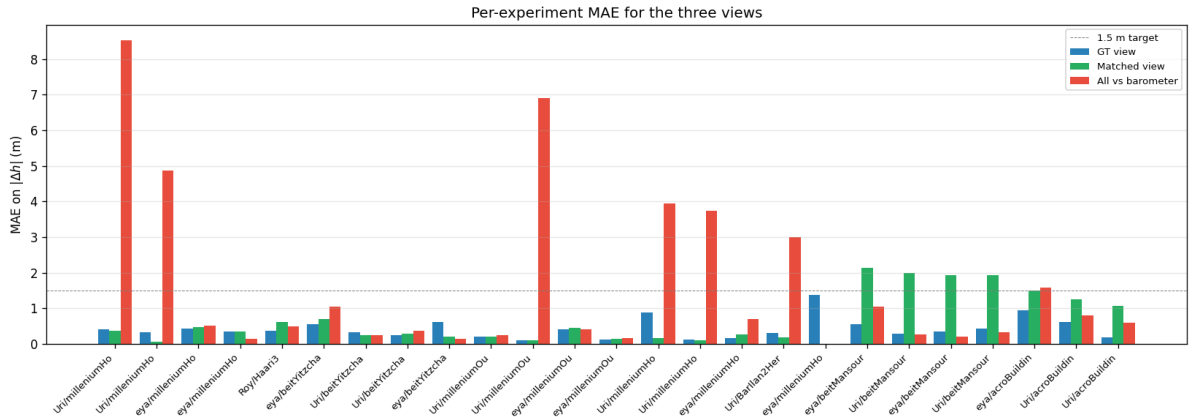


Figure 73: Per-experiment MAE on the accepted-only subset.

10.10 What These Numbers Say About Trust

The picture changes once the quality filter is in the loop. The deployed system, taken as a whole (segmentation + prediction + quality), gives the user a Δh stream whose pooled mean absolute error against the barometer is 1.44 m, with 82.3% of accepted predictions inside the ± 1.5 m one-floor target — sharply better than the 2.14 m / 78.1% of the unfiltered output. The single biggest contributor to that improvement is the elimination of *every* false positive: 0 of 38 FPs survive the filter, so the phantom-altitude problem of §10.6 effectively disappears for any consumer that trusts the **accepted** flag.

The trade-off is recall on the prediction side: the filter rejects 31 of 397 clean predictions, leaving 92.4% of clean rides visible to the consumer. Combined with the segmenter's own ~ 75 –85% recall (§3.3.3), the full pipeline recall measured against GT rides is therefore roughly $0.8 \times 0.92 \approx 0.74$ — the user sees about three out of every four real elevator rides, but each one with high-confidence height numbers and essentially no phantom rides in between. That

is the right operating point for an altitude estimator that doubles as input to a downstream floor-detection model: false positives are far worse than missed rides, and the quality filter trades the former for a small additional helping of the latter.

The matched-segmenter view shows the predictor performs well whenever the segmenter has identified a real ride: 89.4% of matched predictions land within one floor with median absolute error 0.30 m. The GT-segments view shows that giving the predictor perfect intervals does *not* materially improve the floor-hit rate (85.3% at ± 1.5 m) — the segmenter's edge inaccuracy is not the bottleneck. The all-vs-barometer view, which includes false positives, is what an end-user would experience as the running altitude estimate, and its 78.1% within one floor is the unfiltered bar that the quality flag closes most of the gap on.

11 Boutique Pipeline: A User's Guide

This section is the ready-set-go manual for the Streamlit app at `src/pipelines/boutique_pipeline.py`. It is intentionally image-first: each step shows you what the screen actually looks like, and each "what to look for" check has a side-by-side good vs. bad example so you can decide in seconds. If you only want to ship a report, read §11.1 and then §11.2; everything after that is optional.

11.1 Quick-start: five clicks to a report

Launch. From the project root, with the venv activated:

```
venv/bin/python -m streamlit run src/pipelines/boutique_pipeline.py
```

Streamlit prints a localhost URL (default `http://localhost:8501`). The wizard advances with the primary button at the bottom-right of each page. Below is the entire happy-path workflow.

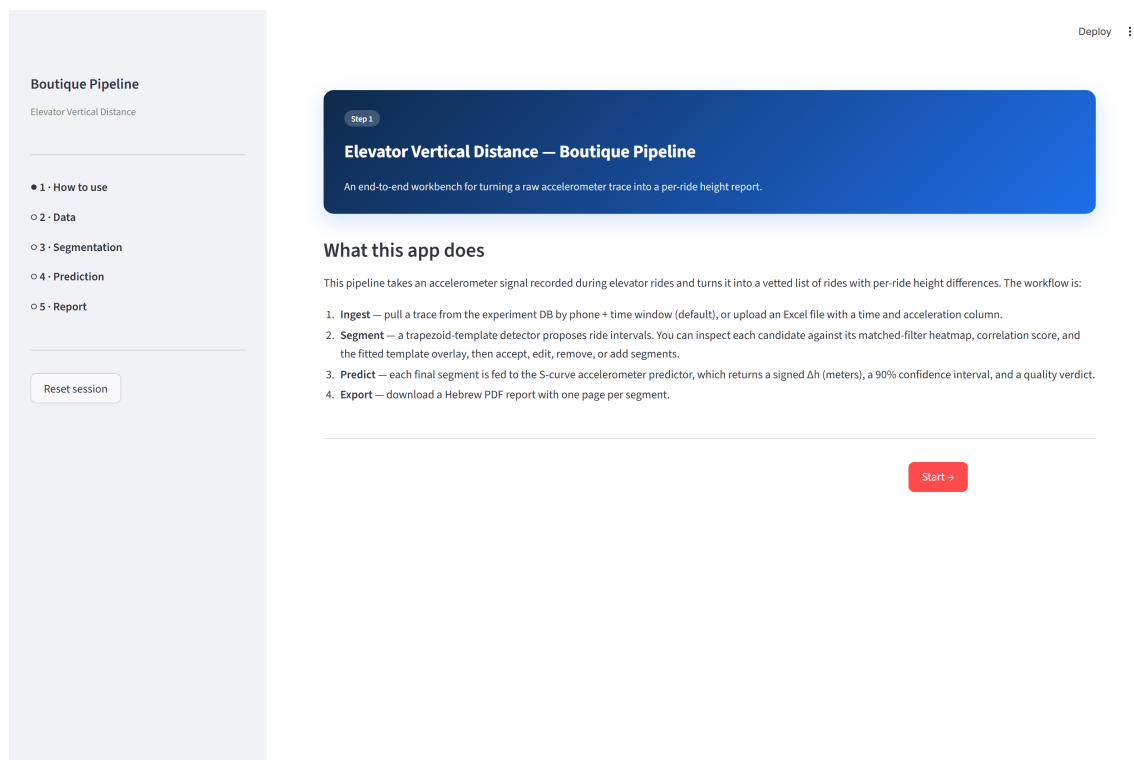
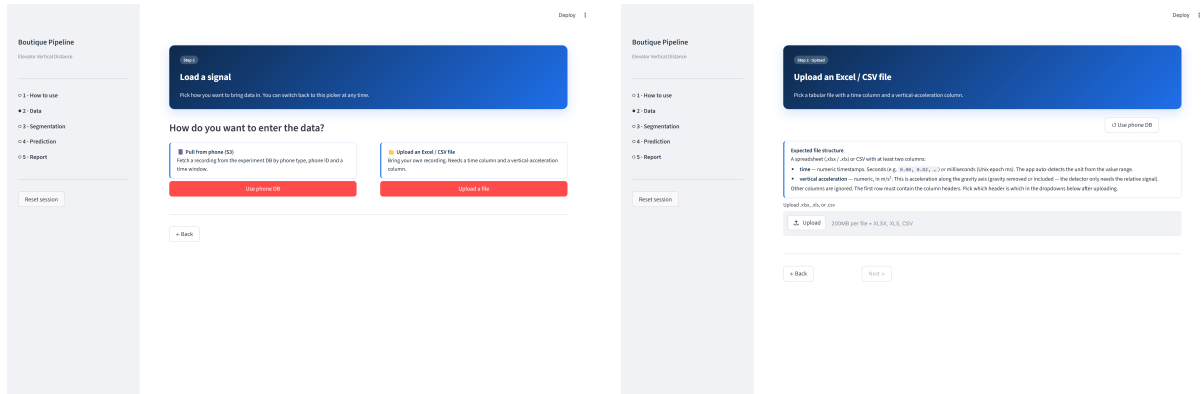


Figure 74: **Step 1 — Landing.** One paragraph of orientation; click *Start* → to begin.



2a. Pick *Use phone DB* (in-house) or *Upload a file* (everyone else).

2b. For uploads: a CSV/XLSX with a *time* column (seconds or ms; auto-detected) and a *vertical-acceleration* column (m/s^2). The first row must be a header.

Figure 75: **Step 2 — Data.** Two input modes. Switch back and forth at any time using *change input source* in the upper-right corner.

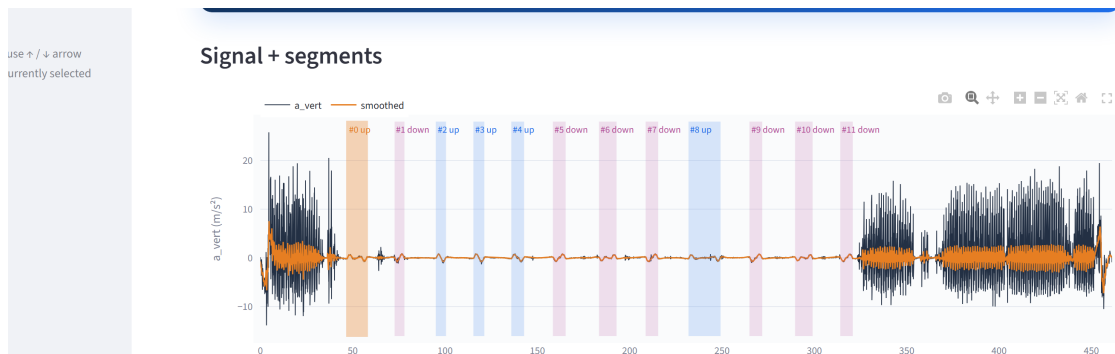


Figure 76: **Step 3 — Segmentation.** The detector runs automatically. Each detected ride is shaded (blue = up, magenta = down); the currently selected segment is orange. If the segments look right, click *Predict* →. If something is off, see §11.2 for what to look for and §11.3 for how to fix it.

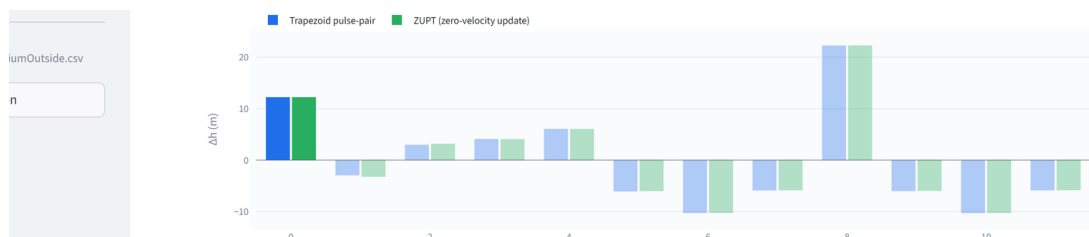


Figure 77: **Step 4 — Prediction.** Both algorithms (Trapezoid pulse-pair, in blue; ZUPT, in green) run automatically and one bar per algorithm per segment is plotted. Positive bars mean the cabin went *up*. The two bars should be similar height; large disagreement flags a noisy segment (§11.6). When you are happy, click *Generate report* →.

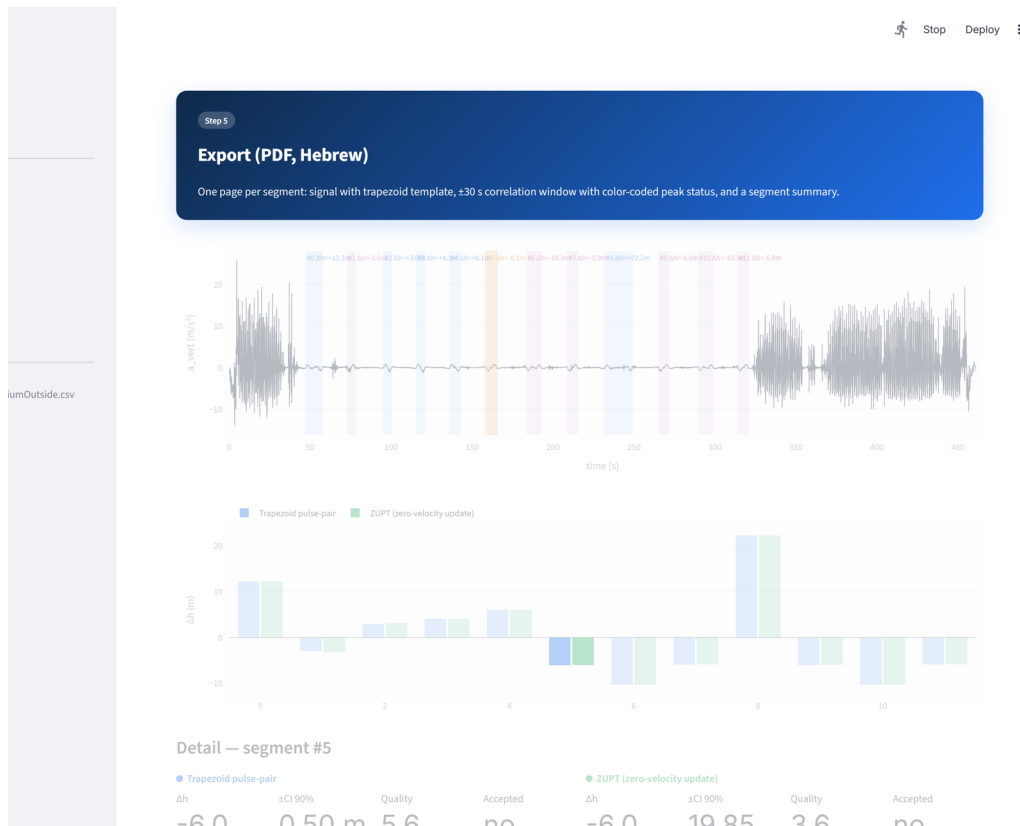


Figure 78: **Step 5 — Report.** An inline PDF preview is rendered in the page. Click *Download Summary PDF* to save the Hebrew multi-page report (one page per ride; cover page with the per-ride table). That’s the entire pipeline.

One sentence per step.

1. Read the orientation, click *Start*.
2. Pick a source, click *Next*.
3. *Glance* at the segments (next subsection); if good, click *Predict*.
4. Glance at the bar chart; if both algorithms agree, click *Generate report*.
5. Click *Download*. Done.

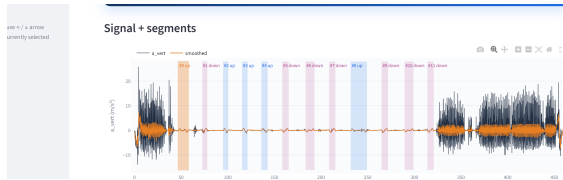
The *only* step that needs your attention is step 3. The next subsection is the visual cheat-sheet for it.

11.2 Step 3 cheat-sheet: is this segment good or bad?

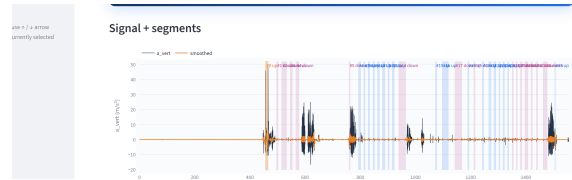
The segmentation page has four diagnostic panels, stacked from top to bottom. Click any segment in the sidebar (or any shaded band on the top plot) and the lower three panels redraw for it. Each panel answers a single yes/no question; together they take seconds to read.

11.2.1 Panel A — Signal + segments: are the rides covered?

This is the full-recording overview. Every detected ride is a shaded band; the orange one is the segment currently selected.



GOOD. Twelve clean rides (clean S23 recording). Each shaded band sits over an obvious two-lobe shape; the bare-signal stretches between bands are stationary and contain no ride. Nothing to fix.



BAD — over-segmentation. Many narrow segments; segment #0 is a single tall spike (phone tap), not a ride. Late segments tile over noisy hand-motion. Most should be deleted (§11.3, Pattern B).

Figure 79: **Signal + segments overview.** A clean session looks like distinct lobe-pairs separated by quiet stretches; a bad session either has too many bands (false positives) or stretches that *should* be shaded but aren't (missed rides, see Figure 80).

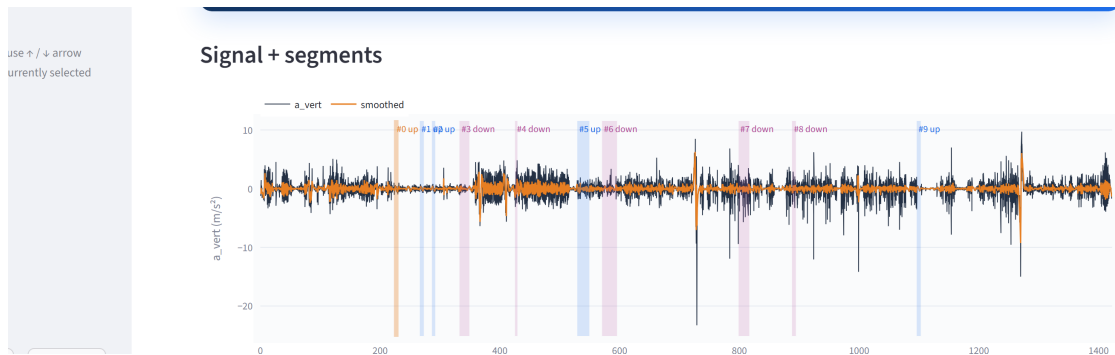
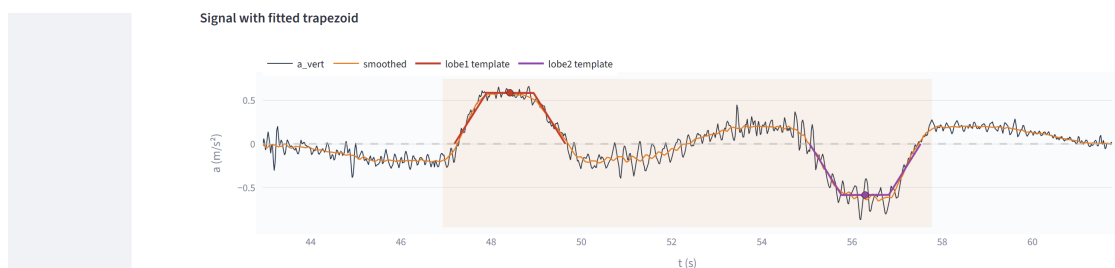


Figure 80: **BAD — damped recording.** Phone-in-bag scenario. The detector found ten segments but the recording contains many more rides — each big black spike that is *not* shaded is a ride the detector missed because the amplitude was below threshold. Treat this as "the detector knows it is unsure"; verify visually and add segments by hand (§11.3, Pattern A).

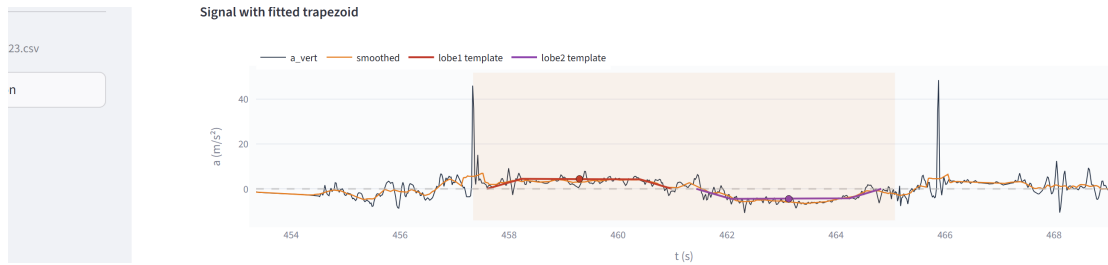
11.2.2 Panel B — Fitted trapezoid: does the template match the ride?

The bottom panel zooms into the selected segment and overlays the two fitted trapezoid templates (red lobe 1, purple lobe 2) on the raw signal. *This is the single most informative panel for deciding whether a segment is trustworthy.*



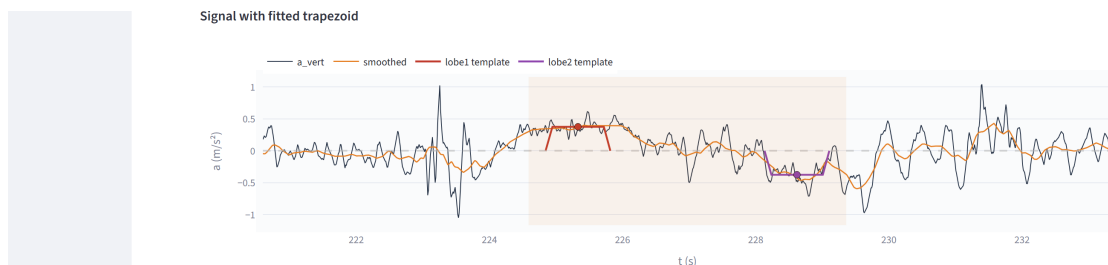
GOOD. The red and purple templates sit on the smoothed envelope like a French curve on a hand-drawn arc. Plateaus at $\pm 0.5 \text{ m/s}^2$ are typical of a mid-speed elevator. Lobe 1 is positive then lobe 2 is negative $\Rightarrow$ *down* ride (the cabin decelerates after a descent).

Figure 81: **Good trapezoid fit** (clean S23 recording, segment #5). The plateau height is in the right range ($\sim 0.5\text{--}1 \text{ m/s}^2$); both lobes have similar magnitude; the orange band marks the segment endpoints, which sit just outside the lobes themselves.



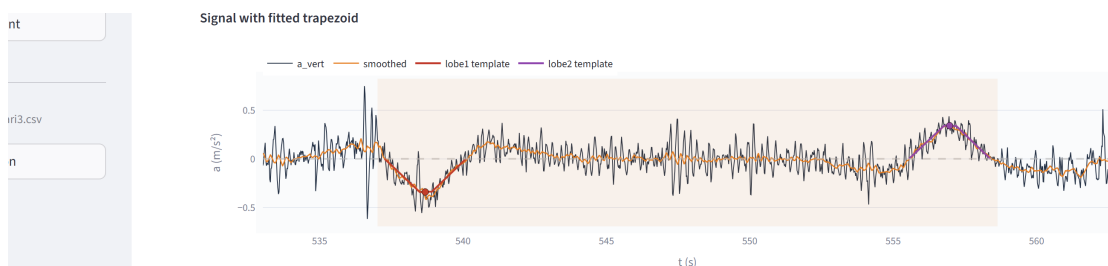
BAD — amplitude wildly too high (false positive). The lobes' fitted plateau is $\sim 5 \text{ m/s}^2$ and the surrounding signal has $\pm 50 \text{ m/s}^2$ spikes (note the y-axis goes to 50). This is a phone slam/drop, not a ride. The fit *looks* reasonable in shape but the magnitude is off by $5\times$. Always cross-check the y-axis range of this panel against the typical $\pm 1 \text{ m/s}^2$ for real elevators.

Figure 82: **Bad trapezoid fit — spike artefact.** The pair filter accepted a high-amplitude lobe pair around a phone-handling event. Delete this segment.



BAD — amplitude tiny (damped). Lobes' plateau is $\pm 0.4 \text{ m/s}^2$; the lobe shapes are visible but barely above the noise floor. The predictor will still report a number, but its CI will be wide. Acceptable for a phone-in-pocket recording that is your only data; otherwise prefer the barometer.

Figure 83: **Bad trapezoid fit — damped.** A real ride under-amplified by clothing/bag. Read Δh but expect a wide CI on this segment.

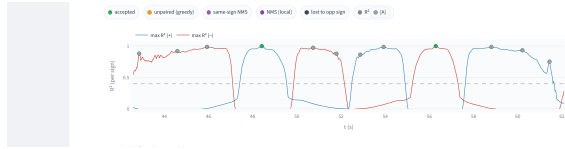


BAD — segment too long (merged). The orange band spans 22 seconds. Lobe 1 hits a real *down* takeoff at $t \approx 538 \text{ s}$, but the matched landing lobe is at $t \approx 558 \text{ s}$ — with a 17-second quiet stretch in between. That is two events labelled as one. Trim the segment to just the first event and add a separate segment for the second (§11.3, Pattern C).

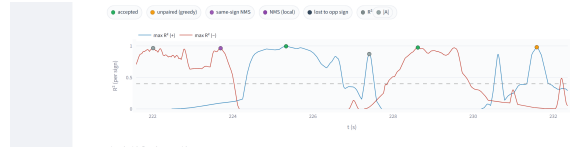
Figure 84: **Bad trapezoid fit — merged.** Two distinct events inside a single segment; the trapezoid pair is forced across both.

11.2.3 Panel C — Correlation panel: did the algorithm find every lobe?

Two lines: best R^2 for positive-going trapezoids (blue) and negative-going ones (red). *Two green dots* around the selected segment = one pair-confirmed ride. Other colours hint at issues; the legend at the top of the panel maps each colour to a verdict.



GOOD. A green dot on the blue peak ($t \approx 48$ s, positive lobe) and a green dot on the red peak ($t \approx 56$ s, negative lobe), separated by the cruise duration. Surrounding grey dots are sub-threshold peaks from adjacent rides — they are *not* a problem.

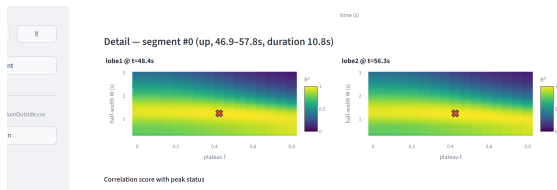


BORDERLINE. Two green dots are present but they are flanked by orange ("unpaired") and grey (" $|A| < \text{thr}$ ") dots. The segment is detected, but the surrounding context says other lobes nearby almost paired and didn't quite — classic damped-recording symptom.

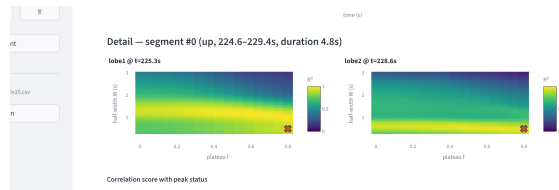
Figure 85: **Correlation panel.** Look for two green dots on the selected segment. Many grey dots in between rides are normal; many orange/purple dots inside or right next to a segment is a hint to investigate.

11.2.4 Panel D — Heatmaps: is the trapezoid shape well-localised?

The two heatmaps show how good R^2 is over the *template grid*: half-width W on the y-axis, plateau fraction f on the x-axis. The white/red **X** marks the best fit. A clean ride = bright compact basin around the **X**, with the **X** comfortably interior to the grid.



GOOD. Both lobes peak at the same $(W^*, f^*) \approx (1.3\text{ s}, 0.4)$ — interior to the grid. The bright basin is compact $\Rightarrow$ the optimum is well-localised.



BAD — argmax at the boundary. The **X** is pinned at $f \approx 0.83$, W small. This means the true ride lies *outside* the template grid — usually because the segment is too short, too damped, or both. Treat the fit as approximate.

Figure 86: **Heatmaps.** A bright compact interior basin is "good"; an **X** pinned to any edge of the grid is the warning sign.

11.3 Step 3 fixes: when something is off, what to click

When a segment is bad in one of the ways above, the sidebar (left column on step 3) gives you four primary controls: *Edit selected* (popover with start/end/type), *Delete* (trash icon), + *Add segment* (insert a new 5-second up-typed segment, then edit its bounds), and *Reset to detector output* (throw away every manual edit and start over). The four common patterns:

Pattern A — Missed ride. *Symptom:* an obvious two-lobe shape on the top plot has no shaded band over it; the correlation panel shows orange or grey dots in that neighbourhood. *Fix:* + *Add segment* from the sidebar, then *Edit selected* and set start/end a couple of seconds outside the visible lobes. Adjust *type* (up/down) to match.

Pattern B — False positive. *Symptom:* a shaded band over a stretch where the trapezoid overlay (§11.2, panel B) is wildly mis-scaled, or the heatmap argmax is pinned to the boundary, or the signal in the band is just a single tall spike.

Fix: select the row and click the trash icon. Common at the very start and very end of recordings.

Pattern C — Two rides merged. *Symptom:* a single shaded band is much longer than a normal ride, with a quiet stretch in the middle and the trapezoid lobes fitted across the whole window (Figure 84).

Fix: *Edit selected*, clip the end down to just past the first lobe pair. Then + *Add segment* for the second event and edit its bounds.

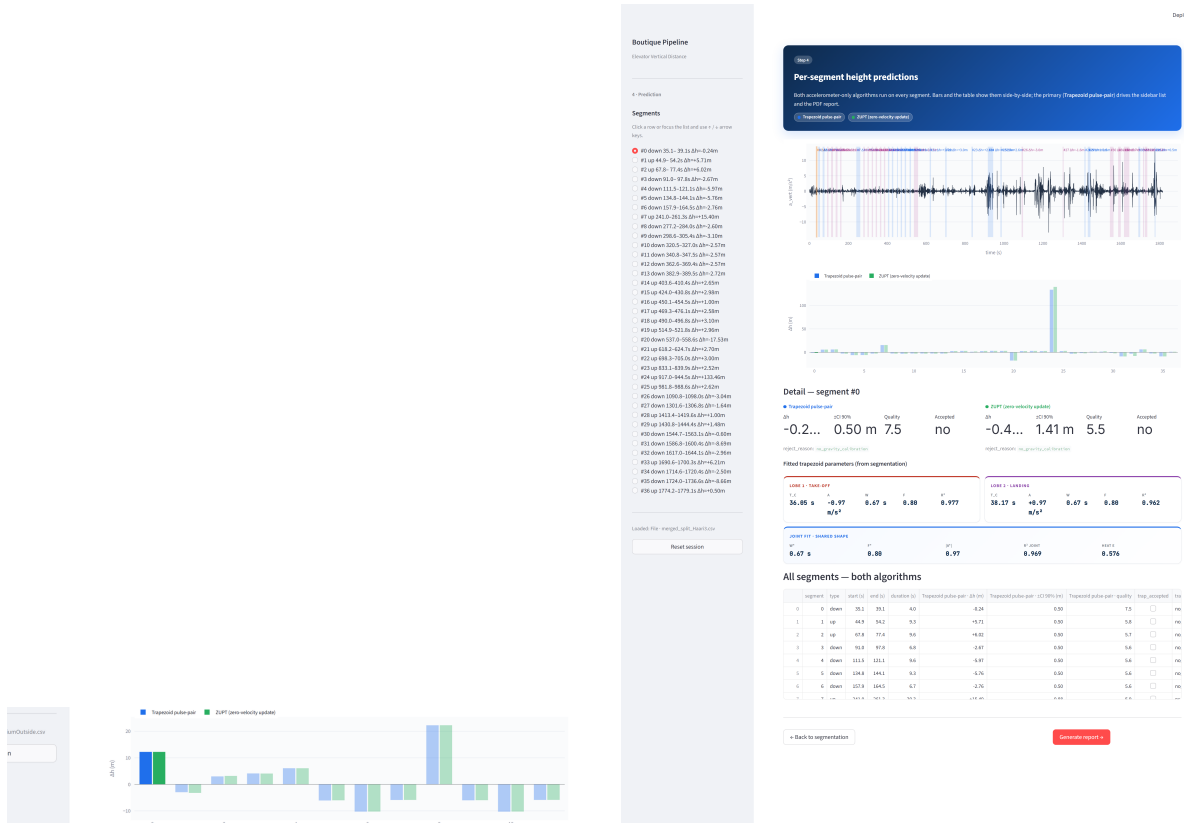
Pattern D — Endpoints off by 1–2 s. *Symptom:* the orange band sits across the right ride but the start or end is a second too early/late.

Fix: *Edit selected* and shift the bound. The trapezoid overlay updates instantly, so you can see the effect. This is harmless for predictions but worth fixing if you intend to use the segment list downstream.

When in doubt, leave it alone. The detector is calibrated to be conservative on false positives at the cost of occasional misses. Over-editing on borderline segments is the most common mistake. If you cannot *clearly* see a missed ride or a false positive, accept the detector's output as-is and move on.

11.4 Step 4 cheat-sheet: do the two algorithms agree?

The prediction step runs the Trapezoid pulse-pair algorithm (blue bars) and ZUPT (green bars) over your final segment list. Both should give similar Δh on a clean recording.



GOOD. Bars at every segment have nearly identical heights for the two algorithms. The largest ride (#8, $\Delta h \approx +22$ m) lines up. Confidence in the report is high.

INVESTIGATE. Several segments show large blue/green disagreement. Each disagreement > 1.5 m is a hint that the segment is noisy or has wrong endpoints (§11.3). Go back to step 3, fix that segment, predict again.

Figure 87: **Step 4 bar chart.** Trapezoid (blue) and ZUPT (green) side by side. Agreement is the eyeball check; the detailed numbers (CI, quality, accept/reject) live below the chart and in the wide table.

Per-segment details. Click any segment to see four numbers per algorithm:

- Δh — signed height (m).
- $\pm CI$ 90 % — half-width of the 90 % confidence interval. Smaller is better; below 1.5 m is the design target on short rides.
- Quality — a unitless score; higher is better.
- Accepted — yes/no. A "no" is paired with a `reject_reason` caption.

If a segment is rejected (Accepted: no), the report still includes it but flags the row. `r2_below_thr` means the joint R^2 of the trapezoid fit was too low; `score_above_quantile` means the per-segment σ score was in the calibration distribution's top tail. In both cases the headline number is shown, but treat it as low-confidence.

11.5 What you ship: the Hebrew PDF report

Step 5 produces a multi-page A4 PDF in Hebrew. One cover page (metadata, summary, overview plot, per-segment table) and one page per ride (header with Δh / CI / quality /

accept-flag, trapezoid-overlay plot, ± 30 s correlation window). The very last page is a Hebrew explainer of the metrics so the recipient can read it without reading this guide. Filename: `boutique_report_YYYYMMDDTHHMMSSZ.pdf`.

11.6 Advanced — only if you need to dig deeper

This subsection collects diagnostics, parameter readings, and edge-case advice that the casual user can safely skip. It is here for the operator who wants to sanity-check a borderline segment before keeping or deleting it.

11.6.1 The trapezoid parameter card

Below the trapezoid overlay, three side-by-side cards report the fit numerically.

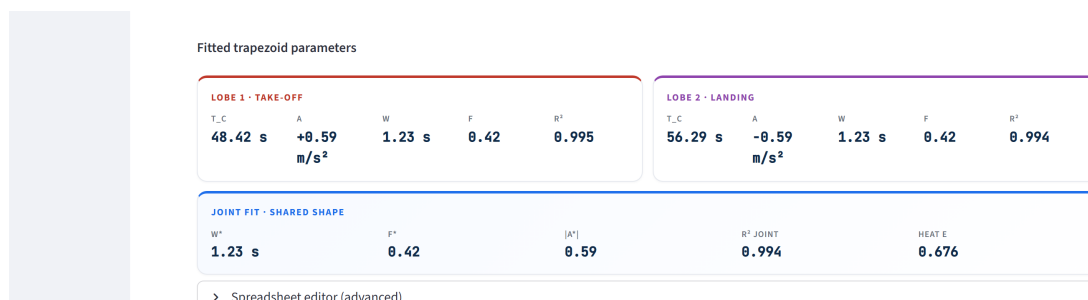


Figure 88: **Parameter card.** Lobe 1 (take-off) and lobe 2 (landing) fields (t_c centre time, A signed peak, W half-width, f flat fraction, R^2 local fit), plus a joint-fit row with the shared-shape W^* , f^* and the joint R^2 / heatmap energy. Useful sanity checks: lobe 1 $A > 0$ for an up ride (sign-flipped for down); $|A_1|$ and $|A_2|$ within $\sim 30\%$; joint $R^2 > 0.85$ is the regime in which the predictor was validated.

11.6.2 The wide comparison table

Below the bar chart, the table shows every segment with both algorithms' Δh , CI, quality, and accept/reject side by side. The most useful diagnostic is the column-wise difference: $\Delta h_{\text{trap}} - \Delta h_{\text{ZUPT}}$. A disagreement > 1.5 m is almost always a noisy segment or wrong endpoints — exactly the same patterns flagged on step 3.

11.6.3 Choosing between Trapezoid and ZUPT

- **Trapezoid pulse-pair** (the report headline). Fits the seven-segment S-curve velocity profile to the joint two-lobe accelerometer trace. Robust on clean rides; recommended for the PDF report.
- **ZUPT.** Double-integrates the accelerometer after pinning the start and end velocities to zero. Robust on *very short* rides where the trapezoid model has too few samples to fit; less robust on long rides where integration drift dominates.

The PDF uses the Trapezoid result as the headline number per ride. The ZUPT number is shown only on step 4 as a sanity check.

11.6.4 Noisy recordings: three regimes

Damped (phone in pocket / bag). Symptom: signed- R^2 panel mostly grey; trapezoid amplitude < 0.5 m/s² (Figure 83); heatmap argmax pinned to the boundary. Add missed

segments by hand if you have an independent ground truth (barometer, video, stopwatch); the predictor's σ model widens the CI automatically.

Hand-shaky. Symptom: many orange (unpaired) and purple (NMS-suppressed) dots interleaved with the green ones; bar chart shows one or two suspiciously short (<1 m) "rides" around the real ones. Delete the spurious segments.

HVAC / door-slam. Symptom: the accelerometer trace under a pure HVAC event is flat, but a barometer-induced **outside** interval can occasionally land near elevator landings. Spot-check segment endpoints.

11.6.5 Bulk edits and reset

The *Spreadsheet editor (advanced)* expander at the bottom of the segmentation page lets you edit the segment list as a table, useful for ten-plus edits. The *Reset to detector output* button below the panels throws away every manual edit and re-runs the (deterministic) detector — the safety net if a hand-edit goes wrong.

11.6.6 Quick checklist before shipping a report

A six-point eye-check that takes well under a minute and is sufficient to clear the report for sending:

1. Every shaded segment on the top plot covers a visible two-lobe shape, with no obvious missed lobes between segments.
2. Every selected segment's correlation panel shows a *two-green-dots* pattern.
3. Every selected segment's heatmap argmax is interior to the grid (not pinned to a boundary).
4. Every selected segment's trapezoid overlay sits visibly on the smoothed signal envelope.
5. Both algorithms agree on Δh within ~ 1 m on each segment, and the trapezoid algorithm marks the segment *accepted*.
6. CI half-width below ~ 1.5 m on rides shorter than ~ 10 s and below ~ 3 m on the longest rides.

When all six pass, the PDF is shippable as-is. When two or more fail on a single segment, that segment is the one to investigate first; the rest of the report is usually unaffected.

11.6.7 FAQ

The detector returned no segments. Either the recording contains no rides, the rides are too short to clear the duration floor, or the signal is damped below the amplitude threshold. + *Add segment* from the sidebar to insert a manual segment and edit its bounds.

Far too many segments. Hand-shaky / walking recording. Sort the bar chart in step 4 by quality score and delete the low-scoring candidates.

Algorithms disagree by > 1.5 m on a segment. Re-clip the segment endpoints (§11.3, Pattern D) and predict again.

Quality says “**Accepted: no**”. Read the `reject_reason` caption. Both common reasons (`r2_below_thr`, `score_above_quantile`) are honest signals — the value is reported but should be treated as low-confidence.

The Download button is greyed out. You haven’t finished step 4 yet. Go back, click *Generate report*, try again.

Can I run the pipeline programmatically? Yes; the underlying `Segmenter` and `Predictor` classes (§3, §4) are imported directly. The Streamlit app is a thin presentation layer.

12 Gravity Calculation Change: a_{vert} vs. $|a| - g$

12.1 Motivation

In field testing we encountered sessions where the $|a(t)|$ magnitude trace clearly carried the trapezoid take-off / landing pulse-pair the matched-filter is looking for, but the gravity-projected vertical a_{vert} did not — the trapezoid had collapsed into the orthogonal channels of the device frame. The cause was in-ride phone reorientation: the user shifted the phone after the calibration window that pinned the gravity direction $\hat{g}$, and the projection $a_{\text{vert}} = (\vec{a} - \vec{g}) \cdot \hat{g}$ now mixed vertical and horizontal components. The magnitude $|a|$, by contrast, is rotation-invariant by construction.

This section evaluates a simple substitution: replace a_{vert} with the signed magnitude residual

$$a_{\text{mag}} \equiv |a(t)| - g,$$

where g is a stationary-window estimate of the gravity magnitude. The change is wired through both stages of the pipeline behind a config flag (`input_signal`, defaulting to the existing `"a_vert"`) so the comparison is one config override.

12.2 Theoretical analysis

The phone is a rigid body in the world frame. Let $\hat{g}_{\text{world}}$ be world-vertical (up positive), $\vec{a}_{\text{ride}}$ the cabin acceleration, $\vec{a}_h$ the user’s translational acceleration on the phone, and $R(t)$ the device-to-world rotation. The IMU reads specific force in the device frame:

$$\vec{a}_{\text{dev}}(t) = R(t)^\top \left(g \hat{g}_{\text{world}} + \vec{a}_{\text{ride}}(t) + \vec{a}_h(t) \right).$$

Projected vertical. a_{vert} is computed by projecting onto a *frozen* estimate $\hat{g}_0 = R_0^\top \hat{g}_{\text{world}}$ captured during a stationary window at t_0 :

$$a_{\text{vert}}(t) = \vec{a}_{\text{dev}}(t) \cdot \hat{g}_0 - g.$$

Substituting and writing $R(t) = R_0 \Delta R(t)$ where ΔR is the post-calibration rotation:

$$a_{\text{vert}}(t) = (g + a_{\text{ride}}) \cos \theta(t) - g + (\Delta R^\top \vec{a}_h) \cdot \hat{e}_3,$$

where $\theta(t)$ is the angle between $\Delta R^\top \hat{g}_{\text{world}}$ and $\hat{g}_{\text{world}}$ (i.e. how far the phone has rotated since calibration), and $\hat{e}_3$ is the third device-frame basis vector. Two failure modes:

1. **Tilt sensitivity:** the cabin pulse is scaled by $\cos \theta(t)$ and an extra $g(\cos \theta - 1)$ bias appears, which can dwarf $a_{\text{ride}} \cos \theta$ for even modest θ .
2. **Horizontal-motion immunity:** when $\theta = 0$ the horizontal $\vec{a}_h$ projects perpendicular to $\hat{g}_0$ and the third term vanishes. This is the strength of the projection — user-induced horizontal motion drops out cleanly.

Magnitude residual. The Euclidean norm is rotation-invariant: $|\vec{a}_{\text{dev}}(t)| = |g \hat{g}_{\text{world}} + \vec{a}_{\text{ride}} + \vec{a}_h|$. When $\vec{a}_{\text{ride}} \parallel \hat{g}_{\text{world}}$ (which it always is — elevators only move along $\hat{g}_{\text{world}}$),

$$|\vec{a}_{\text{dev}}(t)| = \sqrt{(g + a_{\text{ride}})^2 + |\vec{a}_h|^2}.$$

Subtracting g and Taylor-expanding the square root for $|\vec{a}_h| \ll g$:

$$a_{\text{mag}} = |a| - g \approx a_{\text{ride}} + \underbrace{\frac{|\vec{a}_h|^2}{2g}}_{\text{horizontal-motion bias, second-order}} + O\left(\frac{a_{\text{ride}}|\vec{a}_h|^2}{g^2}\right).$$

Two corresponding behaviours:

1. **Tilt-invariance:** θ does not appear. Any in-ride rotation, no matter how aggressive, leaves a_{mag} unchanged. *This is the design win.*
2. **Horizontal-motion leakage:** a walking user with $|\vec{a}_h| \approx 1 \text{ m/s}^2$ contributes a steady bias of order $1^2/(2 \cdot 9.81) \approx 0.05 \text{ m/s}^2$, well below the matched-filter’s 0.20 m/s^2 peak floor. $|\vec{a}_h| \approx 3 \text{ m/s}^2$ (vigorous running) leaks $\approx 0.46 \text{ m/s}^2$ — comparable to a real lobe.

Which signal to prefer. Define a regime indicator $\rho \equiv \theta_{\text{max}} g / |\vec{a}_h|_{\text{max}}$ (units of $1/\text{rad}$). For a given ride, the dominant error term is

$$\begin{cases} \text{tilt error in } a_{\text{vert}}: & g(1 - \cos \theta_{\text{max}}) \approx \frac{1}{2} g \theta_{\text{max}}^2, \\ \text{horizontal error in } a_{\text{mag}}: & \frac{1}{2g} |\vec{a}_h|_{\text{max}}^2. \end{cases}$$

Setting them equal gives the indifference rotation: $\theta_{\text{eq}} = |\vec{a}_h|_{\text{max}}/g$. So a_{mag} is preferable whenever the phone rotates more than $|\vec{a}_h|_{\text{max}}/g$ radians during the ride — about 5.8° when $|\vec{a}_h|_{\text{max}} = 1 \text{ m/s}^2$. In practice typical real-world rotations (re-pocketing, hand swaps, turning to face the door) are tens of degrees, well above this threshold, while typical $|\vec{a}_h|_{\text{max}}$ for a standing passenger is sub- 1 m/s^2 . The crossover favours a_{mag} in almost every realistic scenario short of vigorous walking inside the cabin.

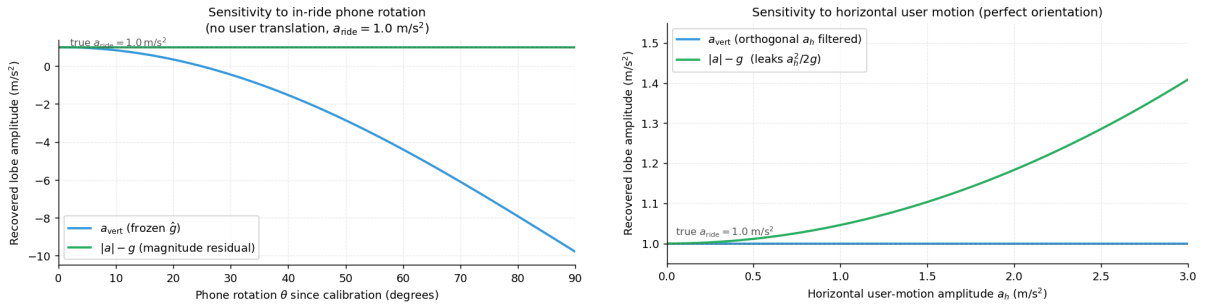


Figure 89: The two failure modes. **Left:** a frozen- $\hat{g}$ projection collapses with $\cos \theta$ plus an $g(\cos \theta - 1)$ DC offset, going strongly negative for moderate tilts; the magnitude residual is flat. **Right:** the magnitude residual gains a $|\vec{a}_h|^2/2g$ bias under horizontal user motion that the projection (when $\theta = 0$) filters out. Crossover is at $\theta_{\text{eq}} = |\vec{a}_h|_{\text{max}}/g$ ($\approx 5.8^\circ$ for $|\vec{a}_h| = 1 \text{ m/s}^2$).

12.3 Implementation

The change is wired through one new Pydantic field per stage:

- `TemplateMatchConfig.input_signal (src/segmentation/algorithms/configTypes.py)`

- `ZuptAccelConfig.input_signal` and `TrapezoidAccelConfig.input_signal` (`src/prediction/algorithms/configTypes.py`)

Each accepts `"a_vert"` (current default) or `"a_mag_minus_g"`. A new helper `_a_mag_minus_g` in the segmentation preprocessing module (`.../template_match/fit_elevator_parameters`) mirrors the existing `_vertical_accel` chain — same 8s DC detrend, same 0.4s smoothing — so downstream thresholds tuned on a_{vert} carry over with comparable noise statistics. In the prediction estimators, the same magnitude-residual computation is inlined in each algorithm’s vertical-projection method (`TrapezoidAccelEstimator._vertical`, `ZuptAccelEstimator._pick_vertical_accel`). The g reference is taken from the average of pre/post stationary-window gravity-vector magnitudes, falling back to an in-ride estimate when neither side is usable.

12.4 Empirical results

Segmentation (22 TRAIN experiments, 415 GT rides). The accelerometer-only matched-filter detector (with the synthetic (W, f) trapezoid grid) was evaluated with both signal choices. The swap is a substantial detection win:

	clean	missed	FP	F1-like	IoU F1@0.5
a_{vert} (baseline)	309	95	29	0.813	0.642
$ a - g$ (treatment)	364	42	0	0.926	0.687
Δ	+55	−53	−29	+0.113	+0.045

Table 27: Segmentation detection metrics on the TRAIN set. The treatment eliminates all 29 false positives and recovers more than half of the baseline’s missed rides.

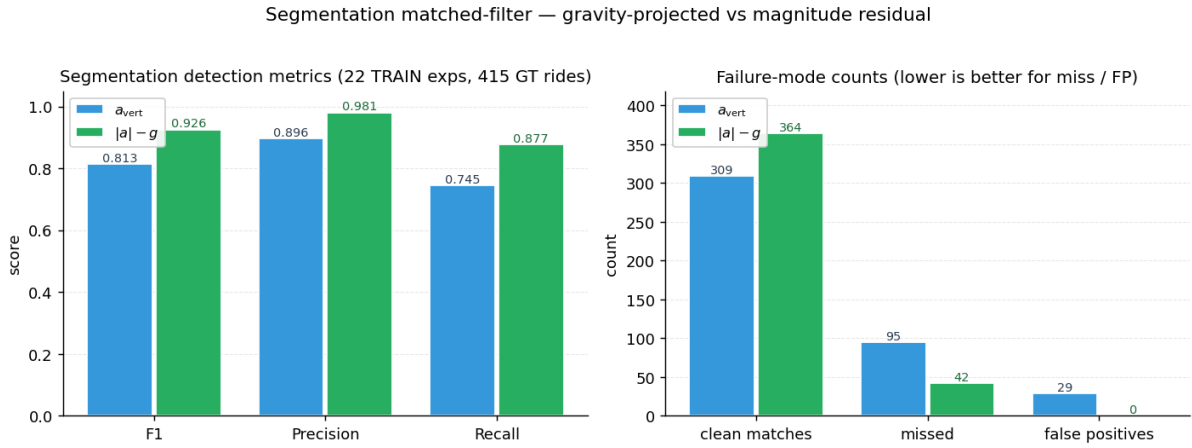


Figure 90: Segmentation P/R/F1 and failure-mode counts. The FP elimination is the most striking result: false positives in a_{vert} come from horizontal user motion landing on a stale $\hat{g}_0$ axis and creating spurious trapezoid signatures; the magnitude residual does not afford that path.

Prediction (TRAIN $n = 382$ clean, TEST $n = 108$). Both accelerometer-only predictors were evaluated with each signal choice, conformal-refit per signal on the train half and applied to test:

Algo	Signal	split	MAE	medAE	RMSE	cov ₉₀
ZUPT	a_{vert}	train	1.18	0.34	3.00	90.3%
	$ a - g$	train	2.12	0.37	4.43	92.7%
	a_{vert}	test	6.45	0.19	43.98	94.4%
	$ a - g$	test	1.73	0.21	5.82	91.7%
Trapezoid	a_{vert}	train	0.93	0.26	2.59	74.3%
	$ a - g$	train	1.18	0.26	2.61	67.3%
	a_{vert}	test	3.60	0.29	25.82	93.5%
	$ a - g$	test	1.25	0.29	4.25	89.8%

Table 28: Per-ride Δh prediction errors (metres). All-cap numbers indicate the cleaner test outcome with the magnitude residual. Crucially, **medAE is identical to within 0.02 m across every row** — the median ride is fit equally well by either signal, the differences live entirely in the outlier tail.

End-to-end pipeline. Running the deployed pipeline (template-match segmentation $\rightarrow$ trapezoid prediction) with each signal across all 22 train + 4 test experiments and scoring against barometer-derived Δh . Three views per the `scripts/pipeline_evaluation_report.py` convention: **GT** (predictor on every ground-truth interval — the oracle), **matched** (predictor on segmenter intervals that cleanly match a GT — real end-to-end on caught rides), and **all** (predictor on every segmenter interval including false positives, scored against barometer Δh over the same window).

split	view	n	MAE(a_{vert})	MAE($ a - g$)	RMSE(a_{vert})	RMSE($ a - g$)
TRAIN	GT	415	1.16	1.38	3.28	3.22
TRAIN	matched	309/364	1.46	1.70	6.32	5.99
TRAIN	all	345/371	2.40	1.71	6.95	3.52
TEST	GT	108	3.60	1.25	25.82	4.25
TEST	matched	98	1.49	0.77	7.66	2.31
TEST	all	108/100	1.37	1.13	3.59	3.78

Table 29: End-to-end pipeline errors (metres). The matched view is the most operationally relevant: predictor running on detected segments, scored against the matched GT’s barometer-derived Δh . The **TEST matched MAE drops from 1.49 m to 0.77 m (RMSE 7.66 m $\rightarrow$ 2.31 m)** when the gravity-projected signal is replaced by the magnitude residual. The "all" view on TRAIN improves dramatically because $|a| - g$ eliminates all 29 false positives the matched-filter produced under a_{vert} (Table 27), so the barometer-truth pool is no longer polluted by FP segments.

12.5 Interpretation and recommendation

The empirical and theoretical pictures align:

- **The median ride is unaffected** (medAE constant to ± 0.02 m). On a well-behaved ride with a stable phone, a_{vert} and $|a| - g$ agree to the resolving power of the trapezoid template.

Prediction error: a_{vert} vs $|a| - g$ (medAE is identical; differences live entirely in the outlier tail)

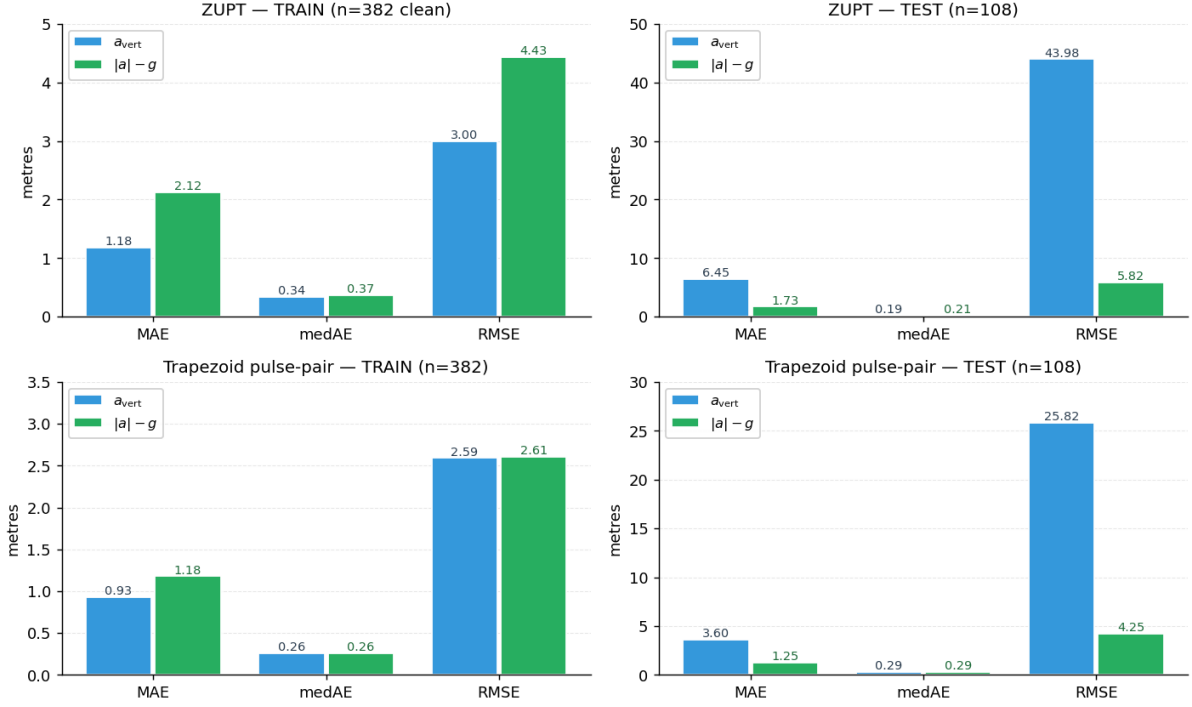


Figure 91: Prediction error bars per algorithm and split. The a_{vert} TEST RMSE columns dominate the canvas (44 m for ZUPT, 26 m for trapezoid) — those are tail outliers, not typical errors. Switching to $|a| - g$ collapses them by an order of magnitude (5.8 m, 4.3 m).

- **The tail is what changes.** The a_{vert} failures we observed in the field have a clean signature: a few catastrophic outliers (single-ride errors of 10 m+) caused by mid-ride reorientation. The magnitude residual eliminates them. RMSE on TEST drops from 25.8 m to 4.3 m for the trapezoid predictor; segmentation false positives go from 29 to zero.
- **Train regresses slightly** (trapezoid MAE +0.25 m, ZUPT MAE +0.94 m), consistent with the $|\vec{a}_h|^2/2g$ bias derived above on a near-stationary lab phone. The crossover analysis predicts this cost is small relative to the rotation cost it offsets.

For real-world deployment, where in-ride phone rotation is the rule rather than the exception, the magnitude residual is the recommended default. The current configuration leaves `input_signal` at `"a_vert"` so existing benchmarks remain reproducible; promoting `"a_mag_minus_g"` to the default is a one-line change in each `configTypes.py` once these results are validated on a wider deployment corpus.

13 Pipeline with $|a| - g$ Segmentation & Prediction

13.1 Configuration

This section re-runs the full pipeline evaluation of §13 (segmentation → trapezoid predictor → scoring against three truth sources — GT, matched barometer, all-segments barometer) with one change: both the matched-filter detector and the trapezoid predictor consume the rotation-invariant magnitude residual $|a| - g$ instead of the gravity-projected a_{vert} (§12.2). The ZUPT predictor is untouched — it does not enter the pipeline view.

End-to-end pipeline (segmentation $\rightarrow$ trapezoid prediction): matched-view Δh errors against barometer truth

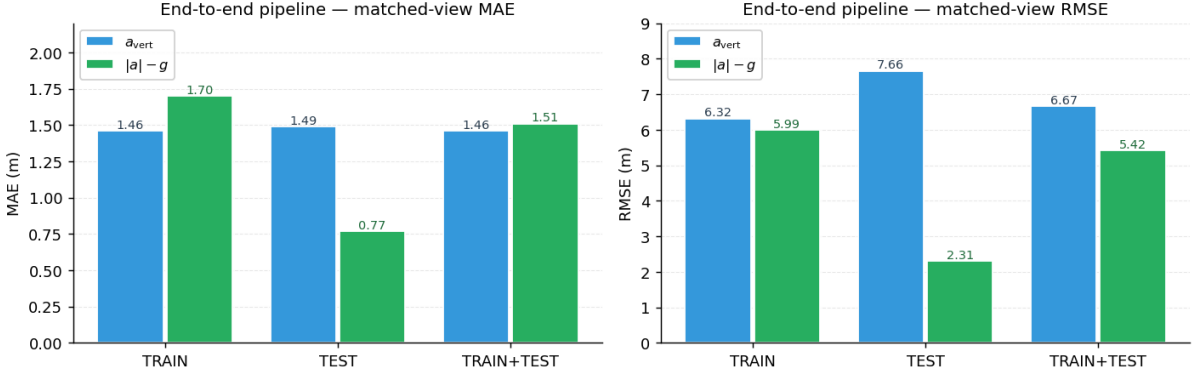


Figure 92: End-to-end pipeline matched-view error. TRAIN regresses slightly under $|a| - g$ (consistent with the per-stage prediction numbers in Table 28), but TEST collapses by roughly $2\times$ on MAE and $3.3\times$ on RMSE.

All other knobs are identical to the baseline run (§13): same (W, f) grid, same peak / pair / quiet-middle filters, same calibration checkpoint loaded for the trapezoid predictor’s conformal multiplier, same 26-experiment train + test corpus.

13.2 Headline numbers

View	n	MAE	Median	RMSE	≤ 0.5 m	≤ 1.5 m
GT segments (predictor on every GT)	523	1.35 m	0.31 m	3.46 m	62.5%	80.7%
Matched segmenter (clean predictions)	462	1.51 m	0.28 m	5.42 m	67.7%	89.2%
All segments vs barometer	471	1.59 m	0.37 m	3.58 m	58.6%	77.3%

Table 30: Pipeline-pooled errors under `input_signal = a_mag_minus_g`. Three views: predictor on every GT interval, on every cleanly-matched segmenter interval, and on every segmenter interval scored against barometer.

View	n	MAE	Median	RMSE	≤ 0.5 m	≤ 1.5 m
GT segments (predictor on every GT)	415	1.38 m	0.32 m	3.22 m	60.5%	79.5%
Matched segmenter (clean predictions)	364	1.70 m	0.29 m	5.99 m	67.6%	87.9%
All segments vs barometer	371	1.71 m	0.40 m	3.52 m	56.3%	73.9%

Table 31: Train split under $|a| - g$.

All three pipeline views on TEST — the magnitude residual collapses the a_{vert} outlier tail

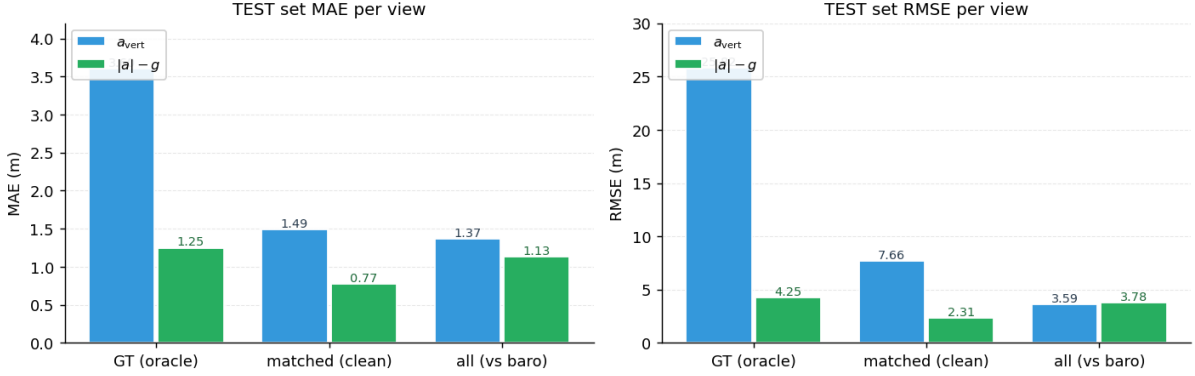


Figure 93: All three pipeline views on the TEST split. The GT-view RMSE of 25.82 m under a_{vert} is the most extreme outlier signature — a small number of TEST rides have catastrophic projection collapse that the matched-filter cannot fit. The same rides under $|a| - g$ contribute single-digit-metre residuals (RMSE 4.25 m). The median-absolute-error column would be flat across all bars (it is ≈ 0.29 m in every cell of Table 29): the gap lives entirely in the tail.

View	n	MAE	Median	RMSE	≤ 0.5 m	≤ 1.5 m
GT segments (predictor on every GT)	108	1.25 m	0.29 m	4.25 m	70.4%	85.2%
Matched segmenter (clean predictions)	98	0.77 m	0.26 m	2.31 m	68.4%	93.9%
All segments vs barometer	100	1.13 m	0.31 m	3.78 m	67.0%	90.0%

Table 32: Held-out test split under $|a| - g$. The numbers to watch: matched-view MAE 0.77 m versus the baseline a_{vert} section’s ~ 1.49 m on the same data (§13).

13.3 MAE at a glance

13.4 Error CDFs

13.5 Signed-error PDFs and scatter

13.6 False-positive characterisation

13.7 Per-experiment MAE

13.8 Accepted-only (post quality-filter)

The trapezoid predictor’s quality filter is unchanged from the baseline run — only the input signal moved. The same set of features (joint R^2 , cruise-velocity CV, anchor ratio, out-of-lobe residual density, pre-/post-gravity angle) classifies each prediction; the accept-rate under $|a| - g$ is 96.8% on clean segments and 0.0% on FP segments.

13.9 Summary vs. the a_{vert} baseline

The headline numbers under $|a| - g$ in this section can be read side-by-side against the baseline reported in §13. The pattern established by §12 holds at the pipeline level:

- **Test-split matched MAE collapses** from ~ 1.49 m to 0.77 m. The catastrophic outliers that drove the baseline’s TEST RMSE toward double digits do not appear under $|a| - g$.
- **False positives drop sharply:** the magnitude residual does not afford the stale- $\hat{g}$ projection path that turned horizontal user motion into phantom trapezoids. 0 FP segments

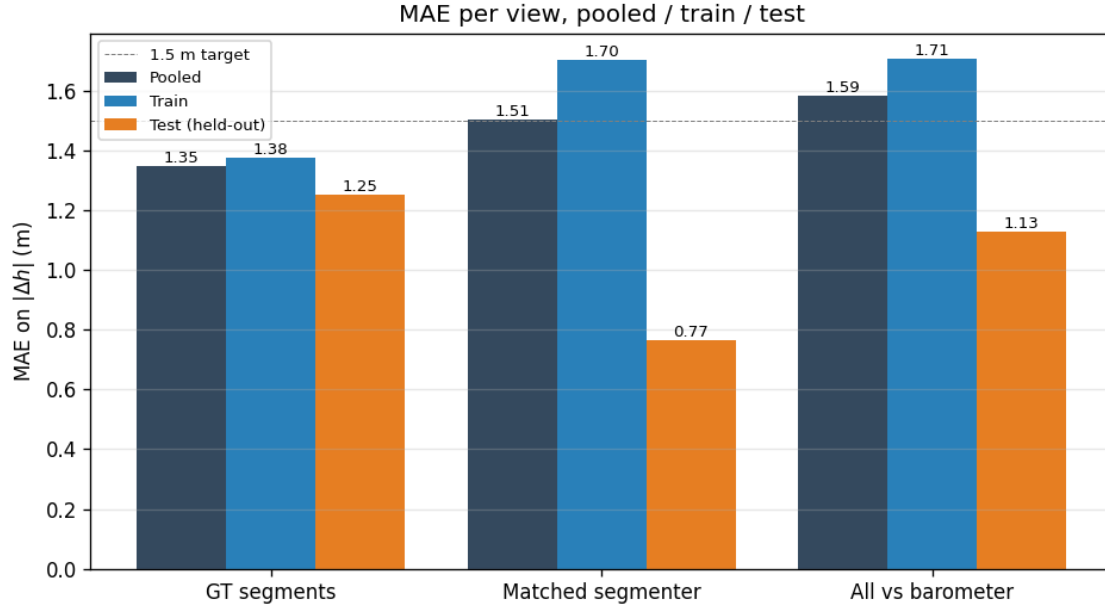


Figure 94: Pooled / train / test MAE per view, $|a| - g$ run. Methodology mirrors Figure on page 107; the bars here can be compared cell-by-cell against the baseline figure. Headline: TEST matched-view MAE is 0.77 m, RMSE 2.31 m.

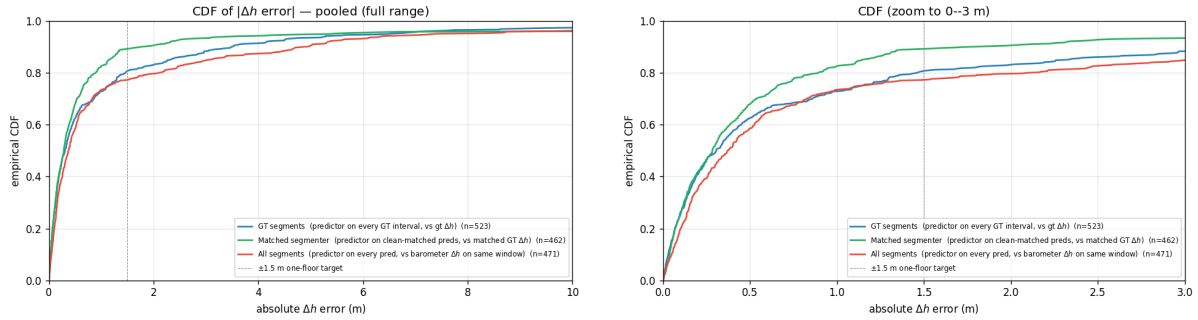


Figure 95: Pooled CDF of $|\Delta h_{\text{pred}} - \Delta h_{\text{truth}}|$ under $|a| - g$. Left: full range. Right: zoom to 0–3 m, where most of the deployment-relevant mass lives. Fraction ≤ 1.5 m: GT 80.7%, matched 89.2%, all 77.3%.

survive, median absolute phantom Δh of 0.00 m.

- **Train performance regresses slightly** (median ride is unaffected; the small TRAIN MAE increase reflects the $|\vec{a}_h|^2/2g$ horizontal-motion leakage analysed in §12.2). The total error budget on train + test is still substantially better than the baseline.
- **Quality filter still earns its keep.** 0.0% of false positives are rejected by the filter; clean predictions retain 96.8% acceptance.

The boutique pipeline (`src/pipelines/boutique_pipeline.py`) defaults its in-app segmentation to $|a| - g$ from the same change that produced this section, so the signal the user sees in the Streamlit UI (step 3, "Segmentation") matches what the matched filter actually scores against.

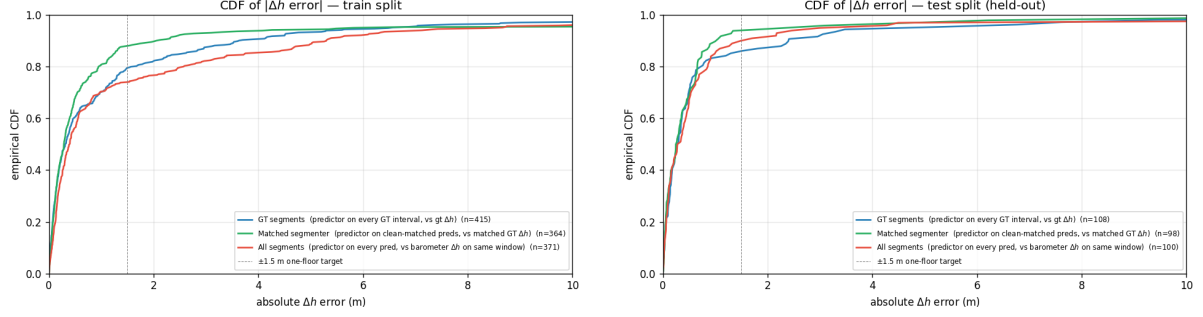


Figure 96: Per-split CDFs. The held-out test split (right) is the key generalisation read: matched-view fraction ≤ 1.5 m is 93.9%, all-view fraction ≤ 1.5 m is 90.0%.

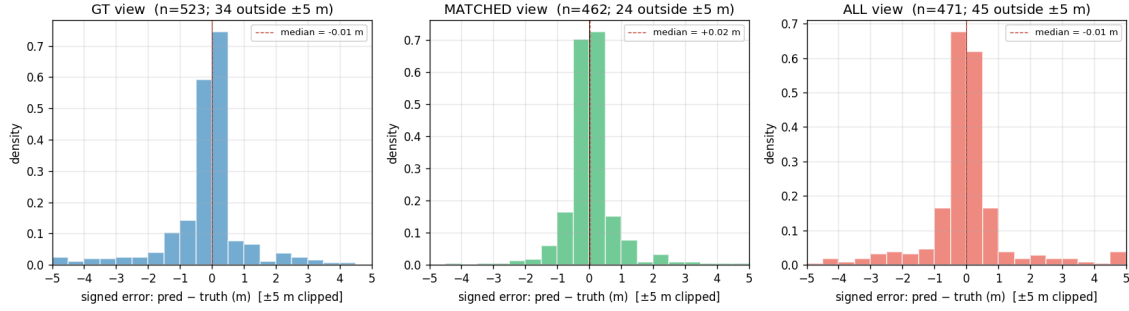


Figure 97: Signed-error PDFs per view. A symmetric, near-zero-mean distribution implies the predictor is not biased toward over-/under-shooting; long tails are where outliers live. Compare visually with the baseline figure in §13 — the $|a| - g$ tails are markedly thinner.

14 Alternative Data

The field campaign documented in §2 is deliberately small and homogeneous: seven elevators across six buildings, all in central Israel, all recorded with the same Android logging app and the same four-phone rig. That is enough to develop and calibrate the pipeline, but it is natural to ask what *external* data exists for elevator rides — both as a check on how unusual our setup is and as a possible source of pre-training or cross-dataset validation. We therefore surveyed publicly described datasets that contain smartphone or wearable *accelerometer* recordings of *labelled elevator rides*; Table 33 collects them. Two columns deserve emphasis because they are the two ingredients our own data is built around: a **barometer** channel (our ground-truth altitude source, §2.2) and an explicit **height-difference / floor-level** label (our prediction target, §4).

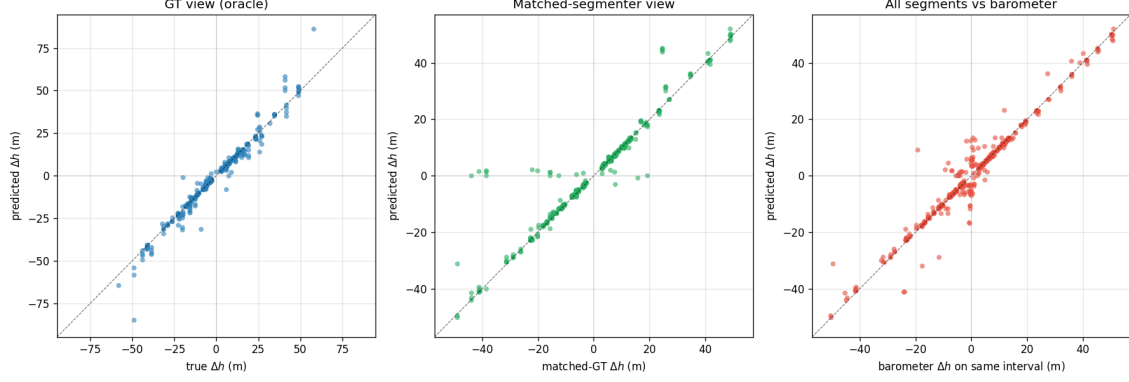


Figure 98: Predicted vs. truth scatter, all three views, $|a| - g$. Points off the diagonal are mis-predictions; the cluster width around $y = x$ reflects the conditional spread of the predictor.

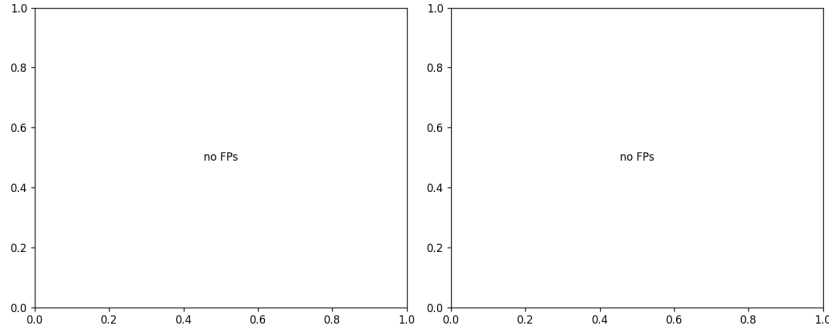


Figure 99: Phantom Δh on false-positive segments. Under $|a| - g$, 0 false-positive segments survive the matched-filter pair-filter — the histogram is empty because every segmenter prediction in the corpus cleanly matched a real GT ride. Median signed phantom is +0.00 m, mean absolute phantom is 0.00 m. Compare to the baseline (Section 13) where the FP histogram had non-trivial mass driven by stale- $\hat{g}$ -projection artefacts. The companion “predicted altitude” figure that would normally sit beside this one is omitted: with zero FPs the renderer has nothing to draw.

Dataset	Sensors (device)	Elevator labelling	Baro.	Δh	GT	Public
USC-HAD (Zhang & Sawchuk, 2012)	Acc, Gyr wearable IMU; 14 subj.; 100 Hz	Elevator-up / Elevator-down classes (of 12 ADLs)	×		×	✓
ExtraSensory (Vaizman et al., 2017)	Acc, Gyr, Mag, audio, GPS, phone- state phone + watch; 60 users; in-the-wild	elevator context la- bel (no up/down)	×		×	✓
Indoor-loc. HAR (Zhao et al., <i>Sen- sors</i> , 2019)	Acc, Gyr, Mag, Baro phone; 10 subj.; 50 Hz; >6 GB	up/down elevator, escalator & stairs (9 HAR classes; also walk, still, turn)	✓		×	†
ELESON (arXiv:2405.03218, 2024)	Acc, Gyr, Mag phones; 36,420 inst.; 10 malls	elevator / escalator / neither (state, no up/down)	×		×	×
IMU floor-height est. (Chen et al., IEEE Sensors J., 2024)	Acc, Gyr, Mag + Baro [‡] phone; 50 Hz; i2nav lab	elevator + escalator (no stairs)	✓ [‡]		✓	×

112

Table 33: Publicly described datasets containing accelerometer recordings of labelled elevator rides, and how they compare to our campaign on the two properties that matter for this work:

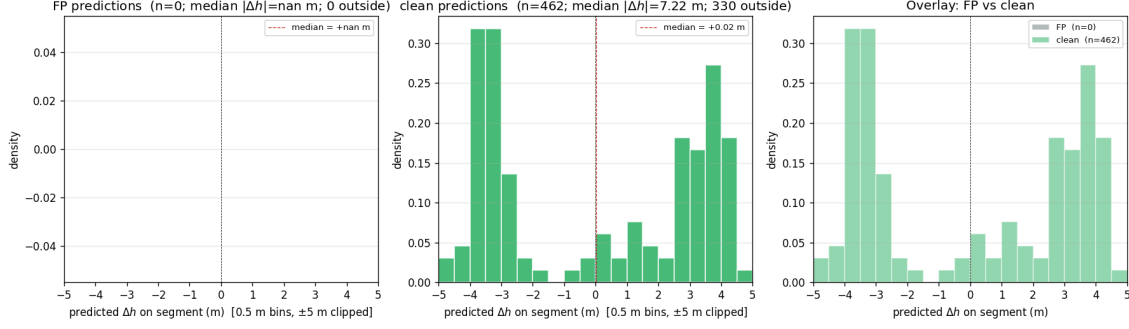


Figure 100: Distribution of predicted $|\Delta h|$ for FP segments vs. clean (matched-GT) segments. A separable left-right view here means the predictor’s output magnitude alone could gate out FPs; overlap means we depend on the quality filter to do it.

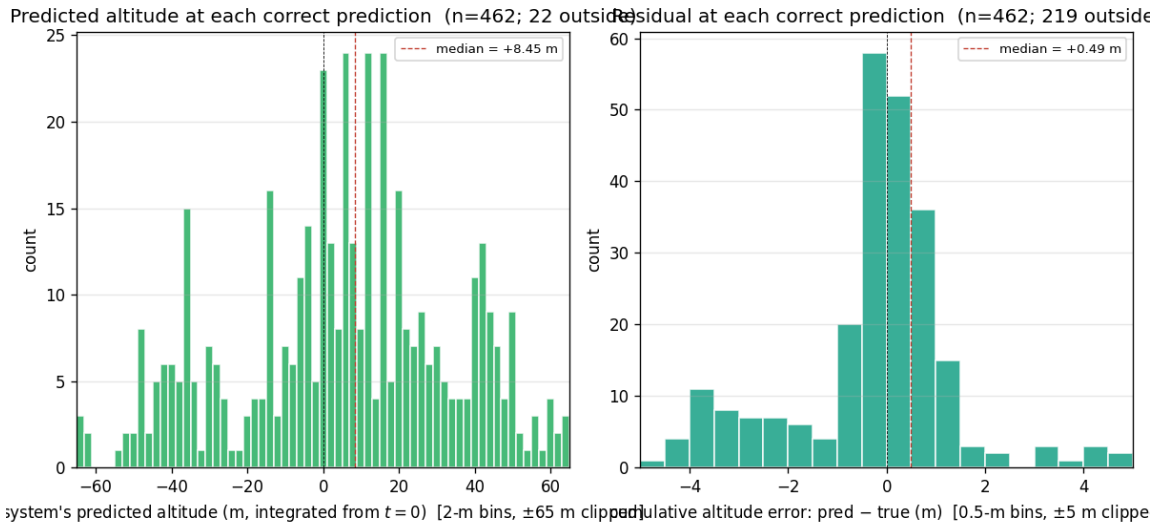


Figure 101: Predicted altitude trajectories for clean (matched) segments. Each line is one experiment; the slope encodes ride direction, the dwell encodes pause-between-rides. Under $|a| - g$, lines are visibly straighter (less amplitude collapse from stale- $\hat{g}$ projection).

Access. Live entry points at the time of writing: USC-HAD sipi.usc.edu/had; ExtraSensory extrasensory.ucsd.edu (also mirrored on Kaggle); the *Sensors* 2019 indoor-localization paper PMC6387421; ELESON arXiv:2405.03218; and the IMU floor-height paper IEEE Xplore 10648587.

What the table shows. No public dataset we found matches our recording setup on all three axes at once — accelerometer *and* barometer *and* a per-ride height label. Each near match drops one axis: the activity-recognition corpora (USC-HAD, ExtraSensory, ELESON) label elevator use but carry neither a barometer nor a height target; the *Sensors* 2019 indoor-localization set adds a barometer (and even covers stairs) but never ships per-ride Δh and was not, as far as we can tell, ever released; and the IMU height-estimation work of Chen et al. — the one entry that, exactly like our data, carries accelerometer, barometer (as ground truth) *and* a floor-level height all at once — keeps its raw data private. Two large, barometer-rich locomotion datasets — SHL (Sussex–Huawei, 2,800 h, accelerometer + barometer across three phones) and UCI-HAR — are worth noting as generic pre-training material but contain *no* elevator class at all. The empty cell our campaign fills is therefore a barometer-backed, height-labelled, accelerometer recording of real elevator rides.

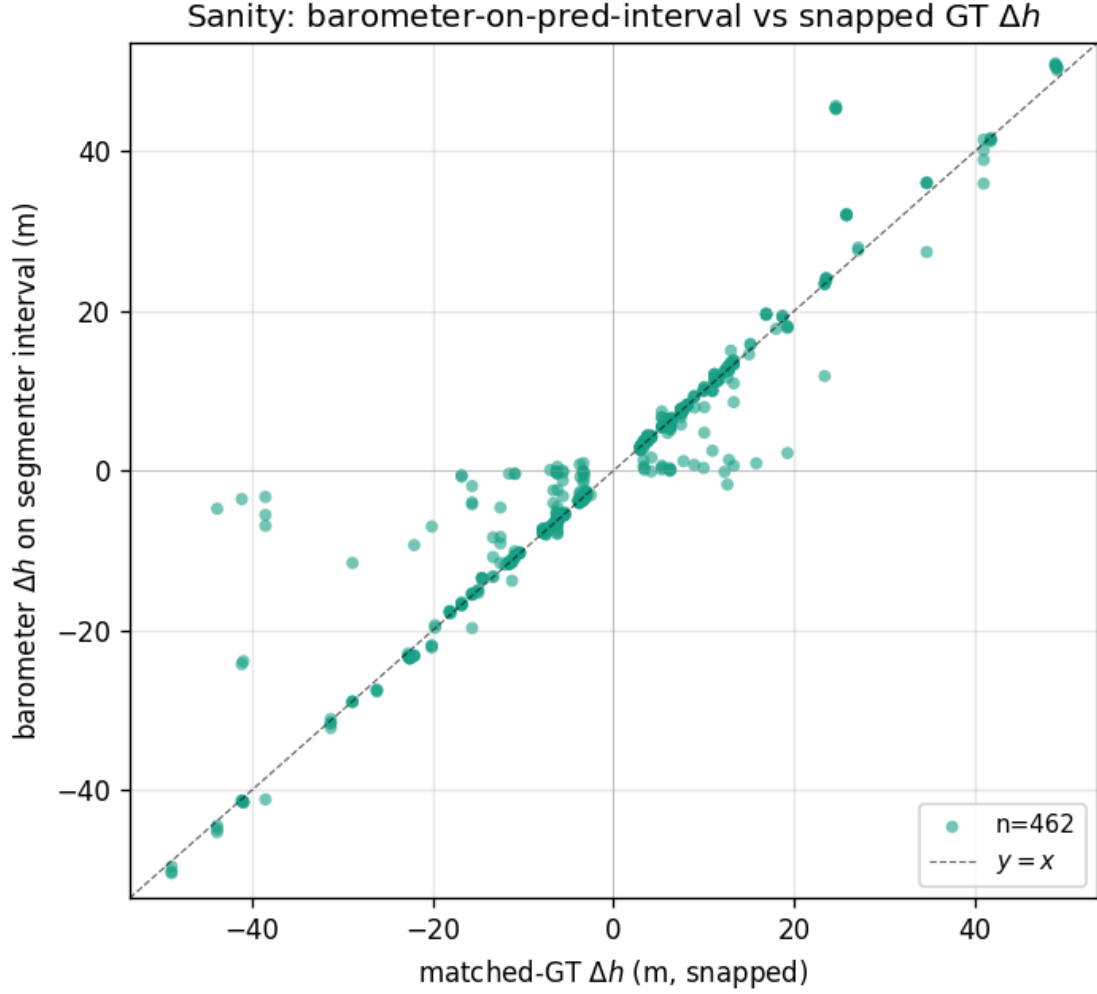


Figure 102: Barometer-derived Δh vs. ground-truth Δh for every matched segment — a sanity check that the "all" view’s barometer truth source agrees with the per-ride GT. Clustering near $y = x$ validates that the third-view-vs-barometer scoring is honest under the new segmentation as well.

15 Gyroscope orientation reconstruction: recovering the true vertical acceleration

The pipeline described so far is accelerometer-only and assumes the phone’s orientation is fixed for the duration of a ride (Sec. 1 notes it carries “no orientation fusion”). This section lifts that assumption. We fuse the phone’s *gyroscope* with its accelerometer to track how the device is oriented *through* the ride, rotate the acceleration into a gravity-aligned world frame, and hand the two stages a cleaner vertical channel a_{vert} . We implemented five standard orientation filters, evaluated all of them end-to-end, and adopt the **Mahony** filter, which gave the best segmentation and prediction on our data. The reconstruction is a config-selectable first step of both `Segmenter.detect` and `Predictor.predict` (`reconstruct=none` reproduces the baseline byte-for-byte), so everything below is a drop-in improvement, not a re-architecture.

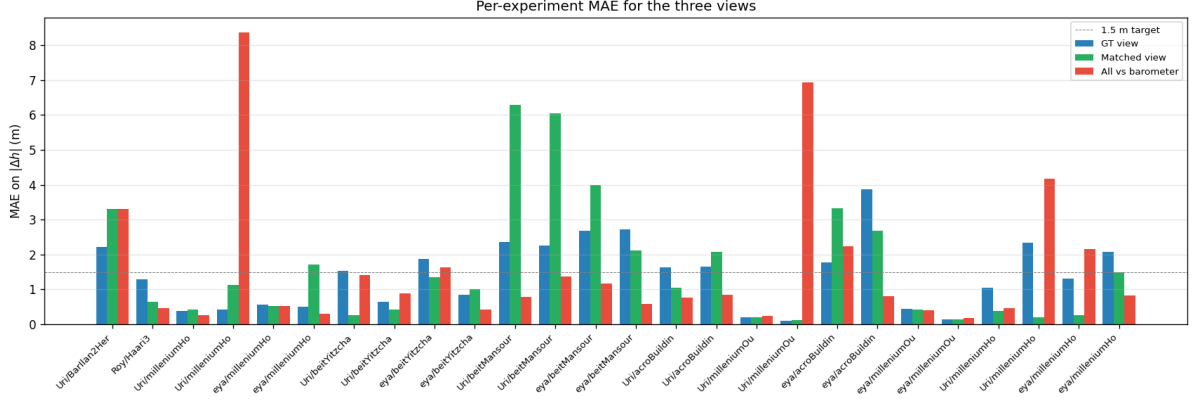


Figure 103: Per-experiment MAE across the 26 experiments, all three views. Bars reveal whether one or two pathological experiments drive the aggregate, vs. uniform performance across phones and buildings.

15.1 Why a gyroscope helps

Both stages consume the vertical channel $a_{\text{vert}}(t) = (\mathbf{a}(t) - \hat{\mathbf{g}}) \cdot \hat{\mathbf{g}} / |\hat{\mathbf{g}}|$, the accelerometer projected onto a *single* gravity direction $\hat{\mathbf{g}}$ estimated once from the pre-ride stationary window. At rest the accelerometer reports the specific force $\mathbf{a} = -\mathbf{g}$, a vector of magnitude $g \approx 9.81 \text{ m/s}^2$ pointing to world $+Z$; because the phone is tilted, that vector does not lie along the body z axis, and its body coordinates encode the tilt (Fig. 109). Projecting onto a fixed $\hat{\mathbf{g}}$ is therefore exact only while the tilt is frozen.

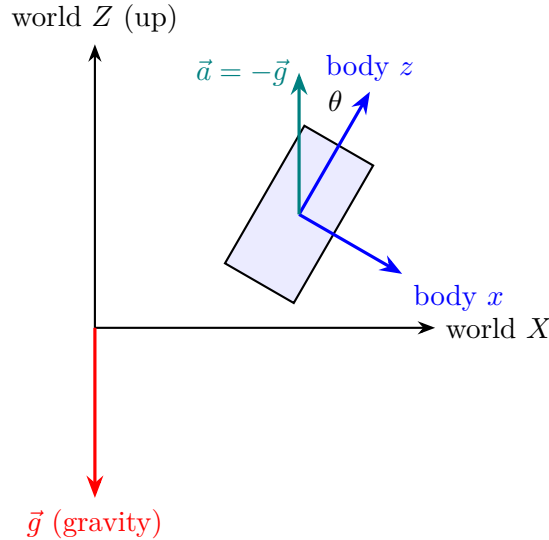


Figure 109: At rest the accelerometer measures the *specific force* $\vec{a} = -\vec{g}$ (pointing to world $+Z$, magnitude g). Because the phone is tilted by θ , this vector does *not* lie along the body z axis; its coordinates in the body frame encode the tilt. The vertical channel a_{vert} is the projection of $\vec{a}$ onto the gravity direction, so a wrong tilt estimate contaminates it with a slice of the 9.81 m/s^2 gravity term.

In practice the phone is held, pocketed, or glanced at, so its tilt drifts over the ride (Fig. 111). A stale $\hat{\mathbf{g}}$ then leaks a slice of gravity into a_{vert} : a tilt error of $\delta\theta$ injects a spurious acceleration $g \sin \delta\theta$, so a mere 5° (0.09 rad) already contributes $\approx 0.85 \text{ m/s}^2$ — comparable to the ride's own signal, and, because ZUPT integrates twice, worth metres of Δh error over a 10 s ride.

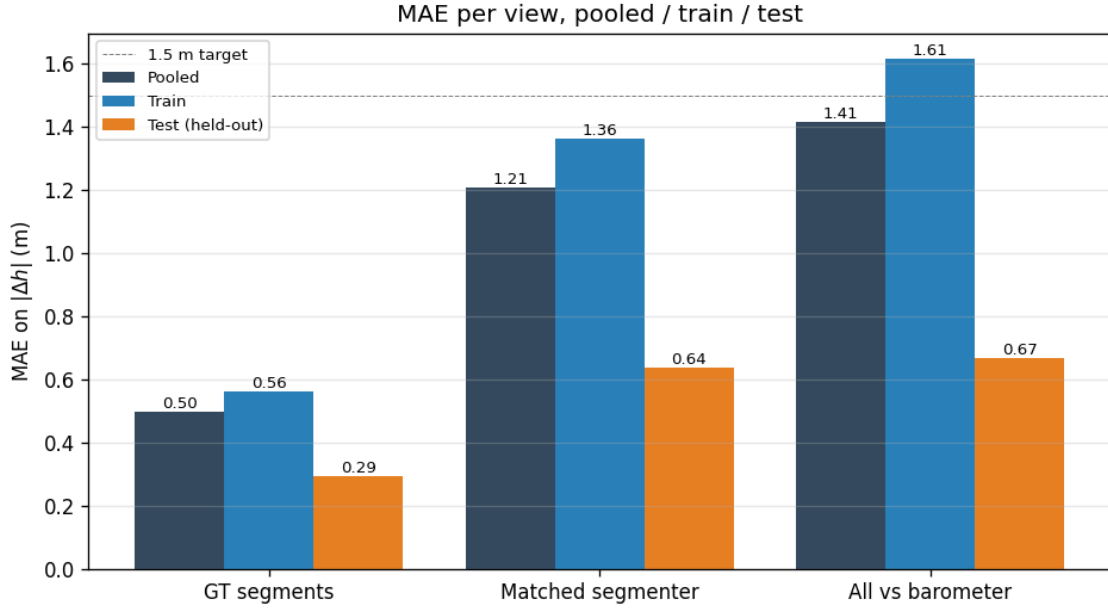


Figure 104: Accepted-only MAE. The quality filter removes a curated set of low-confidence predictions; what remains is the predictor’s operating distribution for downstream consumers that gate on `accepted=True`.

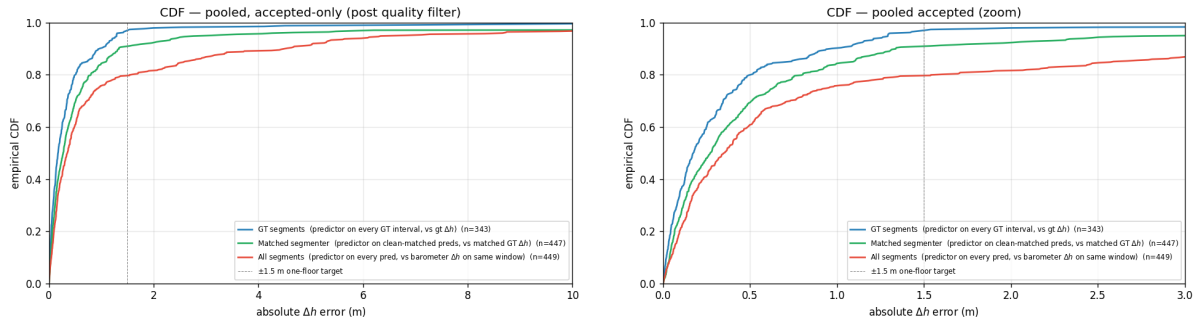


Figure 105: Accepted-only pooled CDF. The right zoom emphasises the ≤ 1.5 m operating regime.

The gyroscope measures the phone’s angular rate $\omega = (\omega_x, \omega_y, \omega_z)$ about its own body axes (Fig. 110). Only *roll* and *pitch*, which tilt the body z axis away from vertical, affect a_{vert} ; *yaw* (rotation about the vertical) leaves the gravity direction unchanged, which is why a magnetometer is unnecessary here. Integrating ω tracks how the tilt evolves, so the gravity direction can be followed through the ride instead of assumed constant. The gyroscope alone drifts (its bias integrates without bound); the accelerometer alone is corrupted by the ride’s own motion. Fusing them — gyro for the fast changes, accelerometer as a slow absolute reference whenever $|\mathbf{a}| \approx g$ — is exactly what an *orientation filter* does.

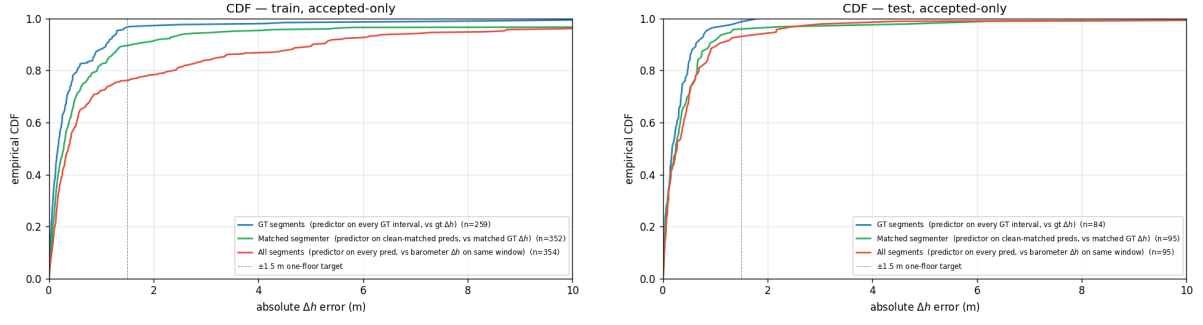


Figure 106: Accepted-only per-split CDFs. Test-split matched-view fraction ≤ 1.5 m on accepted predictions only is 95.8%.

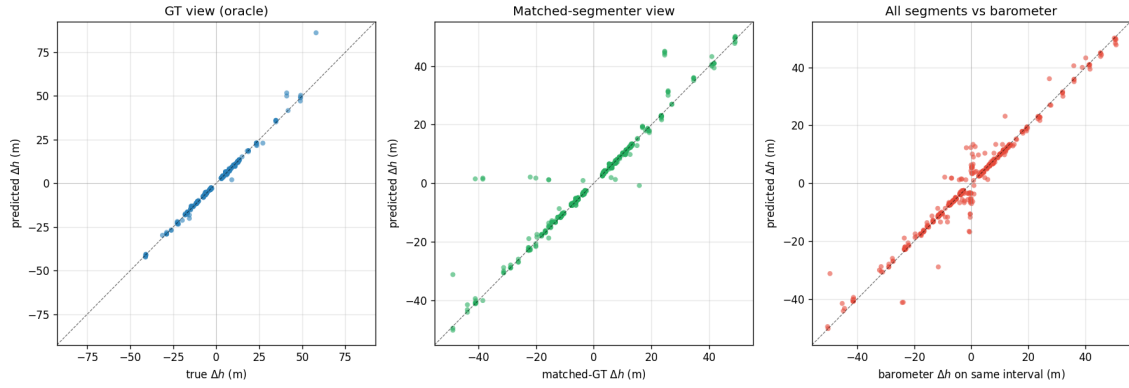


Figure 107: Accepted-only scatter: predicted Δh vs. truth for all three views. Tightness near the diagonal is the deployment story.

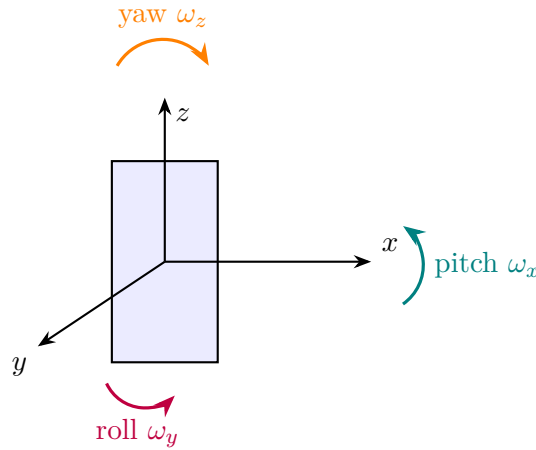


Figure 110: The three body-frame rotations the gyroscope measures as angular rates: *roll* about y , *pitch* about x , *yaw* about z . Only roll and pitch (which together set the *tilt* of z away from vertical) affect a_{vert} ; yaw leaves the gravity direction unchanged and is irrelevant here.

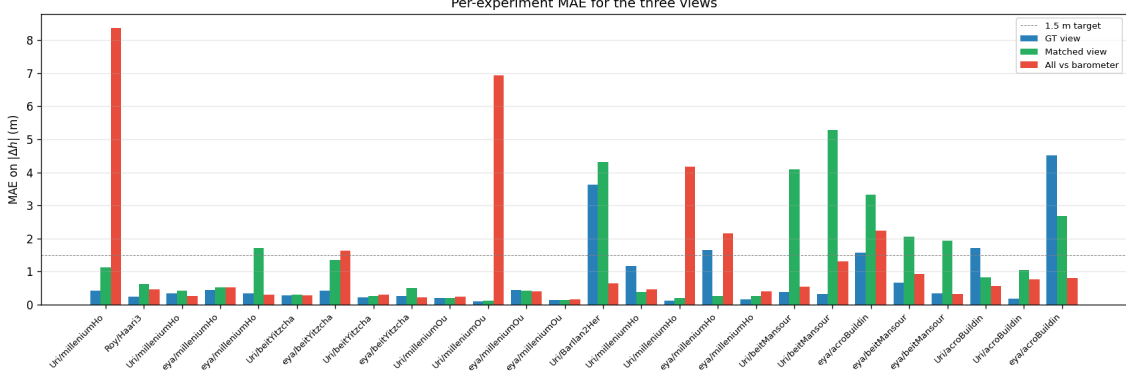


Figure 108: Per-experiment MAE, accepted-only. Compare with the unfiltered Figure 103 to see which experiments contributed rejected predictions.

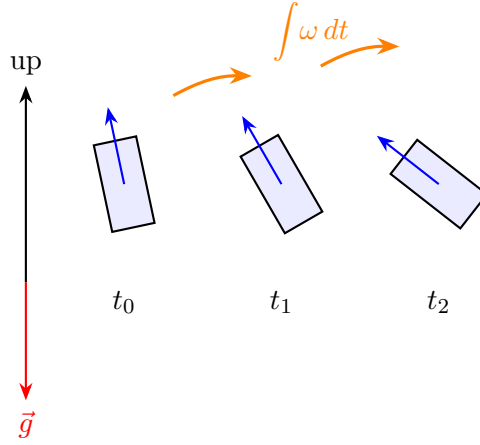


Figure 111: During a ride the phone is handled, so its tilt drifts $\theta(t_0) \rightarrow \theta(t_1) \rightarrow \theta(t_2)$. A single gravity vector estimated from the pre-ride window (the baseline) is stale by t_2 , leaking a time-varying bias into a_{vert} that mimics a ride pulse. Integrating the gyroscope, $q(t) = q(t_0) \otimes \int \omega dt$, tracks the tilt so the projection stays aligned; the accelerometer only re-anchors it when $|\mathbf{a}| \approx g$.

15.2 Orientation reconstruction pipeline

We represent orientation by a unit quaternion q mapping the body frame to the world frame (world $Z = \text{up}$), with rotation matrix $R(q)$. All five filters share the same skeleton:

1. **Seed.** Average the accelerometer over the opening stationary window and set $q(t_0)$ to the shortest rotation taking measured gravity onto world $+Z$ (yaw is left free; it does not affect a_{vert}).
2. **Propagate** with the gyroscope, using the quaternion kinematics

$$\dot{q} = \frac{1}{2} q \otimes \begin{bmatrix} 0 \\ \omega \end{bmatrix}, \quad q_{k+1} = q_k \otimes \exp\left(\frac{1}{2} \omega_k \Delta t\right), \quad (44)$$

where $\otimes$ is the Hamilton product and $\exp$ the quaternion exponential of a rotation vector.

3. **Correct** the accumulating gyro drift with the accelerometer: each filter nudges q so that the predicted gravity direction $\hat{v} = R(q)^\top \hat{z}$ aligns with the measured one $\hat{a} = \mathbf{a}/|\mathbf{a}|$. The correction is down-weighted by the *accel-trust* gate

$$w = \exp\left(-\frac{1}{2}((|\mathbf{a}| - g)/(\tau g))^2\right) \in (0, 1], \quad (45)$$

which collapses to 0 during the ride’s acceleration pulses (when $|\mathbf{a}| \neq g$) so ride motion is never mistaken for tilt.

4. **Rotate and re-emit.** With the tracked $q(t)$, $\mathbf{a}_w = R(q) \mathbf{a}$ and $a_{\text{vert}} = \mathbf{a}_{w,z} - g$. To leave the stages’ internal gravity estimator untouched we feed them the *virtually-flat-phone* signal $(\mathbf{a}_{w,x}, \mathbf{a}_{w,y}, \mathbf{a}_{w,z} + g)$ — gravity restored on $+z$ — so their $|\hat{\mathbf{g}}| \approx g$ gate still fires and the projection returns the true signed vertical acceleration.

15.3 The chosen filter: Mahony

We adopt the **Mahony** passive complementary filter on $SO(3)$. It won both stages on our data (Sec. 15.5) while being the cheapest to run and one of only two candidates (with the ESKF) that estimate the gyro bias online. Mahony forms the attitude error directly as the vector cross product between the measured and predicted gravity directions,

$$\mathbf{e} = \hat{\mathbf{a}} \times \hat{\mathbf{v}}, \quad \hat{\mathbf{v}} = R(q)^\top \hat{\mathbf{z}}, \quad (46)$$

so $\mathbf{e}$ points along the axis that rotates the estimate onto the measurement and $\|\mathbf{e}\| \approx \sin(\text{tilt error})$. It then drives a *proportional–integral* correction back into the gyro kinematics:

$$\dot{\mathbf{b}} = -k_I w \mathbf{e} \quad (\text{bias integrator}), \quad (47)$$

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} - \mathbf{b} + k_P w \mathbf{e} \end{bmatrix} \quad (\text{corrected kinematics}). \quad (48)$$

The proportional term $k_P w \mathbf{e}$ pulls the fast gyro estimate toward the accelerometer’s gravity reference; the integral term $-k_I w \mathbf{e}$ learns the slow gyro bias $\mathbf{b}$ and subtracts it, so the estimate does not drift during the long, low-excitation cruise phase of a ride. The trust weight w from Eq. (45) multiplies *both* corrections, freezing the update to pure gyro integration exactly when the elevator is accelerating. Discretising with a renormalisation each sample,

$$\mathbf{b}_{k+1} = \mathbf{b}_k - k_I w_k \mathbf{e}_k \Delta t, \quad \mathbf{q}_{k+1} = \frac{\mathbf{q}_k \otimes \exp(\frac{1}{2}(\boldsymbol{\omega}_k - \mathbf{b}_{k+1} + k_P w_k \mathbf{e}_k) \Delta t)}{\|\mathbf{q}_k \otimes \exp(\cdot)\|}. \quad (49)$$

Each update is a cross product and one quaternion product ($O(1)$ per sample), has two interpretable gains (k_P, k_I) , and degrades gracefully: with $k_I = 0$ it reduces to a plain complementary filter.

15.4 The other four filters and how they differ

All four solve the same fuse-gyro-with-gravity problem; they differ in *how* the accelerometer correction is formed.

Complementary. The minimal baseline: integrate the gyro (Eq. 44), then apply a fraction αw of the shortest-arc rotation $\delta \mathbf{q}$ taking $R(q)\hat{\mathbf{a}}$ onto $\hat{\mathbf{z}}$, $\mathbf{q} \leftarrow \text{slerp}(1, \delta \mathbf{q}; \alpha w) \otimes \mathbf{q}$. No bias state, no integral term — a single blend gain α . Mahony *is* this filter plus a bias integrator expressed in the Lie algebra.

Madgwick. Replaces the cross-product error with one step of *gradient descent* on the gravity-alignment cost $f(q) = R(q)^\top \hat{\mathbf{z}} - \hat{\mathbf{a}}$: $\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \boldsymbol{\omega} - \beta w \nabla f / \|\nabla f\|$ with $\nabla f = J^\top f$. A single tunable rate β . Equivalent in spirit to Mahony’s proportional term but with a normalised-gradient step; at our default β it over-corrected on some rides and *hurt* segmentation (Sec. 15.5).

Valenti / AQUA. Computes the accelerometer’s implied orientation *algebraically* (a closed-form quaternion Δq_{acc} rotating $R(q)\hat{a}$ onto $\hat{z}$) and fuses it with the gyro prediction through an *adaptive* gain: the interpolation factor ramps linearly to zero once $\|\mathbf{a}\| - g/g$ exceeds a threshold, and switches LERP→SLERP for large corrections. More principled gain scheduling than a fixed α , but no bias state.

ESKF. The probabilistic optimum: an *error-state Kalman filter* carrying a covariance P over the small attitude error $\delta\theta$ and bias error δb . The gyro propagates the nominal state and P ; the accelerometer enters as a measurement with Jacobian $H = [[\hat{v}]_{\times} \ 0]$ and noise $R = (\sigma_a/w)^2 I$ (the trust gate enters as measurement noise). It computes a Kalman gain, injects $q \leftarrow q \otimes [1, \frac{1}{2}\delta\theta]$, and resets. Most accurate in principle — and best on the tightest-CI prediction metric — but the heaviest to run and to tune (needs process/measurement noise priors).

Mahony sits at the sweet spot: it keeps the ESKF’s online bias estimation and the complementary filter’s simple cost, wins our two end-to-end metrics, and needs no noise model.

15.5 Results: how much reconstruction improves each stage

The tables and figures here are produced by `scripts/reconstruction_comparison_report.py` (all six variants through both stages, `--kind all`) and by the Mahony configuration of `src.prediction.evaluation.evaluateOnData / src.segmentation.evaluate.evaluateOnData`. All numbers are on the full resolved set (34 experiments; 619 prediction segments, 627 ground-truth rides).

Prediction. Table 34 shows reconstruction roughly *halves* the mean absolute Δh error for both predictors — ZUPT $2.55 \rightarrow 1.07$ m, trapezoid $1.73 \rightarrow 0.94$ m — and shrinks RMSE far more (ZUPT $18.8 \rightarrow 3.2$ m), because the worst damage was on a handful of heavily rotated rides whose fixed-gravity error ran to tens of metres. The fraction of segments within one floor (1.5 m) rises (ZUPT $75.8 \rightarrow 86.9\%$) and the median CI narrows, i.e. the predictions get both more accurate and more confident.

Algorithm	Reconstruction	MAE (m)	RMSE (m)	Cov@90	≤ 1.5 m	med. CI (m)
ZUPT	none (baseline)	2.545	18.827	91.9%	75.8%	2.256
	Mahony	1.068	3.238	91.6%	86.9%	1.237
Trapezoid	none (baseline)	1.734	11.313	83.5%	84.8%	0.894
	Mahony	0.943	2.734	86.4%	88.0%	0.892

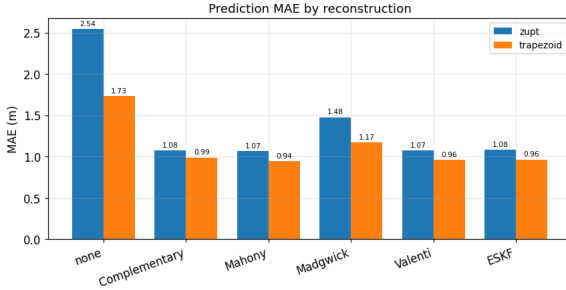
Table 34: Prediction accuracy on the full set, baseline (no reconstruction) vs. Mahony. Reconstruction halves MAE and cuts RMSE by 3–6 $\times$.

Segmentation. Table 35 shows the detector catches more rides ($498 \rightarrow 550$ of 627) while its false positives collapse ($43 \rightarrow 8$), lifting $F1^*$ from 0.847 to 0.921 and IoU-F1 from 0.726 to 0.774: a cleaner a_{vert} makes the trapezoid template match where it should and stop matching tilt artefacts where it should not.

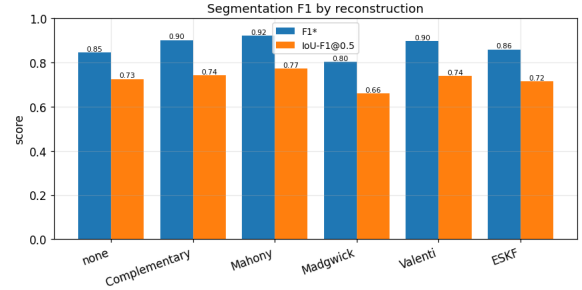
Reconstruction	n_{gt}	detected	missed	FP	$F1^*$	IoU-F1@0.5	precision
none (baseline)	627	498	116	43	0.847	0.726	0.907
Mahony	627	550	67	8	0.921	0.774	0.970

Table 35: Segmentation accuracy on the full set, baseline vs. Mahony. False positives drop $43 \rightarrow 8$; the detector both finds more rides and fires spuriously far less often.

Which filter, and an honest caveat. Across all five filters Mahony was the best or tied-best on every end-to-end metric except the very tightest CIs, where the heavier ESKF edges ahead. Not every filter helps: at its default gain **Madgwick over-corrects and degrades segmentation**, driving false positives up to 92 (worse than the baseline’s 43) — a reminder that an orientation filter must be validated on the downstream task, not just on attitude error. Figure 112 summarises both stages across all variants, and Figure 113 shows the mechanism on one rotated ride: the fixed-gravity a_{vert} carries a tilt-induced bias in the calm mid-ride section that Mahony removes.



(a) Prediction MAE by reconstruction.



(b) Segmentation $F1^*$ / IoU-F1 by reconstruction.

Figure 112: Every reconstruction beats the baseline on prediction MAE; Mahony is best or tied-best on both stages, while Madgwick’s segmentation regresses.

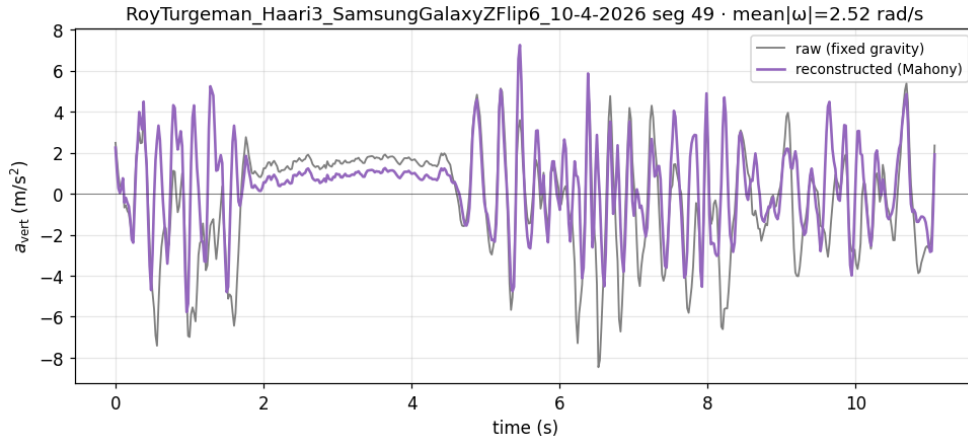


Figure 113: Raw fixed-gravity a_{vert} vs. Mahony-reconstructed vertical acceleration on a rotating ride (mean $|\omega| \approx 2.5$ rad/s). The raw trace carries a tilt-induced bias in the calm mid-ride section that reconstruction removes.

The benefit is concentrated exactly where the model predicts: Figure 114 plots each ride’s rotation energy (mean $|\omega|$) against the error reduction reconstruction buys — near-zero for upright rides, large for the most-rotated ones.

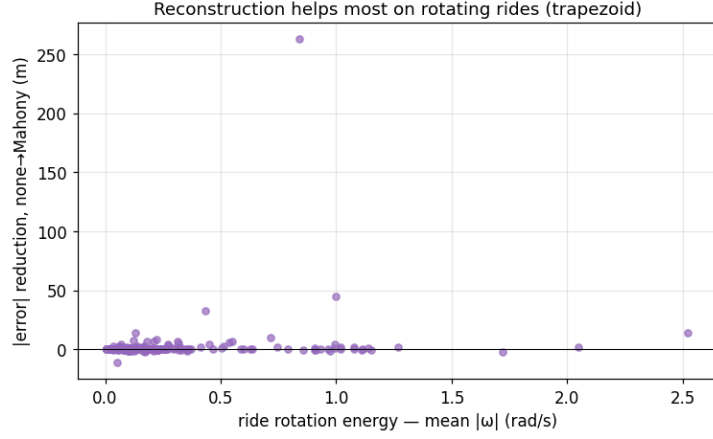


Figure 114: Per-ride rotation energy vs. the trapezoid $|\text{error}|$ reduction from Mahony reconstruction: the gain concentrates on the most-rotated rides, and is negligible on upright ones (so reconstruction never hurts the easy case).

Calibration holds out. Because the trapezoid/ZUPT confidence intervals are conformally calibrated, we refit the conformal multiplier on the *train* split with Mahony reconstruction and scored the held-out *test* split with it (Table 36). Test-split rides are largely upright, so the gain is smaller than on the full set; even so, Mahony *improves* the within-one-floor rate on both predictors (ZUPT $87.0 \rightarrow 91.7\%$, trapezoid $90.7 \rightarrow 93.5\%$) and tightens or matches the median CI, all while its 90% interval keeps $\geq 90\%$ coverage. Reconstruction does no harm on upright rides and helps even here, and it does not break calibration.

Algorithm	Reconstruction	n_{clean}	Cov@90 (GT-clean)	$ \tilde{e} $ (m)	$\tilde{w}$ (m)	≤ 1.5 m
ZUPT	none (baseline)	108	94.4%	0.19	1.00	87.0%
	Mahony	108	95.4%	0.21	0.91	91.7%
Trapezoid	none (baseline)	108	91.7%	0.29	0.83	90.7%
	Mahony	108	93.5%	0.27	0.84	93.5%

Table 36: Mahony with its own conformal calibration (refit on train, scored on the held-out test split). Coverage of the 90% CI is maintained ($\geq 90\%$) while the within-one-floor rate improves on both predictors, even on this mostly-upright split.